SPRING BOOT 4 · SPRING FRAMEWORK 7 · JAVA 21

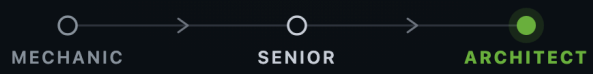
# THE SPRING BOOT MASTERY BOOK

*From your first bean to the architect's chair.*



SOLVED ONE TURN AT A TIME

THE ARC OF EVERY CHAPTER




*"Show me the mechanism. Then show me the system."*

OTMANE OMRY

v2.4 · SEVENTEEN INCIDENTS · ONE HOSTILE REVIEW

The Spring Boot Mastery Book
Part I: The Craft
Chapter 1: The Review
Part II: The Container
Chapter 2: The Container
Chapter 3: Smoke and Mirrors
Chapter 4: The Magic Show
Part III: The Request
Chapter 5: The Front Door
Chapter 6: The Pool
Part IV: The Data
Chapter 7: The Thousand Queries
Chapter 8: The Silent Commit
Chapter 9: The Proxy You Didn't Write
Part V: Production
Chapter 10: The Breach That Wasn't
Chapter 11: Forty Minutes
Chapter 12: Three A.M.
Chapter 13: The Cascade
Part VI: The Architect
Chapter 14: The Split
Chapter 15: Faster
Chapter 16: The Architect's Chair
Capstone
Chapter 17: The Hostile Review
Appendices: The map behind the story

# The Spring Boot Mastery Book

***Show me the mechanism. Then show me the system. From your first bean to the architect's chair, in seventeen incidents and one hostile review. Spring Boot 4 / Spring Framework 7 / Java 21.***

**Otmane Omry**

Software Engineer & Développeur Full Stack · Java, Spring Boot, React, Angular

otmaneomry.github.io · linkedin.com/in/omryotmane

**v2.4, November 10, 2026** · pure-Java edition · runnable labs in `labs/` · version history in Appendix I · version-sensitive facts audited in `docs/VERSION-CLAIMS.md`

**What this is.** This book follows the roadmap.sh/spring-boot tree, with every topic weighted toward what a senior architect needs in 2026. Some things are left out on purpose: Appendix H lists the omissions and where to study them. The book is taught as one continuous story, built to carry a reader from a first Spring bean to senior-architect judgment. The protagonist, Kenza, spent eight years writing frameworkless Java at Meridian Freight, a logistics firm allergic to frameworks: object graphs hand-wired in `main()`, raw JDBC, an embedded Jetty configured line by line. She knows Java well and Spring not at all when she joins the team behind a bug-bounty platform. One year of outages, audits, lost updates and 3 a.m. pages rebuilds her from someone who hand-wires everything into a principal architect. Every chapter is a realistic production incident that forces one roadmap topic to senior depth. Then the chapter climbs to the system-level call behind it: which quality attribute was at stake, what the decision record says, which alternatives lost and why. You remember the concepts through what happened to Kenza, and you practice deciding what she should do next before the text tells you.

**Who this book is for.** This book is written for two kinds of reader. If you know core Java (generics, collections, lambdas, basic concurrency) and basic SQL but nothing about Spring, you sit in Kenza's seat. You learn every mechanism with her, from first principles. Core Java is the only hard prerequisite, but the climb is fast, and the Part primers are your handholds. Each Part opens with a half-page primer in plain prose on its core nouns. Read it if Spring is new to you, and skip it without loss if Spring is your day job. If you already run Spring in production, read the same incidents for what sits above the mechanism: styles, boundaries, consistency, the -ilities, the decision reps. Every chapter opens at a level a beginner can reach and climbs to the architecture on top, and it talks down to neither reader. The arc inside every chapter is the arc of the book: *mechanic → senior → architect*.

**Lineage.** This is the pure-Java edition. Its prose, cast, incidents, four-beat debriefs and architect layer evolved from earlier drafts that are not published here. It ships with companion artifacts. `examples/` holds commented `.java` drill files, one folder per topic. `labs/` is a multi-module Maven project (Spring Boot 4 · Java 21) with six labs: a first application you can run, then five of this book's traps as green tests you can break yourself (`cd labs && ./mvnw test`).

---

## How to read this book

**Four passes, each at its own speed:**

1. **The story pass:** read it like a novel. The book asks one thing of you: at every ⏸ **Your call** beat, stop and commit to an answer, out loud or in a margin note, before you read

the reveal. The plot pauses exactly where an architect would have to decide. Predicting is the rep: retrieval practice disguised as suspense.

2. **The debrief pass:** go back to the `## The debrief` sections only. Each one opens with the chapter's kill-shot (the interview question the chapter prepares you to answer) and its follow-ups. Answer aloud for 90 seconds before you read on, then tick each of the four beats you said: claim, mechanism, trade-off, failure mode. Three ticks pass. With fewer, re-read the scene named in the story anchors and try again the next day. The model answer follows below a rule, in the same four beats (Claim → Mechanism → Trade-off → Failure mode). After it come **THE ARCHITECT'S LENS** (which -ility was at stake, what the ADR records, which alternatives were rejected and why), the story anchors and the interview ammo. Every chapter has one kill-shot, numbered by chapter: kill-shot 3 is Chapter 3's question.
3. **The decision pass:** replay only the ⚖ **Your move, architect** forks. Each is a realistic decision with three options. Commit, then check your reasoning against the revealed trade-offs. Most forks replay a decision you just watched. For those, argue the losing options at full strength before the reveal. Ch.15 hands you the rubric instead of the answer, and Ch.16 and 17 hand you the pen. One warning from the index in Appendix D.1: the balanced option wins often but not always. It loses outright at least twice, and one winning call is overturned at the hostile review. If you catch yourself picking the moderate option by pattern instead of analysis, you've stopped doing the rep. Watch which kind of door each decision is. Two-way decisions reward keeping options open. One-way doors (the public contract, the consistency model) reward committing decisively on today's information. At least two forks in this book are one-way doors where "keep it purchasable" is the documented trap.
4. **The anchor drill:** cover the page, recall the scene (*the 28-second dashboard…*), re-derive the mechanism, then name the -ility it threatened. If the scene won't surface both, re-read that chapter.

**The four-week path.** If you have an interview date, read, run and say aloud on this schedule; a chapter takes about 35 minutes to read and 10 to drill.

| Week | Read | Run | Say aloud |
|---|---|---|---|
| 1 | Parts I and II (Ch.1 to 4) | lab-00, lab-01 | kill-shots 1 to 4 |
| 2 | Parts III and IV (Ch.5 to 9) | lab-02, lab-05 | 5 to 9, then 1 to 4 again |
| 3 | Part V (Ch.10 to 13) | lab-03, lab-04 | 10 to 13, then 5 to 9 again |
| 4 | Part VI and Ch.17 | nothing new | all 17, cold, twice |

**The recall hooks:**

- **B.E.A.N.S.:** Bootstrap · Endpoints · Access-to-data · Non-functional · Scale.
- **The Request Journey:** socket → connector → filter chain → DispatcherServlet → controller → `@Transactional` proxy → repository → persistence context → DB → back. It is born on the Wall in Chapter 5, annotated all year, and diagrammed in Appendix E.
- **The senior answer skeleton:** Claim → Mechanism → Trade-off → Failure mode. It is Rachid's whole curriculum, in four beats.
- **The -ilities ledger:** scalability · availability · evolvability · performance · security · operability · cost, plus the ones that specific incidents add (integrity, maintainability, testability). They are the architect's subject: every major decision in this book names which of them it serves and which it taxes. An answer that names no -ility is a developer answer.
- **The ADR trail:** Glasshouse numbers its Architecture Decision Records, and so does this book. When a chapter mints one (context → decision → consequences, half a page, dated), it joins the trail in Appendix F. A decision you can't show a record for gets relitigated forever.
- **The story anchors:** every gotcha is welded to a scene. Recall the scene, and the mechanism comes with it.

**A note on temperature.** The chapters run hot by design, and the incident prose stays hot. The flat ground is deliberate and lives elsewhere: the plain-prose primers before each Part and the debriefs after each chapter are the rest stops. Use them that way, because the narration will not slow down for you.

**Spaced callbacks.** From Chapter 2 onward, each chapter deliberately brings back one or two earlier incidents (*"this is February's proxy ghost wearing a new coat"*). When you hit a callback, pause one beat and re-derive the original mechanism from memory before reading on. That is the spacing schedule at work.

---

## The cast

| Who | Role | Why they matter |
|---|---|---|
| **Kenza** | Protagonist. Eight years of frameworkless Java at Meridian Freight: hand-wired object graphs, raw JDBC, Jetty configured by hand. Strong Java, zero Spring | Her plain-Java instincts are the book's recurring device: sometimes a head start, sometimes the trap. Her arc: humbled → mechanic → debugger → teacher → incident commander → system decider → architect. Watch how early she starts making calls about Atrium instead of about code. |

| Who | Role | Why they matter |
|---|---|---|
| **Rachid** | Principal architect, employee #4, stepping back at year's end | Reviews in green ink and answers questions with smaller questions. His catchphrase ("Show me the mechanism") is the book's thesis. His quieter second question is *"and which -ility does it spend?"* |
| **Imane** | Junior engineer, joins in June | The mirror arc: she asks the questions everyone avoids and ships the book's second self-invocation bug. She ends the year receiving the Chapter-1 treatment from Kenza. |
| **Youssef** | SRE / platform engineer | Owns Kubernetes, on-call, and the Monday brick ceremony. The voice of operational debt, and so of **operability**, the -ility developers forget until 3 a.m. |
| **Leila** | Product manager | Every technical failure reaches her as money: the **cost** column of the ledger. She argues the user's side of every trade-off and teaches Kenza that stating a trade-off is a negotiation, not a lecture. |
| **@sable** | Top external researcher on the platform's own program | Never appears in person, only through reports, retest comments and one late-night email. The book's conscience: the outside world, always testing your claims. |
| **Malika** | Triage lead | The human face of the workflows the code keeps breaking. Because of her, "data integrity" always has a name attached. |

**The company:** Glasshouse is a bug-bounty and vulnerability-disclosure platform (a mini HackerOne). Companies run Programs, Researchers submit Reports, Triagers validate and route them, and Admins manage programs and payouts. The core system is a seven-year-old Spring monolith codenamed Atrium (Spring Boot 4.0.x, Java 21, Maven, PostgreSQL, Kubernetes). It is later fronted by a gateway, with one service split out where the seam is real. Glasshouse dogfoods itself through its own public program, Stonethrow, because people who live in glass houses should invite the stones.

**The furniture** (recurring memory anchors):

- **The Wall:** the do-not-erase whiteboard where the Request Journey is born (Ch.5) and annotated all year. It also accumulates the context map: the borders inside Atrium that eventually decide what splits and what stays.
- **"Show me the mechanism.":** Rachid's catchphrase. It passes down the succession chain until Imane hears it from Kenza in the book's final line.
- **The Canary:** the dented rubber canary on the desk of whoever owns the gnarliest bug. You explain the problem to it, out loud, before you may escalate to a human.

- **The Brick:** the pager-shaped relic handed over every Monday with whatever on-call wisdom the outgoing person has the dignity to admit.
- **The green pen:** Rachid reviews on paper, in green. Where the pen ends up is how the book ends.
- **The ADR binder:** a thin, unglamorous folder in the repo, `docs/adr/`. It starts this book nearly empty and fills as you read.

---

## Contents

**Part I: The Craft**

1. **The Review.** Her first PR gets 47 green-ink comments. Clean code, SOLID, constructor injection and DTOs at the boundary, with the dependency rule, the first seam of a modular monolith, and ADR-001 underneath them.

**Part II: The Container** 2. **The Container.** A startup crash and a cross-user data leak. IoC/DI, scopes, the bean lifecycle and circular dependencies, then architecture as something you enforce (fitness functions). 3. **Smoke and Mirrors.** Triage updates half-save in production. AOP, proxies and self-invocation, then the boundary between leaning on the framework and fighting it. 4. **The Magic Show.** A dependency bump makes a bean vanish, only in prod. Auto-configuration, conditions and config precedence, then evolvability as a supply-chain property.

**Part III: The Request** 5. **The Front Door.** @sable reports that the API leaks its floor plan. The Request Journey, validation and `ProblemDetail`, then API design as contract governance. 6. **The Pool.** Launch-day 503s with idle CPUs. Thread-per-request, pool exhaustion and timeouts, then availability math: queues, bulkheads, blast radius.

**Part IV: The Data** 7. **The Thousand Queries.** The 28-second dashboard. Persistence context, N+1 and `LazyInitializationException`, then performance as a designed property. 8. **The Silent Commit.** Half-saved failures and lost updates. Transactions, rollback rules and locking, then consistency models chosen on purpose. 9. **The Proxy You Didn't Write.** Spring Data, projections, migrations and pool sizing, then the aggregate boundary, where DDD starts paying rent.

**Part V: Production** 10. **The Breach That Wasn't.** @sable's critical against Glasshouse itself. The filter chain, JWT and CSRF/CORS, then security architecture: token boundaries, trust zones, zero-trust posture. 11. **Forty Minutes.** The CI suite nobody trusts. Slices, context caching and Testcontainers, then the test pyramid as an architectural fitness function. 12. **Three A.M.** Paged blind, and later Kubernetes kills healthy pods. Actuator, probes and metrics, then observability as a designed property with an SLO attached. 13. **The Cascade.** One vendor outage takes down everything. Caching, async and circuit breakers, then resilience composition: backpressure, idempotency, honest degradation.

**Part VI: The Architect** 14. **The Split.** "Microservices by Q3." Bounded contexts, the strangler fig, sagas and the outbox, the modular-monolith defense: argued from both sides, on the record. 15. **Faster.** Idle threads burn money. Virtual threads vs reactive, native image, then runtime selection as a decision memo on cost per -ility. 16. **The Architect's Chair.** A vague spec, a final design review, a green pen changing desks. System design, the five debug drills, reading framework source and mentoring.

**Capstone** 17. **The Hostile Review.** Six months later, an acquirer's hired guns run a due-diligence architecture review. Kenza defends the whole system, every chapter's decision attacked in one room, and the reader sits the exam with her.

**Appendices:** A. coverage map · B. story-anchor index · C. the five debug drills · D. cadence · E. the Wall, rendered · F. the ADR trail · G. the Architect's Principles & flashcard index · H. the omissions & where to study them · I. version history

# Part I: The Craft

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Bean:** an object whose construction and lifetime Spring manages. You declare what your class needs; the container builds it and hands it in. **The container** (`ApplicationContext`): the registry that scans your code for annotated classes, constructs each bean (once, by default), injects its dependencies, and serves it wherever it's needed. **Dependency injection:** a class does not create its collaborators with `new`; it receives them. In this book it always receives them through constructor parameters, into `final` fields. **Controller:** a bean whose methods answer HTTP requests. `@GetMapping("/hello")` on a method routes `GET /hello` to it, and in a `@RestController` the value the method returns becomes the response body. **Jackson:** the JSON library Spring Boot uses by default. When a controller returns a record or an entity, Jackson writes it to the response as JSON (serialization) by calling its getters. **Entity:** a class mapped to a database table. **DTO:** a plain object (usually a `record`) that carries data across a boundary, such as your API. **JPA** (Jakarta Persistence): the standard Java API for mapping classes to database tables and for loading and saving their objects. **Hibernate:** the library that implements JPA, and the implementation Spring Boot's JPA starter brings in. **Session:** the object through which an application uses Hibernate. It loads and saves entities and holds the set of entities it is currently managing, which JPA calls the *persistence context*. A session is opened, used and closed. **Lazy loading:** a fetch strategy. An association (a field that refers to another entity, or to a collection of them) is loaded on first access rather than up front, and that first access needs the session to be open. **Proxy** (a preview): the container may hand back a *wrapper* around your bean that adds behavior such as transactions, security and caching. Chapter 3 looks at it up close. **`@Transactional`:** an annotation that puts a method inside a transaction, the all-or-nothing unit of database work. A proxy around the bean opens the transaction before the method runs and commits it when the method returns.

A first Spring Boot application fits in one source file: one class starts the container, one class is a bean, and one controller receives that bean through its constructor. Start it, request `http://localhost:8080/hello?name=Kenza`, and the response is `Hello, Kenza`.

```java
@SpringBootApplication                    // scan this package, auto-configure, start
        the container
public class FirstBeanApp {
    public static void main(String[] args) {
        SpringApplication.run(FirstBeanApp.class, args);
    }
}

@Component                                // a bean: Spring constructs it once and
        hands it out
class Greeter {
    String greet(String name) { return "Hello, " + name; }
}

@RestController                           // also a bean; its dependency arrives
        through the constructor
class HelloController {
    private final Greeter greeter;
    HelloController(Greeter greeter) { this.greeter = greeter; }

    @GetMapping("/hello")
    String hello(@RequestParam(defaultValue = "world") String name) { return
        greeter.greet(name); }
}
```

**run it:** `labs/lab-00-first-bean`

# Chapter 1: The Review

The applause was real. That was the part Kenza kept replaying afterward: actual applause, in the Thursday sprint demo, three weeks into her first Spring job.

The ticket had been ATR-4112: *monthly disclosure digest for program admins*. That meant one page per month per program, with new reports, status changes, public disclosures and a severity rollup. Kenza had built it the way she'd built forty features in eight years of frameworkless Java at Meridian Freight: read the schema, write the service, wire the endpoint, ship. Four days, soup to nuts. When she paged through a live digest for the Stonethrow program (Glasshouse's own bug-bounty program, run against itself, because people who live in glass houses should invite the stones), Leila said, "Northbeam has been asking for this since October," and somebody actually clapped.

Friday morning there was a printed diff on her desk.

It was on paper, forty-some pages of her own code, annotated in green ink in a small, patient hand. She counted the comments while her coffee went cold: forty-seven. The handwriting belonged to Rachid, principal architect, employee #4, a man she had so far exchanged eleven words with.

Comment #1 sat beside a line that worked. It had run in the demo, in front of everyone, flawlessly:

```
@GetMapping("/api/programs/{id}/digest")
public List<Report> digest(@PathVariable Long id, @RequestParam String month) {
    return digestService.getDigest(id, month);   // JPA entities, straight onto the
        wire
}
```

The green ink said:

> *Works is not done. What happens when this entity meets Jackson after the session closes?*

Kenza read it three times. *At Meridian this was safe. An object held whatever I'd copied out of the ResultSet and nothing else; data never reached back to the database on its own. What session? What closes?* Her instinct said the comment was pedantry, and by lunch she would learn that the instinct was the trap.

## The investigation

She read all forty-seven before going to find him. It felt important, like reading the stack trace before asking for help.

They sorted into species. There were naming comments. Her 612-line `ReportDigestService` had methods called `process`, `handleEntries`, and `getData2`, and beside `getData2` the ink said only: *Names are the first documentation. This one documents despair.* Beside a 130-line method: *If you have to scroll, it's three methods wearing one name.* There was comment #23, beside a helper that caught an exception, logged it, and returned `null`: *A caught exception is a decision. We'll come back to that one. And in this codebase, absence is an `Optional` or a thrown exception, never null, and `Optional` is a return type, not a field.* Comment #47 sat beside nothing at all, because there was nothing there: *Tests?*

There was a long bracket around the service's field declarations:

```
@Service
public class ReportDigestService {   // 612 lines: SQL to HTML, all of human experience

    @Autowired private ReportRepository reportRepository;
    @Autowired private ProgramRepository programRepository;
    @Autowired private ResearcherRepository researcherRepository;
    @Autowired private ActivityRepository activityRepository;
    @Autowired private JavaMailSender mailSender;   // unused; "for later"

    public List<Report> getDigest(Long programId, String month) {
        // ...580 lines of querying, severity math, date juggling, HTML assembly...
    }
}
```

The bracket's annotation: *Five dependencies, one of them unused, none of them visible from outside. How would you construct this class? Try it. Literally try.*

> **Your call.** Do what the green ink asks before you read on. In a plain unit test, with no Spring running, write the line that constructs `ReportDigestService`. What do you pass to the constructor, and what do the five fields hold once the object exists? Core Java is enough to answer.

(If you committed an answer at the pause: `new ReportDigestService()` compiles, because a class that declares no constructor gets a no-argument one. There is nothing to pass, and the object exists with five `null` fields. They are `private` and have no setters, so only reflection can reach them, and the first method that uses a repository throws `NullPointerException`. Rachid explains at the Wall how Spring fills those fields.)

Then comment #1 again. Rachid had circled the return type (`List<Report>`, raw JPA entities) and *dated it*, like a surveyor marking a sinkhole: *This pattern pages someone at 2 a.m. Not necessarily you. Not necessarily here. But it's in the codebase now, and code is copied more often than it's read. R., Jan 16.*

He found her first, two coffees in hand, and pulled a chair to her desk.

"The digest is good," he said, and meant it. "The reporting-window math is correct, including the month-boundary case everyone gets wrong. The severity rollup matches what triage needs. You shipped in four days something we've deferred for two quarters." He set down her coffee. "Now let me show you the difference between code that runs and code that survives."

"Forty-seven comments," Kenza said.

"Forty-seven questions," Rachid said. "Some I'm asking the code. Some I'm asking you. Walk with me."

## What the senior sees

The Wall was a floor-to-ceiling whiteboard by the kitchen with a sun-bleached `DO NOT ERASE` note taped to its frame, dated 2019. Rachid uncapped a marker.

"Eight years of hand-wired Java at Meridian," he said. "So you've paid for dependency injection by hand. Tell me what the container is for."

"Doing my old job," Kenza said. "At Meridian I wired the graph myself, in a two-hundred-line `main()`: repositories before services before handlers, in dependency order. The container builds that graph so I don't `new` everything by hand and ripple every constructor change through forty call sites."

"Good. Inversion of control: the container does the wiring you used to do by hand. That's the half you brought with you, and here's the half you didn't. Component scan finds your classes. The `ApplicationContext` constructs them, wires them, decides their lifetime and, this matters later, reserves the right to hand back something *wrapped*. Transactions, security, caching all arrive through that wrapping. Every `new` on a service class opts that object out of all of it. Your `main()` handed back exactly what it constructed. This container is something you *inhabit*."

*So the ApplicationContext is my Meridian `main()`, generalized, and it writes its own wiring.* "Fine," she said. "But then why pick on my `@Autowired` fields? Field injection is less ceremony. The container's doing the wiring either way."

Rachid capped the marker. "Show me the mechanism."

"The… annotation injects the field."

"That's the claim wearing a trench coat. *How?* When, in the object's life? By whom? What state is the object in beforehand?"

She opened her mouth and closed it. She didn't have it.

"Then here's the mechanism, and it changes the verdict. Spring instantiates your class through its constructor, the no-arg one, since you declared nothing. At that instant your object *exists*, with five null fields. Then a piece of container machinery

(`AutowiredAnnotationBeanPostProcessor`, if you want its name) writes your private fields by reflection: after construction, from outside, an assembly step only the container can perform. Constructor injection inverts that. Dependencies are parameters, and the fields can be `final`: set once, safely visible to every thread, no setter anyone can abuse. So the object is *born complete* or not born at all."

Kenza sat with that. "And the trade-off?" she said, because forty-seven comments had taught her there was always one.

"Honesty hurts. A constructor with five parameters is ugly *on purpose*. The ugliness is the Single Responsibility Principle telling you the class does too much. Your `ReportDigestService` queries, aggregates, renders, and apparently almost sends email. Field injection let it sit at 612 lines, because a sixth dependency would have cost one invisible line. That *anesthesia* is the smell."

He tapped the printed diff, comment #1. "Now the circle I dated. Mechanism again. Your controller returns `Report` entities, and Jackson serializes by walking their getters at response-writing time, after your service has returned. `Report.activity` is a collection, and JPA loads to-many associations lazily by default. Whoever mapped `Report.researcher` marked it lazy too, correctly, because to-one associations default to *eager*, which is its own trap. Lazy means Hibernate handed you proxies: placeholder objects that fetch their data only when a getter is called, and only while their session is still open. Your raw JDBC never produced one of those. A row copied out of a `ResultSet` was complete the moment you held it. Here, every lazy association is an IOU against an open session, and the session is a thing that closes."

Kenza saw it then. The proxies needed an open session to load anything, and serialization ran after her service had already returned. Her Meridian instinct, that a getter is a getter and calling one costs nothing, had been the landmine this time. "But it worked in the demo."

"It worked," Rachid said, "because Atrium runs with open-session-in-view switched on. It's a convenience: it keeps the database session open all the way through rendering, so lazy loads during serialization still succeed. It papers over the boundary instead of designing it."

"And the failure mode?"

"The day someone copies your controller's style into code that *isn't* a web request, say a scheduled job, there's no view to hold the session open, and they meet `LazyInitializationException` at whatever hour scheduled jobs run. Which is why I dated it. Entities are your schema with behavior attached, and the API boundary is a *contract*. Couple them, and every schema change becomes a breaking API change, and every serialization walks a graph you never meant to publish. DTOs at the edge. Records, ideally: immutable, six lines."

On the way back he gestured at her source tree: `controllers/`, `services/`, `repositories/`. "One more. You packaged by layer, like the oldest parts of Atrium. The newer parts package by feature: everything about `digest` lives in `digest`. Mechanism: package-private is the only access boundary the language gives you for free, short of full modules, and it only works when collaborators share a package. Package-by-layer forces every class `public` and scatters one feature across three directories. Package-by-feature gives a change one address and lets your assembler stay invisible to the rest of the monolith. Layers tell me what code *is*, and features tell me what it's *for*."

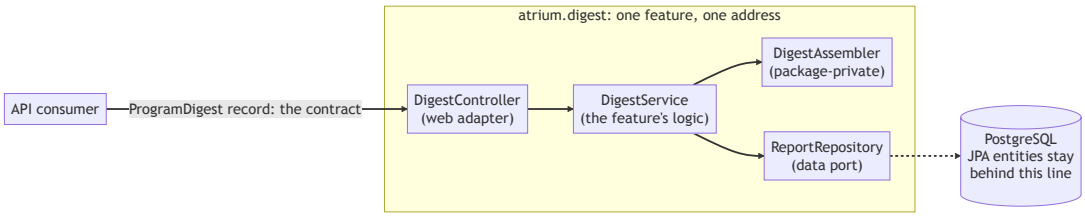
He was right about the directories. The digest lived at three addresses, every class `public` because the layout left no other option.

## The architecture under the review

Kenza looked at the source tree a long moment. "You're not actually reviewing my code, are you. You're reviewing the *system* my code teaches people to build."

Rachid uncapped the marker again, visibly pleased. "Say more."

"Forty-seven comments, and underneath them there are… what, three rules? Dependencies point inward and arrive through constructors. The boundary speaks contracts, never schema. A feature lives at one address with its internals hidden." She took the marker and drew it before she could doubt herself:




"Three rules," Rachid agreed, "and one direction. What you've drawn has older names than Spring. People say *hexagonal*, *ports and adapters*, *clean architecture*, and the differences matter less than the invariant they share: source-code dependencies point toward the domain, and the domain knows nothing about the web or the database. Your `DigestService` depends on a repository *interface*, a port. JPA is an adapter behind it, and the controller is an adapter in front of it. We don't add ceremony: no `ports/` and `adapters/` folders, no four-layer structure for a CRUD feature. That's how cathedrals get built over bus stops. But the dependency rule itself is non-negotiable, because of which -ility it protects. Name it."

"Evolvability," Kenza said slowly. "Schema can change without breaking the API. Persistence can change without touching the digest math. Each feature package is… a wall we could someday cut along."

"And that," Rachid said, "is the only honest definition of a *modular monolith*: one deployable whose internal borders are real enough to cut. Every boundary you respect in January is an option you still own in November. Every boundary you blur is an option you've sold, usually without noticing the sale, which is why we write the sale down." He tapped the repo's thin `docs/adr/` folder on her screen. "Architecture Decision Record. Context, decision, consequences, in half a page. Comment forty-eight: your rewrite ships with ADR-001."

*At Meridian we had wiki pages nobody read,* Kenza thought. *A dated half-page next to the code is just a commit message that survived.* It was the first time all day something had felt easier than home.

## The fix

The rewrite session started with a test. Rachid insisted it come first, as proof.

"Unit test for your digest service," he said. "Plain JUnit. No Spring."

Kenza typed `var service = new ReportDigestService();` and stopped. There was nothing to pass. The dependencies were private fields only reflection could reach. She could boot a whole container to test date arithmetic, or inject mocks by reflection hackery. *Eight years of Java and I cannot construct my own class.* "Okay," she said. "I can't. The class can't exist outside the container. Constructor injection wins on mechanism, not taste."

"Write that sentence down," Rachid said. "It generalizes."

The rewrite took two days, the best two of her month. The god service split along its responsibilities: a `DigestService` that orchestrated, a `DigestAssembler` that aggregated, and the HTML rendering deleted entirely because the frontend wanted JSON. Dependencies arrived through constructors and landed in `final` fields. The service declared what it needed and stayed ignorant of how those needs were built. That was Dependency Inversion made physical. The null-returning helper became two honest signatures: an `Optional` where absence was an answer, a thrown exception where absence was a bug. The string-typed `month` became a `YearMonth`. And the digest entries, which she'd been modeling as one bloated class with a `type` string and a `switch`, became a closed set the compiler could police:

```
public sealed interface DigestEntry
        permits DigestEntry.Submitted, DigestEntry.StatusChanged, DigestEntry.Disclosed
        {

    record Submitted(String reportRef, String title, Severity severity, Instant at)
            implements DigestEntry {}
    record StatusChanged(String reportRef, ReportStatus from, ReportStatus to, Instant
        at)
            implements DigestEntry {}
    record Disclosed(String reportRef, String title, Instant at)
            implements DigestEntry {}
}
```

Sealed plus records made the digest's vocabulary finite, immutable, and pattern-matchable. Add a fourth entry kind, and every exhaustive `switch` would fail to compile until someone handled it. It was the discriminated union she'd been faking with enums since 2019. The boundary got its own record, `ProgramDigest`, owing nothing to JPA. And the service shrank to something that read like the feature description:

```
@Service
public class DigestService {

    private final ReportRepository reports;
    private final DigestAssembler assembler;

    public DigestService(ReportRepository reports, DigestAssembler assembler) {
        this.reports = reports;        // final: born complete, testable with plain new
        this.assembler = assembler;
    }

    public ProgramDigest monthlyDigest(long programId, YearMonth month) {
        var activity = reports.findActivityFor(
                programId, month.atDay(1), month.atEndOfMonth());
        return assembler.assemble(programId, month, activity);
    }
}
```

The controller shrank further, to one route and one record out, with no entity anywhere near the wire:

```
@RestController
@RequestMapping("/api/programs/{programId}/digest")
class DigestController {

    private final DigestService digest;

    DigestController(DigestService digest) {
        this.digest = digest;
    }

    @GetMapping
    ProgramDigest monthly(@PathVariable long programId, @RequestParam YearMonth month)
        {
        return digest.monthlyDigest(programId, month);
    }
}
```

The tests were almost insultingly easy now: `new DigestService(stubRepo, new DigestAssembler())`, no container, and they ran in milliseconds. The whole feature lived in one package, `atrium.digest`, with the assembler package-private. And one more file rode along in the PR, half a page, dated:

```
# docs/adr/0001-feature-seams-in-the-atrium-monolith.md

* Status: accepted   * Date: 2026-01-20   * Deciders: Rachid, Kenza

## Context
Atrium is a seven-year-old monolith with layer-packaged sediment; new work
keeps coupling API contracts to JPA schema and scattering features across
directories.

## Decision
New features package by feature with package-private internals; dependencies
arrive by constructor into `final` fields; the API boundary speaks records,
never entities; the domain depends on ports, not adapters.

## Consequences
Each feature is independently testable without the container and forms a seam
we could later extract along (or deliberately never extract: the option is
the point). Costs accepted: more types at the boundary, mapping code, and
arguments with the parts of the codebase that predate the rule.
```

The PR merged Tuesday with one green comment on the printed copy: *Done. Not just working.*

Rachid stopped by as she closed the ticket and set something on her monitor stand: a dented yellow rubber canary, paint worn off one wing.

"What's this?"

"House rule. Whoever owns the gnarliest open bug keeps the Canary, and you must explain your problem to it, out loud, in full sentences, before you're allowed to escalate to a human. You'll want this. Everyone does eventually." He left an index card beside it, green ink: **Claim → Mechanism → Trade-off → Failure mode.** "Every answer you give me from now on, in those four beats. And Kenza, that god service has at least three more services hiding in it. Keep going. Split it." He nodded toward the kitchen. "The Wall's open to you. Standing invitation."

### ⚖ *Your move, architect: the seams fork*

Rachid's parting assignment (*"that god service has at least three more services hiding in it; keep going"*) left a fork, because the digest wasn't the only reporting code in Atrium. How should Kenza structure the rest of the reporting work? You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Extract classes, keep the layer packages.** Minimal churn, and it fits the team's muscle memory: `services/` just gets four classes where it had one.
- **B. Package-by-feature with enforced seams.** Reporting becomes `atrium.reporting.*` feature packages, with internals package-private, the boundary rules from ADR-001 and (next chapter) a build-time check that *fails CI* when someone violates the dependency direction.

- **C. Extract a reporting microservice now.** The digest is read-only and bursty; a separate deployable could scale and fail independently, and the seam is freshest today.

**The trade-offs, since you've committed: A** spends almost nothing and buys almost nothing. It preserves every-class-public coupling, so the *evolvability* it protects is the codebase's freedom to keep tangling. In six months the four classes have re-fused. **B** is what Kenza chose. It taxes the team a naming argument and some mapping code, and pays out a monolith whose borders are real: cuttable later, testable now, cheap to operate tonight (one deployable, one pager). **C** buys independent scaling *nobody asked for* (the digest serves dozens of admins monthly) at the price of a network boundary's full failure menu: partial failure, latency, versioned contracts, distributed debugging, a second pipeline. The architect's tell is noticing that B keeps C purchasable later while costing a tenth as much today. (When the real split argument arrives in Chapter 14, it will be fought along seams that ADR-001 started cutting in January. And when someone finally does violate these seams, and they will, it won't be malice; it'll be a Tuesday. That's Chapter 2's problem.)

She pinned the index card to her monitor, between the canary and the cold coffee, and realized late, the way you notice a magician's second hand, that his entire review of her `@Autowired` fields had run in exactly those four beats. Humbled, she admitted to herself, and hooked. She had no way of knowing that the decomposition he'd just blessed would take staging down at 6:50 on a late-January morning, or that the failure report would be the politest thing to happen to her all quarter.

## The debrief: Ch.1, The Review

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 1.** Why is field injection a smell, and what does constructor injection actually buy you?

Then the follow-ups, cold:

1. Why does Spring exist; what does IoC buy you over `new`?
2. Why DTOs instead of entities from controllers?
3. When is field injection acceptable?
4. Where does hexagonal architecture earn its cost in a Spring monolith, and where doesn't it?
5. Why not setter injection?

Tick what you said:

- ☐ Claim: Constructor injection into `final` fields is the only injection style that should survive review.

- [ ] Mechanism: Field injection builds the object empty, then the container writes private fields by reflection. Constructor arguments are resolved at instantiation.
- [ ] Trade-off: A constructor shows the true dependency count, which works as an SRP alarm, for a few lines of boilerplate.
- [ ] Failure mode: With field injection the class cannot be built in a plain unit test: fields stay null and the first call throws `NullPointerException`.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Constructor injection into `final` fields is the only injection style that should survive review. **Mechanism:** Spring resolves constructor arguments at instantiation, so the bean is born complete. Field injection instead constructs the object empty and has container machinery (`AutowiredAnnotationBeanPostProcessor`) write private fields by reflection afterward, so only the container can assemble the class. **Trade-off:** Constructors expose the true dependency count: a five-argument constructor is ugly on purpose, an SRP alarm that field injection silences. The cost is a few lines of boilerplate. **Failure mode:** A field-injected class can't be constructed in a plain unit test (fields stay null → `NullPointerException`), can't make its dependencies `final`, and hides creeping coupling until the class is a 612-line god service nobody can instantiate outside production.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Evolvability** is the chapter's subject: DTO boundaries keep schema changes from becoming API breaks, and feature seams keep extraction purchasable. **Testability** is its leading indicator: code you can't construct in a unit test is architecture you can't change cheaply. OSIV trades a hidden **operability** debt (sessions held across rendering) for developer convenience: the framework papers over a boundary the architect should design. The architect's question, every time: *which -ility does this convenience spend, and did anyone approve the purchase?*
- **The ADR:** ADR-001, feature seams in the monolith: package-by-feature, constructor injection into `final` fields, records at the boundary, domain depends on ports. Half a page, dated, next to the code.
- **Rejected alternatives:** field injection + entities-on-the-wire + layer packaging (the path of least ceremony: it optimizes for the first four days and taxes every day after); a ceremonial hexagonal layout with `ports//adapters/` liturgy everywhere (the invariant matters, the folder religion doesn't); extracting a reporting service now (buys unneeded scaling with a distributed system's full failure menu, and B keeps it purchasable).

- **Fight or lean:** *Lean* on the container: it's the wrapping (transactions, security, caching) that makes Spring worth inhabiting. *Fight* OSIV: it's the framework extending your session (the persistence context) through rendering for you. It does not extend the transaction, and a request that touched the database keeps its connection until rendering ends. The architect decides which is which, feature by feature.

**Story anchors**

- The printed diff with 47 green comments, #1 on a line that *worked* → "works is not done": senior code is judged by how it survives, not whether it runs.
- The unit test Kenza couldn't write (`new ReportDigestService()` with nothing to pass) → field injection's mechanism (reflection after birth) and why constructor injection wins on testability, not taste.
- The circled, dated return type `List<Report>` → entities at the API boundary meet Jackson after the session closes; DTO records decouple contract from schema (the landmine detonates in Chapter 7).
- Kenza's marker drawing on the Wall (three rules, one direction) → the dependency rule; a modular monolith is a monolith whose borders are real enough to cut.

**Interview ammo**

- *Follow-up: Why does Spring exist; what does IoC buy you over `new`?* The container owns construction order, lifetime, and wiring of the whole object graph, and gets the chance to decorate beans on the way out (transactions, security, caching). Every `new` opts that object out of all of it.
- *Follow-up: Why DTOs instead of entities from controllers?* Jackson walks getters at response-writing time, after your transaction has returned, so lazy associations either explode (`LazyInitializationException`) or drag your whole schema onto the wire. Records at the boundary make the API a deliberate contract.
- *Follow-up: When is field injection acceptable?* Essentially never in production code. The senior answer names the costs (untestable without the container, non-final state, hidden dependency count) and concedes only test classes, where the container already runs.
- *Architect follow-up: Where does hexagonal architecture earn its cost in a Spring monolith, and where doesn't it?* The dependency rule (domain depends on ports; web and persistence are adapters) earns its cost wherever the domain logic outlives its delivery mechanism. The folder ceremony doesn't: enforce the direction, skip the liturgy, and write the decision down so the next feature doesn't re-litigate it.
- *Follow-up: Why not setter injection?* A setter needs a field that is not `final`, and any caller can use it to replace the dependency after construction. Constructor parameters go into `final` fields: set once, safely visible to every thread, and the object is born complete or not born at all.

**Pointers** → `examples/ch01_core/` (`ContainerAndInjection.java`, `SelfInvocationTrap.java` foreshadows Ch.3)

---

# Part II: The Container

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Scope:** how many instances of a bean exist. The default is **singleton**: one shared instance for the whole application, used by every request at the same time, so any field you put on a singleton is shared by every concurrent request at once. **Bean lifecycle:** the fixed sequence the container walks for every bean: construct → inject dependencies → run init callbacks → wrap in proxies if needed → serve → destroy at shutdown. **BeanPostProcessor:** container machinery that may inspect or replace each bean as it is built. Most "magic" annotations are implemented by one. **AOP proxy:** a generated wrapper that runs logic around your method (open a transaction, check a permission). Calls reach your method through the wrapper. AOP stands for aspect-oriented programming: behavior that many methods need, such as a transaction, is written once and applied around them. **`@Transactional`:** the annotation from Part I that puts a method inside a transaction. What this part adds: if the method throws an unchecked exception, the proxy rolls the transaction back, and nothing written inside it is kept. **Spring Data repository:** the bean you call to load and save one kind of entity, such as `ReportRepository`. Its built-in methods, `save()` and `findById()` among them, join the transaction that is already open. When nothing has opened one, each call runs in a transaction of its own, which commits before the call returns. **Managed entity:** an entity that the current session is tracking. Change a field of a managed entity inside a transaction and Hibernate writes the change to the database at commit, with no `save()` call. That check at commit is called dirty checking. **Auto-configuration:** Boot's conditional defaults, meaning pre-written configuration that activates only when your classpath and properties say it applies. **Starter:** a dependency bundle that pulls in one feature's libraries at tested, compatible versions. **Profile / configuration properties:** named environment overlays (`dev`, `prod`) and typed objects bound from YAML or environment variables.

# Chapter 2: The Container

Staging died at 6:50 a.m., which was, Kenza would later admit, the politest possible time for it.

The nightly CI run had deployed her branch, and the pod never went `Ready`. The branch was the decomposition Rachid had blessed, splitting the 600-line god service into `ReportService` and `ProgramService`. The digest rewrite had taken only the digest out of that class, as `DigestService` and `DigestAssembler`, and this branch split what remained. At the bottom of the log sat her first Spring failure that wasn't about her code's logic at all. The last warning line ended with the message of a `BeanCurrentlyInCreationException`:

```
Error creating bean with name 'reportService': Requested bean is currently in creation: Is there an unresolvable circular reference or an asynchronous initialization dependency?
```

Boot printed no stack trace for it. Below the warning, under the banner `APPLICATION FAILED TO START`, its failure report drew the loop as a small diagram, almost conversational: `reportService` needs `programService`, which needs `reportService`. The container had walked her dependency graph, found a loop, and refused to play. The loop was in her own diff:

```
@Service
public class ReportService {
    private final ProgramService programs;   // digest rows need program names

    public ReportService(ProgramService programs) {
        this.programs = programs;
    }
}

@Service
public class ProgramService {
    private final ReportService reports;     // program summaries need report counts

    public ProgramService(ReportService reports) {
        this.reports = reports;
    }
}
```

Each constructor demanded the other bean, finished and whole, before its own first line could run. No order could satisfy that. *My old `main()` at Meridian would never have let me type this,* she thought. *Hand-wiring the graph meant choosing the construction order yourself, and two `new` calls that each*

*demand the other's result first is a line that can't be written. The container had just rediscovered, at boot, what my fingers used to know.*

It was also, she registered distantly, a Tuesday. ADR-001 was a week old (*dependencies point one direction, arrive through constructors, the boundary speaks contracts*), and Rachid's prophecy had been specific: the first violation wouldn't be malice; it would be a Tuesday. Here it was, written by the constitution's own author. A cycle leaves the direction rule with no direction at all. The one mercy: because both dependencies arrived through constructors, the container had to face the loop while building the graph. It died on the launch pad instead of in a heap dump.

Youssef read over her shoulder, deadpan. "Spring failing at startup is the system doing you a favor. Prod failing at noon never writes you a failure report this polite. Frame it."

She didn't get the chance. At 11:20, support escalated a P1, and the week stopped being funny.

A researcher attached a screenshot of their report-preview pane: an internal draft of a vulnerability write-up, detailed and severity-scored, belonging to a *different researcher*, against a *different company's* program. On Glasshouse, whose entire product was the promise that disclosure stays controlled, that was the apocalypse with a ticket number. Leila appeared at Kenza's desk, laptop held like a subpoena: the affected customer was threatening to pull their program. "I need a sentence I can say out loud. Preferably one with the word *contained* in it."

## The investigation

The preview pane was rendered by `DraftPreviewService`, a class older than half the team. Kenza's stomach dropped before she consciously knew why:

```
@Service
public class DraftPreviewService {

    private final DraftRepository drafts;
    private final PreviewTemplate template;
    private Draft currentDraft;   // 2019: "temp cache, avoids double-load" (git blame)

    public DraftPreviewService(DraftRepository drafts, PreviewTemplate template) {
        this.drafts = drafts;
        this.template = template;
    }

    public PreviewHtml render(Long draftId, ResearcherId viewer) {
        this.currentDraft = drafts.load(draftId);      // step 1: stash on a field
        applyRedactionsFor(viewer);                    // step 2: mutate the field
        return template.render(this.currentDraft);     // step 3: read it back
    }
}
```

> ⏸ **Your call.** Kenza's stomach knows before she does. Before the next line names it: what is wrong with this class, how does researcher A's draft reach researcher B's screen, and which -ility just failed? Commit.

It was a mutable field on a `@Service`, written at the top of every call and read at the bottom.

*And at Meridian this would have been fine,* she thought, and made herself finish the thought, because it was the whole bug. *There I'd have written `new DraftPreviewService(...)` inside the handler: one object per call, the field dying with the stack frame. "I'll just new it" was correct in a hand-wired world. Spring never hands you a fresh one. The default is one shared instance for everything.*

She said it out loud: "At Meridian I'd have `new`-ed one per request. Here there is exactly one. Forever."

One instance, one `currentDraft` field, every Tomcat worker thread writing to it. Researcher A's request stashes draft A. Researcher B lands on another thread and stashes draft B into *the same field*. Thread A reaches step 3 and renders whatever is there now. The field is shared because the object is shared; nothing more is required for the race. The bug was seven years old, so she hadn't written it. She had built it a highway: her own digest work had added a preview prefetch that multiplied calls to `render()` by fifty.

(If you called the field at the pause, the loud -ility down is *security*: wrong researcher, wrong company. A quieter one hides behind it, and Rachid will make her name it at the Wall.)

The Canary was already on her desk, where Rachid had set it the week before. House rule, the one he'd issued with the bird itself (whoever owns the gnarliest open bug explains it to the bird, out loud and in full sentences, before escalating to a human): *you'll want this; everyone does eventually.* Eventually was a Tuesday. She picked up the dented yellow bird, felt ridiculous for four seconds, and explained in full sentences: "You are a singleton. There is one of you for the whole application. Your field is a whiteboard bolted to the kitchen wall: every request that walks past writes its secrets on it and reads whatever the last person wrote. The fix is not a better whiteboard. It's to stop owning a whiteboard."

Saying it out loud surfaced the second half: fixing this class prevented nothing about the next one.

## What the senior sees

Rachid was at the Wall when she found him, green pen already out. He drew a tall empty column next to the sun-bleached `DO NOT ERASE` note.

"Two failures, one week," he said. "A cycle and a leak. Tell me what they have in common."

"They're both… me not knowing what the container actually does between `main()` and the first request."

"Good. So let's watch it work." He wrote *IoC* at the top of the column. "Inversion of control means you never call `new` on a collaborator. You declare what you need, and the container builds the graph: construction order, lifetime, all decided for you. Both of this week's failures happened inside decisions you didn't know it was making."

Component scan came first. `@ComponentScan` walks the packages and finds the stereotypes: `@Component` and its specializations `@Service`, `@Repository`, `@Controller`, plus `@Configuration` classes with their `@Bean` methods. What it registers is not objects but *recipes*: `BeanDefinitions`.

"Then the first hook," he said. "`BeanFactoryPostProcessors` run against those definitions, before any bean exists. That's how `@Configuration` classes get parsed and property placeholders resolved. Later, `BeanPostProcessors` run against the *instances*. One edits the recipe, the other decorates the cake. Now, why did your cycle kill staging at boot instead of prod at noon?"

"Because the `ApplicationContext` doesn't wait. It builds every singleton eagerly at startup."

"Yes. A plain `BeanFactory` is just a lazy lookup: ask, receive. The `ApplicationContext` is a `BeanFactory` with opinions. It pre-instantiates every singleton during refresh, registers the post-processors automatically, publishes events, owns the `Environment`. Eager creation means wiring mistakes detonate on the launch pad, where they cost nothing." *So the `ApplicationContext` is my old `main()`,* she thought, *the dependency-ordered wiring I used to type by hand, plus lifetimes, plus a biography for every object it builds.*

"Youssef gave you the operations version at six fifty," Rachid added. "Here's the architecture version. Eager creation is a free integration test on every deploy: the whole singleton graph, every constructor and every `@PostConstruct`, must succeed before the pod's readiness probe goes green, so a wiring mistake structurally *cannot* reach a user. That's **availability**, purchased at boot. We never sell it back. Sprinkle `@Lazy` to speed up restarts and the first failure moves from the pipeline to the first unlucky request at noon."

He made her build the biography down the new column, in her handwriting, question by question, until she could recite it cold:

instantiate → populate dependencies → `*Aware` callbacks → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` / `afterPropertiesSet` → init method → `BeanPostProcessor.postProcessAfterInitialization` → in service → `@PreDestroy` / `DisposableBean`.

Beside each stage she wrote one clause:

- **instantiate:** the container calls the constructor, and constructor arguments are resolved here.
- **populate dependencies:** `@Autowired` fields and setters are filled in on the object that now exists.
- **`*Aware` callbacks:** a bean that implements `BeanNameAware` or `BeanFactoryAware` is handed its own name or its factory (`ApplicationContextAware` is served one step later, by a `BeanPostProcessor`).

- **`postProcessBeforeInitialization`:** every `BeanPostProcessor` gets a first look at the bean once its dependencies are set.
- **`@PostConstruct` / `afterPropertiesSet`:** the bean's own init code, by annotation or by implementing `InitializingBean`.
- **init method:** the method named in `@Bean(initMethod = "...")`, for a class you cannot annotate.
- **`postProcessAfterInitialization`:** every `BeanPostProcessor` sees the initialized bean and may hand back a different object in its place.
- **in service:** the finished singleton is cached and injected wherever it is needed.
- **`@PreDestroy` / `DisposableBean`:** cleanup callbacks, run when the context shuts down.

"Where does `@PostConstruct` come from?" he asked.

"The annotation?"

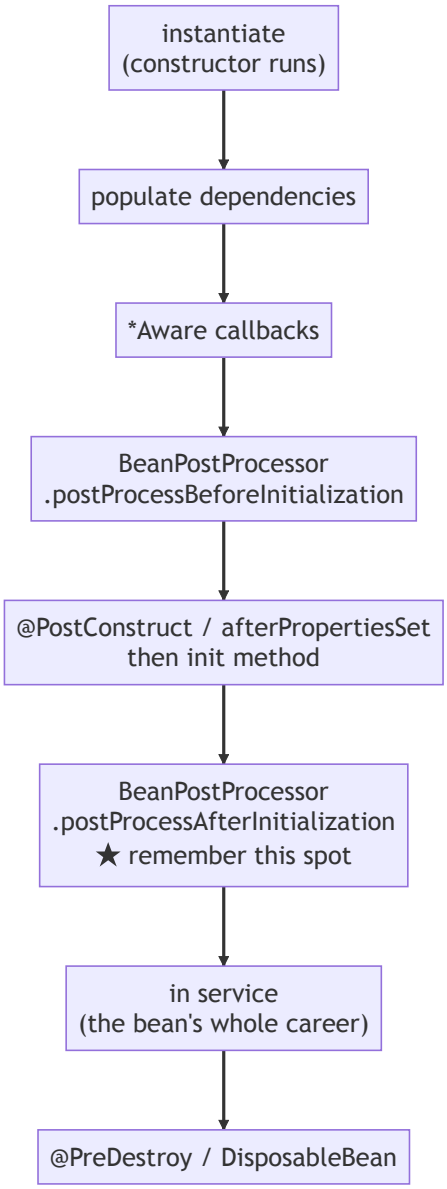
He just looked at her. "Show me the mechanism."

She opened the framework source instead of guessing, and found it: `CommonAnnotationBeanPostProcessor`, Spring's own `BeanPostProcessor`. It scans each bean for the annotation and invokes the method during the *before-initialization* phase.

"So `@PostConstruct` isn't a feature of Java," she said slowly. "It's a customer of the lifecycle, same as anything I could write."

"Everything is. That's the design." Then he tapped the last entry before *in service*, `postProcessAfterInitialization`, and held the pen there. "Remember this spot. Something important is born here. Not today's lesson. Soon."

He drew a star beside it. By standup the column ran the height of the Wall, a vertical spine beside the sun-bleached note:




They did the cycle next. Spring keeps three caches while building singletons: finished beans in `singletonObjects`, half-built beans exposed early in `earlySingletonObjects`, and `singletonFactories`, factories that *can* hand out a reference to a bean that exists but isn't initialized yet. That machinery means a setter or field cycle is resolvable. A is instantiated, registers its early factory, starts populating, and needs B. B is built; when *it* needs A, it receives A's early reference from the cache. Both finish, because the early reference breaks the deadlock.

"But your cycle was in the constructors," he said. "An early reference requires the object to *exist*. A constructor cycle needs each object before the other's first line can run. There is nothing for the cache to hand out."

"And the catch," he added, "is that the early reference escapes *before* initialization finishes: your neighbor receives a bean whose `@PostConstruct` hasn't run yet. If a post-processor replaces the bean *after* init, the reference your neighbor holds and the bean the container keeps are two different objects, and Spring throws an exception rather than let that stand. One reason Boot has shipped with circular references *disabled by default* since 2.6: even resolvable cycles fail fast unless you opt back in. The framework decided your design smell wasn't worth its cleverness, and it's right."

Thirty seconds of quick-fire, pen popping against the board:

- Two beans, one type? `@Primary` declares the default; `@Qualifier` picks by name at the injection point.
- A bean whose *job* is manufacturing another bean? `FactoryBean<T>`: ask for it, get the product; prefix the name with `&` for the factory itself.
- Scopes beyond singleton? `prototype`, a new instance per injection point or lookup, and `@RequestScope`/`@SessionScope` for web state.

"With one famous catch," he said. "Injection happens once, at startup, so a prototype injected into a singleton is resolved a single time and held forever. Unless you go through an `ObjectProvider` or a scoped proxy that re-resolves per use, the narrow scope drowns in the wide one."

"So I *could* slap `@RequestScope` on `DraftPreviewService`," Kenza said. "Make it behave like an object I'd `new`-ed inside the handler."

"You could. Why won't you?"

"Because the state shouldn't exist at all. Scope would be a patch over a field that has no reason to live. It needs to be stateless, not differently stateful."

"One more -ility and you pass," he said. "Name what statelessness buys at the system level."

She thought of Youssef's pod dashboards. "Scalability. The load balancer assumes any replica can serve any request, which is only true if no instance holds state between calls. Today the field lied between two threads in one JVM; three pods behind a balancer and it lies between *machines*."

"So the stateless-singleton rule is a *precondition* for horizontal scale." Rachid capped the pen. That was apparently the whole exam.

## The fix

The cycle first. Rachid made her try `@Lazy` before rejecting it, because rejecting a tool you've never held is superstition:

```
public ReportService(@Lazy ProgramService programs) { ... }
```

Staging booted. A breakpoint showed `programs` was no `ProgramService` at all but a proxy that would resolve the real bean on first use. “So it works by lying to the constructor,” she said. “It hands me an IOU. The cycle’s still in the design. I’ve just hidden it, and whatever goes wrong cashing the IOU goes wrong at first use, in production, instead of at startup, for free.”

“Break-glass,” Rachid agreed. “Legacy code you can’t reshape, a boot-order knot. Not a refactor you control.”

The honest fix was the one the cycle had been pointing at: a third thing that both services needed and neither of them was. She extracted it:

```
@Service
public class ProgramDirectory {          // the thing they were both reaching for
    private final ProgramRepository programs;
    private final ReportRepository reports;

    public ProgramDirectory(ProgramRepository programs, ReportRepository reports) {
        this.programs = programs;
        this.reports = reports;
    }

    public ProgramSummary summaryFor(Long programId) { ... }   // names + counts
}
```

`ReportService` and `ProgramService` each took a `ProgramDirectory`. Neither took the other. The graph was a DAG again, and staging went green. *A circular dependency is rarely two classes that need each other,* she wrote in her notes. *It’s one missing class that both of them are trying to be.*

Then the leak. The fix was almost insultingly small. Values move down the call stack as locals and parameters. The whiteboard comes off the wall:

```
@Service
public class DraftPreviewService {
    private final DraftRepository drafts;
    private final PreviewTemplate template;

    public DraftPreviewService(DraftRepository drafts, PreviewTemplate template) {
        this.drafts = drafts;
        this.template = template;
    }

    public PreviewHtml render(Long draftId, ResearcherId viewer) {
        Draft draft = drafts.load(draftId);
        Draft redacted = redactFor(draft, viewer);   // pure function, no field
        return template.render(redacted);
    }
}
```

The collaborator fields are `final`, set once in the constructor, and safe to share across every thread because they never change. Per-request data lives and dies on the stack. The diff was a dozen lines, and it kept the customer.

That evening Rachid stopped at her desk and set the printed postmortem draft back down. "Your first review caught forty-seven problems because I happened to read forty pages of your code. Somebody reviewed 2019's `currentDraft` field and let it *through*. And your cycle sailed past me wearing my own blessing. What does that tell you about review as enforcement?"

"That it's vigilance," Kenza said. "And vigilance decays with headcount and deadlines, and with whoever's tired on a Thursday."

"So. Architecture you can't enforce is architecture you hope for. The question is what enforcement costs."

### ⚖ Your move, architect

How do you stop the *next* violation, the mutable field or constructor knot not yet written, by an engineer not yet hired? Before reading on, commit.

- **A. Review vigilance, sharpened.** A checklist in the PR template, the green pen's rules taught at onboarding. Free and immediate, with zero infrastructure.
- **B. Real module walls.** Spring Modulith or JPMS boundaries: illegal dependencies are rejected by the module verifier, which makes them *unexpressible* rather than merely detectable.
- **C. Fitness functions in CI.** ArchUnit tests assert the constitution itself (field injection, mutable singleton state, dependency direction, entities on the wire) and fail the build, by rule name, on violation. (The build-time check Chapter 1's fork promised.)

**The trade-offs, since you've committed: A** costs nothing and decays: forty-seven catches in the first review, zero this week. The **evolvability** it guards lives in the heads of whoever hasn't quit yet. **B** buys the strongest **enforceability** at the heaviest **cost**, a quarter of migration for a seven-year monolith, bought before Atrium's borders have settled. **C** is what they chose. A fitness function asserts a *property of the architecture*, not a behavior of the code. It makes the constitution executable, and its enforcement survives personnel, which is what evolvability requires. The tax is rule maintenance, documented escape hatches, and the occasional morning arguing with a robot. The move rhymes with Chapter 1's seams fork, where package-by-feature was chosen over a microservice cut because it kept the split purchasable later. **C** keeps **B** purchasable at a tenth of the cost tonight: every ArchUnit rule today is a border Modulith could harden later.

Small fixes don't prevent recurrence, so she made it structural. The new rule was plain: singleton beans hold no mutable state. Collaborators are `final`, request data flows through parameters, and anything genuinely stateful must justify its scope in review. The rule joined ADR-001's rules in a constitution that went into the same PR as tests. Youssef wired the ArchUnit rules into CI, failing the build on non-final instance fields in `@Service`/`@Component` classes, each failure naming its law:

```
@ArchTest
static final ArchRule singletonsHoldNoMutableState =
        fields().that().areDeclaredInClassesThat().areAnnotatedWith(Service.class)
                .or().areDeclaredInClassesThat().areAnnotatedWith(Component.class)
                .should().beFinal()
                .because("ADR-002: singleton beans hold no mutable state");
```

The rule has two limits: `final` stops reassignment, so a `final` list can still be mutated, and `areAnnotatedWith` sees only the annotation written on the class itself, so `@RestController` and `@Repository` classes are covered only by `areMetaAnnotatedWith(Component.class)`.

Beside it went the constitutional core: ADR-001's direction, the invariant from the Wall. ArchUnit can assert that kind of rule too. The first rule says which fields must be `final`; this one says which packages a class is allowed to depend on. Kenza read it twice before approving. It was the first time *the domain knows nothing about the web* had existed anywhere a deadline couldn't erase it:

```
@ArchTest
static final ArchRule dependenciesPointInward =
        noClasses().that().resideInAPackage("..digest..")
                .and().areNotAnnotatedWith(RestController.class)
                .should().dependOnClassesThat()
                .resideInAnyPackage("..web..", "jakarta.servlet..")
                .because("ADR-002: the domain knows nothing about the web (ADR-001's
        rule, enforced)");
```

> **ADR-002: Executable architecture (fitness functions). Context:** ADR-001's rules were violated within a fortnight, innocently, by their own author. Review vigilance decays and doesn't survive turnover. **Decision:** The constitution becomes a test suite: ArchUnit rules in CI fail the build on field injection, non-final singleton state, dependency-direction violations, and entities returned from controllers. Eager singleton startup is treated as a free integration test and never traded away. **Consequences:** Violations fail the build with the rule named, instead of surfacing as a P1 with a customer attached. Costs accepted: rule maintenance, documented escape hatches, and arguing with a robot instead of with Rachid.

She wrote PM-31, her first postmortem, owning both halves of the week under one title: *Two faces of the same ignorance.* Leila got her sentence with *contained* in it, plus one she liked better: *structurally can't recur*.

The decomposition branch merged Thursday and deployed clean. Buried in it, unremarked by anyone, its author included, `bulkTriage()` now called `this.updateStatus()`, a method wearing a `@Transactional` annotation no caller would ever reach the way the container intended. Nothing failed, and nothing would for a while.

Friday, a calendar invite from Rachid appeared for late February: "The proxy talk." Below it, one sentence: *Bring the lifecycle column. We're going back to the spot I told you to remember.*

## The debrief: Ch.2, The Container

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 2.** Walk the bean lifecycle: where exactly do `BeanPostProcessors` run, and how is `@PostConstruct` implemented?

Then the follow-ups, cold:

1. How does Spring resolve a circular dependency, and when can't it?
2. Why did Boot turn circular-reference resolution off by default?
3. How do you safely use a prototype or request-scoped bean inside a singleton?
4. How do you keep architectural rules alive after the architect leaves the room?
5. `BeanFactoryPostProcessor` or `BeanPostProcessor`, with one real example of each?

Tick what you said:

- ☐ Claim: A bean's life is a fixed, hookable state machine driven by the container, and the hooks hold every magic feature.

- ▢ Mechanism: The order from instantiation to `@PreDestroy`, with `CommonAnnotationBeanPostProcessor` invoking `@PostConstruct` in the before-init phase.
- ▢ Trade-off: Eager singleton creation makes wiring mistakes explode at startup, paid for with boot-time work and a container you must understand.
- ▢ Failure mode: The circular dependency: an early reference whose `@PostConstruct` has not run yet, and a constructor cycle that fails at startup.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** a Spring bean's life is a fixed, hookable state machine driven by the container, and the hooks are where every "magic" feature lives. **Mechanism:** instantiate → populate dependencies → `*Aware` callbacks → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct/afterPropertiesSet` → init method → `BeanPostProcessor.postProcessAfterInitialization` → in service → `@PreDestroy/DisposableBean`. `@PostConstruct` is no JVM feature: `CommonAnnotationBeanPostProcessor`, a BPP Spring ships, finds the annotation and invokes the method in the before-init phase. `BeanFactoryPostProcessors` run earlier still, against bean *definitions* rather than instances. **Trade-off:** eager singleton creation plus uniform hooks means wiring mistakes explode at startup (cheap) and frameworks get a universal decoration point, paid for with boot-time work and a container you must understand. **Failure mode:** a circular dependency hands out a bean before the sequence has finished. To resolve a setter or field cycle the container gives the neighbor an early reference, a bean whose `@PostConstruct` has not run yet. A constructor cycle cannot be resolved at all and fails at startup with `BeanCurrentlyInCreationException`. Since Boot 2.6 even resolvable cycles fail fast unless you opt back in.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Availability** as fail-fast integrity: eager singleton creation means a pod that can't construct its graph never goes `Ready`, and a whole failure class moves from 2 a.m. to the pipeline. **Scalability** as precondition: any-replica-serves-any-request is only true when singletons hold no instance state. The chapter's main subject is **evolvability through enforceability**: architecture that a failing build can prove and review can only hope for.
- **The ADR:** ADR-002, executable architecture: the ADR-001 constitution plus the stateless-singleton rule, enforced by ArchUnit fitness functions in CI; eager startup never

traded away.

- **Rejected alternatives:** review vigilance alone (free, but it decays with headcount: 47 catches in the first review, zero this week); `@RequestScope` on the leaky service (differently stateful, not stateless: a patch over a field with no reason to live); full Spring Modulith/JPMS walls now (strongest enforcement, a quarter-long migration; kept purchasable, with ADR-002's rules as the down payment).
- **Fight or lean:** *Lean* on the strict container: eager refresh, constructor-cycle refusal, circular references off by default since 2.6. The framework declining to be clever for you is a gift. *Fight* the reflex to make bad shapes boot: `@Lazy` IOUs and exotic scopes convert launch-pad failures back into 2 a.m. ones.

**Story anchors**

- The 6:50 a.m. staging corpse and Youssef's "failure report this polite" → eager singleton instantiation in the `ApplicationContext`: constructor cycles fail at boot, and start-up-time failure is a feature.
- One draft field, two researchers, the whiteboard bolted to the kitchen wall → singleton is Spring's default scope (the #1 flip for hand-wired instincts: a bean is not an object you just `new`-ed for this call); singleton beans must be stateless.
- Rachid's pen tapping `BPP.after`: "something important is born here" → `BeanPostProcessors` decorate every bean during initialization; `@PostConstruct` is invoked by one of them (`CommonAnnotationBeanPostProcessor`), and proxies are born one step later.
- The red CI build naming its own law: *"ADR-002: singleton beans hold no mutable state"* → a fitness function asserts a property of the architecture, not a behavior of the code.

**Interview ammo**

- *Kill-shot: Walk the bean lifecycle; where do BPPs run; how is `@PostConstruct` implemented?* Instantiate → populate → `*Aware` → BPP.before → `@PostConstruct`/init → BPP.after → ready → destroy callbacks. `@PostConstruct` is executed by `CommonAnnotationBeanPostProcessor` in before-init, and AOP proxies are minted in after-init.
- *Follow-up: How does Spring resolve a circular dependency, and when can't it?* Setter/field cycles between singletons resolve via the three-level cache: a factory in `singletonFactories` mints an early reference, promoted through `earlySingletonObjects` into `singletonObjects`. Constructor cycles can't, because each object must exist before the other's constructor runs; since Boot 2.6 even resolvable cycles fail fast unless you opt back in.
- *Follow-up: Why did Boot turn circular-reference resolution off by default?* The early reference escapes before initialization completes, so the neighbor holds a bean whose `@PostConstruct` hasn't run, and post-init wrapping makes the early and final references diverge

(Spring detects this and throws). Failing at startup beats debugging a half-initialized object, and the honest fix is extracting the third collaborator both sides need.

- *Follow-up: How do you safely use a prototype or request-scoped bean inside a singleton?* Injection happens exactly once at startup, so a plain reference freezes the first instance forever. Inject an `ObjectProvider<T>` (or use a scoped proxy) so resolution happens per use instead of per wiring.
- *Architect follow-up: How do you keep architectural rules alive after the architect leaves the room?* Make them executable: fitness functions (ArchUnit, module verification) that fail the build with rule and reason named. Vigilance decays with headcount, while a red build argues back at every commit. The cost is rule maintenance and adjudicating exceptions.
- *Follow-up: `BeanFactoryPostProcessor` or `BeanPostProcessor`, with one real example of each?* A `BeanFactoryPostProcessor` runs against the bean definitions before any bean exists, which is how `@Configuration` classes get parsed and property placeholders resolved. A `BeanPostProcessor` runs against each instance: `CommonAnnotationBeanPostProcessor` finds `@PostConstruct` and invokes the method in the before-initialization phase.

**Pointers** → `examples/ch01_core/` (`BeanLifecycleAndPostProcessors.java`, `ScopesAndCircularDeps.java`)

---

# Chapter 3: Smoke and Mirrors

Malika did not believe in ghosts, which made it worse that she had brought printouts of three.

"GH-2907," she said, laying the first page on Kenza's desk like evidence at a trial. "Tuesday. I moved it to Triaged and bumped severity to Critical. The severity stuck. The status didn't. It's been sitting in New for two days with a Critical badge on it, and the researcher thinks we're stalling his payout."

The second page was the same haunting on a different report: severity raised to High on Monday, status still New. "And I know I moved that one, because I remember it. You don't forget a hardcoded AWS key."

The third was the strangest. "The audit log says I triaged this one at 14:06. I believe the audit log, because I remember doing it. The report itself says nobody ever touched it."

"An audit row for a change that never happened," Kenza said slowly.

"Severity drives the bounty math," Malika said. "When severity and status disagree, the payout calculator picks one and the researcher disputes it. Leila has nine disputes in her queue. They're not angry yet. They're *confused*, which on a trust product is the step before angry."

Kenza pulled the production timeline. The first ghost was dated four weeks back, and her stomach dropped before her eyes finished confirming it. Four weeks back was the day her decomposition of the digest god service had merged. It was her refactor, the clean one, lifecycle-literate and postmortem-hardened.

*In eight years at Meridian I never once shipped a bug that corrupted data politely,* she thought. *They crashed. This one is filing paperwork.*

## The investigation

She reproduced it in staging by lunchtime: queue a bulk triage of twenty reports, make the fourteenth contain an invalid status transition, run it. The batch threw, as expected. The rest she had not expected: thirteen reports fully updated, the fourteenth *half* updated (severity saved, status not, audit row present), and six untouched.

A report had been half updated inside a `@Transactional` method.

The code was hers. Before her refactor, the triage controller had called `updateStatus` directly on the injected service. Her decomposition had added a bulk entry point in the same class, by then `ReportServiceImpl`, the one implementation of what had become the `ReportService` interface (trimmed here to its triage code):

```
@Service
public class ReportServiceImpl implements ReportService {
    private final ReportRepository reports;
    private final AuditLogRepository auditLog;

    public ReportServiceImpl(ReportRepository reports, AuditLogRepository auditLog) {
        this.reports = reports;
        this.auditLog = auditLog;
    }
    public void bulkTriage(List<TriageDecision> decisions) {
        for (TriageDecision d : decisions) {
            updateStatus(d.reportId(), d.status(), d.severity()); // ==
        this.updateStatus(...)
        }
    }

    @Transactional
    public void updateStatus(long reportId, ReportStatus status, Severity severity) {
        Report report = reports.findById(reportId).orElseThrow();
        auditLog.save(AuditEntry.of(report, status, severity));
        report.applySeverity(severity);
        reports.save(report);
        report.transitionTo(status); // throws on an illegal transition
        reports.save(report);
    }
}
```

> **Your call.** Thirteen committed, one half-saved, six untouched, inside a method that says `@Transactional` two lines up. Before the breakpoint speaks: what is `this.updateStatus(...)` really calling, why did the same annotation work for the controller, and which -ility is failing without a single exception? Commit.

She set a breakpoint inside `updateStatus` and evaluated `TransactionSynchronizationManager.isActualTransactionActive()`.

`false`.

The annotation was sitting right there, two lines above the breakpoint, doing nothing. *So `@Transactional` isn't ambient,* she thought. *At Meridian a transaction was the `Connection` in my hand: `setAutoCommit(false)`, and everything I ran on it was inside, no matter which method did the running. I assumed the annotation was that with the plumbing hidden. Paint it on the method and the transaction is just… present.* The instinct had felt so safe she'd never examined it, and it was the bug.

Without an active transaction, every repository call ran in its own. `SimpleJpaRepository`'s methods carry their own `@Transactional`, so each `save()` opened a transaction, committed, and closed. The audit row committed first, and that was Malika's third ghost. Then severity. Then, if `transitionTo` threw, nothing rolled back, because there was nothing left to roll back: every earli-

er commit was already permanent. Her "atomic" method was three independent commits in a trench coat.

But *why* was the annotation inert? It had worked when the controller called `updateStatus`. It died when `bulkTriage` called the same method, under the same annotation.

She did the thing she should have done an hour earlier. In the debugger, paused in the controller, she evaluated the injected reference. Then, paused inside `bulkTriage`, she evaluated `this`.

```
// Debugger > Evaluate Expression, Thursday, 16:40
reportService.getClass()
// => class com.glasshouse.atrium.triage.ReportServiceImpl$$SpringCGLIB$$0

this.getClass()   // evaluated inside bulkTriage()
// => class com.glasshouse.atrium.triage.ReportServiceImpl
```

They were two different classes, and two different *objects*. The controller had never once held her object. It held something Spring had built. The subclass with the machine-generated name wrapped her object, and the wrapper was what had been injected everywhere, in place of the original. It stood in front of `ReportServiceImpl` like a body double.

Transactions, security, and caching live on the wrapper, so they apply only to calls that arrive through it. Every `new` opts out. So, she now saw, does every `this.` January surfaced whole, Rachid at the Wall in her first month: *the container reserves the right to hand back something wrapped.* She had filed that, at the time, under philosophy. It was a spec, and her loop had spent four weeks walking around the wrapping from the inside.

Rachid appeared at her shoulder with the timing he always seemed to have. She pointed at the screen.

"I've been calling methods on a ghost."

"No," Rachid said. "You've been *bypassing* the ghost. The ghost is the only one doing real work."

(If you committed at the pause: self-invocation, around the proxy. Notice what almost nobody commits to: *nothing threw* while atomicity dissolved. Rachid names the silence after the fix.)

## What the senior sees

They took it to the Wall, where the bean lifecycle column from the singleton-leak postmortem (one instance app-wide; a mutable draft field leaking between researchers) still ran down the left edge in Kenza's own handwriting: instantiate, populate, the post-processor sandwich. Rachid's February invite, "The proxy talk" (*bring the lifecycle column*), was still pending; production had set an earlier date. Rachid uncapped the green marker and tapped one line near the bottom: `BeanPostProcessor.postProcessAfterInitialization`.

“Last month I told you to remember this spot,” he said. “Something important is born here. You’ve just met it. So what is it, mechanically? Not ‘a proxy.’ Show me the mechanism.”

Kenza took the marker. “There’s a `BeanPostProcessor`…” She checked the notes from her source dive. “`AnnotationAwareAspectJAutoProxyCreator`, one of the `AbstractAutoProxyCreator` family. After my bean finishes initializing, it inspects it. Mine has a `@Transactional` method, which matches the transaction advisor’s pointcut. So instead of handing the container my object, it hands back a wrapper. Every bean that asked for `ReportService` got the wrapper injected. The wrapper opens the transaction, delegates to my object, and commits or rolls back on the way out.”

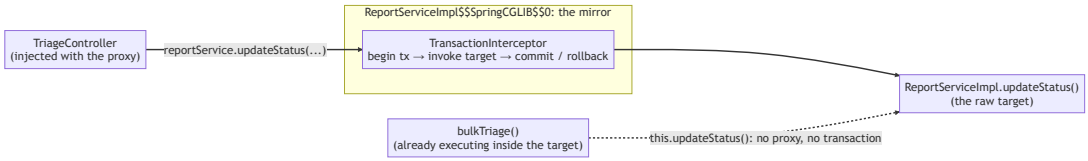
It also explained something from January’s cycle talk: the three-level cache is clever enough to mint AOP proxies early, so those survive the loop. When a neighbor asks for a bean that is still being built, the factory in `singletonFactories` asks this same post-processor for the reference.

“Vocabulary, so you can talk to other adults about it,” Rachid said. “The transaction concern is an *aspect*. The wrapping behavior, `TransactionInterceptor` here, is *advice*. The rule deciding which methods get it is a *pointcut*. And Spring applies all of it by *weaving at runtime, with proxies*, not by rewriting your bytecode. That last clause is your whole bug. Why?”

She stared at the loop in her head: `bulkTriage` calling `updateStatus`, plain Java, an `invokevirtual` on `this`. “Because the advice lives on the proxy, and the proxy is only in the call path when the call comes through the reference *other beans hold*. Inside my object, `this` is just… me. The raw object. The call never leaves the building, so it never passes the front desk.”

Rachid held out the marker. “Draw both call paths. Boxes first, metaphors afterward.”

She drew it under the lifecycle column:




The drawing said it flatly: the advice lives on the proxy, so only calls arriving through the injected reference are intercepted. A call on `this` never enters the left half.

It was the decorator pattern, applied by the container. She had hand-wrapped a logging decorator around a shipment service at Meridian once, and it had carried exactly this blind spot: the inner object, calling itself, never passes back out through the outer one. Meridian had never needed to warn her, because the object her `main()` built was the object everyone held. A call was a call, with nothing in between to skip. *This wraps the object,* she thought, *and objects can call themselves.*

“Two more questions,” Rachid said. “Why is the proxy a subclass and not an interface?”

“It could be either,” Kenza said. “A JDK dynamic proxy implements the bean’s interfaces and routes every call through an `InvocationHandler`. CGLIB subclasses the class itself and overrides its methods. Mine has an interface, so a JDK proxy was possible. But Boot has shipped

`spring.aop.proxy-target-class=true` for years, and still does as of Boot 4.0, so it builds CGLIB class proxies by default and injecting the concrete type keeps working." She paused, watching the consequence arrive. "Which means a `final` method can't be overridden. So a `final` method can't be advised. `@Transactional` on a `final` method gets no transaction."

"And it fails louder than yours did," Rachid said. "The call runs on the proxy instance itself, and the proxy's injected fields are null, so it usually ends in a `NullPointerException` on the first field it touches. Spring logs a warning when it builds a proxy for a class with a public `final` method. Last question. Suppose a `@PostConstruct` method stashes a reference to `this` in a registry, and suppose that bean is transactional. What does it stash?"

Kenza looked at the lifecycle column and the answer was simply *there*, in the ordering she had memorized for entirely different reasons. "`@PostConstruct` runs in the *before*-initialization phase; `CommonAnnotationBeanPostProcessor` invokes it. The proxy isn't created until *after*-initialization. So `@PostConstruct` always sees the raw bean. If you leak `this` from there, you've published the unproxied object, and anyone calling through that reference skips every aspect, forever."

Rachid capped the marker. "The lifecycle was load-bearing. It usually is."

## The fix

The three of them argued the fix at the Wall the next morning: Kenza, Rachid, and Youssef, who arrived with coffee and an opinion.

"Self-injection," Youssef said. "Seen it work. The bean asks the container for its own proxy: `ObjectProvider<ReportService>`, call `getObject()`, invoke `updateStatus` through that. Three lines. Ships today."

Rachid looked at Kenza. "Argue all three. Out loud."

She took a breath. "Option one: move `@Transactional` up to `bulkTriage`, the entry method the proxy sees. Honest and simple, but it changes the semantics. The whole batch becomes one transaction, so one bad decision rolls back nineteen good ones, and Malika's team loses a morning's work to a single typo. And since the controller still calls `updateStatus` directly, the annotation has to stay there too, which leaves a method that looks transactional and is still trap-armed for the next internal caller. We'd fix this bug and replant it.

"Option two: Youssef's self-injection. It does work: you go back out through the proxy. But the class now injects itself, which reads like a riddle. Every future reader stops and frowns, then either cargo-cults it or 'simplifies' it back into the bug. It treats the symptom and preserves the bad shape: one class doing two jobs.

"Option three: extract the per-report transaction into its own bean. The transaction boundary becomes a *bean* boundary, visible in the architecture instead of hidden in a calling convention. It costs a new class and a longer import list."

"And which do you ship?"

“Three. The class was doing two jobs anyway: orchestration and the unit of work. The bug was the design telling me that.”

Youssef shrugged amiably. Rachid nodded once. “Self-injection is clever,” he said. “Clever is what you write *before* the postmortem.”

```
@Service
public class TriageWorkflow {

    private final ReportRepository reports;
    private final AuditLogRepository auditLog;

    public TriageWorkflow(ReportRepository reports, AuditLogRepository auditLog) {
        this.reports = reports;
        this.auditLog = auditLog;
    }

    @Transactional   // entry method, always reached through the proxy
    public void apply(TriageDecision decision) {
        Report report = reports.findById(decision.reportId()).orElseThrow();
        auditLog.save(AuditEntry.of(report, decision)); // written first, as before
        report.applySeverity(decision.severity());
        report.transitionTo(decision.status()); // unchecked → the whole unit unwinds
        // managed entity: dirty checking flushes at commit, all of it or none of it
    }
}
```

`ReportServiceImpl.bulkTriage` shrank to a loop over `workflow.apply(d)`, so every call crossed a bean boundary and went through the proxy. Three details went in on purpose. The audit row is still saved first, before the transition that can throw, so the test below has a write for the rollback to undo. There is no explicit `save()` for the report: inside the transaction the entity is managed, and dirty checking flushes the changes at commit. And `IllegalTransitionException` extends `RuntimeException` deliberately, because the proxy’s default rollback rule only watches unchecked exceptions; a checked one would have sailed straight through to a commit. Kenza filed that detail away. It would come back for her in the spring.

Per-decision atomicity was a decision too: one illegal transition fails one report, and the other nineteen land. That trade-off went into the ADR instead of into folklore.

Then the test, because a fix without a regression test is just an apology:

```
@SpringBootTest
class TriageAtomicityTest {

    @Autowired TriageWorkflow workflow;
    @Autowired ReportRepository reports;
    @Autowired AuditLogRepository auditLog;

    @Test
    void failedTransitionLeavesNoPartialState() {
        Report r = reports.save(TestData.report(ReportStatus.NEW, Severity.LOW));
        var illegal = new TriageDecision(r.getId(), ReportStatus.AWARDED,
         Severity.CRITICAL);

        assertThatThrownBy(() -> workflow.apply(illegal))
            .isInstanceOf(IllegalTransitionException.class);

        Report reloaded = reports.findById(r.getId()).orElseThrow();
        assertThat(reloaded.getSeverity()).isEqualTo(Severity.LOW); // severity didn't
         stick
        assertThat(auditLog.findByReportId(r.getId())).isEmpty();   // no ghost audit
         row
    }
}
```

She wrote it against the old path first, pointing the same assertions at the old `bulkTriage` with a batch of one: ghost audit row, half-saved severity, red. Pointed at `TriageWorkflow.apply`: green. Then she deleted `@Transactional` from `apply` once, to be sure the test noticed: red again, because the audit row committed alone. On the Wall, next to the lifecycle column, she drew her object as a small square and the proxy around it as a billowing cloud of marker smoke, with an arrow labeled `this.` curving around the outside of the cloud and a caption beneath: *this. goes around the mirror.*

PM-32 went out under Kenza's name, with the timeline starting at her own merge commit. Owning it cost less than she'd feared and bought more than she'd expected. Malika read it and sent back one line: *"Thank you for believing my ghosts."* Kenza also wrote the team's first proxy-traps review note, a new section in the review checklist: any annotated method called via `this.` from inside its own class is a finding, not a style nit.

## The architecture under the silence

She took the checklist line to Rachid for sign-off and got a question instead. "Good line. Now tell me what actually failed this month. I mean the system, not the transaction."

Kenza thought of Malika's printouts. "Nothing failed, and that's the indictment. Thirteen commits and an audit row, with no exception, no log, no alert. Atrium kept agreeing with itself, wrongly. Our only monitor was Malika's memory."

"A thrown exception is a gift," Rachid said. "It carries a timestamp, a stack trace and a blast radius you can measure. A silent no-op has a blast radius too. You just can't see its edges until a triager brings you printouts. A failure mode that hides is an architecture problem, not a code problem, because architecture decides where failure is allowed to be invisible. The correct answer: nowhere you haven't signed for."

He tapped the smoke cloud on the Wall. "Which makes the proxy a leaky abstraction, and a leak allows two honest responses. *Operationalize* it, with your checklist line and boundaries on entry points only. Or *eliminate* it, with real weaving in the bytecode and no mirror to walk around. Both are defensible. The only wrong move is the one this codebase made for seven years: choosing by default. So name what the annotation buys, and what it spends."

"It buys maintainability: cross-cutting concerns declared once instead of hand-rolled into every method. And it spends visibility: control flow I can't see in the file I'm reading."

"Then it's a purchase," Rachid said, "and purchases get receipts." The receipt went into `docs/adr/` that afternoon, half a page, dated:

> **ADR-003: Declarative cross-cutting, with eyes open. Context:** A self-invoked `@Transactional` method ran with no transaction at all. The failure was invisible: no exception, no log, atomicity dissolved into independent commits (PM-32). **Decision:** Keep annotation-driven transactions, caching, and security (the wrapping is why we inhabit the container), but adopt the proxy discipline: any annotated method invoked via `this.` is a review *finding*, and transaction boundaries live on service entry points only (per-decision atomicity for bulk triage rides with this record). **Consequences:** We lean on the framework where its behavior is observable and fight it where it hides control flow. The checklist line costs review attention forever, a tax paid knowingly, because the alternative was paid in silence.

### ⚖ *Your move, architect*

ADR-003 settled the bug, and it named a strategy for a codebase that carries a couple hundred `@Transactional` methods. What is the transaction strategy going forward? You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Declarative `@Transactional` plus the proxy discipline.** Keep the idiom and enforce the checklist: boundaries on service entry points, and `this.`-invocations are review findings.
- **B. Programmatic `TransactionTemplate` in the hot paths.** The boundary becomes a lambda you can see in the method body. With no proxy in the call path, there is no trap.
- **C. AspectJ compile-time weaving.** Advice is woven into the bytecode itself, so self-invocation is intercepted like any other call and the trap ceases to exist.

**The trade-offs, since you've committed:** **B** makes the boundary visible, but the demarcation code moves into the business methods themselves and the codebase loses its one convention: a **maintainability** cost on every method that adopts it, which is exactly what the annotation exists to avoid. It is right *surgically*, where a boundary deserves to be seen, and wrong as a default. **C** genuinely kills the trap: woven advice rides inside the compiled method, so even `this.` calls are advised. But it adds the AspectJ compiler or agent to every build and hands every new hire bytecode nobody can debug: permanent **cost** and **operability** against an occasional, now-*named* trap. **A** is what they chose, and the reasoning is the architect's tell: it converts an invisible failure into a visible review category (**operability** bought with review attention forever) while keeping C purchasable if the findings recur. (They will. The checklist line meets its first real test late that summer, at the hands of a June hire who hasn't met the mirror yet.)

Rachid read the postmortem, signed it in green, and left a sentence hanging in the air on his way out that she would not fully understand until late September:

"One more thing. `@Cacheable`, `@Async`, `@PreAuthorize`: same mirror, same trap. You'll meet it again."

At standup the next day, someone joked that at least the *startup* side of Spring never pulled tricks like this: the auto-configuration just works. Everyone laughed, including Kenza. Nobody wrote it down.

## The debrief: Ch.3, Smoke and Mirrors

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 3.** Why does `@Transactional` silently do nothing when a method calls another `@Transactional` method on the same bean?

Then the follow-ups, cold:

1. Couldn't Spring just intercept internal calls too?
2. Does the interface matter, and what about `final` methods?
3. Declarative `@Transactional`, programmatic `TransactionTemplate`, or AspectJ weaving as the codebase-wide strategy?
4. Why does the circular-reference cache have three levels and not two?

Tick what you said:

- ☐ Claim: Declarative transactions are applied by an AOP proxy, and a self-invocation never reaches the proxy.
- ☐ Mechanism: The proxy is created at `postProcessAfterInitialization`, the `TransactionInterceptor` advice sits on it, and `this.method()` is a plain call on the raw

target.

- ▢ Trade-off: Proxy weaving needs no build step or agent, and it intercepts only calls that cross the bean boundary.
- ▢ Failure mode: The method runs with the caller's transaction, usually none, so each repository call commits independently and leaves half-saved state.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Because Spring's declarative transactions are applied by an AOP proxy, and a self-invocation never reaches the proxy. **Mechanism:** During bean creation, at `postProcessAfterInitialization`, an `AbstractAutoProxyCreator` wraps the bean in a proxy (a CGLIB subclass by default in Boot, a JDK dynamic proxy for interface-based setups), and the `TransactionInterceptor` advice lives on that proxy. Other beans are injected with the proxy, but inside the class `this.method()` is a plain JVM call on the raw target, so the interceptor chain never runs. **Trade-off:** Proxy-based runtime weaving needs no bytecode manipulation, build step, or agent, which is why Spring defaults to it. The cost is that it intercepts only calls that cross the bean boundary. Full AspectJ weaving lifts that limit but brings the build/agent complexity back. **Failure mode:** The annotated method runs with whatever transaction the caller had, usually none. Each repository call then commits independently, so a mid-method exception leaves half-saved state, which is silent data corruption. The same shape hits anything published from `@PostConstruct`, which runs before the proxy exists. A `final` method fails differently: CGLIB can't override it, so the call runs on the proxy instance, whose injected fields are null, and usually ends in a `NullPointerException`.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Maintainability** of invisible cross-cutting: declarative annotations buy clean business code and pay with control flow you can't see in the file you're reading. The bigger casualty is **operability**: a silent no-op has no alarm, no stack trace and no visible blast radius, and a failure mode that hides is an architecture problem, not a code problem.
- **The ADR:** ADR-003, declarative cross-cutting, with eyes open: keep annotation-driven transactions, caching, and security; adopt the proxy discipline, under which `this.`-invoked annotated methods are review findings and transaction boundaries sit on service entry points only.

- **Rejected alternatives:** self-injection (works, reads like a riddle, preserves the two-job class); `@Transactional` hoisted onto `bulkTriage` (changes batch semantics, leaves a trap-armed method behind); `TransactionTemplate` as default (explicit, but demarcation leaks into business code); AspectJ weaving (kills the trap, taxes every build; kept purchasable).
- **Fight or lean:** *Lean* on declarative cross-cutting: it is observable at bean boundaries and is the reason the container is worth inhabiting. *Fight* it where it hides control flow: operationalize the leak with discipline rather than eliminate it with weaving. Choosing by default is the only wrong move.

**Story anchors**

- The debugger printing `ReportServiceImpl$$SpringCGLIB$$0` next to a plain `ReportServiceImpl` → callers hold the proxy; `this` is the raw, unadvised bean.
- The smoke cloud on the Wall and its caption, *"this. goes around the mirror"* → self-invocation bypasses every proxy-applied aspect, so `@Cacheable`, `@Async`, `@PreAuthorize` share the trap with transactions.
- Malika's audit row for a change that never happened → an inert `@Transactional` dissolves one atomic unit into N independent commits and raises no error doing it.
- Rachid tapping the smoke cloud, *"a thrown exception is a gift"* → silent no-ops have blast radii you can't see. ADR-003 is the receipt: operationalize the leak or eliminate it, but never choose by default.

**Interview ammo**

- *Kill-shot: Why does `@Transactional` silently do nothing on a self-invoked method?* The advice lives on a runtime proxy that callers are injected with; `this.method()` is a direct call on the target object, so the interceptor chain is never in the call path and the method runs with whatever transaction (usually none) the caller had.
- *Follow-up: Couldn't Spring just intercept internal calls too?* Only by abandoning proxies for real weaving: AspectJ compile-time or load-time weaving modifies the bytecode itself and does advise self-calls, but costs a build step or agent. Spring supports that mode and deliberately doesn't default to it.
- *Follow-up: Does the interface matter, and what about `final` methods?* With an interface a JDK dynamic proxy is possible, though Boot defaults to CGLIB class proxies (`spring.aop.proxy-target-class=true`) so concrete-type injection keeps working. CGLIB advises by overriding, so a `final` method can't be advised: the call runs on the proxy instance, whose injected fields are null, and usually ends in a `NullPointerException`. Spring logs a warning for a public `final` method when it builds the proxy.

- *Architect follow-up: Declarative `@Transactional`, programmatic `TransactionTemplate`, or AspectJ weaving as the codebase-wide strategy?* Declarative as the default with the proxy discipline enforced in review, `TransactionTemplate` surgically where the boundary deserves to be visible, weaving only when recurring findings prove the discipline doesn't hold. The abstraction leaks either way; the architect chooses which tax to pay, on the record.
- *Follow-up: Why does the circular-reference cache have three levels and not two?* The third level, `singletonFactories`, holds a factory for each bean that is still being built. When a neighbor asks for such a bean, the factory asks the auto-proxy creator for the reference, so the AOP proxy is minted early and survives the loop.

**Pointers** → `examples/ch01_core/SelfInvocationTrap.java` (the same trap generalizes: `examples/ch12_extensions/AopAnnotationTraps.java`) · **run it:** `labs/lab-01-self-invocation` (a self-invoked `@Transactional` call proven to run with no transaction)

---

# Chapter 4: The Magic Show

The failover drill was scheduled for 6:30 on a Tuesday morning in March, which meant Youssef had been awake since five, drinking coffee the consistency of roofing tar and narrating his own actions like a bomb-disposal video. The plan was boring on purpose: kill the Postgres replica in prod, watch the platform degrade gracefully, watch the dashboard go amber, watch the alert fire, bring the replica back, write down the timings. Atrium had survived seven years; the drill was supposed to confirm the folklore.

Youssef killed the replica at 6:31. Then he waited for the dashboard to notice.

It didn't. The health panel glowed the same serene green it had glowed all night. He refreshed it, then curled the actuator endpoint directly. It answered `"status":"UP"`, smooth as a salesman. Meanwhile, two terminals over, the replica was (his word in the incident channel) *extremely* down. For forty minutes a real degraded dependency was invisible to every pane of glass the company owned, and the only reason anyone knew was that Youssef had broken it himself, on purpose, with a calendar invite.

At standup he projected two JSON payloads side by side, component details shown in both. Staging's health endpoint showed a `db` component, dutifully `UP`. Prod's showed Boot's stock components, `diskSpace` and `ping` among them, and nothing about a database: no `db`, no replica check. The top-level status was a green lie.

"Last week's platform dependency bump went out Thursday," Youssef said. "Version numbers only. Reviewed and approved." He did not look at Kenza. He didn't have to; she'd approved it herself. The diff had been green and the bump tidy, the kind of PR you wave through between meetings.

Rachid studied the two payloads for a long moment. Then he asked the room, mildly, "Why does this bean exist in staging and not in prod?"

Nobody answered. The room held eight engineers and seven years of folklore, and not one person could say why any given health bean existed anywhere.

"Then nobody owns it," Rachid said. He capped his pen. "Magic is just a mechanism you haven't read yet. Roadmap work is cancelled this afternoon. Bring lunch."

## The investigation

Kenza started where she always started: grep. `ReplicaLagHealthIndicator` appeared nowhere in Atrium's source. It wasn't in `config/` or `monitoring/`, or in any `@Configuration` class she could find. No code the team had written in this repository defined the bean that was supposed to watch the database.

*At Meridian this would have been a line in main(),* she thought. *new ReplicaLagHealthIndicator(dataSource, maxLag), registered by hand: code I wrote, in a file I owned, that a dependency bump could not silently delete.* The instinct that wiring lives in user code was exactly why she'd never thought to look anywhere else. This time the hand-wired reflex had been a blindfold instead of a head start.

The class lived in a separate internal repo: `glasshouse-monitoring-spring-boot-starter`, written in 2021 by an engineer who'd left in 2023. Inside was a small auto-configuration:

```
// glasshouse-monitoring-spring-boot-starter, migrated from
// spring.factories for Boot 3, imports patched for Boot 4;
// nobody read it either time.
@AutoConfiguration
@ConditionalOnClass(ReplicaHealthProbe.class)   // compiled against osprey-metrics 2.x
@ConditionalOnProperty("glasshouse.monitoring.replica-probe.enabled")
public class ReplicaMonitoringAutoConfiguration {

    @Bean
    HealthIndicator replicaLagHealthIndicator(
            DataSource dataSource,
            @Value("${glasshouse.monitoring.replica-probe.max-lag:5s}") Duration
        maxLag) {
        return new ReplicaLagHealthIndicator(new ReplicaHealthProbe(dataSource),
        maxLag);
    }
}
```

Next to it in the repo sat a fossil: an empty `META-INF/spring.factories` with a three-year-old comment, `moved to AutoConfiguration.imports for Boot 3, delete after merge`. The actual registration now lived in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, one fully-qualified class name on one line. That file was the bean's birth certificate, and nobody at Glasshouse had ever read it.

"Reproduce it," Rachid said. "Locally. Don't theorize."

> **Your call.** Two conditions gate that bean; Thursday's bump changed only version numbers. Which gate failed in prod, why was the failure *silent*, and which -ility just died with the dashboard? Commit.

Kenza booted Atrium with the prod property set and `--debug` on the command line. The ConditionEvaluationReport filled her terminal: every auto-configuration Boot had considered, hundreds of them, with the conditions each had evaluated and the verdict. She scrolled past `DataSourceAutoConfiguration matched`, past Jackson, past Tomcat, hunting. Then:

```
ReplicaMonitoringAutoConfiguration:
   Did not match:
      - @ConditionalOnClass did not find required class
        'io.osprey.metrics.jdbc.ReplicaHealthProbe' (OnClassCondition)
```

Youssef and two others had drifted over, and the room audibly exhaled. The dependency bump had taken osprey-metrics from 2.9 to 3.0. In 3.0 the vendor had repackaged: `io.osprey.metrics.jdbc` became `io.osprey.metrics.probe.jdbc`. The starter jar, compiled against 2.x, asked the classpath for a class that no longer existed at that address. And `@ConditionalOnClass` did precisely what it was designed to do: it backed off, with no `ClassNotFoundException`, no warning and no log line. Nothing had malfunctioned. The silence was the feature, pointed the wrong way.

(If you called `@ConditionalOnClass` at the pause, take the second beat too. The -ility that died wasn't availability: the replica was *allowed* to fail, and that was the drill. The casualty gets named at the Wall and billed to the supply chain.)

"That explains prod," Youssef said slowly. "Why did staging never catch it?"

Kenza pulled up the profiles. There it was, the second condition:

```
# application-prod.yml: the only place the module was ever switched on
glasshouse:
  monitoring:
    replica-probe:
      enabled: true        # staging never sets this: "staging has no replicas"

management:
  health:
    db:
      enabled: false       # 2021: "the custom check is better", and prod kept no
        fallback
```

Staging had never run the custom probe at all, because the activating property existed only in the prod profile. So staging fell through to Boot's default `db` health contributor. That check is a connection-validity ping, `Connection.isValid()` against the primary, and it knows nothing about replicas. Staging therefore showed a database check, just not the one prod depended on. And prod had been left with no fallback: the original author had disabled Boot's default there because the custom check was "better," so when the custom probe stopped registering, no database check remained at all. Both environments were green, and each was wrong in its own way.

Kenza stared at the `enabled: false` line and thought of the sinkhole Rachid had circled and dated in January: *code is copied more often than it's read*, inked beside a return type that leaked raw JPA entities across an API boundary. The note covered more than that one return type. It marked any working-but-wrong pattern that propagates by paste. Config was worse: yaml is copied with no compiler to object and no type to mismatch. This line had outlived its author by three years, and it had meant nothing to anyone who'd pasted profiles around it since.

"Two conditions and a profile," Rachid said. "A murder weapon in three parts. Now, to the Wall. All of you."

## What the senior sees

Rachid drew two doors on the Wall, side by side, both opening into a box labeled ApplicationContext.

"Every bean enters through one of these. Door one." He tapped it. "Kenza, what's inside `@SpringBootApplication`?"

"It's…" She caught herself before saying *the magic annotation*. "Three annotations. `@SpringBootConfiguration`, which is a `@Configuration`, so the class can define beans, and which is the marker `@SpringBootTest` searches for. `@ComponentScan`, which walks downward from the main class's package. That's why `AtriumApplication` sits at `com.glasshouse.atrium` and everything lives beneath it. And `@EnableAutoConfiguration`."

"Door one is yours," Rachid said. "Your packages, your stereotypes, scanned from the root down. Move the main class and half your beans vanish, but at least they vanish loudly. Door two is the one nobody here had read." He wrote `AutoConfiguration.imports` over the second door. "Boot does not scan jars hoping to find configuration. It reads a list. Every jar may carry one file at `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. The import selector behind `@EnableAutoConfiguration` collects every such file on the classpath and gets a roll-call of candidate classes. Before Boot 2.7 the list lived in `spring.factories`, your fossil. A starter that still registers only there is silently dead on anything modern. Sound familiar?"

"Then the conditions," Kenza said. "A candidate still has to pass its gates."

"Name the gates."

"`@ConditionalOnClass` asks whether this type is on the classpath. The annotation itself is read from the class file's metadata with ASM, never loaded, which is why an auto-configuration can safely name a class that isn't there, and why its absence makes no sound. `@ConditionalOnProperty` asks whether the operator turned this on. `@ConditionalOnBean` and `@ConditionalOnMissingBean` ask whether the context already contains, or lacks, a bean." She paused, seeing the shape of it. "And those last two only make sense if there's an order. They're evaluated against the beans registered *so far*."

"Which is why ordering is part of the contract," Rachid said. "`@AutoConfigureAfter`, or the `after` attribute on `@AutoConfiguration`. Condition on a `DataSource` before `DataSourceAutoConfiguration` has run and your gate fails for a bean that was ten milliseconds from existing. And the back-off gate, `OnMissingBean`, is the whole social contract of Boot. Why does your `@Bean` beat Boot's default, every time, with no flag?"

Kenza worked it through aloud. "User configuration is processed before auto-configuration is evaluated. So when Boot's default `DataSource` bean reaches its `@ConditionalOnMissingBean`

check, my bean is already in the registry. Boot sees it and yields. I win by declaring, not by disabling." She looked at the yaml still on the screen. "Which is what 2021-guy should have done, instead of turning the default `db` check off with a property and leaving prod with no net."

*So a starter is a Maven dependency that arrives already wired,* she thought. *At Meridian a jar was just bytes on the classpath, and the wiring was always mine to write. This is dependencies plus the code that registers them, and the registration travels inside the jar.* The naming convention got two minutes: `glasshouse-monitoring-spring-boot-starter`, company-first, because `spring-boot-starter-*` belongs to Boot itself. The parent BOM (bill of materials) got five. `spring-boot-dependencies` pins hundreds of third-party versions, tested together as one matrix. That was exactly why the bump of everything *in* the BOM had been safe, and why osprey-metrics, which wasn't, had walked in unread.

Rachid capped his pen. By now the team knew it meant the mechanism was settled and the bill was coming. "Name what this incident spent. Youssef."

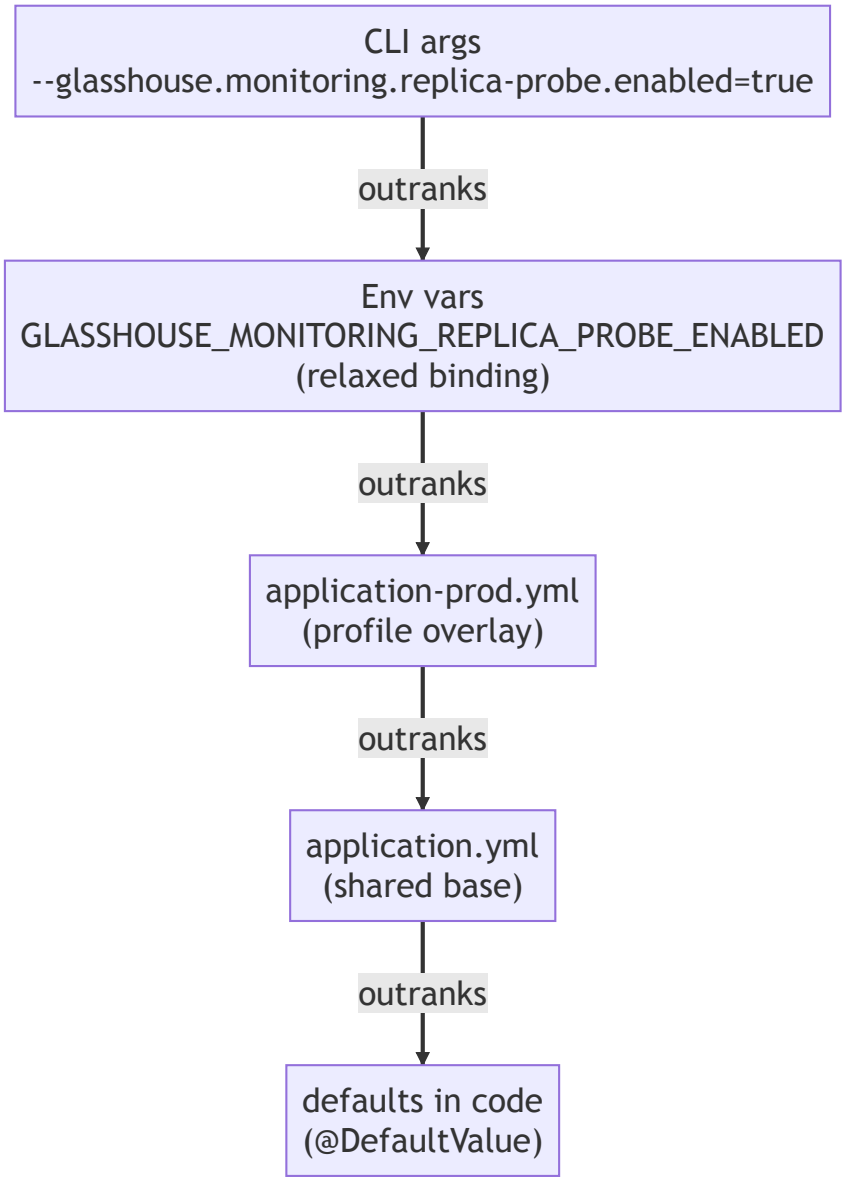
"Operability. For forty minutes the system couldn't tell us the truth about itself."

"That's the symptom's -ility. Kenza, the cause's."

"Evolvability," she said, slower. "But not ours. The supply chain's. We treated Thursday's bump as a chore; it was an architecture event. The blast radius ran from osprey's `jdbc` package, through a condition in a starter nobody had read, into prod's only database health check. And the diff that detonated it was two characters of version number."

"Because the BOM is a contract," Rachid said. "Inside its tested matrix, a bump moves you between tested points. Outside, where osprey lived from the day it got its own version line, the testing burden is yours whether or not anyone signs for it, and the upgrade gets a reviewer who asks what the jar promises the classpath and which conditions, in which repos, lean on that promise." He nodded at the condition report still on Kenza's screen. "The second rule you've already run. The container will explain its own composition, every candidate with its gates and its verdict: `--debug` at boot, `/actuator/conditions` while it runs. A system you can *interrogate* is an operability property like any other. It exists only if you design the habit of asking. This morning the answer took four minutes to read and three years to think of reading."

Then Rachid drew a ladder on the Wall and labeled the rungs top to bottom: CLI args > env vars > profile yml > application.yml > defaults. He drew it low, under the lifecycle column and February's smoke cloud (the proxy drawn as a cloud around the raw bean; the advice lives on that wrapper, and a `this.` call never crosses it). The Wall now read like the container itself, first how a bean is born, then who decides whether it may be:




Resolution starts at the top: the first rung that defines a property wins. A lower rung speaks only when every rung above it stays silent.

“The other half of your murder weapon. A bean can be gated on configuration, and configuration is a layered fight. Higher rung wins. Youssef, prove the ladder.”

Youssef, without ceremony, set `GLASSHOUSE_MONITORING_REPLICA_PROBE_ENABLED=true` on the staging deployment and let the pod restart. The env var outranked the profile file, and relaxed

binding mapped the screaming snake case onto the kebab-case property without anyone writing a translation. The probe module woke up in an environment whose yml had never heard of it.

*Meridian's homegrown loader had a crude version of this,* Kenza thought. *Read the properties file, then let -D flags stomp it: two rungs, hand-coded, in a class from 2014. This is the same ladder grown five rungs tall, with the case-translation I'd never have thought to write. For once the hand-wired instinct is a head start instead of a trap.*

"One more thing," she said, and it came out before she'd decided to confess it. "At Meridian I wrote all of this wiring myself in main(), and I complained about the boilerplate constantly. Boot deleted the boilerplate, and I never once asked where it went."

"Every convenience is a debt of understanding," Rachid said. "Today you paid one back."

## The fix

The rewrite was small and merciless. Every condition became explicit and carried a comment saying why. The property binding moved out of stringly-typed `@Value` into a validated record. The difference is mechanical. `@Value` resolves one string at one injection point and fails wherever that bean happens to live, while `@ConfigurationProperties` binds the whole tree: typed, relaxed, validated at startup, and visible to the IDE's metadata. And the starter's own bean now wore the back-off gate, so any future app could override it by declaring:

```
@Validated
@ConfigurationProperties("glasshouse.monitoring.replica-probe")
public record ReplicaProbeProperties(
        boolean enabled,
        @DefaultValue("5s") @DurationMin(seconds = 1) Duration maxLag) { }

@AutoConfiguration(after = DataSourceAutoConfiguration.class)  // the DataSource must
        exist first
@ConditionalOnClass(name = "io.osprey.metrics.probe.jdbc.ReplicaHealthProbe") // 3.x
        home, by name
@ConditionalOnBean(DataSource.class)
@ConditionalOnProperty(prefix = "glasshouse.monitoring.replica-probe",
                       name = "enabled", havingValue = "true")
@EnableConfigurationProperties(ReplicaProbeProperties.class)
public class ReplicaMonitoringAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean(name = "replicaLagHealthIndicator")  // override by
        declaring
    HealthIndicator replicaLagHealthIndicator(DataSource ds, ReplicaProbeProperties
        props) {
        return new ReplicaLagHealthIndicator(ReplicaHealthProbe.of(ds),
        props.maxLag());
    }
}
```

The `management.health.db.enabled: false` line was deleted from the prod profile; Boot's default primary check now ran everywhere as the baseline, with the replica probe layered on top. The fossil `spring.factories` was finally buried. Because a silent absence had cost them forty blind minutes, Kenza added the test the incident had shown was missing: a startup smoke test that boots the context with prod-shaped properties and asserts, by name, that the critical beans exist. It says nothing about whether they work. Absence had just been promoted to a failure class of its own.

It was, she realized while naming the test class, ADR-002's move aimed at a new target. The fitness functions from the morning staging died politely had turned architecture rules into executable claims that fail the build the moment they stop being true; this one made the container's composition the same kind of claim.

The runbook gained its shortest and most useful page: *"My bean didn't get created": run with `--debug`, read the ConditionEvaluationReport, find your class under negative matches, read which condition said no. The report is the answer; everything else is guessing.*

One more file rode along in the PR, the binder's fourth entry, half a page, dated:

> **ADR-004: Config governance. Context:** A transitive repackage inside a routine version bump silently removed prod's only database health bean; at standup nobody could say why any bean existed anywhere. **Decision:** Feature configuration binds through typed `@Validated @ConfigurationProperties` only, with no raw `@Value` in feature code; profiles overlay a shared base, never fork it; every third-party version rides the BOM or gets a named owner; a startup smoke test asserts critical beans exist by name; reading the `--debug` condition report is a documented runbook skill. **Consequences:** Config becomes a schema, validated at boot; the BOM constrains upgrade freedom in exchange for a tested matrix; absence is now a failure class with a test that catches it.

### ⚖ *Your move, architect*

The PR was easy to approve. The harder decision sat underneath it: what is Atrium's config strategy, now that everyone has watched a bean vanish without a sound? You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Fix the bean, keep ad-hoc `@Value` and tribal knowledge.** Document the gotcha on the wiki; zero process cost today.
- **B. Govern it.** Typed `@Validated @ConfigurationProperties` only, profiles that overlay rather than fork, all versions on the BOM, the bean-existence smoke test, condition-report reading as a runbook skill.
- **C. Centralize now.** Stand up Spring Cloud Config Server: one source of truth, runtime refresh, every deployment pulls its config at boot.

**The trade-offs, since you've committed: A** costs nothing today and prices the next vanishing bean at a day of blind dashboards. The tribal knowledge it leans on resigned in 2023, and operability stays folklore. **B** is what Kenza wrote into ADR-004. It charges a schema tax (every knob needs a record, a constraint and a metadata entry) and buys *interrogability*: config that fails loudly at boot instead of lying silently at runtime, and bean existence asserted by a test instead of assumed from memory. **C** buys runtime refresh nobody asked for. The price is a new failure domain in front of every boot (config server down, fleet can't start), plus a consistency question nobody was ready to answer in March: which pods hold which config? The shape is the same as January's move. B keeps C purchasable, because typed records relocate to any backend, while C bought now is infrastructure in search of a requirement. (Chapter 14 prices central config honestly, once there's more than one deployable to feed.)

"Where does Kubernetes point at this thing, now that it's back?" Youssef asked, mostly to himself, while reviewing the PR. "Liveness, readiness, we should fight about that properly someday." Nobody took the bait. It was March; that fight was a September problem, and nobody knew it yet.

On Friday, Kenza gave her first-ever brown-bag, standing at the front of a room instead of the side of one. She titled the deck *The Magic Show* and walked the whole chain: `@SpringBootApplication` to its three annotations, the two doors into the context, the `.imports` roll-call, the conditional gates, and the precedence ladder Youssef had inked permanently onto the Wall beneath February's smoke-cloud proxy drawing and its caption, *this. goes around the mirror*. The penultimate slide was the one nobody had asked for and everybody photographed: the greenfield path, the same machinery with nothing on fire. New service → `start.spring.io` → pick the starters. The parent BOM pins every third-party version, so you declare dependencies and almost never a version number; auto-configuration conditions in a working baseline off your classpath. Your whole job is a typed `@ConfigurationProperties` record for your own knobs, an `application-{profile}.yml` per environment, and one calm read of the condition report before the first deploy.

Her closing slide was a single sentence: *A bean is never magic. It is a list entry that survived its conditions.*

Rachid sat in the second row and wrote nothing in green the entire hour. Afterward, he stopped at her desk and pointed his pen toward the controllers package.

"You know how the beans are born," he said. "Now learn what they do all day."

## The debrief: Ch.4, The Magic Show

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 4.** How does `@EnableAutoConfiguration` find configurations, and how would you debug one that didn't apply?

Then the follow-ups, cold:

1. Why does your own `@Bean` override Boot's default, and when doesn't it?
2. Why is `@ConditionalOnBean` dangerous without `@AutoConfigureAfter`?
3. The platform team proposes Spring Cloud Config Server "so this never happens again." Your call?
4. `@ConfigurationProperties` or `@Value`?
5. How do you write a custom starter?

Tick what you said:

- ▢ Claim: Boot never scans jars for configuration: it reads a declared list and lets conditions veto it.
- ▢ Mechanism: The `AutoConfiguration.imports` files, the `@Conditional` gates, and user configuration processed first so `@ConditionalOnMissingBean` defaults yield.
- ▢ Trade-off: Silent back-off lets your `@Bean` win without flags, and the price is that absence produces no error and no log.
- ▢ Failure mode: A renamed class or an unset profile property deletes a bean with zero output. `--debug` and the ConditionEvaluationReport show it.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Boot never scans jars for configuration; it reads a declared list and then lets conditions veto it. **Mechanism:** `@EnableAutoConfiguration` imports a selector that reads every `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file on the classpath (`spring.factories` before Boot 2.7/3.0), then evaluates each candidate's `@Conditional` gates: `OnClass` against ASM-read metadata so an absent class can't blow up the read, `OnProperty` against the environment, and `OnBean`/`OnMissingBean` against the beans registered *so far*, which is why `@AutoConfigureAfter` ordering is part of correctness. User config is processed first, so Boot's `@ConditionalOnMissingBean` defaults yield to your declarations. **Trade-off:** Silent back-off is the design. It is what lets your `@Bean` win without flags, and the price is that absence produces no error and no log. **Failure mode:** A renamed class or an unset profile property deletes a bean with zero output. The diagnostic is `--debug` (or `/actuator/conditions`) and reading the negative matches in the ConditionEvaluationReport. Grepping your own code finds nothing, because the bean was never in your code.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Evolvability as a supply-chain property**: the BOM is a tested-matrix contract. Bumps inside it move between tested points, and a dependency outside it makes every upgrade an architecture event with a blast radius someone must own. And **operability as interrogability**: a container that can explain its own composition (`--debug`, `/actuator/conditions`) gives you that property only if reading the explanation is a designed habit. Silent back-off spends operability to buy override-by-declaration; ADR-004 buys it back at the beans that matter.
- **The ADR:** ADR-004, config governance: typed `@Validated @ConfigurationProperties` only, profiles overlay (never fork), versions ride the BOM or get an owner, critical beans asserted by name in a startup smoke test, the condition report a runbook skill. Config becomes schema, validated at boot.
- **Rejected alternatives:** ad-hoc `@Value` plus tribal knowledge (free today; the next vanishing bean costs a blind day, and the tribe resigned in 2023); Spring Cloud Config Server now (runtime refresh nobody asked for and a new failure domain in front of every boot, re-priced in Ch.14); disabling auto-configuration and hand-wiring everything (fighting the whole social contract to punish one unread jar).
- **Fight or lean:** *Lean* on conditions and the BOM: silent back-off is what lets your `@Bean` win by declaring, and the matrix is tested so you don't have to. *Fight* the silence wherever your 3 a.m. depends on a bean existing. The framework can't know which absences are fatal to you, so promote those to existence that a smoke test asserts by name.

**Story anchors**

- The green dashboard over a dead replica → `@ConditionalOnClass` backing off silently after the class moved packages in a routine bump.
- The two doors on the Wall → `@ComponentScan` from the root package (your beans) vs `AutoConfiguration.imports` (the jars' beans), two distinct entrances into the context.
- Youssef's env-var flip on the staging deployment → the config-precedence ladder (CLI > env > profile yml > application.yml > defaults) plus relaxed binding mapping `SCREAMING_SNAKE` onto kebab-case.
- Rachid's standup question, *"Why does this bean exist in staging and not in prod?"*, met by eight engineers' silence → a bean nobody can explain is a bean nobody owns; ADR-004 turns existence into an asserted, smoke-tested claim.

**Interview ammo**

- *Kill-shot: How does `@EnableAutoConfiguration` find configurations, and how would you debug one that didn't apply?* It reads the `AutoConfiguration.imports` files from every jar (not classpath scanning) and filters each candidate through its `@Conditional` gates. To debug one, use `--debug` or `/actuator/conditions` and read which condition vetoed your class.
- *Follow-up: Why does your own `@Bean` override Boot's default, and when doesn't it?* User config registers before auto-config evaluates, so Boot's `@ConditionalOnMissingBean` sees your bean and backs off. It fails to back off when your bean's type/name doesn't match what the condition checks, and then you get both.
- *Follow-up: Why is `@ConditionalOnBean` dangerous without `@AutoConfigureAfter`?* Conditions evaluate against bean definitions registered so far, so without ordering you can condition on a bean that's defined ten milliseconds later and wrongly back off. Only condition on beans from configs you're explicitly ordered after.
- *Architect follow-up: The platform team proposes Spring Cloud Config Server "so this never happens again." Your call?* Wrong -ility. This incident was about interrogability, not centralization. Typed validated properties plus a bean-existence smoke test buy that without a new failure domain in front of every boot. Central config gets priced when there are multiple deployables and a real runtime-refresh requirement.
- *Follow-up: `@ConfigurationProperties` or `@Value`?* `@Value` resolves one string at one injection point and fails wherever that bean happens to live. `@ConfigurationProperties` binds the whole tree into one typed object, with relaxed binding, validation at startup and metadata the IDE can read.
- *Follow-up: How do you write a custom starter?* Name it company-first, as in `glasshouse-monitoring-spring-boot-starter`, because `spring-boot-starter-*` belongs to Boot. Put the beans in an `@AutoConfiguration` class and list its fully qualified name, on a line

of its own, in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Make every condition explicit, order the class with `after`, bind its knobs through a `@ConfigurationProperties` record, and put `@ConditionalOnMissingBean` on its beans so an application overrides one by declaring its own.

**Pointers** → `examples/ch04_autoconfig/` + `examples/ch03_starters/`

---

# Part III: The Request

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Servlet:** Java's standard unit of HTTP handling. All of Spring MVC runs as one servlet. **Filter:** code that sees every request before (and after) the servlet runs. Security lives here, before your code is ever reached. **DispatcherServlet:** Spring's front controller. It receives every request and routes it to the matching controller method. **Controller:** the annotated class whose methods handle routes. Arguments are bound from the request, and validated only where the method declares it (`@Valid` on a body, a constraint on a parameter); return values are serialized to JSON by message converters. **Worker thread / pool:** the embedded server (Tomcat) gives each request one thread from a bounded pool. While a thread is busy with one request, it serves no other. **Timeout:** an upper bound you can set on how long an outbound call may wait.

# Chapter 5: The Front Door

The report landed in the Stonethrow queue on a Tuesday morning, and Malika escalated it before her coffee went cold. Reporter: @sable, the number-one name on Glasshouse's own leaderboard, the researcher whose real identity was a running office bet. Severity: Medium, information disclosure. Title: *Your front door leaks your floor plan.*

Leila read it aloud at triage, every word, because she wanted the room to hear the prose. It was clinically polite. The reproduction steps were better than the team's own runbooks: numbered, deterministic, with expected-versus-actual for each request. Step three was the one that made the room go still:

```
POST /api/reports HTTP/1.1
Host: glasshouse.io
Content-Type: application/json

{"programId": 6093, "title": null, "body": "x", "severity": "HIGH"}

HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "timestamp": "2026-03-24T09:41:17.882Z",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "org.springframework.dao.DataIntegrityViolationException: ...
      at org.hibernate.engine.jdbc.spi.SqlExceptionHelper.convert...
      ... HikariPool-1 ... jdbc:postgresql://atrium-prod-db:5432/atrium ...",
  "message": "could not execute statement [...] [insert into report ...] ...",
  "path": "/api/reports"
}
```

A null title. That was all it took. Atrium answered with a full stack trace: Hibernate class names, the failing SQL, and, riding along in a wrapped cause's message, a fragment of the production JDBC URL. Malformed JSON got a 400 with the same `trace` field. An invalid severity enum, likewise. Every failure shape on the public API was a guided tour of the internals.

The report closed with a single lowercase signature: *s.*

"Our best customer," Leila said quietly, "just told us our porch light shines straight onto the safe." She looked around the table. "We sell trust to security researchers. This is the one bug we don't get to have."

Rachid slid the printout across the table to Kenza. "It's yours."

She took it with both hands and felt January arrive again: comment #1 beside a line that *worked, works is not done.* Same lesson, new postmark. This time the green ink had come from outside the building.

> **Your call.** Before Kenza opens a single file, commit: who, or what, assembled that error JSON, what let a null title reach Postgres, and which -ility just failed in front of the platform's best researcher?

## The investigation

Reproducing it took ninety seconds. Understanding it took the afternoon, and the understanding was worse than the bug.

First question: who wrote that error body? No code in Atrium built that JSON. The `DataIntegrityViolationException` left the repository uncaught, so the servlet container performed an *error dispatch* to Boot's `/error` endpoint, where an auto-configured `BasicErrorController` assembled a response from `DefaultErrorAttributes`. The answer was *nobody*, and that was the problem. The stack trace was in it because of five lines of YAML:

```
# application.yml, git blame: 2019, "temporary, debugging the CSV importer"
spring:
  web:
    error:
      include-stacktrace: always
      include-message: always
```

*Temporary.* The default for `include-stacktrace` was `never`; someone had opted into the leak, once, for a CSV importer that no longer existed. The Boot 4 migration had renamed the key mechanically, from `server.error` to `spring.web.error`, and nobody had questioned its value, so the blame behind that rename still said 2019. She read `ErrorMvcAutoConfiguration` until she knew exactly which bean built which field. It was the new reflex, kicking in without her noticing: read the machinery until it stops being magic. The commit was old enough to start school.

Second question: why was a null title reaching the database at all? She opened the submission controller and found 2019 staring back:

```
@RestController
public class ReportSubmissionController {

    @Autowired
    private ReportRepository reports;            // 2019 vintage

    @PostMapping("/api/reports")
    public Report submit(@RequestBody Report report) {  // the JPA entity, raw off the
        wire
        return reports.save(report);             // no @Valid: the database is the
        validator now
    }
}
```

The JPA entity was bound straight from the wire and saved with nothing checking it in between; the only validation was whatever constraint the database happened to enforce. That meant the *error messages* were written by Postgres and Hibernate, for an audience of attackers.

It was comment #1 run in reverse, she realized. In January, an entity had leaked *out* through Jackson after the session closed, the contract coupled to the schema on the way to the wire. Here, one came *in* off the wire before anything checked it: the same unguarded boundary, crossed in the opposite direction.

*At Meridian I parsed request JSON by hand, and the null check lived in the same ten lines as the parsing. Bound and checked were one motion,* she thought. *My instinct says a framework that binds the object has surely checked it too.* In Spring MVC, binding and validation are separate steps, and the second is opt-in: no `@Valid` on the parameter means no validation, no exception, no 400. Her instinct was the trap. The framework had been doing exactly what the code asked, which was nothing.

(If you called it at the pause: both answers are *nobody*, and the -ility is more than security: it's contract integrity. Rachid gets there.)

Third: the wider she looked, the worse it got. Three other public controllers had hand-rolled `try/catch` blocks returning three different error shapes: one `{"error": ...}`, one `{"message": ...}`, one a bare string. And when she replayed Sable's request with `Accept: text/html`, the same failure came back as the Whitelabel error page. The error path chose an HTML view or a JSON body depending on the header. It had *content-negotiated its own leak*. Nobody had designed the API layer. It was sedimentary.

## What the senior sees

"Before you fix the door," Rachid said, uncapping a marker at the Wall, "learn the house." He drew a socket on the far left and a database cylinder on the far right, then handed her the marker. "One request. Socket to DB and back. You talk, I interrupt."

"A request hits the pod. Tomcat's connector accepts the connection and a worker thread picks it up."

"Then?"

"The filter chain. Servlet filters, before any Spring MVC exists." She drew boxes. "Then the `DispatcherServlet`."

"Which is what?"

"A servlet. Just a servlet, registered at `/`, sitting inside Tomcat like any other." She heard how the sentence demystified itself as she said it. "The entire framework happens inside one servlet's `service()` method. Front controller pattern: one entry point that dispatches to everything else."

*So the DispatcherServlet is Meridian's biggest hand-rolled servlet, finished: the routing table, the JSON parsing, the dispatch switch, the response writing, everything that servlet grew inside one `service()` method, redone as pluggable strategies,* she thought, and for once the hand-wired years were a head start instead of a trap.

"Who calls your controller method?" Rachid asked. "Name it."

"The DispatcherServlet…"

"Too coarse. Name the hop."

She worked it out at the board, source open on her laptop. The `HandlerMapping` (concretely, `RequestMappingHandlerMapping`) matched path and method. It returned a `HandlerExecutionChain`: the handler plus the `HandlerInterceptors` wrapped around it. Atrium's own `ApiTimingInterceptor` ran there: `preHandle` before the handler, `afterCompletion` on the way out even when things failed, `postHandle` only on success. Then the `HandlerAdapter` invoked the method. *It* was the one that knew how to build the arguments.

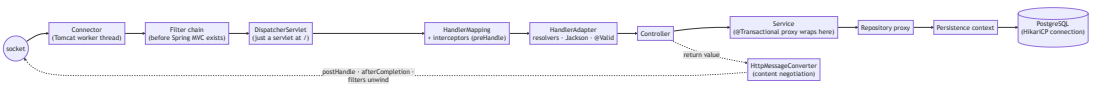
"What turns Sable's JSON into your object, and where does it live?"

"An argument resolver. `RequestResponseBodyMethodProcessor` sees `@RequestBody`, asks the registered `HttpMessageConverters` who can read `application/json`, and Jackson's converter deserializes it." She tapped the spot. "And if the parameter is annotated `@Valid`, Bean Validation runs right here, after binding, *before* the controller is ever entered. Failure throws `MethodArgumentNotValidException`, and the controller never sees the garbage."

"And the way back?"

"`@RestController` means `@ResponseBody`, so the return value skips view resolution and goes back through the converters, chosen by content negotiation against the `Accept` header." She drew the unused branch anyway: `ViewResolver`, where a `ModelAndView` would have gone looking for a JSP, the older model of server-rendered HTML. Atrium hadn't rendered a view since 2021; the SPA drew the screens, the API served the data, and the JSP branch on the diagram got labeled *the road not taken*. "Then interceptors unwind, filters unwind, response flushes."

She stepped back. The drawing ran the full width of the Wall, socket to cylinder, beneath February's smoke-cloud proxy (the `@Transactional` wrapper a `this.` call walks straight past, dispatching on the raw object inside) and March's precedence ladder (property sources outranking one another in fixed order, command line down to packaged defaults):



(The compact version, born that afternoon; the full render lives in Appendix E.)

Looking at it, she understood why Rachid had cancelled an afternoon for this, as he had earlier in March. That afternoon, the ConditionEvaluationReport had made bean absence *interrogable*: every auto-configuration candidate listed, every vetoing condition named. This drawing gave a request the same property. Systems you can ask, instead of systems you must believe.

Rachid studied the wall. "Now the question that matters. Where would you catch an exception thrown by a *filter*?"

"`@RestControllerAdvice`," she said. "Global exception handling, catches everyth…"

He waited, which with Rachid meant *no*.

*A `try/catch` at the outermost frame catches everything beneath it. At Meridian, the hand-rolled logging filter had been that frame.* She'd assumed the advice sat where her old catch-all had. Her own diagram said otherwise. The advice was no layer in front of anything: it was consulted by `ExceptionHandlerExceptionResolver` *inside* the DispatcherServlet's dispatch loop. A filter that threw did so *before the servlet ran for that request*, so the exception fell to the container, which error-dispatched it to `/error`, straight into the auto-configured controller with the loose YAML. The hand-wired reflex was the trap, and that path was the actual anatomy of Sable's 500.

"Advice lives inside the servlet's world," she said slowly. "Filters live in front of it. My advice can never see a filter's exception. They're different jurisdictions."

Rachid drew three brackets under the diagram. "Filter: sees the raw `ServletRequest` and `ServletResponse`, doesn't know what a handler is, can wrap or short-circuit everything. Interceptor: inside the servlet, knows *which* handler was chosen, brackets the invocation. Advice: narrowest scope, richest context, the exception phase of dispatch only. People file this choice under style," he added. "It's jurisdiction: a layering decision about which layer is *allowed to know what*. A filter that imports your domain types has annexed territory it can't defend. Draw the borders on purpose, and every cross-cutting concern has exactly one legal home." Then he tapped the empty whiteboard to the left of the servlet, out past the filters. "Security lives out here, in front of the front door. Remember the geography. One day the worst week of your year will start in this exact spot."

She wrote it down. It would take until July to learn what he meant.

## The fix

Kenza wrote the rebuild plan as four invariants, because rules outlive patches.

**One: DTOs at the boundary. Entities never.** This finished the lesson from her first month, comment #1 in green ink: *"Works is not done. What happens when this entity meets Jackson after the session closes?"* Records, immutable, with Jakarta Bean Validation doing the gatekeeping declaratively:

```
public record SubmitReportRequest(
        @NotNull Long programId,
        @NotBlank @Size(max = 200) String title,
        @NotBlank String body,
        @NotNull Severity severity) {}

@RestController
@RequestMapping("/api/{version}/reports")
class ReportSubmissionController {

    private final ReportSubmissionService service;

    ReportSubmissionController(ReportSubmissionService service) {
        this.service = service;
    }

    @PostMapping(version = "2")
    ResponseEntity<ReportCreatedResponse> submit(
            @Valid @RequestBody SubmitReportRequest request) {
        ReportCreatedResponse created = service.submit(request);
        return ResponseEntity
                .created(URI.create("/api/v2/reports/" + created.id()))
                .body(created);
    }
}
```

That `version = "2"` was Framework 7's new native versioning. The attribute names a version but never says where one lives in a request, and the framework will not guess: start the app with no strategy registered and the context fails fast, with *API version specified, but no ApiVersionStrategy configured*. On its own the annotation is half a sentence; one registration completes it, and Kenza put hers where the next person would go looking for the policy:

```
@Configuration
class ApiVersionConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer versioning) {
        versioning.usePathSegment(1,                    // /api/v2/reports → version "2"
                          path -> path.value().matches("/api/v\\d+/.*"))
                  .addSupportedVersions("1", "2")
                  .setDefaultVersion("1");              // no version in the path →
        version 1
    }
}
```

`setDefaultVersion("1")` was ADR-005's "explicit defaults" clause made executable: a request with no version in its path was treated as version 1, the shapes integrators already depend-

ed on, and could never land on a newer mapping by accident. The predicate kept the resolver off the routes that predated versioning, such as January's `/api/programs/{id}/digest`, where a bare `usePathSegment(1)` reads `programs` as a version and answers 400. A version outside the supported list earned a designed 400.

A correct submission now returned what the HTTP spec had always asked for: 201 Created with a `Location` header pointing at the new resource, as the contract requires. The detail endpoint alongside it took its id as a `@PathVariable` and an `includeActivity` flag as a `@RequestParam`, each bound by its own resolver; only `@RequestBody` ever touched Jackson.

**Two: one error shape for the whole API.** A single `@RestControllerAdvice` producing RFC 9457 `ProblemDetail`, a standard error format that has been first-class in Spring since Framework 6, machine-readable `type` URIs and all:

```
@RestControllerAdvice
class ApiProblemHandler extends ResponseEntityExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(ApiProblemHandler.class);

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        var pd = ProblemDetail.forStatusAndDetail(
                HttpStatus.BAD_REQUEST, "Request validation failed.");
        pd.setType(URI.create("https://glasshouse.io/problems/validation"));
        pd.setProperty("errors", ex.getFieldErrors().stream()
                .map(f -> Map.of("field", f.getField(), "reason",
        f.getDefaultMessage()))
                .toList());
        return ResponseEntity.badRequest().body(pd);
    }

    // Spring Security's denial: rethrow it, and the security filter answers 401 or
        403.
    @ExceptionHandler(AccessDeniedException.class)
    void handBack(AccessDeniedException ex) {
        throw ex;
    }

    @ExceptionHandler(Exception.class)   // last resort: log everything, tell nothing
    ProblemDetail fallback(Exception ex, HttpServletRequest request) {
        log.error("Unhandled exception at {}", request.getRequestURI(), ex);
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error occurred.");
    }
}
```

`MethodArgumentNotValidException` became a structured 400 listing each failing field and its reason: the client's own data, nothing internal. `HttpMessageNotReadableException`, Sable's broken-JSON case, kept its 400 and took the `ProblemDetail` shape from the base class. `ResponseEntityExceptionHandler` maps Spring MVC's own exceptions to their statuses, and the handler declared for the closest exception type wins, so a 404, a 405 and an unsupported version never reach the catch-all, which would have answered 500 to each of them. A constraint placed directly on a parameter, such as a `@Min` on a `@RequestParam`, raises `HandlerMethodValidationException`, and the base class answers that with a 400 as well. And the 2019 override came out of `application.yml` entirely, returning `include-stacktrace` to its default `never`. Even the container's error dispatch, the one path her advice could never reach, now had nothing left to leak.

**Three: versioning as a first-class citizen.** As of Boot 4, with Spring Framework 7 underneath, API versioning is native to the web framework: a `version` attribute on the mapping annotations, with the resolution strategy (path segment, header, or query parameter) registered once in a `WebMvcConfigurer`, the `ApiVersionConfig` above. Atrium chose the path segment; public routes moved to `/api/v2/**`, the v1 shapes were frozen and marked deprecated, and `RequestMappingHandlerMapping` now matched on version the same way it matched on path and method. No more hand-rolled `if (version.equals(...))` archaeology.

**Four: one Jackson, configured once.** A single builder-customizer bean, Boot's hook for shaping the auto-configured mapper, set non-null inclusion and ISO-8601 timestamps in one documented place, replacing the per-controller `ObjectMappers` scattered through the codebase. Content negotiation stayed, but every negotiable representation now came from the same, deliberate configuration. A `MockMvc` test pinned each failure shape so the contract couldn't drift back.

## The contract, governed

The invariants were done by Wednesday evening when Leila appeared with the bounty sheet. "Triage scored it Medium. Sanity-check me: my gut says we got off cheap."

"Your gut's right," Kenza said. "Medium scores the *data*; it's the wrong score for what it proved. A stack trace in an error body isn't cosmetic. It's a contract violation. The error surface is part of the API; Sable consumed a response we never designed, which means we'd spent seven years publishing a contract nobody wrote."

Rachid, passing with coffee, stopped. "Say the -ilities."

"Interoperability: one error grammar instead of four accidents. Evolvability: a version policy, so the contract can change without breaking whoever built against it. And security, because every undesigned response is attack surface. An API is a governed product surface: who may change what, with what notice. Deprecation should be a process with dates, not an outage with apologies."

"Then write the governance down," Rachid said, "or you've described a mood."

> **ADR-005: API contract governance. Context:** A malformed request leaked a stack trace (Hibernate internals and a production JDBC URL fragment) to an external researcher, because Atrium's error responses were accidental: per-endpoint folklore in three shapes, plus Boot's `/error` running under a 2019 debug flag. **Decision:** RFC 7807/9457 `ProblemDetail` is the only error shape on the public API; all responses are records, never entities (per ADR-001); validation happens at the binding layer via `@Valid`; the versioning policy is deliberate: Boot 4 native versioning, path-segment resolution, explicit defaults, and a published sunset process for deprecated versions. **Consequences:** Every error is now a designed response and every contract change a governed one; the version policy taxes each new endpoint with a decision it never used to owe, but it makes deprecation a process instead of an outage.

### ⚖ *Your move, architect*

Invariant three was the contested clause. The design review ran forty minutes over, because everyone had a versioning religion. Boot 4 makes any strategy a one-line registration, and ADR-005 demanded a sunset policy whichever way the room voted, so the forty minutes went to the resolution mechanism itself: where, physically, does the version live in a request integrators script against? You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Path segment.** `/api/v2/**`: every log line, curl reproduction, and cache key agrees on what was called.
- **B. Header / media-type.** Canonical URLs forever; the version travels invisibly in `Accept` or a custom header.
- **C. Query parameter.** `?version=2`: the cheapest thing an integrator can flip; ugly in caches and bookmarks.

**The trade-offs, since you've committed: A**, Atrium's choice, buys debuggability and cache-friendliness (every log line, cache key, and support ticket agrees on what was called) and pays in URL proliferation: the version infects every link and tempts teams into copying whole route trees. **B** buys clean, stable URLs and pays in invisibility: curl reproductions omit the header and caches can't vary without `Vary` discipline, so debugging is harder by design. **C** buys the gentlest migration (integrators change one parameter and keep their URL structure) and pays at the cache and at the contract: query strings fragment cache keys, leak into referrer logs and bookmarks, and make the version read like an option instead of a promise. None of the three rides free. Any scheme applied uniformly beats per-team improvisation, and Atrium's failure was having four of them.

And the mechanism is the *small* decision; pick wrong and you pay for a migration. The decision underneath is the public contract: the `ProblemDetail` shape and the URL grammar integrators build against. That one is a one-way door. Once Sable's scanners parse your error bodies and

Northbeam's scripts hard-code your routes, every "we'll firm it up later" becomes a breaking change you inflict on people who did nothing wrong. Shipping loose to "keep the contract flexible" means deciding later under *worse* information (live integrators, frozen mistakes) and paying twice: once to build loose, once to break everyone. Hence ADR-005's consequences clause: deprecation is a process with dates, because nothing else reopens the door. So commit deliberately, version from day one, and spend the review effort *before* first publish, when forty minutes in a room is the cheapest the decision will ever be.

The retest request went out Thursday. Sable's reply was one line: *Better. s.*

On Friday, Kenza walked the full team through the Wall: socket to DB and back, every component named, every jurisdiction marked. The Request Journey diagram stayed up; the sun-bleached *DO NOT ERASE* note acquired a neighbor. The ADR binder took its fifth entry, and Malika taped Sable's report beside the diagram, the outside world permanently auditing the drawing. Rachid decreed, in the tone he used for things that were already true, that every future hire would trace it in their first week.

"You'll be giving the tour," he added to Kenza, and then, almost to himself: "I won't be the one giving this tour forever."

She didn't catch the weight of it. Leila was already talking about how marketing wanted the newly presentable platform front and center for a spring campaign, and that felt like good news.

## The debrief: Ch.5, The Front Door

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 5.** Trace an HTTP request from the socket to the DB and back.

Then the follow-ups, cold:

1. Where do you catch an exception thrown by a filter, and why not `@ControllerAdvice`?
2. What exactly builds your `@RequestBody` argument, and what fires when validation fails?
3. How do you version a public API without breaking integrators, and how much does the mechanism matter?
4. What fires when a constraint on a `@RequestParam` fails?

Tick what you said:

- ☐ Claim: One servlet, the `DispatcherServlet`, dispatches every request through pluggable strategies: a front-controller pipeline.
- ☐ Mechanism: In order: connector, filter chain, `DispatcherServlet`, `HandlerMapping`, `HandlerAdapter` with argument resolvers, controller, then the converter on the way back.

- [ ] Trade-off: The uniform pipeline centralizes cross-cutting concerns cheaply, but each layer sees only its own jurisdiction.
- [ ] Failure mode: Exceptions thrown in front of the servlet never reach `@ControllerAdvice`, and a permissive `/error` configuration publishes your stack traces.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Spring MVC is a front-controller pipeline: one servlet, the `DispatcherServlet`, dispatches every request through pluggable strategies. **Mechanism:** Tomcat's connector accepts the socket and assigns a worker thread → the servlet filter chain runs (security lives here, before Spring MVC exists) → `DispatcherServlet` → `HandlerMapping` selects the handler plus its interceptor chain → `preHandle` → `HandlerAdapter` invokes the method via argument resolvers (`@RequestBody` through Jackson's `HttpMessageConverter`, then `@Valid` via Bean Validation; `MethodArgumentNotValidException` fires before the controller runs) → controller → service (where the `@Transactional` proxy wraps) → repository proxy → `EntityManager` and persistence context → a HikariCP connection → the database; the return value goes back through a converter chosen by content negotiation, `postHandle/afterCompletion` fire, filters unwind, the response flushes. **Trade-off:** the uniform pipeline centralizes cross-cutting concerns (advice, converters, versioning) cheaply, but each layer sees only its own jurisdiction. **Failure mode:** exceptions thrown in front of the servlet never reach `@ControllerAdvice`; they fall to the container's error dispatch, and a permissive `/error` configuration will happily publish your stack traces, which is exactly how a security platform leaked its floor plan.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Contract integrity**, meaning interoperability (one error grammar every integrator can parse) and **evolvability** (one deliberate version policy), plus **security of the error surface**: every response shape you didn't design is attack surface you don't control.
- **The ADR:** ADR-005, API contract governance: `ProblemDetail` as the only error shape, records at the boundary (per ADR-001), validation at the binding layer, versioning with explicit defaults and a published sunset process.
- **Rejected alternatives:** per-endpoint `try/catch` folklore (the sedimentary API, three error shapes and counting); leaving validation to the database (Postgres writing client-facing error messages for an audience of attackers); ungoverned header/media-type ver-

sioning (clean URLs bought with invisibility); and no version policy at all, which is the same decision left unrecorded.

- **Fight or lean:** *Lean* on the framework's contract machinery: `ProblemDetail`, `@Valid`, advice, native versioning are first-class; hand-rolling them is fighting for no prize. *Fight* your own overridden defaults (`include-stacktrace: always` was the team opting *out* of safety and forgetting the sale), and never fight the geography: harden `/error` rather than pretending advice can reach a filter's exception.

**Story anchors**

- *Sable's 500 with the JDBC URL in it* → unhandled exceptions falling through to Boot's `/error` with `include-stacktrace: always`; honest `ProblemDetail` from a `@RestControllerAdvice` is the cure.
- *"Who calls your controller method? Name it."* → the DispatcherServlet pipeline: HandlerMapping → interceptors → HandlerAdapter → argument resolvers → converters.
- *Kenza reaching for advice to catch a filter's exception* → jurisdictions: Filter (in front of the servlet, raw streams), HandlerInterceptor (inside, knows the handler), advice (exception phase of dispatch only).
- *Leila's bounty sheet: "it's the wrong score for what it proved"* → the error surface is security architecture: information disclosure is a contract violation, not a cosmetic bug. Hence ADR-005, every error a designed response.

**Interview ammo**

- *Kill-shot: Trace an HTTP request from the socket to the DB and back, naming every component.* The four-beat answer above; the differentiator is naming the hops (`RequestMappingHandlerMapping`, `HandlerAdapter`, argument resolvers, `HttpMessageConverter`) and knowing what wraps where.
- *Follow-up: Where do you catch an exception thrown by a filter, and why not `@ControllerAdvice`?* Advice is invoked by `ExceptionHandlerExceptionResolver` inside the DispatcherServlet's dispatch; a filter throws before the servlet runs, so you handle it in the filter layer itself (or accept the container's error dispatch to a hardened `/error`).
- *Follow-up: What exactly builds your `@RequestBody` argument, and what fires when validation fails?* The `RequestResponseBodyMethodProcessor` argument resolver delegates to the Jackson `HttpMessageConverter` selected by content type, then runs Bean Validation for `@Valid`; failure throws `MethodArgumentNotValidException` before the controller method is entered.
- *Architect follow-up: How do you version a public API without breaking integrators, and how much does the mechanism matter?* Less than the policy: pick one resolution strategy (Boot 4's `version` attribute makes path, header, or query first-class), publish explicit defaults and

a dated sunset process, and pin it in tests, because integrators can plan around a process.

- *Follow-up: What fires when a constraint on a `@RequestParam` fails?* A constraint placed directly on a parameter raises `HandlerMethodValidationException`. The `@Valid` body of `submit` raises a different one, `MethodArgumentNotValidException`. An advice that extends `ResponseEntityExceptionHandler` answers both with a 400 `ProblemDetail`.

**Pointers** → `examples/ch10_mvc/`

---

# Chapter 6: The Pool

The "Spring into Security" banner went live at ten o'clock on a Tuesday, and for twenty glorious minutes it worked as promised: sign-ups tripled. Leila stood at the back of the exec room watching the load balancer's traffic line, the only graph Atrium had, climb like a good quarterly slide.

At 10:31, Cortexa began answering in thirty seconds instead of three hundred milliseconds. This was the third-party CVE-enrichment vendor that Atrium called synchronously inside every report submission. Their status page would later say *elevated latency in one region*. Glasshouse's status page said nothing, because at 10:32 Glasshouse stopped saying anything at all.

The whole site 503'd. The submission form went down, and so did login, the leaderboard, and the static marketing page the promo pointed at, which was served through the same app. The brick screamed on the desk of whoever held it that week, and Youssef appeared at Kenza's shoulder, already having checked the obvious things.

"CPU?" Kenza asked.

"Six percent." He turned his laptop around: six healthy-looking pods, idling. "Memory's fine. Disk's fine. Postgres is bored. The site is down and every machine involved is having a restful morning."

In the war room, an exec said the sentence execs say: "Can we add more pods?" Youssef added more pods. Six became twelve became eighteen, and the traffic graph didn't move. The new pods came up green, accepted connections, and went exactly as silent as the old ones.

"We just bought more chairs," Youssef said, "in front of the same locked door."

Leila was on the bridge, watching launch conversions flatline in a window two executives could also see. "Someone tell me what the door is."

> **Your call.** Idle CPUs, eighteen green pods, a site answering 503. Before the thread dump, name the exhausted resource, what's holding it, and which -ility is failing. Commit.

## The investigation

Eight years of production Java at Meridian Freight said: find the exhausted resource, because something always is. CPU, memory, and database connections all checked out, so Kenza went looking for the resource nobody graphs.

She picked a pod and took a thread dump.

```
"http-nio-8080-exec-117" #341 daemon prio=5 RUNNABLE
   java.lang.Thread.State: RUNNABLE
        at sun.nio.ch.SocketDispatcher.read0(java.base/Native Method)
        at sun.nio.ch.NioSocketImpl.implRead(java.base)
        ...
        at org.springframework.web.client.DefaultRestClient$DefaultResponseSpec.body
        at com.glasshouse.atrium.enrichment.EnrichmentService.enrich
        at com.glasshouse.atrium.report.ReportSubmissionService.submit
```

She grepped the dump. One hundred and ninety-six threads named `http-nio-8080-exec-*` sat in the same native socket read, all waiting on `api.cortexa.io`. None had crashed, and none was erroring. The JVM reports a thread blocked in a native read as `RUNNABLE`, which is its idea of a joke. Four workers were doing actual work. The rest were parked, each holding open a request the load balancer had long since abandoned.

(If you called the worker pool: right resource, and the right -ility is **availability**. Hold your fix; the napkin math is about to price it.)

She pulled up the code she'd walked past a dozen times without seeing:

```
@Service
public class EnrichmentService {

    // Built ad hoc years ago, on HttpURLConnection. No timeouts configured anywhere.
    private final RestClient cortexa = RestClient.builder()
            .baseUrl("https://api.cortexa.io")
            .requestFactory(new SimpleClientHttpRequestFactory())
            .build();

    public CveEnrichment enrich(SubmittedReport report) {
        // Called synchronously from ReportSubmissionService.submit(...)
        return cortexa.post()
                .uri("/v1/enrich")
                .body(EnrichmentRequest.from(report))
                .retrieve()
                .body(CveEnrichment.class); // parks this Tomcat worker as long as
        Cortexa pleases
    }
}
```

*A read blocks until it returns. That's just the network,* she thought. It was the Meridian reflex, from eight years of raw sockets behind a hand-configured Jetty where a blocking read had never once cost more than its own wait. The model itself held no surprises. In Tomcat's default model, exactly as in the Jetty pool she had sized by hand, the thread *is* the request: one worker carries it from connector to filter chain to controller to this socket read and back. While Cortexa held the socket, that worker could do nothing else, and everything it carried waited with it. She had never set a

timeout at Meridian either. No library had set one on her behalf, and the freight traffic had never made her pay for the omission. This morning Cortexa sent the invoice.

On the bridge, Youssef was explaining why the site hung instead of failing fast. “Tomcat’s NIO connector keeps accepting sockets after the workers are gone, up to `max-connections`, eight thousand-odd by default. They sit with the poller, queued for a worker that’s never coming back. Past that there’s `accept-count`, a backlog of about a hundred. Past *that*, connection refused. And the probes and ingress health checks are HTTP requests too. They queue behind the same dead pool and time out, the pod drops from the Service, and the front door answers 503: no healthy upstream. Eighteen pods, declared dead for telling the truth too slowly.”

Rachid had been silent in the corner of the war room. Now he spoke, mostly to the exec.

“The server didn’t run out of CPU. It ran out of patience tokens. It has two hundred of them per pod, every request borrows one for its whole lifetime, and a vendor in another region is holding a hundred and ninety-six of them hostage, per pod. Kenza. Show them the math.”

## What the senior sees

Kenza took a napkin, because the Wall was three rooms away and the bridge was waiting.

“Little’s law, informally: requests in flight equals arrival rate times how long each one takes. Each pod is taking about seven submissions a second right now. That’s the promo working. Normally a submission takes three hundred milliseconds, so seven times zero point three is about two threads busy per pod. Trivial.

“Cortexa goes to thirty seconds. Same arrivals, hundred-times duration: seven times thirty is two hundred and ten concurrent, and the pool is two hundred. That’s `server.tomcat.threads.max`, the default, untouched since this repo was created. The submission endpoint alone now demands more than the whole pool, so it absorbs all of it, and login and the leaderboard starve behind it. One slow downstream doesn’t slow down one feature. In thread-per-request, it confiscates the threads every feature shares.”

“And the pods?” the exec asked. “Eighteen is triple the capacity.”

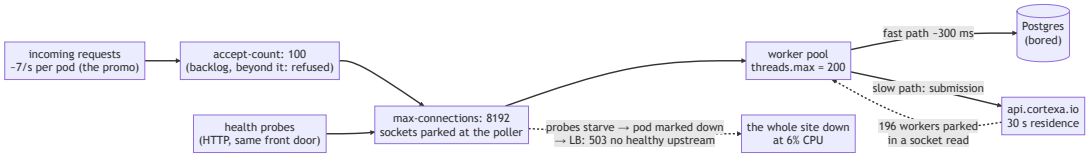
“It would be, if arrival rate were a constant, but it’s a crowd. Every user staring at a spinner hits refresh, the load balancer re-sends what timed out, and demand climbs to meet whatever capacity we add. And every new pod opens hundreds more concurrent calls into a vendor that’s already drowning. We’re making Cortexa slower by scaling at it.”

One mercy was January’s: the eighteen pods had at least come up *interchangeable*. The whiteboard-bolted-to-the-kitchen-wall bug, the singleton stashing a draft on a field every thread shared, was the kind of state that breaks scale-out, because identical pods stop being identical. Statelessness had bought the right to scale out, and the pool was the next bottleneck up.

She drew the connector where it sat on the Request Journey, *left* of everything they’d traced on the Wall in March, when the team had walked one HTTP request from socket to database and back. Socket → connector → filter chain → `DispatcherServlet` → onward. Nothing on March’s

line was failing: the filters were idle and the `DispatcherServlet` barely entered, because a request only reaches them on a worker thread, and no workers were free. The outage lived in the Journey's *first box*, drawn as a single word and never opened. "Everything we traced last month happens *on* a worker thread. The connector is the toll booth where the thread gets assigned, and we've never once looked at it, because Boot embedded it and it worked."

She sketched the rest on a second napkin, transcribed that evening onto the Wall, under the Journey's far-left box:




Under it, the law in one line: *in-flight = arrival rate × residence time*.

"Run it backwards," Rachid said from the doorway.

Kenza flipped the napkin. "Pool fixed at two hundred, residence thirty seconds: two hundred divided by thirty. Six point six requests a second, per pod, for *everything*."

"That's the postmortem's first line. As of this morning, availability is division, and division forces a design question: how much of the pool may one dependency hold? Today, all of it. A bulkhead, a small pool per dependency, makes that a number somebody chose. Size it stingy and it rejects honest traffic, so the number is a negotiation. Blast radius is a design property, and we never designed ours."

At Meridian she had built the server inside the app by hand, an embedded Jetty wired line by line in `main()`, so Boot's fat jar (`java -jar` and the server comes up with the code) was the one part of Spring she had trusted from day one. Atrium's `pom.xml` even had a `<packaging>war</packaging>` profile from the years when it deployed into a standalone Tomcat. That was the old order, where the server owned the app instead of the app carrying its server. The profile had been dead since the move to Kubernetes, kept around like an appendix.

Someone on the bridge asked the perennial question: "Is Tomcat the problem? I've read Jetty is faster."

Rachid looked at Kenza. She shook her head. "Swapping is one starter exclusion: drop `spring-boot-starter-tomcat`, add `spring-boot-starter-jetty`, and Boot's `ServletWebServerFactory` auto-configuration wires whichever container is on the classpath. Undertow used to be the third door. It's off the menu as of Boot 4.0; it never caught up to the Servlet baseline. And it would change nothing, because the servlet model is the model: one thread per in-flight request, bounded pool. We'd be renaming the pool. The problem is that we let a thirty-second stranger borrow our threads with no return date."

"There *is* a structural answer," Rachid added quietly. "The JVM grew threads that stop being precious, cheap enough that blocking one is nearly free. That conversation is worth having." He

checked his watch. “Not at eleven twenty, mid-incident, in front of finance.” He wrote `virtual threads, later` in the napkin’s corner; that corner would wait until November.

## Two seconds or thirty

Mitigation came first: Youssef flipped Cortexa enrichment off behind a config property, the parked threads drained as their socket reads finally returned, and the site came back in ninety seconds. The exec left. Then came the argument about what *normal* should be.

Kenza proposed timeouts: one second to connect, two seconds to read, on every outbound call Atrium made, starting with Cortexa.

Leila pushed back hard, and she wasn’t wrong to. “Researchers *wait* for those scores. Enrichment is half of why submission feels smart. So if Cortexa has a slow afternoon, we just… fail it?”

“Today Cortexa had a slow morning and we failed *everything*: login, leaderboard, your launch. The trade-off isn’t enrichment-works versus enrichment-fails; it’s one feature degrading honestly versus the whole site degrading dishonestly. A timeout is us deciding in advance how much of our capacity a vendor may hold.”

“Then don’t make the researcher wait at all,” Leila said, and that, everyone agreed afterward, was the actual fix. If enrichment could fail without failing the submission, it shouldn’t be *inside* the submission. Accept the report, commit it, return 201, and enrich afterward; the score appears when it appears.

Leila lost the argument she’d started and won the one that mattered. Kenza was learning that this is what a good trade-off discussion looks like.

## The fix

The timeouts went in as policy. As of Boot 4.0, the `spring.http.clients.*` properties set connect and read timeouts for every client built through Boot’s auto-configured `RestClient.Builder`. That gave Atrium a global floor, committed to `application.yml` before any named client got its own numbers:

```
spring:
  http:
    clients:
      connect-timeout: 2s   # the global floor: a forgotten client may be slow,
      read-timeout: 5s      # never unbounded; named clients override stricter
```

The Cortexa client then chose stricter numbers, with a name attached:

```
@Configuration
class CortexaClientConfig {

    @Bean
    RestClient cortexaClient(RestClient.Builder builder) {
        var settings = HttpClientSettings.defaults()
                .withConnectTimeout(Duration.ofSeconds(1))
                .withReadTimeout(Duration.ofSeconds(2));
        return builder
                .baseUrl("https://api.cortexa.io")

        .requestFactory(ClientHttpRequestFactoryBuilder.detect().build(settings))
                .build();
    }
}
```

The numbers had a derivation, because Rachid asked for one. “Where does two seconds come from?”

“The submission route promises four seconds at p99,” Kenza said. “Our own work spends about half a second: Postgres, serialization, the filter chain. Connecting to Cortexa may spend one more. What’s left is what guests may borrow. Two seconds is Cortexa’s share of a budget, not a mood.”

“Then the budget goes next to the route,” Rachid said. “Budgets nobody can see get spent twice. And when Leila wants the timeout raised, and she will, that’s a product negotiation over who gets the missing second.” The Request Journey, it turned out, was also a ledger, with every hop spending from the same four seconds.

By evening the ledger entry sat where reviewers would trip over it, in two comment lines above the mapping. ADR-006 would make that convention house style:

```
// Route budget (ADR-006): POST /api/v2/reports promises p99 4s
//   = ours 0.5s + connect 1s + Cortexa 2s + headroom 0.5s; raising a share is a
        product call
@PostMapping(version = "2")
ResponseEntity<ReportCreatedResponse> submit(
        @Valid @RequestBody SubmitReportRequest request) { /* ... */ }
```

Later that week the enrichment call moved out of the request path, and its three seconds came off the budget comment above the route. `submit()` now persisted the report and recorded enrichment as pending. A background worker on a small, bounded, *named* executor called Cortexa afterward and filled in the score. A two-second timeout on a deferred call costs a retry, where the missing timeout on the inline call had cost a launch.

Then came the connector itself, configured deliberately for the first time in seven years. The numbers were mostly the same, but now they were decisions instead of dust:

```
server:
  shutdown: graceful           # the default since Boot 3.4, pinned explicitly
  http2:
    enabled: true              # multiplexes streams; NOT more workers
  tomcat:
    threads:
      max: 200                 # the pool: hard ceiling on concurrent requests
      min-spare: 10
    max-connections: 8192      # sockets the NIO poller will hold open
    accept-count: 100          # backlog once max-connections is full
    connection-timeout: 5s     # slow/idle clients can't squat on a connection
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s   # the drain window: how long "finish" gets
```

Graceful shutdown hadn't needed enabling (it has been Boot's default since 3.4), yet the mid-incident rolling restarts had still killed in-flight requests, an error spike on top of the outage. Graceful only waits out the drain window before abandoning what's left. Threads parked thirty seconds on Cortexa outlived it. What was new was that window, `timeout-per-shutdown-phase`, now a number somebody owned. SIGTERM still meant *stop accepting, finish what you hold, then die*. Youssef scheduled a drill against the cluster's `preStop` hooks, because he does not believe YAML until he has watched it. HTTP/2 got a one-line caveat in the commit message: *multiplexing reduces connection pressure, not thread pressure; every request still borrows a worker.*

"Deploys are availability events," Youssef said, taping the drill date to the Wall. "One real outage a year, forty deploys a quarter. Get the drain window wrong and the deploys *are* the outage, sliced thin." The runbook gained one line in Kenza's hand: *the drain window must outlive your slowest in-flight request, or graceful is a longer word for kill.* Youssef added the other bound under it: the window also has to fit inside the pod's `terminationGracePeriodSeconds`, which defaults to 30 seconds and starts counting before the `preStop` hook runs, so a 30-second drain needs a longer grace period in the pod spec.

The deploys got faster too. Every image build had been shipping the 200-megabyte fat jar as one Docker layer. The Dockerfile now ran `java -Djarmode=tools -jar atrium.jar extract --layers --launcher`, splitting dependencies, loader, and application code into separate layers. A one-line change now shipped kilobytes instead of the world, and mid-incident that difference is minutes.

Kenza wrote the decision note that afternoon, three sentences long: *Every outbound call gets a timeout. Every timeout gets a named owner who chose the number. Someday this call gets a circuit breaker: a timeout decides how long one request waits; something still needs to decide when to stop sending requests at all.* That last sentence was a promissory note that came due in September. By the time it reached the binder, the note had grown a number:

> **ADR-006: The latency budget. Context:** One slow vendor consumed every worker thread in a bounded pool; the whole site 503'd with CPUs idle. **Decision:** Every outbound call carries an explicit timeout derived from a per-endpoint latency budget; budgets sum along the Request Journey and are recorded next to the route; no unbounded waits anywhere, with `spring.http.clients.*` as the global floor. **Consequences:** Slow dependencies now fail fast and partially instead of globally, and degraded modes become explicit product negotiations. This is the Leila conversation, institutionalized: every new dependency starts with an argument about its share of the budget.

### ⚖ *Your move, architect*

Rewind to 11:20, with the site down, finance in the room, and the mitigation flag not yet found. The capacity question had three honest answers. You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Raise `threads.max` and keep adding pods.** No code change, and it is visible action: the exec is already nodding.
- **B. Queue-and-async the vendor call.** Enrichment never belonged in the request path; accept, commit, return 201, enrich later.
- **C. Timeouts plus a per-dependency bulkhead, now.** Cap how long any request may wait on Cortexa and how many may wait at once; config and one small bean.

**The trade-offs, since you've committed: A** spends **cost** to buy minutes, not availability. Eight hundred threads at 30 s residence are just eight hundred parked threads; retries raise the arrival rate to meet whatever capacity you add; and every new pod ships its own Hikari pool, opening fresh connections against a Postgres that had no part in this outage (that bill arrives in Chapter 9). **B** is the right shape for the long term: Chapter 13 builds it properly, with a bounded queue, retries, and idempotency, and Chapter 14 makes it durable. Mid-incident, though, it is a redesign wearing a fix's clothes, and it spends **operability** exactly when the team has none to spare. The deferred worker shipped that week was a first step toward B, affordable once C had made failure cheap. **C** is what the team chose: it buys **availability** by capping blast radius *at the slow dependency*, so Cortexa's worst afternoon now costs an enrichment outage while the rest of the site stays up. The price is hard conversations with Leila about what "degraded" shows users, and those conversations are a feature. C is a bridge that buys the quarter: the structural answer to the economics of blocking a thread arrives in November as ADR-015's virtual threads.

A backlog ticket appeared, titled "Real application metrics: we graphed the LB because we had nothing else," filed with no sprint attached, where it began to age. And Kenza walked over to Youssef's desk and put her name on the on-call rotation.

"You sure?" he said. "The brick doesn't care what you were doing at three a.m."

“I stood in a war room today and the only person with evidence was me. I’d rather be the person with evidence.”

Youssef considered the thread dump still open on her laptop. It was the first time, he told Rachid later, that a developer had brought him evidence instead of vibes. He slid the rotation spreadsheet across.

## The debrief: Ch.6, The Pool

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 6.** How does the thread-per-request model work, and how does one slow downstream take a whole Spring Boot service offline?

Then the follow-ups, cold:

1. Would swapping Tomcat for Jetty, or adding pods, help?
2. What do `threads.max`, `max-connections`, and `accept-count` each bound?
3. How do you pick a timeout that isn’t a guess?
4. Where do you set the connect timeout and the read timeout?
5. What happens on SIGTERM under graceful shutdown?

Tick what you said:

- ☐ Claim: Thread-per-request: each request occupies one thread of a bounded worker pool (200 by default), so the pool is the concurrency ceiling.
- ☐ Mechanism: The connector hands each request to a worker, a blocking call parks it, and Little’s law prices it: arrival rate × duration.
- ☐ Trade-off: Simple and debuggable, with real stack traces, but request duration is partly other people’s systems: capacity is hostage to your slowest dependency.
- ☐ Failure mode: One slow downstream consumes the entire pool, health probes starve, pods get marked down, and the whole site returns 503 while CPU idles.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Boot's default embedded Tomcat is thread-per-request: a bounded worker pool (200 by default) where each request occupies one thread for its entire lifetime, so the pool, not CPU, is the concurrency ceiling. **Mechanism:** The NIO connector accepts sockets (up to `max-connections`, with an `accept-count` backlog behind that) and hands each ready request to a worker. Every blocking operation in the handler, such as JDBC or a synchronous `RestClient` call, parks that worker until it returns, and Little's law prices it: threads needed ≈ arrival rate × request duration. **Trade-off:** The model is simple and debuggable, with real stack traces and no async contagion, and the bounded pool is a crude built-in load shed. But request duration is partly controlled by *other people's systems*, so your capacity is hostage to your slowest dependency. **Failure mode:** One downstream going from 300 ms to 30 s multiplies one endpoint's thread demand a hundredfold and consumes the entire pool. Health probes starve in the same queue, pods get marked down, and the whole site 503s while CPU idles. Adding pods just parks more threads against the same dependency. The first defense is a timeout on every outbound call; the structural fixes are circuit breakers or virtual threads.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Availability**, revealed as arithmetic: a bounded pool divided by residence time *is* the throughput ceiling, and residence belongs to your slowest dependency until a timeout takes it back. Blast radius sits on top: bulkheads exist to keep one dependency's failure down to a feature outage. **Operability** rides along. The drain window is availability at the *end* of the lifecycle, because deploys are the most frequent availability events a system has.
- **The ADR:** ADR-006, the latency budget: every outbound call carries a timeout derived from a per-endpoint budget; budgets sum along the Request Journey and live next to the route; no unbounded waits anywhere.
- **Rejected alternatives:** scale-out under fire (multiplies parked threads, vendor pressure, and DB connections; that bill arrives in Chapter 9); swapping Tomcat for Jetty (renames the pool, keeps the model); full queue-and-async mid-incident (the right shape at the wrong hour; Chapter 13 builds it properly).
- **Fight or lean:** *Lean* on Boot's connector defaults and the `spring.http.clients.*` global floor: the numbers are sane, and the floor catches the client somebody forgot. *Fight* the silence around defaults: nobody had read `threads.max: 200`, so nobody had decided on it, and it sat as dust for seven years until a vendor priced it.

**Story anchors**

- 196 of 200 threads parked in the same socket read on `cortexa.io` → thread-per-request exhaustion: workers are the scarce resource, and the CPU sits idle.

- "We just bought more chairs in front of the same locked door" → horizontal scaling can't outrun a slow dependency: new pods' threads park too, and retries raise the arrival rate.
- The napkin: 7/s × 30 s = 210 > 200 → Little's law: concurrency demanded = arrival rate × duration, and the pool is the ceiling.
- The flipped napkin: 200 threads ÷ 30 s ≈ 6.6 req/s → availability run backwards: pool divided by residence time is the pod's throughput ceiling, whoever owns the residence.

**Interview ammo**

- *Kill-shot: Explain how a single slow third-party API can 503 your entire Spring Boot service while CPU sits idle.* Each request holds one of ~200 Tomcat workers for its full duration. When a downstream stretches duration to 30 s, arrival rate × duration exceeds the pool and every worker parks in a socket read. New connections queue at the poller until health checks fail and the load balancer drops the pods. That is thread exhaustion, with the CPU idle.
- *Follow-up: Would swapping Tomcat for Jetty, or adding pods, help?* No. The swap (exclude `spring-boot-starter-tomcat`, add the Jetty starter; Boot wires whichever `ServletWebServerFactory` is on the classpath; Undertow is gone as of Boot 4.0) changes the container and keeps the blocking servlet thread model. More pods just multiply parked threads against the struggling vendor. The fixes are outbound timeouts, deferring the call, circuit breakers, or virtual threads.
- *Follow-up: What do `threads.max`, `max-connections`, and `accept-count` each bound?* `threads.max` caps concurrent in-flight request processing. `max-connections` caps sockets the NIO poller holds open (cheap while idle: keep-alive connections don't hold workers). `accept-count` is the listen backlog once connections are full; beyond it, refusals.
- *Architect follow-up: How do you pick a timeout that isn't a guess?* Derive it from a latency budget: the route promises a p99, subtract what your own hops spend, and the remainder is the most any dependency may borrow. Record it next to the route, give it a named owner, and renegotiate it as a product decision when someone wants it raised.
- *Follow-up: Where do you set the connect timeout and the read timeout?* In two places. `spring.http.clients.connect-timeout` and `spring.http.clients.read-timeout` are the global floor for every client built from Boot's auto-configured `RestClient.Builder` (2 s and 5 s at Atrium). A named client overrides them with `HttpClientSettings.defaults().withConnectTimeout(...).withReadTimeout(...)`, handed to its request factory: the Cortexa client gets 1 s to connect and 2 s to read.
- *Follow-up: What happens on SIGTERM under graceful shutdown?* The server stops accepting new requests, finishes the ones it holds, then exits. `server.shutdown: graceful` has been Boot's default since 3.4. The wait is capped by `spring.lifecycle.timeout-per-`

`shutdown-phase`, the drain window (30 s here): whatever is still running when it ends is abandoned, so the window must outlive the slowest in-flight request and fit inside the pod's `terminationGracePeriodSeconds`.

**Pointers** → `examples/ch06_server/ThreadPoolSaturationTrap.java` · **run it:** `labs/lab-05-timeouts-bulkheads` (a missing read timeout parks the caller's thread; a timeout fails fast)

---

# Part IV: The Data

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Persistence context:** the map of entities Hibernate watches during a transaction. It caches by identity, detects your changes by diffing snapshots (*dirty checking*), and writes them at commit. **Lazy loading:** a fetch strategy. An association is loaded on first access rather than up front. **N+1:** a query pattern. One query loads a list, then one more runs per element as each is touched. **Transaction:** the all-or-nothing unit of work, opened and committed by a proxy around an annotated method. **Propagation:** how nested calls share (or suspend) it. **Isolation:** what concurrent transactions are allowed to see of each other. **Optimistic vs pessimistic locking:** optimistic locking detects a conflicting write at commit via a version column, and you retry or surface it. Pessimistic locking blocks other writers with a database lock while you hold the row. **Repository:** an interface Spring Data implements at runtime; method names and annotations define the queries. **Projection:** fetching exactly the columns a view needs into a record, instead of loading whole entities. **Migration:** versioned, append-only SQL scripts (Flyway) that evolve the schema. **Connection pool:** the small fixed set of database connections (Hikari) every request must borrow from.

# Chapter 7: The Thousand Queries

The brick changed hands on the first Monday of May. Youssef held it out in the kitchen. It was an ancient pager-shaped phone case, dented at one corner, and it came with the ritual's outgoing wisdom: "Nothing here is slow until somebody important is watching."

Kenza took the brick. It was heavier than it looked, like everything else here.

Somebody important was watching on Tuesday. Northbeam Bank was the marquee renewal, and Leila ran the demo for its CISO: a walkthrough of the program dashboard the deck called *real-time triage visibility*. Kenza watched from the back as the page began, gently, to spin.

It spun for twenty-eight seconds. Kenza counted. "While that loads," Leila vamped, "the severity model behind these numbers is, ah… it's… thorough." The CISO said nothing, which was worse. The page eventually arrived, complete, correct, and pointless.

Afterward Leila found Kenza by the kitchen, voice flat. "I can sell slow once. I can't sell slow twice. Tell me what it is by Thursday."

Traffic was the villain everyone wanted: promo volumes were the new normal, and Northbeam's program held four times January's reports. But the page had always been this page; something had been waiting in it.

> **Your call.** The page is correct and complete. It is also twenty-eight seconds late, and dev has never felt it. Before Kenza flips a flag: what lives inside those twenty-eight seconds, why was it invisible for years, and which -ility was never *designed*, only assumed? Commit.

## The investigation

Kenza restored a prod-sized snapshot into dev, flipped on the things nobody flipped on (`spring.jpa.show-sql=true` and `spring.jpa.properties.hibernate.generate_statistics=true`), and loaded the dashboard once.

Her terminal scrolled for eleven seconds.

It was the same `SELECT`, hundreds of times, differing only in the bound ID. At the bottom, Hibernate's session metrics showed 401 JDBC statements executed for one page load. One query fetched the page of two hundred reports (the old repository method returned a plain `List`, so no count query ran beside it). Then came one per report for its researcher and one more per report for the activity rollup: one plus two-N. Each round trip cost seventy-odd milliseconds, and they ran one after another. That was the twenty-eight seconds: hundreds of individually flawless queries.

The shape lived in a service nobody had touched in years:

```
@Entity
public class Report {
    @Id @GeneratedValue private Long id;
    private String title;

    @ManyToOne                          // FetchType.EAGER, the JPA default, crept back
        when a March
                                        // cleanup dropped January's explicit fetch =
        LAZY. Nobody chose it.
    private Researcher researcher;

    @OneToMany(mappedBy = "report",  // inverse side: a mirror, not a column
               cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Activity> activities = new ArrayList<>();
}

// ProgramDashboardService: archaeology, untouched since the dashboard shipped
public List<ReportRow> dashboard(Long programId, Pageable page) {
    return reports.findByProgramId(programId, page).stream()
        .map(r -> new ReportRow(
            r.getTitle(),
            r.getResearcher().getHandle(),   // EAGER: already cost one SELECT per row
            r.getActivities().size(),        // LAZY: costs another, right now
            latest(r.getActivities())))
        .toList();
}
```

She brought the scroll to Rachid.

"Is this a Hibernate problem or a JPA problem?" she asked.

"JPA is the contract. Hibernate is the contractor." He tapped the `@ManyToOne`. "The eager default on to-one associations is in the contract. How the contractor honors it is his business, and after a JPQL list query, his business is one follow-up `SELECT` per row he hasn't already met. Eager doesn't mean *joined*, Kenza. It means *non-negotiable*."

That was what had ambushed her. "At Meridian, a row was the columns I'd asked for," she said, because it deserved air. "If I wanted the researcher, I wrote the `JOIN` myself. Forget it, and I'd have hit the null in dev on day one. Absence by default. Hibernate's default is presence: non-negotiable, one `SELECT` per row, each one too small for dev to feel."

"Both defaults are opinions," Rachid said. "A senior knows whose opinion they're running."

The collection was the other half. `getActivities()` was lazy, correctly lazy, but the mapping loop touched it inside the request, two hundred times, one `SELECT` each. And none of this had ever hurt in dev, because something in the web tier had been keeping the session open, so each lazy touch simply ran one more query. *Raw JDBC had no lazy anything. What came back in the `ResultSet` was all there was, and an object could never reach back to the database after the fact. Here, reach-*

*ing back was routine.* She had never felt the draft. On her hand she wrote `open-in-view`. She would not need to ask.

## 2:07 a.m.

The brick went off at 2:07 on Friday morning, four days into her first rotation. Atrium had no metric worth the name. What paged her was a dead-man's check, which fired if the digest hadn't left the building by 2:05. Once her heart had stopped doing pager-math, the stack trace was almost beautiful in its specificity:

```
@Scheduled(cron = "0 0 2 * * *")
public void sendDisclosureDigest() {
    List<Report> reports = reportRepository.findDisclosedSince(cutoff());
    // ← the transaction ended here; the repository method was the boundary
    for (Report r : reports) {
        mailer.send(digestTemplate.render(r));  // template walks r.getActivities()
    }   // LazyInitializationException: Cannot lazily initialize collection of role
        // 'com.glasshouse.atrium.report.Report.activities' with key '5390' (no
        session)
}
```

A teammate had shipped the digest job in March, copying the house's oldest style: load entities, hand them to a renderer, let it walk whatever it wants. It ran clean for weeks, because the template printed only titles and severities, fields already loaded. On Wednesday someone added an activity timeline. The repository call had opened and closed its own transaction, so the entities came back *detached*, and at 2 a.m. the new section reached for a lazy collection and found no session behind it. There was no web request, so there was no open-session-in-view safety net. Nothing on the scheduled thread held the session open, and the draft finally came through.

The digest could be late. She disabled the schedule and went back to bed without dignity.

At the morning postmortem, Rachid slid a photocopied page across the table: a printed diff, green ink in the margin, dated January 16. He asked her to read comment #1 into the record.

"Works is not done," Kenza read. "What happens when this entity meets Jackson after the session closes?"

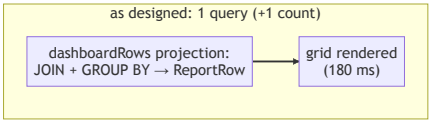
Below it sat the circle she had half-dismissed as pedantry in January: *This pattern pages someone at 2 a.m. Not necessarily you. Not necessarily here. R., Jan 16.* The mechanism was exactly as the note described: an entity outlives its transaction and comes back detached, and whatever walks its lazy getters, Jackson then and a mail template now, finds no session left to load with. It was a prophecy with a postmark and a four-month fuse.

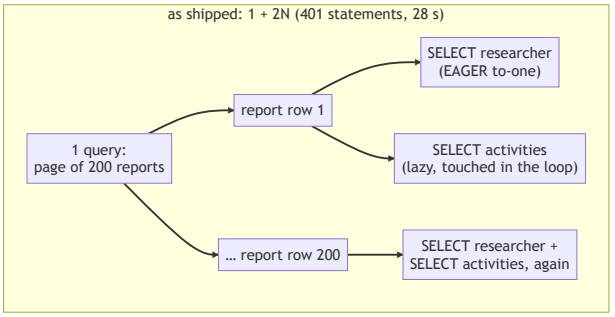
The diff was from her first review, and the code that died at 2 a.m. wasn't even hers. Her version had been rewritten in January, but the *style* had survived, copied forward by someone who'd never seen the green ink. The review had fixed a file. Only a mechanism, taught, would have fixed the team.

## What the senior sees

That afternoon the Wall gained a new panel, the Canary presiding from the marker tray. House rules: explain it out loud, in full sentences, before you escalate.

Kenza opened the panel herself and chalked it for the Canary: the shape as shipped and, beside it, a fetch plan finally written down.




Rachid drew a box and labeled it *persistence context*. "Tell the Canary what this is. Use your old words first."

*So the persistence context is the per-request `Map<Long, Report>` she'd hand-rolled at Meridian to spare the database repeat lookups. Except hers only remembered objects, and this one watches them. Hers lived as long as she chose, and this one lives exactly as long as a session.* She said it nearly that way, and Rachid wrote the line onto the panel, where it stays to this day: *an identity map that watches what it holds, and it lives in a session, and the session closes.*

Then the states, in a column. Transient: `new`, unknown to the context. Managed: loaded or persisted inside it, held in the identity map, and watched. Detached: the session closed, the object lives on, the watching stopped. Removed: scheduled for deletion at flush. The identity map was the first-level cache: `find()` the same ID twice in one session and you get one `SELECT` and the *same instance*, both times.

"And managed means watched," Rachid said, gesturing at the codebase. "Which is why nobody here ever calls a save method to update anything. Why does that work?"

"Dirty checking. The context snapshots the entity at load. At flush it diffs every managed entity against its snapshot and writes `UPDATE` for the differences." *At Meridian an `UPDATE` existed because she had typed it; nothing ever wrote behind her back. Here the commit* is *the `UPDATE`.* "And flush happens at commit, or earlier, before a JPQL query that might need to see my pending changes."

"Good." He tapped the `mappedBy`. "Two sides of this relationship. Which one does Hibernate believe?"

"The owning side. `Activity.report` holds the foreign key, and that column is the truth. `mappedBy` marks the `@OneToMany` as the inverse side: a mirror, not a column." While they were there, two more facts went onto the panel. `cascade = ALL` rippled persist and remove down to

activities. `orphanRemoval = true` turned removal from the collection into a `DELETE`: correct on a true composition like an activity log, catastrophic if ever copy-pasted onto `researcher`.

Last, the question she'd written on her hand. "Why was the dashboard's laziness invisible for years, and the digest job's fatal in one night?"

"Because the web tier has been anesthetized. Open-session-in-view holds the session open through rendering, so a controller, or Jackson, touching a lazy field just silently runs another query. Your scheduled job had no anesthetic. Three fixes for the exception, with a trade-off each. Tell the Canary, in order."

She faced the bird. DTO projection: select exactly what the view needs, inside the transaction. It is the best fix, because *what this screen loads* gets answered once, in the query, where it's visible. Per-query fetching, with `JOIN FETCH` or `@EntityGraph`, when you genuinely need the entity graph: precise, but you ship whole entities to a renderer that wanted four fields. And OSIV: zero code, global, and it hides the missing fetch plan instead of fixing it. The hidden N+1 ships to production, and every request that touched the database keeps its pooled connection until rendering ends, lazy load or not: *a connection held hostage through rendering*.

"One more," Rachid said. "Why not just make everything eager and be done?"

"Because you can make a lazy mapping eager for one query. You can't make an eager mapping lazy for one query. One of those is a decision. The other is a sentence."

## The fix

The rebuild took the rest of the sprint. The principle was the panel's last line: *choose the fetch per view, in the query, where it's visible.*

```
public interface ReportRepository extends JpaRepository<Report, Long> {

    // List view: declarative, reusable fetch plan; survives pagination
    @EntityGraph(attributePaths = "researcher")
    Page<Report> findByProgramId(Long programId, Pageable page);

    // Detail view: precise and query-local; one screen, one fetch plan
    @Query("select r from Report r join fetch r.researcher " +
           "left join fetch r.activities where r.id = :id")
    Optional<Report> findDetail(@Param("id") Long id);

    // Dashboard grid and digest job: fetch what you serialize, nothing else
    @Query(value = """
        select new com.glasshouse.atrium.report.ReportRow(
            r.title, res.handle, count(a), max(a.createdAt))
        from Report r join r.researcher res left join r.activities a
        where r.program.id = :programId
        group by r.id, r.title, res.handle
        """,
        countQuery = "select count(r) from Report r where r.program.id = :programId")
    Page<ReportRow> dashboardRows(@Param("programId") Long programId, Pageable page);
}
```

Each tool earned its place by trade-off. `JOIN FETCH` went to the detail view, where one report's full graph was the point. The review note named the risks. Join-fetch two `List` collections in one query and Hibernate refuses outright with a multiple-bag error. Join-fetch a collection under a `Pageable` and Hibernate pages *in memory*, logging a warning while it loads the world. `@EntityGraph` took the list view: declarative, reusable, safe alongside real pagination on a to-one path. DTO projection went to the grid and the rebuilt digest job: a constructor expression straight into a record, now selecting four columns instead of entity graphs. It carried an explicit count query, because a derived count over a `GROUP BY` counts the wrong thing. And `@ManyToOne` got `fetch = LAZY` across the codebase, with the line that anchored the day's ADR: *lazy everywhere; loading is the query's job.*

Then came the debate. Kenza proposed `spring.jpa.open-in-view=false`, which would make every latent lazy touch in every untested path start throwing instead of quietly querying. Boot had been flagging the default at every startup since 2019 (`spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering`), a warning nobody had read. Youssef settled it with pool arithmetic: under OSIV, a request that touched the database keeps its Hikari connection after the service method has returned, until rendering ends, lazy touch or not. "We almost died in April because two hundred threads held things they didn't need. Now you want requests holding database connections to write JSON? Turn it off."

It was April's napkin, aimed one layer down: in-flight equals arrival rate times residence, and OSIV adds rendering and serialization to a *connection's* residence. April had priced this pattern for

request threads; here it was again, against their own database. “Open-in-view is the framework choosing your session boundary for you,” Rachid said. “It chose in 2019, and nobody was in the room. Take the boundary back.”

```
spring:
  jpa:
    open-in-view: false              # ADR-007: fail loudly in the service, not slowly
        in the pool
    properties:
      hibernate:
        default_batch_fetch_size: 32 # safety net: a missed N+1 becomes N/32 IN-
        queries, not a fix
```

The trade-off went into the ADR in plain language: *we accept loud `LazyInitializationException`s in staging as the price of never again shipping a silent N+1 that bills the connection pool through rendering*. The batch size was a smoke alarm, not a sprinkler: a global mitigation that collapses a missed N+1 into ceiling-of-N-over-32 `IN`-list queries, which is better and never good.

The PR carried one more thing, because Rachid asked the question that turns a fix into a property: *“It’s one query today. What keeps it one in November?”* Nothing did, so the hot paths got a guard, and January’s ADR-002 move (the constitution as a test suite: each architectural rule written as a test that fails the build when violated) reached the data layer:

```
@DataJpaTest(properties = "spring.jpa.properties.hibernate.generate_statistics=true")
class DashboardFetchPlanTest {

    @Autowired EntityManagerFactory emf;
    @Autowired ReportRepository reports;
    Statistics statistics;

    @BeforeEach
    void unwrap() { statistics = emf.unwrap(SessionFactory.class).getStatistics(); }

    @Test
    void dashboardIsExactlyTwoStatements() {     // rows + count: the fetch plan,
        asserted
        statistics.clear();
        reports.dashboardRows(NORTHBEAM, PageRequest.of(0, 200));   // the seed fills
        this page
        assertThat(statistics.getPrepareStatementCount()).isEqualTo(2);
    }
}
```

The slice flipped on the same `generate_statistics` flag her investigation had used, scoped to the test and never riding to production. It unwrapped past the JPA interface to Hibernate’s native `Statistics` and ran against a prod-shaped seed. The seed has to fill the page of two hundred: on

a short page Spring Data already knows the total, skips the count query, and the test would see one statement. Drift the query back toward entities, or resurrect an eager default, and two becomes two-hundred-and-something: CI red, regression named. "Performance as a designed property," Rachid said. "A fetch plan you haven't asserted is a fetch plan you're renting."

He tapped `ReportRow` on his way out. "Notice what you built. The entity graph is your write model: cascades, orphan removal, dirty checking, shaped for *changing* things. `ReportRow` is a read model: flat, purpose-shaped, owing the entities nothing. Two models, starting to separate; the seam is load-bearing." She didn't know yet the seam had a name.

Staging flipped that week, production waited out a quarter of archaeology, and the decision went into the binder under its real name:

> **ADR-007: The fetch policy. Context:** The dashboard ran 401 individually flawless statements per page; a 2 a.m. digest job died on `LazyInitializationException`; open-session-in-view had anesthetized the data boundary since 2019. **Decision:** All associations lazy; every read path declares its fetch plan (DTO projection by default, `JOIN FETCH`/`@EntityGraph` where the view needs entities); OSIV off behind a migration window, not a flag-flip; hot paths assert SQL statement counts in CI. **Consequences:** The data boundary is explicit: January's circled pattern can no longer reach serialization, because entities no longer reach it. The cost is a projection type per view and a quarter of archaeology turning hidden N+1s into visible fetch plans.

### ⚖ *Your move, architect*

Rewind to the table where Youssef said "turn it off." One boolean stood behind seven years of anesthesia, and there were three honest positions on it. You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Leave OSIV on.** Every tutorial's default, and Atrium's since 2019; the dashboards keep working tomorrow, and there's a renewal to support.
- **B. Off everywhere, today.** Honest: every latent lazy touch in every untested path throws at once, and you fix what screams.
- **C. Off behind a deprecation window.** Dev and staging flip now, and the exceptions become an inventory. Fixes land per feature seam, and production flips when staging goes quiet.

**The trade-offs, since you've committed: A** is the only indefensible option for a team that just paid this invoice. The exception stays hidden until the next non-web path reaches through a detached entity, and every request that touched the database keeps holding a pooled connection through rendering, after its transaction has ended: **availability** spent quietly and forever, by April's arithmetic. **B** is honest, but it converts years of unknown N+1s into a week of production

incidents: **availability** spent loudly, for a lesson staging would have taught cheaper. **C** is what the team chose: boring, staged, reversible, with **operability** preserved. Staging screams for free; ADR-001's feature packages give the archaeology a unit of work and a definition of done; production flips when the inventory is empty. The flag is global, but the migration never has to be.

The dashboard came back at 180 milliseconds against the prod-sized snapshot. Leila re-demoed for Northbeam the following week and didn't have to say "thorough" once. By the retro, query-plan questions had started routing themselves to Kenza's desk; the 2 a.m. page had cost her a night and bought her a specialty.

One thing nagged. Profiling triage writes, she'd seen commits landing where she couldn't predict. They were small and separate, scattered through what looked like single operations, with audit rows committing on their own schedule. It didn't smell like a fetch problem. She wrote *triage commit boundaries?* on a sticky note and stuck it to the brick.

## The debrief: Ch.7, The Thousand Queries

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 7.** What causes `LazyInitializationException`, and what are three different fixes with different trade-offs?

Then the follow-ups, cold:

1. If OSIV makes the exception disappear, why turn it off?
2. When does `JOIN FETCH` itself become the bug?
3. How do you stop N+1 regressing once fixed?
4. What are the four entity states, and which one does Hibernate watch?
5. When does Hibernate flush, and what does a flush write?

Tick what you said:

- ☐ Claim: A lazy association was touched after its persistence context closed, and the fix is deciding where data loading ends.
- ☐ Mechanism: The Hibernate proxy or uninitialized collection, the session that dies when the transaction ends, and the detached entity.
- ☐ Trade-off: Three fixes with a cost each: DTO projection, `JOIN FETCH` or `@EntityGraph`, and OSIV keeping the connection until rendering ends.
- ☐ Failure mode: OSIV in production: the exception vanishes, the N+1 ships, and the connection pool drains under load.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** `LazyInitializationException` means a lazy association was touched after its persistence context closed; the fix is deciding where data loading *ends*, not silencing the exception. **Mechanism:** A lazy association is a Hibernate proxy (to-one) or an uninitialized persistent collection. Touching it triggers a `SELECT` through the session that owns it, and that session dies when the transaction ends, unless open-session-in-view is holding it open for a web request. So an entity that escapes the service boundary is *detached*, and when Jackson or a mail template reaches through it, there is no session left to load with. **Trade-off:** Three fixes, each with a cost. DTO projection fetches exactly what the view needs inside the transaction (the best: the fetch plan is visible in the query, at the cost of a type per view). Per-query fetching via `JOIN FETCH` or `@EntityGraph` keeps real entities and loads precisely, but risks cartesian blowup on multiple collections and in-memory pagination. OSIV keeps the session open through rendering: zero code, but every request that touched the database keeps its pooled connection until rendering ends, and every missing fetch plan ships as a hidden N+1. **Failure mode:** OSIV in production: the exception vanishes, the N+1 ships, and under load the connection pool drains while request threads serialize JSON with a database connection in their pocket.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Performance as a designed property** means the fetch plan is part of the read model's contract, declared in the query and asserted in CI, so nobody has to discover it in front of a CISO. **Availability** rides along: under OSIV, every request that touched the database keeps its connection until rendering ends, which is Chapter 6's residence arithmetic aimed at the Hikari pool.
- **The ADR:** ADR-007, the fetch policy: all associations lazy; every read path declares its fetch plan (DTO projection by default, `JOIN FETCH`/`@EntityGraph` where entities are required); OSIV off behind a migration window; hot-path statement counts asserted in CI.
- **Rejected alternatives:** eager-everywhere (a sentence, not a decision: lazy can go eager per query, never the reverse); OSIV left on (hides the missing fetch plan, bills the pool for the cover-up); an immediate production flag-flip (years of unknown N+1s as a week of incidents); `default_batch_fetch_size` as the fix (a smoke alarm, not a sprinkler).
- **Fight or lean:** *Fight* OSIV, on the warrant Chapter 1's lens signed: it is the framework choosing your session boundary, and the architect takes it back. *Lean* on the persistence

context inside it: dirty checking, the identity map, declarative fetch plans and the batch-size net are the machinery that makes the container worth inhabiting.

**Story anchors**

- The 28-second spinner in front of Northbeam's CISO → N+1: one query for the page, then two per report, for 401 perfect statements in the session metrics.
- The green-ink comment, dated in January and read into the May postmortem → `LazyInitializationException`: an entity that meets Jackson after the session closes is detached, and no one is left to load for you.
- "A connection held hostage through rendering" on the Wall → OSIV: the trap dressed as a convenience. It hides the missing fetch plan and bills the connection pool for the cover-up.
- The statement-count test pinned at two → performance as a designed property: a fetch-plan regression fails the build before it reaches a renewal demo.

**Interview ammo**

- *Kill-shot: What causes `LazyInitializationException`, and what are three fixes with different trade-offs?* The persistence context closed before a lazy load; fix with DTO projection inside the transaction (best), per-query fetching via `JOIN FETCH`/`@EntityGraph` (precise, entity-shaped), or OSIV (global anesthetic: trades the exception for held connections and hidden N+1).
- *Follow-up: If OSIV makes the exception disappear, why turn it off?* Because under OSIV every request that touched the database keeps its pooled connection until rendering ends, and every missing fetch plan ships as a silent production N+1. With it off, fetch bugs fail loudly at the boundary instead of slowly in the pool.
- *Follow-up: When does `JOIN FETCH` itself become the bug?* Fetching two `List` collections in one query fails outright (multiple-bag fetch), and join-fetching a collection under `Pageable` forces Hibernate to paginate in memory after loading everything. Use `@EntityGraph` on to-one paths, fetch IDs first, or batch-size the collection.
- *Architect follow-up: How do you stop N+1 regressing once fixed?* Make the fetch plan part of the read model's contract: hot paths pin SQL statement counts in slice tests via Hibernate `Statistics`, so a resurrected eager default or new lazy touch fails CI with the number named. These are ADR-002's fitness functions at the data layer.
- *Follow-up: What are the four entity states, and which one does Hibernate watch?* Transient: created with `new`, unknown to the persistence context. Managed: loaded or persisted inside it, held in the identity map, and watched. Detached: the session closed, the object lives on, the watching stopped. Removed: scheduled for deletion at flush.
- *Follow-up: When does Hibernate flush, and what does a flush write?* At commit, or earlier, before a JPQL query that might need to see the pending changes. At flush the context

compares every managed entity with the snapshot it took at load and writes an `UPDATE` for the differences, which is why an update needs no save call.

**Pointers** → `examples/ch07_jpa/NPlusOneAndLazyInitializationTrap.java`, `EntityLifecycleAndDirtyChecking.java` · **run it:** `labs/lab-02-n-plus-one` (counts the 1+N selects, then one with a fetch plan)

---

# Chapter 8: The Silent Commit

The spreadsheet had three columns and one impossible row.

It was late May. In a conference room with the blinds half down, the finance lead turned her laptop around with the careful neutrality of someone presenting evidence. Report #5817 was an authentication bypass in a customer program: validated, severity Critical, bounty five thousand dollars. The Atrium database said the report was Awarded. The payouts table held a payout row, timestamped and complete. The audit log said `AWARD reportId=5817 amount=5000.00 status=OK`. The fourth column, typed in by hand from the payment provider's portal, said *no such transfer*.

"The database says we paid," the finance lead said, "the bank says we didn't, and the log says *handled*."

Leila had her arms folded. "The researcher has been politely asking about their bounty for nine days. Researchers talk to each other, Kenza. On day ten, politeness becomes a screenshot."

Kenza was already scrolling the audit log, and a cold familiarity was settling in. While chasing the thousand queries earlier that month she'd noticed triage writes committing in odd little bursts, transactions ending in places she couldn't predict. She'd written *triage commit boundaries?* on a sticky note, stuck it to the brick, and buried the question under the N+1 work. The sticky note had just invoiced her for five thousand dollars.

"I've seen the edge of this," she said. "Give me the morning."

From the doorway, where he had apparently been standing a while, Rachid said, "A caught exception is a decision. Most people make it by accident." He left before anyone could ask him to elaborate, which Kenza had learned was his way of assigning homework.

## The investigation

The award path lived in `BountyService.award()`, and the method looked respectable: annotated, tidy, reviewed. The rot was one call down, in a helper that `git blame` dated to 2021 with the commit message *temporary: don't let provider blips fail awards*.

```java
@Service
public class BountyService {

    @Transactional
    public void award(long reportId, BigDecimal amount) {
        Report report = reports.findById(reportId).orElseThrow();
        report.setStatus(ReportStatus.AWARDED);            // write 1: dirty-checked
        payouts.save(new Payout(report, amount));          // write 2
        auditLog.record("AWARD", reportId, amount, "OK");  // write 3: same transaction
        paymentGateway.transferSafely(report.getResearcherId(), amount);
    }
}

// PaymentGatewayHelper.java, "temporary", 2021
void transferSafely(UUID researcherId, BigDecimal amount) {
    try {
        client.initiateTransfer(researcherId, amount);     // throws
        PaymentProviderException (checked)
    } catch (PaymentProviderException e) {
        log.warn("transfer failed: handled, will reconcile later", e);
    }
}
```

> ⏸ **Your call.** Annotated, tidy, reviewed, and the bank is empty. Where exactly does the rollback signal die, and which -ility has already failed by the time the proxy decides?

She reconstructed the night of the award. The provider had a four-minute API outage. `initiateTransfer` threw its checked `PaymentProviderException`; the helper caught it, logged the word *handled*, and returned normally. Control passed back up to `award()`, which also completed normally. The proxy saw a normal return and committed. The status, the payout row, and the audit row confirming success were now all durable. The only thing that hadn't happened was the money.

*Her raw-JDBC instinct said the transaction should have noticed. At Meridian she'd written the rollback herself, a `conn.rollback()` in the very `catch` that saw the failure. The instinct was the trap. Spring's transaction isn't ambient awareness; it's a proxy that inspects how the method exits and watches for exactly one signal, and that signal had been eaten two stack frames down.*

(If you committed at the pause: the signal dies in the 2021 `catch`, because the proxy reads only what crosses the method boundary, and nothing exceptional ever did. The -ility is integrity. Its price list arrives at the Wall.)

It was February's proxy bug in a new form (annotations live on the container's wrapper; a `this.` call dispatches on the raw object behind it). February's self-invoked call never *entered* the proxy, so no transaction wrapped it at all; tonight's exception never *exited* through it. The rollback signal travels out through the same proxy, and a `catch` two frames down meant it never arrived.

The persistence context from the thousand-queries week (the N+1, one lazy load per row) had done its job correctly: dirty checking, a snapshot-diff at commit, decided *what* to flush. Nobody in seven years had decided *whether* it should commit.

She was still writing it up when Malika appeared with the sprint's second gift. That morning, two triagers had been working the same hot report on Northbeam's program. Malika had raised the severity to Critical with a justification note; a colleague, reading it at the same moment, had lowered it to Medium. Both saves succeeded, and both got green toasts. The severity was whatever the second click said, the first click's reasoning was gone, and a researcher whose bounty rode on that number had opened a dispute.

"No error," Malika said. "It just… picked one of us. We'd like to know the rules of the game we're apparently playing against each other."

Two failures in one sprint had the same cause: nobody at Glasshouse had ever decided what a write boundary promised. Kenza grepped the codebase and winced. `@Transactional` sat on controllers, on repository fragments, and on helpers, sprinkled like salt, by feel.

## What the senior sees

At the Wall, Rachid handed her the marker instead of taking it. "Walk me through what the annotation *does*, not what it means."

She drew it under the smoke-cloud proxy from February, which still said *this. goes around the mirror* in her own handwriting, shorthand for a self-call landing on the raw object behind the annotated wrapper. "The call hits the CGLIB proxy. The transaction interceptor asks the `PlatformTransactionManager` to begin: a connection comes out of Hikari, a database transaction opens, and the connection and the `EntityManager` get bound to the current thread (`TransactionSynchronizationManager`, ThreadLocals). That's why every repository call inside the method lands in the same transaction without being told: they look up the thread's bindings."

"Name the propagation that makes that work."

"`REQUIRED`, the default. Join the caller's transaction if one exists, start one if not. It's why three repositories and a helper all wrote into one transaction without knowing it."

"And on the way out?"

"The interceptor looks at *how the method ended*. Normal return: commit. Exception: it consults the rollback rules. `RuntimeException` or `Error`, roll back. Checked exception, *commit anyway*." She stopped, hearing it. "And a caught exception isn't an ending at all. The method returned normally, so the proxy never knew."

"So where did your rollback die?"

"In a `catch` block in 2021. The proxy can only see what crosses the method boundary."

She wrote it on the Request Journey diagram, to the right of the service layer, where it has stayed ever since: *commit/rollback happens on the way out*.

"Why is the default unchecked-only?" she asked. "That's the part that feels like a trap."

"History, not law," Rachid said. "EJB decided last century that checked exceptions were *business conditions* the caller would handle, like a rejected order or a missing file, and that unchecked ones were system failures. Spring inherited the convention. You may overrule it per method: `rollbackFor = Exception.class`. What you may not do is fail to choose."

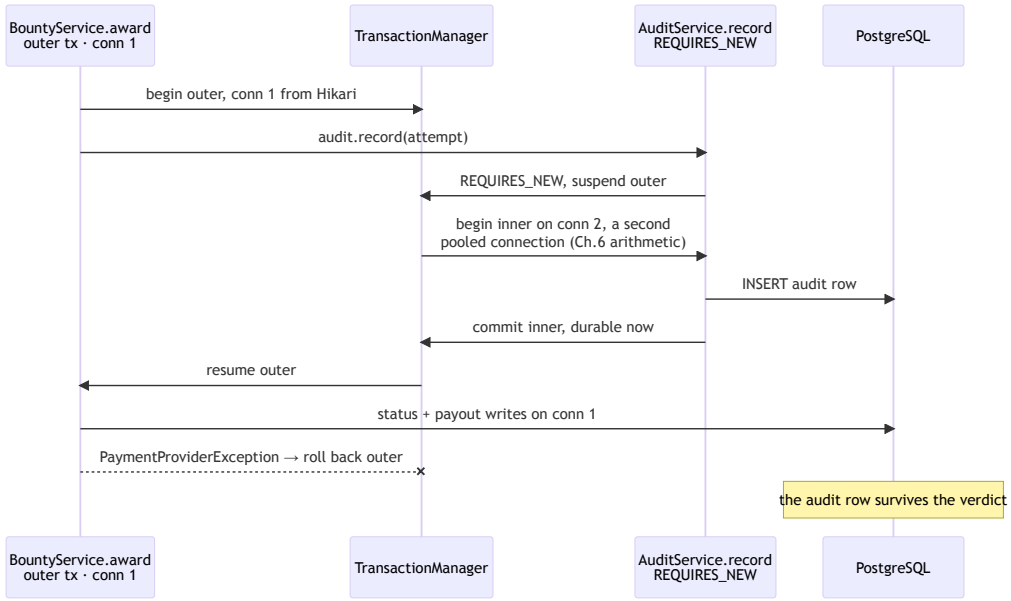
Then he tapped the audit line in her drawing. "Your audit log participates in the very transaction it's supposed to witness. Tonight it confirmed a lie. What does it do when a legitimate award *fails* and rolls back?"

Kenza followed it through. "The audit row rolls back with it. We'd have no record the attempt ever happened. The witness dies with the defendant."

"So?"

"So the audit needs its own transaction. `REQUIRES_NEW`: the interceptor suspends the outer transaction, takes a *second* connection, runs the audit write, commits it independently, then resumes the outer. The audit survives whatever the business logic decides." She paused, because the four-beat skeleton on her index card demanded it. "Cost: every `REQUIRES_NEW` call holds two pool connections at once, so under load it can double our Hikari draw. When every pooled connection is held by an outer transaction waiting for its second, they all stall until the pool's connection timeout. And the inner transaction can deadlock against rows the outer one still holds."

She drew it before he could ask, as two lifelines with the suspension explicit:




"And `NESTED`?"

“Savepoints inside one physical transaction: partial rollback without abandoning the whole. But savepoints are a JDBC affair; JPA has no savepoint API, so our `JpaTransactionManager` would refuse `NESTED` at runtime anyway. And we don’t want a *partial* anything here; we want an independent witness.” Rachid nodded, and she understood she’d just passed something.

The lost update came last. Kenza had arrived assuming it was an isolation problem; Postgres runs `READ_COMMITTED` by default, and surely some stricter level would have caught it. Rachid just waited while she argued herself out of it. Malika’s read and Malika’s write were *two different transactions*, separated by forty seconds of human thinking across stateless HTTP requests. Isolation governs what concurrent transactions see of each other, and these two never overlapped; the bug lived between requests, where no transaction existed. Raising the level to `SERIALIZABLE` would add contention on the hot tables without touching it.

“Say the diagnosis precisely,” Rachid said. “Most people reach for an isolation level here, because the bug smells like the database’s fault.”

“It’s not an isolation problem,” Kenza said slowly. “It’s an application-level versioning problem. The two transactions never overlap, so isolation has nothing to referee. The version has to live in the data and travel with the request.”

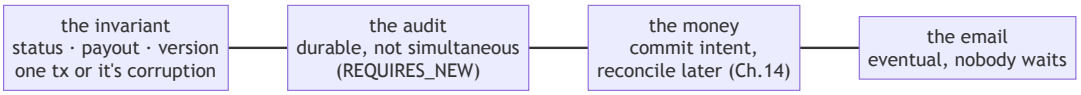
“That sentence is a tell. Recite the ladder anyway. You’ll be asked.” She did, marker up, one level to a row. “And Postgres?” She added a column.

| Level | What it allows or rules out | What Postgres does |
|---|---|---|
| `READ_UNCOMMITTED` | Dirty reads: data another transaction may yet roll back. | Refuses to do dirty reads. Ask for this level and you silently get `READ_COMMITTED`. |
| `READ_COMMITTED` | Rules dirty reads out, but a re-read can differ: the non-repeatable read. | Runs at this level by default. |
| `REPEATABLE_READ` | Pins rows already read. New rows can still appear: phantoms. | Snapshot isolation, so no phantoms either. |
| `SERIALIZABLE` | Rules out all three. | Serializable-snapshot (SSI): fast because it lets transactions run, then aborts any it can’t order. Whatever asks for it must retry. |

Rachid drew one long horizontal line beneath her ladder and labeled the ends. “Consistency is a spectrum, and somebody has to *price* it. This end is the invariant: status, payout, version. One transaction or it’s corruption. That end is the eventual: the notification email, and arguably the transfer itself, if we ever commit *intent* and reconcile. Between them, your audit: durable, not simultaneous. Integrity is tonight’s -ility, and you pay for it in throughput. Every guarantee you

strengthen holds a lock longer, draws a second connection, or retries under load. Nobody upstairs priced it, so the defaults did, and defaults don't know which row is the money."

She copied the line into her notebook before the marker dried:




*Consistency priced per write: every step rightward buys throughput and sells simultaneity.*

"Version your data, or lock it and hold," Rachid said. "Choose by contention and hold time. Now go let Malika break it on purpose."

## The reenactment

On Thursday, in staging, Malika was on one laptop and a fellow triager on another, both with report #5817's staging twin open. Malika had brought her own split keyboard. She swam open water before dawn and refused to lose a race, even a staged one, on laptop keys. Kenza counted down from three; both clicked Save. Green toast, green toast, and the database silently kept the second write and discarded the first. It was the lost update, reproduced in one take.

Then she deployed her branch and they did it again.

```
@Entity
public class Report {
    @Id @GeneratedValue private Long id;

    @Version
    private long version;   // UPDATE report SET severity=?, version=2
                            //   WHERE id=? AND version=1  → 0 rows = someone got there
        first
    // ...
}

@RestControllerAdvice
@Order(Ordered.HIGHEST_PRECEDENCE)   // consulted before the catch-all advice, or the
        409 leaves as a 500
class ConcurrencyAdvice {
    @ExceptionHandler(OptimisticLockingFailureException.class)
    ProblemDetail conflict(OptimisticLockingFailureException e) {
        var pd = ProblemDetail.forStatus(HttpStatus.CONFLICT);
        pd.setTitle("Report changed while you were editing");
        pd.setDetail("Another triager updated this report. Review their change, then
        re-apply yours.");
        return pd;
    }
}
```

This time the second save came back as a 409. The merge flow Kenza had sketched with Leila showed the colleague what Malika had changed and offered to re-apply his edit on top. Hibernate did the actual work with one `WHERE` clause: update the row *only if* the version you read is still on disk. Zero rows updated meant somebody got there first, and the exception said so out loud.

*She had hand-rolled this handshake once at Meridian: a hot dispatch table, an `UPDATE … WHERE id = ? AND version = ?`, a check that `executeUpdate()` came back zero rows, a homemade exception to say someone got there first. `@Version` was the same column wearing a JPA annotation, the `WHERE` clause written for her, and for once the hand-wired instinct was a head start instead of a trap.*

"Why not just lock the row?" the colleague asked. "Make my screen wait until hers commits?"

"Contention and hold time," Kenza said. "Two triagers editing the same report in the same minute happens, what, weekly? Optimistic costs nothing until the rare collision, then costs one polite retry. Pessimistic is `PESSIMISTIC_WRITE`, a `SELECT FOR UPDATE` (Hibernate 7 writes it as `for no key update` on Postgres), and it means holding a real database lock across human think time. A row lock ends when its transaction ends, so that transaction has to stay open from the moment you open the report until you save. You go to lunch mid-edit and the row is hostage, and so is the pooled connection under it: the same residence math that took the site down at six percent CPU. We'll reserve pessimistic for short, machine-paced contention, like the payout ledger rollups. A machine writer gets no 409. It catches the optimistic failure, re-reads, and retries with a cap."

Youssef, passing with coffee, did not slow down. "Great. Show me the lock-wait dashboard before you ship it."

There wasn't one. Kenza filed the ticket next to April's still-open *real application metrics* ticket (opened when the site drowned at six percent CPU, pool exhausted), where the operational debt went on aging.

## The fix

The pull request was small. The ADR attached to it was not, and it was the part Rachid read first.

> **ADR-008: The consistency model. Context:** A caught checked exception half-committed an award and left database, bank, and audit log telling three different stories. Two triagers silently overwrote each other across forty seconds of think time. **Decision:** Service-boundary methods declare `rollbackFor = Exception.class` (we opt out of the EJB inheritance); optimistic locking via `@Version` is the default concurrency model, with a retry-or-surface policy named per workflow; pessimistic locks are reserved for short, hot, machine-paced contention; audit writes ride `REQUIRES_NEW`. **Consequences:** Lost updates surface as conflicts users can resolve instead of corruption nobody sees; the cost is conflict UX and the discipline that isolation levels are named, not defaulted, in review.

The operative clauses below were design decisions, and the first was where the transaction boundary sits.

1. Transactions begin and end at the service layer. They do not begin at controllers, which speak HTTP: scheduled jobs and machine writers run the same operations with no controller in front of them, and a controller that answers a failure with an error response has returned normally, which the rollback rules read as commit. They do not begin at repositories either, where five writes become five autonomous commits and a mid-flight failure leaves half of them saved.
2. Any boundary a checked exception can cross declares `rollbackFor = Exception.class`. Catching inside a boundary requires either rethrowing or explicitly calling `setRollbackOnly()`. The word *handled* used to be a log message and is now a design review question.
3. The audit log records on `REQUIRES_NEW`, the attempt before and the outcome after, so the record survives any rollback of the business transaction.
4. `@Version` on `Report`, 409-plus-merge as the standard UX.
5. Query-only service methods get `readOnly = true`. Hibernate skips dirty-checking snapshots and sets flush mode to `MANUAL`, and where the driver supports it the connection gets a read-only hint. It is an optimization, not a security boundary; it will not make a bad query good.

```
@Service
public class BountyService {

    private final ReportRepository reports;
    private final PayoutRepository payouts;
    private final PaymentClient paymentClient;
    private final AuditService audit;

    BountyService(ReportRepository reports, PayoutRepository payouts,
                  PaymentClient paymentClient, AuditService audit) {
        this.reports = reports; this.payouts = payouts;
        this.paymentClient = paymentClient; this.audit = audit;
    }

    @Transactional(rollbackFor = Exception.class)            // ADR-008 §2
    public void award(long reportId, BigDecimal amount) throws PaymentProviderException
        {
        audit.record(AuditEvent.attempt("AWARD", reportId, amount)); // §3: survives
        any rollback
        Report report = reports.findById(reportId).orElseThrow();
        report.setStatus(ReportStatus.AWARDED);
        payouts.save(new Payout(report, amount));
        paymentClient.initiateTransfer(report.getResearcherId(), amount); // last:
        allowed to fail loudly
    }
}
```

The attempt was now recorded up front on its own transaction, and the outcome afterwards by the caller, once `award()` had committed or rolled back, so the audit no longer testified before the verdict. The word *handled* appeared nowhere.

```
@Service
public class AuditService {

    private final AuditRepository auditRepository;

    AuditService(AuditRepository auditRepository) {
        this.auditRepository = auditRepository;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)   // ADR-008 §3
    public void record(AuditEvent event) {
        // second connection, independent commit:
        // the witness survives the business transaction's verdict
        auditRepository.save(AuditRecord.from(event));
    }
}
```

The last file in the diff was §4's machine half. Humans got the 409 and the merge screen; the automated triage writers got a wrapper. The wrapper got the one review comment Kenza addressed to herself, because her first draft had looped *inside* the workflow's own `@Transactional` method. That transaction was already dead by the time the exception surfaced. A `this.`-retry would never have opened a new one: February's mirror, re-firing (the retry dispatches on the raw object, invisible to the transaction-opening proxy).

```
@Component
public class TriageRetry {                          // the machine path, ADR-008 §4
    private final TriageWorkflow workflow;          // separate bean: every attempt
    TriageRetry(TriageWorkflow workflow) { this.workflow = workflow; }   // crosses the
        proxy

    public void apply(TriageDecision decision) {
        for (int attempt = 1; ; attempt++) {
            try {
                workflow.apply(decision);           // fresh tx, fresh read, fresh version
                return;
            } catch (OptimisticLockingFailureException e) {
                if (attempt == 3) throw e;          // capped: surface, don't spin
            }   // NB: never loop INSIDE the failed @Transactional: that context is
        dead,
        }       //     and a this.-retry never opens a new one (February's mirror)
    }
}
```

She presented the postmortem, *The Silent Commit*, to all of engineering the following Wednesday, and walked the room down the same line she'd drawn on the Wall: proxy opens the transaction, method runs, catch swallows the rollback signal, proxy sees a normal return, COMMIT. When the CTO asked why a network call to a payment provider had been inside a database transaction at all, Rachid, in the second row with his green pen capped, simply turned his head and said, "Kenza?"

"It shouldn't be," she said. "Today's fix puts the call last and lets failure roll everything back. That's honest, but it couples our transaction to their uptime. It also has an inverse failure: our writes flush at commit, after the transfer has gone out, so a commit that fails at that point leaves the money sent and the database rolled back. The pattern we want is to commit our *intent* to pay and reconcile the transfer separately. That's a bigger conversation." It was the Wall's spectrum line, recited upward. The CTO wrote something down. So, she noticed, did Rachid.

### ⚖ Your move, architect

ADR-008 §4 was one line in the diff and an hour in the review: there were three defensible ways to stop the silent overwrite, and each was argued out. You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Optimistic: `@Version`, surface the conflict.** A stale write updates zero rows and becomes a 409 with a merge flow for humans, a capped retry for machines.
- **B. Pessimistic: `SELECT FOR UPDATE` on open.** The first triager locks the row; the second waits. No conflict ever happens, because no concurrency ever happens.
- **C. `SERIALIZABLE` isolation on triage transactions.** The database referees everything: any interleaving it can't order as serial aborts, and the application retries.

**The trade-offs, since you've committed:** all three buy the same **integrity**, and they differ in which -ility each one spends. **A**, Kenza's choice, spends almost nothing: contention is weekly (per Malika), a human can resolve the conflicts, and nothing holds a lock across think time. The price is conflict UX and a retry-or-surface policy named per workflow, the discipline ADR-008 imposes. **B** reads like the balanced choice (real locks, no retries, no conflict UX) and is simply *wrong* here, because the hold time is human think time. A pessimistic lock across a coffee break serializes the whole triage team behind one open browser tab, and each lock pins a pooled Hikari connection: Chapter 6's arithmetic, at site-wide cost. The books recommending it assume machine hold times: milliseconds, not lunch. **C** spends **throughput** under load: Postgres SSI referees by *aborting* whatever it can't order, so you inherit retry storms and still write the retry handling A would have demanded. That is A's cost without A's precision. It is worth it only where the invariant spans rows no version column can guard, and then it is named in review, per the ADR.

The choice is a one-way door: the commitment is not a locking annotation but a consistency *contract*. Once Malika's triagers build their workflow around conflict semantics (the 409, the refreshed view, the someone-got-there-first muscle memory), and the merge UI, retraining, and audit trail assume it, switching concurrency models means rebuilding the human workflow. "Pick something now, revisit the locking next quarter" is the documented trap: next quarter the workflow is load-bearing and the revisit costs ten times more, decided under worse information. Commit to the conflict contract deliberately, on day one, with the UX built for it.

The next morning there was a hiring req in her inbox, approved, with a note from Rachid: *The team is about to grow. First candidate interviews Tuesday. You're on the panel: you're the one who'll be teaching them.*

## The debrief: Ch.8, The Silent Commit

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 8.** Why does `@Transactional` only roll back on unchecked exceptions by default, and what's the trap with caught ones?

Then the follow-ups, cold:

1. Your audit log must survive a business rollback: how, and at what cost?

2. Why doesn't raising the isolation level fix the two-triagers lost update?
3. An award writes status, payout, and audit, then sends money and an email: which belong in one transaction?
4. What does each isolation level allow, and what does PostgreSQL do with them?
5. What is `NESTED`, and why does the audit use `REQUIRES_NEW` instead?

Tick what you said:

- ☐ Claim: Commit on checked exceptions, rollback only on `RuntimeException` or `Error`. An exception you catch inside the method never reaches the proxy.
- ☐ Mechanism: The proxy's interceptor inspects how the method ended, and a `catch` looks like a normal return. Rollback-only makes the outer commit throw.
- ☐ Trade-off: The default keeps recoverable business exceptions from nuking whole transactions. `rollbackFor` overrides it per boundary, and `setRollbackOnly()` covers the must-catch cases.
- ☐ Failure mode: A helper catches a checked provider exception, logs handled, and the transaction commits a payout that never happened.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Spring commits on checked exceptions and rolls back only on `RuntimeException/Error`, and any exception you catch inside the method reaches neither rule. **Mechanism:** the AOP proxy's interceptor opens the transaction and binds the connection and `EntityManager` to the thread; on the way out it inspects how the method ended and consults rollback rules inherited from EJB's "checked = business condition, unchecked = system failure" convention. A `catch` inside the method means the proxy sees a normal return and commits. If the exception first crossed an inner `REQUIRED` proxy and matched its rollback rules, the shared transaction is already rollback-only, and the outer commit throws `UnexpectedRollbackException`. **Trade-off:** the default keeps recoverable business exceptions from nuking whole transactions, and `rollbackFor = Exception.class` overrides it per boundary; explicit `setRollbackOnly()` covers the must-catch cases. **Failure mode:** a helper catches a checked provider exception, logs "handled," and the transaction durably commits a payout that never happened, so the database, the bank, and the log all tell different stories.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Consistency/integrity**, chosen on purpose and priced against **throughput**: every guarantee strengthened holds a lock longer, draws a second pooled connection (`REQUIRES_NEW` doubles Hikari draw, which is Chapter 6's arithmetic), or retries under load. The pricing is per write: the invariant (status + payout + version) is transactional, the audit is durable but independent, and the money and the email are candidates for eventual, which is the seed of the outbox conversation. **Operability** rides shotgun: the lock-wait dashboard is still a ticket, aging.
- **The ADR:** ADR-008, the consistency model: `rollbackFor = Exception.class` at service boundaries (opting out of the EJB inheritance); `@Version` optimistic locking as the default, retry-or-surface named per workflow; pessimistic locks only for short, hot, machine-paced sections; audit on `REQUIRES_NEW`; isolation levels named, not defaulted, in review.
- **Rejected alternatives:** pessimistic locking as the default (holds connections across human think time, and Chapter 6's math says no); `SERIALIZABLE` everywhere (SSI aborts under contention; you pay the retry logic anyway); the EJB rollback default (the 2021 `catch` is where that ends); the audit inside the business transaction (the witness dies with the defendant).
- **Fight or lean:** *Lean* on declarative demarcation: the proxy, `REQUIRED` joining every repository call beneath the boundary, dirty checking flushing for you. That machinery keeps the service layer business-shaped. *Fight* the inherited rollback default (`rollbackFor = Exception.class` is a recorded refusal of a 1990s convention) and any review that asks an isolation level to solve a versioning problem.

**Story anchors**

- The spreadsheet that disagreed with the bank → a caught checked exception inside `@Transactional` commits everything around it.
- The audit row that confirmed the lie → audit logging on `REQUIRES_NEW`, so the witness survives the verdict.
- Malika's simultaneous clicks → the lost update; `@Version` turns the silent overwrite into a 409 with a merge.
- The spectrum line under the isolation ladder → consistency is priced, not defaulted: invariant transactional, witness independent, money eventual. The think-time lost update is a *versioning* problem, not an isolation one: the architect's tell.

**Interview ammo**

- *Kill-shot: Default rollback only covers `RuntimeException`: why, how do you change it, and what's the trap with caught exceptions?* It is a convention inherited from EJB (checked = handleable business condition), and you override it with `rollbackFor`. A `catch` swallows the signal entirely, so the proxy commits: rethrow, or call `setRollbackOnly()`. If the ex-

ception already crossed an inner `REQUIRED` proxy that rolls back on it, the transaction is rollback-only and the commit throws `UnexpectedRollbackException`.

- *Follow-up: Your audit log must survive a business rollback: how, and at what cost?* `REQUIRES_NEW`: suspend the outer transaction, open a second physical connection, commit independently. The cost is double pool draw per call and a deadlock risk against rows the outer transaction holds.
- *Follow-up: Why doesn't raising the isolation level fix the two-triagers lost update?* The read and the write are separate transactions with human think time between them; isolation only governs concurrent transactions, so you need version-checked writes (`@Version`: 409-plus-merge for humans, retry-with-cap for machines) or `PESSIMISTIC_WRITE` (high contention, short machine-paced holds).
- *Architect follow-up: An award writes status, payout, and audit, then sends money and an email: which belong in one transaction?* Only the invariant: status, payout, version. The audit rides `REQUIRES_NEW`, and the transfer and email go eventual with reconciliation. Consistency is a spectrum, and you price it per write.
- *Follow-up: What does each isolation level allow, and what does PostgreSQL do with them?* `READ_UNCOMMITTED` allows dirty reads. `READ_COMMITTED` rules them out, but a re-read can differ (the non-repeatable read). `REPEATABLE_READ` pins rows already read, but new rows can appear (phantoms). `SERIALIZABLE` rules out all three. PostgreSQL runs `READ_COMMITTED` by default and gives you that when you ask for `READ_UNCOMMITTED`; its `REPEATABLE_READ` is snapshot isolation with no phantoms, and its `SERIALIZABLE` is SSI, which aborts what it cannot order, so the caller must retry.
- *Follow-up: What is `NESTED`, and why does the audit use `REQUIRES_NEW` instead?* `NESTED` is a savepoint inside one physical transaction: partial rollback without abandoning the whole. JPA has no savepoint API, so `JpaTransactionManager` refuses `NESTED` by default. The audit row must survive a rollback of the business transaction, so it takes `REQUIRES_NEW`: an independent commit on a second connection.

**Pointers** → `examples/ch07_jpa/TransactionalRollbackAndPropagation.java`, `OptimisticLockingAndCascade.java`

---

# Chapter 9: The Proxy You Didn't Write

Imane had been at Glasshouse for exactly one day and four hours when she stopped standup dead.

"Quick one," she said, in the tone of someone who assumed it would be. "I've been reading the triage flow. `ReportRepository` is an interface. Where's the implementation? I've been looking for an hour, and I'd like to know what I'm doing wrong before I spend a second one."

Silence. Eight engineers stood around a kanban board with seven years of accumulated folklore between them, and not one of them could point to a class. Malika's entire triage queue ran through that repository. Half a year of incidents had been debugged on top of it. Nobody had ever opened its implementation, because there wasn't one to open.

Kenza opened her mouth, then caught Rachid's eye across the room. He gave a small shake of the head. Afterward, by the kettle, he was explicit. "Don't tell her. Show her how to find out." He paused. "She's yours, by the way. Onboarding buddy. I've already updated the doc."

*Eight years of Java and half a year of Spring, and my first official act as a mentor is a question I would have flunked in January.* At Meridian, the data layer was the one place "where's the implementation?" always had an answer: every query a string of SQL she'd typed, every row-to-object mapping a loop with her name on the commit. Here, apparently, the repository *was* the magic. Kenza realized she'd been trusting it the way she'd once trusted Boot's startup wiring, the auto-configuration that conjured beans off the classpath: politely, and without evidence.

## The investigation

They did it the honest way: a breakpoint and a debugger, Imane driving.

Inside `TriageWorkflow`, where `ReportRepository` arrived through the constructor, Imane evaluated `reportRepository.getClass()`.

"`jdk.proxy3.$Proxy187`," she read. "That's not a class. That's an apology."

"It's a JDK dynamic proxy," Kenza said. "Spring Data registered a factory bean for your interface at startup. The bean it produces is a proxy that *implements* `ReportRepository`. Nobody wrote it. It was generated. Step into `findById` and keep going past the interceptors."

Imane stepped. Three frames of plumbing, and then a real class with real code: `SimpleJpaRepository`, calling `EntityManager.find`. "Okay. So *that's* the implementation. For `findById`, `save`, the CRUD set."

"For the methods Spring Data ships. Now the interesting half." Kenza scrolled to the methods that existed nowhere at all:

```
public interface ReportRepository extends JpaRepository<Report, Long> {

    // The "latest activity" widget. Compiles. Passed its test.
    Optional<Report> findTopByProgramIdOrderByCreatedAt(Long programId);

    // The reports list. Every page of the infinite scroll pays for this.
    Page<Report> findByProgramIdAndStatus(Long programId,
                                          ReportStatus status,
                                          Pageable pageable);
}
```

"There's no `SimpleJpaRepository` method called `findTopByProgramIdOrderByCreatedAt`," Kenza said. "At startup, Spring Data *parsed the method name*." She dropped a breakpoint in the framework's `PartTree` constructor and restarted the app. They watched the string go in and a parse tree come out: find, top one, by programId, order by createdAt. Spring Data checked every property against the entity metamodel, then compiled the whole thing into JPQL before the first request ever arrived. "The method name is the query. It's not magic. Let me show you the mechanism."

She heard the sentence as it left her mouth, and stopped. Six months of Rachid's green ink, and the catchphrase had changed hands without asking either of them.

Imane, mercifully, was already squinting at the widget code. "Wait. Order by `createdAt`… which direction?"

> **Your call.** The build is green and the one-row test has passed for months. Before Kenza answers: what is `findTopByProgramIdOrderByCreatedAt` actually returning, why has no test objected, and which -ility got sold when a query became a sentence the compiler doesn't speak?

Which is how, on day four, Imane found the bug everyone senior had missed. The "latest activity" widget on every program dashboard had been showing a report from 2019 for months. `OrderByCreatedAt` with no `Desc` is valid in Spring Data's grammar, and the default direction is ascending. Top one of ascending is the *oldest*. The unit test had a single-row fixture, and one row passes any `ORDER BY` ever written.

"The method name compiles," Imane said slowly. "It just answers a different question."

*That* was the part that itched at Kenza's hand-wired instincts. The Java compiler only checks that the method name is legal Java; it knows nothing about what the name means. The startup parser does validate the pieces: misspell a property as `findByTtile` and the context fails fast at boot. But it cannot catch a name that parses cleanly and means the wrong thing. At Meridian a query was a string of SQL the compiler never vouched for, so nobody expected it to: you read the `ORDER BY` with your own eyes and ran it against real rows before you trusted it. Here the green

build had felt like proof, and trusting it was the mistake, because a derived query is a sentence in a language the Java compiler doesn't speak.

(If you committed at the pause: ascending, so the *oldest*; one row blesses every sort. The -ility is the deeper cut, and Kenza names it at the Wall.)

While they were in the logs with `show-sql` on, Imane asked the second question. "Why does the reports list run *two* queries per page?"

There it was, under every `Page<Report>`: the `SELECT` they asked for, and a `SELECT count(*)` over the whole filtered set that nobody had. The count exists so `Page` can report `totalElements` and `totalPages`, and it runs on every request that returns a full page. On the researcher-facing infinite scroll, nothing ever displayed a total. They were paying for a number they threw away, on every scroll event, against the platform's biggest table.

## What the senior sees

At the Wall, under the smoke-cloud proxy drawing with its caption, *this. goes around the mirror*, Kenza gave Imane the stack, top to bottom. "One breath per layer," she promised.

Imane had stopped under the cloud first. She'd read the February postmortem the way new hires read ghost stories. "That's the transaction proxy. The one the bean walked around by calling itself."

"February's ghost in a new coat," Kenza said. "That one was CGLIB around a class we *wrote*, and a `this.` call never crosses the wrapper, so the advice silently never runs. Today's coat has nobody inside it at all. Atrium is full of objects nobody wrote."

A Spring Data repository is three dispatch rules behind one proxy. CRUD methods route to `SimpleJpaRepository`. Methods carrying `@Query` run exactly what you declared (JPQL usually, native SQL when you need Postgres-specific reach), with named parameters bound by `@Param`, not by position. Everything else goes through name derivation: parsed once at startup, validated against entity properties, compiled to JPQL. The boot-time validation is the trade you're making: property names are checked, *intent* is not.

So the team needed a ladder. Derived names stay for the trivial cases. Past that, you write the query:

```
public interface ReportRepository extends JpaRepository<Report, Long> {

    Optional<ReportSummary> findTopByProgramIdOrderByCreatedAtDesc(Long programId);

    // Infinite scroll: Slice fetches size + 1 and answers hasNext, with no COUNT.
    Slice<ReportSummary> findByProgramIdAndStatus(Long programId,
                                                  ReportStatus status,
                                                  Pageable pageable);

    // House rule: past two predicates the query is written, not derived.
    // Ordering is pinned here; the Pageable only supplies the window.
    @Query("""
        select new com.glasshouse.atrium.report.ReportSummary(
            r.id, r.title, r.severity, r.status, r.createdAt)
        from Report r
        where r.program.id = :programId
          and r.status = :status
          and r.createdAt >= :since
        order by r.createdAt desc
        """)
    Slice<ReportSummary> findRecentForTriage(@Param("programId") Long programId,
                                             @Param("status") ReportStatus status,
                                             @Param("since") Instant since,
                                             Pageable pageable);
}
```

`ReportSummary` was a record with five fields, constructed straight from the `select` clause. "A projection," Kenza said. "The query returns exactly these columns, the database does the narrowing, and there's no entity in sight. No entity means no lazy fields waiting to detonate after the session closes, and no over-fetching forty columns to show five. Remember the thousand tiny queries in May? Projections close off that whole family at the boundary." (May, if you skipped it: a loop touched a lazy association on every row, which cost one extra `SELECT` apiece and hundreds of queries for one screen.)

"There are interface projections too," she added, "where Spring generates, yes, another proxy, backed by just the selected columns. Closed ones, where every getter maps to a property, stay efficient. *Open* ones, with `@Value` SpEL expressions, fetch the whole entity to evaluate the expression, which un-fixes everything you fixed. We use records. They can't lie."

Imane pointed at the triage search screen spec: severity, status, program, researcher, date range, any combination. "Derived name for that would be a paragraph."

"That's where derivation stops being clever and starts being a hostage note. Dynamic predicates get a custom repository *fragment*: you write an interface and an `Impl` class, and Spring Data composes it into the same proxy. You build it on Specifications or Querydsl, both of which are just composable `where`-clauses as objects." One breath, as promised.

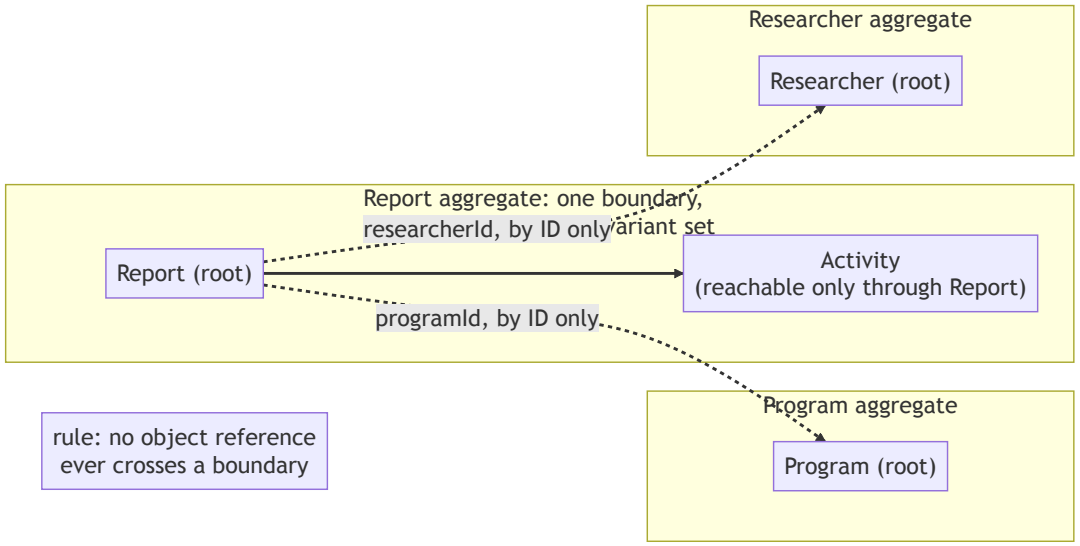
"And why does any of this need a persistence context underneath?" Imane asked. "Why not just… SQL?"

"Because we chose this engine, and it's worth knowing we chose it. Spring Data JPA rides Hibernate, with its persistence context, dirty checking and lazy loading: all the power, and all the traps that come with a session. Spring Data *JDBC* is the same repository grammar with that whole layer deleted. There's no session and no lazy anything. You load and save whole aggregates, and what you fetched is all you have. For a module with small aggregates and no appetite for surprises, that honesty is a feature. Spring Data Mongo is the grammar again over documents. There are no joins, so the document has to *be* the aggregate, modeled for the access pattern, or you suffer. Same interfaces, three engines. Because the grammar is identical, you have to know which one you're holding. Atrium's core is relational and shared, so it stays JPA."

## The architecture under the repositories

Before they left the Wall, Imane pointed at the project tree. "Eleven repositories. Nine tables. Smell, or coincidence?"

"Autopsy," Kenza said. "They grew one per table, and per-table repositories are how the graph wins: every query starts anywhere and reaches anything. That's the soil May's thousand queries grew in. Nobody ever decided what the *unit* was, and the unit we keep consistent, and change, isn't a table." She took the marker and drew three boxes where there had only ever been entity arrows:




"`Report` owns its `Activity` rows. There's an invariant between them (a report's status has to agree with its latest activity), and the lost-update week taught us what guards an invariant: *one* transaction. So they share a boundary, loaded and committed together; the aggregate is the transaction's natural span. `Program` and `Researcher` are other aggregates: a `Report` carries a `pro-`

`gramId`, never a `Program`. The `@ManyToOne` stays mapped; that's just how the foreign key gets written and why `findRecentForTriage` can say `r.program.id`. But the rule governs *navigation*: no code outside a declared read-model query traverses that association across an aggregate line, and domain logic sees only the ID. Inside a boundary you get invariants and one transaction. Between boundaries you get IDs and, when it matters, eventual consistency."

(The lost-update week, for late joiners: two transactions read the same row, both wrote, and the second silently won, until a `@Version` check made the race loud.)

"That's DDD," Imane said, in the tone of someone identifying a diet.

"The ten percent that pays rent here: aggregates, and the words. `Report`, `Triage`, `Disclosure`. The vocabulary on this wall *is* the API of the domain. The day the code says `SubmissionRecordDTO` and Malika says *report*, every meeting pays a translation tax. And Spring Data JDBC? It's this picture *enforced*: whole aggregates in and out, no lazy graph, because it refuses to believe in one. Who polices the boundary, the framework or the review, is an architectural choice, not a library preference. We keep the engine and adopt the worldview: repositories per aggregate *root*, three, not eleven, and reads across a line become projections on purpose. The -ility this week has been circling: **evolvability of the data model**, the aggregate as the unit of consistency *and* of change."

Rachid drifted past with a teabag and looked at the three boxes considerably longer than the tea required. "Leave those lines up," was all he said. Nobody had a name yet for the map the Wall had just started drawing. (Chapter 14 does.)

## Migration day

Friday, 4:55 p.m., the deploy pipeline went red:

```
Migration checksum mismatch for migration version 41
-> Applied to database : 1486912852
-> Resolved locally    :   89004127
Either revert the changes to the migration, or run repair to update the schema history.
```

Someone (in fairness, someone moving fast in a team that was suddenly bigger and shipping faster) had "fixed a typo" in `V41__add_report_indexes.sql`. It was an index name: cosmetic, and already applied to production weeks ago. Flyway records a checksum in `flyway_schema_history` the moment a versioned migration is applied. From that point the file stops being code and becomes a historical record. Edit history, and validation calls you a liar.

"`flyway repair` takes thirty seconds," somebody offered, eyeing the clock.

> ⏸ **Your call.** 4:55 p.m., pipeline red, `flyway repair` is thirty seconds away and so is the weekend. Repair now or investigate first, and name the one thing `repair` can never tell you. Commit.

Youssef put the brick on the table. "We can repair the checksum in thirty seconds, or we can know what we're repairing in thirty minutes. One of those happens again at 3 a.m."

Thirty minutes, then: confirm the edit was genuinely cosmetic by diffing the applied SQL against the file, revert the file to match history exactly, and ship the rename as `V42`, a boring, versioned step forward. The rules went in the runbook before anyone went home: never edit an applied migration; fixes roll forward as new versions; views and functions live in `R__` repeatable scripts, which by design re-run when their checksum changes. ("Liquibase, if you're wondering," Kenza told Imane, "is the same discipline with a changelog abstraction and rollback choreography. We run Flyway because plain SQL is what we'd be reading at 3 a.m. anyway.")

"Append-only," Youssef said, underlining it, "because a versioned migration is a one-way door. Walk through and it locks. Want something different, take the *next* door." Then he made it arithmetic: eight pods, one schema, no maintenance window, every environment replaying the same history in order. Schema evolvability under continuous deploy depends on every database having run the identical sequence of files, which means never rewriting the past. Edit an applied migration and you fork history: a checksum mismatch if you're lucky, silent staging-production drift if you're not.

And since everyone was already in a room being honest, Kenza pulled up the scaling plan and finally did the arithmetic nobody had done in seven years. Each Atrium pod ran Tomcat's 200 worker threads over Hikari's default pool of ten connections. Eight pods × 10 = 80 of Postgres's `max_connections = 100`. The autoscaling proposal said twelve pods. 120 > 100. The new pods wouldn't make the platform faster; they'd make Postgres start refusing connections (*sorry, too many clients already*) for everyone, including the healthy pods.

*At Meridian there had been no pool to size (`DriverManager.getConnection`, used, closed, a fresh handshake per query), and nobody ever counted connections, because freight traffic never made anyone.* The per-pod story was worse. Under burst, 200 threads contend for ten connections. The losers queue at the pool, time out after Hikari's default thirty seconds, and throw `SQLTransientConnectionException`, while the database sits at 4% CPU. The pool is sized by the database's limits and the contention in front of it, not by anybody's CPU count. Pool exhaustion is an outage that *impersonates* a slow database.

It was launch day's equation one layer down, the one behind the 503s with idle CPUs, when every Tomcat worker sat parked on one slow vendor. The same math applied here: 200 threads, a query or three behind each `Page`, funneling into ten connections under a ceiling of a hundred. The queue had simply moved from the connector to Hikari.

## The fix

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10      # 8 pods x 10 = 80; Postgres max_connections = 100,
                                 # minus headroom for migrations, ops psql, and one
        emergency
      connection-timeout: 3000   # starve fast and visibly, not at 30s in silence
  jpa:
    open-in-view: false          # still false; projections made it painless
```

The pull request that closed the week carried more documentation than code. The derived-query rule: explicit `@Query` past two predicates, and every `OrderBy` names its direction. Any test of ordering needs at least two rows, out of order, because one-row fixtures bless every sort ever written. List endpoints return `Slice` unless the UI shows a total; `Page` costs a `COUNT`, so it has to be a choice, not a default. Read paths return projections and entities don't cross the service boundary, the last clause of a lesson Kenza had been learning since her first green-ink review. The Flyway discipline was written down. Hikari was sized by arithmetic, and the scaling plan was corrected before it became somebody's 3 a.m. page.

On top sat half a page in `docs/adr/`, the week's real decision, made durable:

> **ADR-009: Aggregate boundaries. Context:** Repositories had grown one per table, queries reached across the object graph from anywhere to anywhere, and pagination and pool sizing both fell over when real data arrived. **Decision:** Repositories exist per aggregate root, never per table: `Program`, `Report` (owning `Activity`), `Researcher`. Cross-aggregate reads go through projections and read models, never object-graph traversal. References between aggregates are by ID. Flyway migrations are append-only. The Hikari pool is sized from Postgres's `max_connections` and the launch-day thread math, arithmetic recorded here rather than in folklore. **Consequences:** The aggregate becomes the transaction's natural boundary (the invariant unit ADR-008 demanded) and a candidate border for whatever Atrium grows into next. The accepted cost: convenient cross-aggregate `JOIN`s stop being free and become deliberate read-model decisions.

The widget fix itself was one word, `Desc`, and a two-row test. Imane shipped it, four days into the job, with a commit message Kenza didn't edit: "Top of ascending is the oldest. Ask me how I know."

### ⚖ *Your move, architect*

The lines on the Wall created an immediate problem. The program dashboard reads across all three boxes: reports, researcher names, severity rollups. Yesterday that was one cheerful `JOIN` through the object graph. How do dashboards read across aggregates now? You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. A projection per view.** Each dashboard gets its own record read model: one query, exactly its columns, owned by the view.
- **B. Entity-graph traversal with clever fetching.** Keep the object graph for reads; `JOIN FETCH` and `@EntityGraph` the exact path each dashboard needs.
- **C. A separate read database, full CQRS.** Writes hit the aggregates; a synchronized read store serves dashboards, shaped purely for reading.

**The trade-offs, since you've committed: B** re-couples every dashboard to the write model: each new widget needs its own fetch plan against entities whose job is guarding invariants, not rendering. May already paid that bill. The thousand queries were graph-reads doing a view's job. **A** is what the team chose: one `@Query` and one record per view, cheap to add and cheap to delete, with the write model free to evolve underneath. That spends the **evolvability** the aggregates bought. **C** buys the same separation with heavier machinery: synchronization, eventual-consistency UX, a second store to operate and page on. That machinery is an **operability** tax. C becomes defensible when reads run orders of magnitude past writes (think 100×, a saturated replica, rollup queries throttling triage). Until that signal arrives, C is a door A keeps purchasable. Leila's version went in the notes: "We buy the second database when a customer's dashboard is slow, not when a conference talk was good."

The following Wednesday, Imane gave the Wall tour, the onboarding rite: socket to database and back, every hop named out loud. When she reached the repository she added a hop the diagram hadn't had before: "…and the repository is a proxy nobody wrote, which is fine, because now we know who did." Rachid watched Kenza watch Imane, and wrote nothing down.

The team was bigger now, and faster. New `v2` routes were landing weekly, quicker than any one person was reading them. That would matter soon, though not yet.

## The debrief: Ch.9, The Proxy You Didn't Write

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 9.** Where is the implementation of a Spring Data repository, and how does it execute your queries?

Then the follow-ups, cold:

1. Why is `Page` the wrong return type for infinite scroll, and what do you use?
2. How do you size a HikariCP pool, and what does exhaustion look like?
3. Repositories: one per table, or one per what?
4. A search screen filters on severity, status, program, researcher and date range, in any combination: how do you query it?
5. When would you pick Spring Data JDBC over Spring Data JPA?

Tick what you said:

- ☐ Claim: There is no implementation class: Spring Data generates a proxy bean from the interface at runtime.
- ☐ Mechanism: A JDK dynamic proxy dispatches CRUD to `SimpleJpaRepository`, `@Query` methods to declared queries, and the rest through name derivation parsed at startup.
- ☐ Trade-off: Zero boilerplate and property names validated at boot, but nobody checks intent and long names become unreadable.
- ☐ Failure mode: `OrderByCreatedAt` without `Desc` compiles, passes a one-row test, and silently returns wrong data for months.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** There is no implementation class: Spring Data generates a proxy bean from the interface at runtime. **Mechanism:** A repository factory registers a JDK dynamic proxy implementing your interface, and calls dispatch by rule: CRUD methods to `SimpleJpaRepository`, `@Query` methods to your declared JPQL or native SQL, and everything else through method-name derivation, parsed at startup into a query tree, validated against entity property names, and compiled to JPQL. **Trade-off:** Zero boilerplate and fail-fast validation of property *names* at boot. But the query lives in a method name, so the compiler checks Java grammar while nobody checks intent, and past two or three predicates the name becomes unreadable. **Failure mode:** A name that parses but means the wrong thing (`OrderByCreatedAt` without `Desc`) compiles, passes a one-row test, and silently returns wrong data for months. The cure is explicit `@Query` past two predicates and fixtures that can fail.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Evolvability of the data model**, the aggregate as the unit of consistency *and* change: invariants inside one boundary, references by ID, read models keep-

ing dashboards from freezing the write schema. Append-only migrations are the same - ility at the schema layer. The Hikari arithmetic is **availability**, bought with division.

- **The ADR:** ADR-009, aggregate boundaries: repositories per aggregate root (`Program`, `Report+Activity`, `Researcher`); cross-aggregate reads via projections, never graph traversal; references by ID; Flyway append-only; the pool sized from `max_connections` and the thread math.
- **Rejected alternatives:** per-table repositories with free graph reach (the default that grew, coupling nobody decided); entity-graph dashboards (May's bill, paid twice); full CQRS now (kept purchasable, its 100×-reads signal named in advance); wholesale Spring Data JDBC (boundary *enforcement* at the price of remapping the platform's core, when the worldview was adoptable without the migration).
- **Fight or lean:** *Lean* on the generated machinery: derivation for trivial queries, `SimpleJpaRepository` for CRUD, and Flyway's checksum as the guardian of history. When it calls you a liar, believe it. *Fight* the graph: JPA will cheerfully let every entity reach every other forever. The aggregate boundary is discipline the framework never enforces, which is why it lives in review and ADR-009, not in an annotation.

**Story anchors**

- Imane's hour-long hunt for a class that doesn't exist → the repository is a runtime-generated proxy: interface in, JDK proxy out, `SimpleJpaRepository` behind it.
- The 2019 report on the "latest activity" widget → derived-query names compile and parse, but nobody checks intent; missing `Desc` means top-of-ascending = oldest.
- The 4:55 p.m. checksum standoff → applied migrations are history, not code: never edit one; roll forward with a new version, and `repair` only when you can explain what you're repairing.
- Three chalk boxes on the Wall, arrows carrying only IDs → the aggregate is the unit of consistency (invariants inside, one transaction) and of change; cross-aggregate reads are read models, not graph reach. These are the first lines of what Chapter 14 will call a context map.

**Interview ammo**

- *Kill-shot: You write a Spring Data repository interface with no implementation. What actually runs when you call `findByStatus(...)`, end to end?* A startup-built JDK dynamic proxy dispatches the call: derivation parsed the method name into a query tree at boot, validated properties against the metamodel, and compiled JPQL that executes through the shared `EntityManager`. CRUD methods route to `SimpleJpaRepository` instead.
- *Follow-up: Why is `Page` the wrong return type for infinite scroll, and what do you use?* Every full `Page` runs a hidden `COUNT(*)` over the full filtered set to populate totals (Spring Data

skips it only when a short page already gives the total away); `Slice` fetches `size + 1` rows and answers only `hasNext`, so you skip the count you weren't displaying anyway.

- *Follow-up: How do you size a HikariCP pool, and what does exhaustion look like?* By database arithmetic: pods × pool size + headroom ≤ `max_connections`, with contention measured, never by app CPU count. Exhaustion presents as `SQLTransientConnectionException` after `connection-timeout` while the DB idles, an outage impersonating a slow database.
- *Architect follow-up: Repositories: one per table, or one per what?* One per aggregate root, the span of one transaction's invariants and the unit the model evolves by. Cross-aggregate reads are explicit projections and references cross by ID, which makes today's aggregate tomorrow's candidate context, or service, boundary.
- *Follow-up: A search screen filters on severity, status, program, researcher and date range, in any combination: how do you query it?* With a custom repository fragment: you write an interface and an `Impl` class, and Spring Data composes it into the same proxy. Build the predicates there with Specifications or Querydsl, which are composable `where` clauses as objects. A derived name for that screen would be a paragraph.
- *Follow-up: When would you pick Spring Data JDBC over Spring Data JPA?* For a module with small aggregates and no appetite for surprises. It is the same repository grammar with the persistence context deleted: no session, no dirty checking, no lazy loading. You load and save whole aggregates, and what you fetched is all you have. Atrium's core is relational and shared, so it stays on JPA.

**Pointers** → `examples/ch08_data/`

---

# Part V: Production

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Security filter chain:** the ordered filters that authenticate and authorize every request before MVC sees it. **Authentication:** who you are. **Authorization:** what you may do. **JWT:** a signed token the client presents on every request; the server validates the signature and claims instead of keeping a session. **CSRF / CORS:** browser-world protections. CSRF protection guards against requests forged on top of ambient cookies; CORS governs which web origins may call your API. **Test slice:** booting only one layer (web, JPA, JSON) for a test instead of the whole application. **Context cache:** Spring reuses a booted test context across tests with identical configuration. **Testcontainers:** real dependencies (PostgreSQL) in throwaway Docker containers, so tests verify the same database you ship. **Actuator / probes:** built-in operational endpoints. **Liveness:** "restart me." **Readiness:** "route traffic to me." **Metric cardinality:** the number of distinct time series a metric produces; each distinct tag value adds one. **Circuit breaker / bulkhead / idempotency:** a circuit breaker stops calling what keeps failing; a bulkhead caps concurrent calls per dependency; idempotency makes retries safe by making repeats harmless.

# Chapter 10: The Breach That Wasn't

The report arrived on a Tuesday in July, in the Stonethrow queue, between a duplicate XSS and a complaint about markdown rendering. Severity: critical. Author: `@sable`. Title: *"Your guard is posted at a door you no longer use."*

Malika read it twice, then escalated. By ten it was on the war-room screen and nobody was talking.

> Any authenticated researcher token can `GET` and `POST` to `/api/v2/admin/programs/{id}/payout-settings`. I changed the bounty multiplier on a program I do not own. I changed it back. Reproduction below. Separately: your API reflects `Access-Control-Allow-Origin` from whatever `Origin` I send, with `Access-Control-Allow-Credentials: true`. A page I host may call your API from any researcher's browser and read the answer. Today it has no credential to send. The day a cookie rides along, it reads authenticated responses. s.

Leila set her coffee down carefully. "So our best researcher just told us that on our security platform, a researcher can rewrite the payout math for a bank's program. And that we are one cookie away from a website she controls reading other people's data."

"It's valid," Kenza said. She'd already run the reproduction. Her own JWT, a plain researcher token with no admin role, had sailed straight into the admin endpoint and come back `200 OK` with Northbeam's payout configuration.

Youssef was three terminals deep in access logs. "Give me twenty minutes. I want to know if anyone else found it first."

Rachid was at the Wall, at the Request Journey diagram born in March, the week Sable found the leaking stack traces. He picked up the green pen and drew a thick vertical line in front of the `DispatcherServlet` box, then a row of small boxes in front of the line, before any controller and before the front door Kenza had spent the spring rebuilding. The stretch he was drawing on was the one he'd tapped in March and left deliberately empty, *out past the filters*, where the same reporter's floor-plan leak had proved that what happens in front of the servlet never reaches a `@ControllerAdvice`.

"Security lives in front of the front door," he said. "I told you in March that chapter ten of your year would happen out here. Welcome to it."

## The investigation

Kenza pulled up the two suspects. The first was the route guard.

> ⏸ **Your call.** A signed, unexpired researcher token came back `200 OK` from an admin route: no exception, no error log, nothing visibly *broke*. Before the code loads, commit: what class of bug lets a request through without anyone deciding to allow it, and did it get there by error, or by drift?

```java
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/admin/**").hasRole("ADMIN")   // ← still guarding v1
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

She felt the bug before she could name it. *At Meridian the check would have been the first line of the handler itself, an `if` asking who you were before the method did anything. The guard traveled with the code it guarded.* Here the rule lived elsewhere, in a filter, matching a URL string. And the URL had moved out from under it.

"March," she said slowly. "When I versioned the public API. I moved the admin controllers under `/api/v2/admin/**`. The matcher still says `/api/admin/**`. So a request to `/api/v2/admin/programs/{id}/payout-settings` matches no specific rule at all. It falls through to `anyRequest().authenticated()`, and any researcher with a valid token *is* authenticated."

"Authenticated," Rachid said. "Not authorized."

"Right." She wrote it on the Wall under his vertical line. *Authn ≠ Authz.* "The token's real: signed, unexpired, a genuine Glasshouse researcher. Authentication asks *who are you*, and the answer was honest. Authorization asks *are you allowed*, and nobody asked, because the only rule that would have asked was guarding a door we stopped using in March."

(If you called *drift* at the pause: exactly. Nothing malfunctioned; the chain enforced precisely what it said, and what it said had stopped describing the system. Hold that distinction. Rachid prices it at the kitchen, later that afternoon.)

The second suspect was older and uglier: a config class with a 2021 commit and a comment that read `// unblock the SPA team`.

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOriginPatterns(List.of("*"));     // reflects ANY origin
    cfg.setAllowCredentials(true);                  // ...with credentials
    cfg.setAllowedMethods(List.of("*"));
    cfg.setAllowedHeaders(List.of("*"));
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", cfg);
    return source;
}
```

"Whoever wrote this," Kenza said, "thought CORS was an access-control list."

Youssef looked up from the logs. "It isn't?"

"No. CORS is a *browser* contract. For anything non-trivial the browser sends a pre-flight `OPTIONS` first: 'may a page from this origin send a `POST` with an `Authorization` header?' Then on the real response it reads `Access-Control-Allow-Origin` to decide whether the JavaScript that asked ever sees the body. Spring will answer `403` to a cross-origin request whose `Origin` is off the list, but a client that is not a browser can leave that header out or forge it, so the list only tells browsers what to permit." She tapped the screen. "Reflecting every origin *with credentials* tells every browser: let any website read authenticated Glasshouse responses. The only thing missing today is a credential the browser sends by itself. We've hung a sign on the door saying 'all welcome, bring your keys.'"

"No exploitation," Youssef announced, leaning back. "Six months of logs. Nobody reached `/api/v2/admin/programs/*/payout-settings` with a non-admin token except Sable last night, and you, eleven minutes ago. It's a breach that wasn't. She found it first."

The room exhaled. Then the second conversation started, the one Kenza would remember longer.

"We fix this quietly," someone from the exec side said. "Patch the matcher, rotate, move on."

Leila was already shaking her head, and so was Rachid. "We pay this one in full and we say thank you in public," Leila said. "A security platform that hides its own holes is a punchline waiting for the first researcher who *doesn't* report what they find. Trust is the product. The glass house only works if we let people see in."

She won. She usually won when she was right about money, and trust was money.

## What the senior sees

That afternoon Rachid caught Kenza by the kitchen, away from the room.

"Your plan. Change `/api/admin/**` to `/api/v2/admin/**` and ship."

"It would work," she said.

"You're fixing the hole. Fix the *class* of hole." He uncapped the green pen. "Next quarter someone adds `/api/v3`, or a new admin controller nobody updates a matcher for, and the silent fall-through to `authenticated()` opens again. The model is wrong, not the string."

Her Meridian instinct was no shelter either. Scattered per-handler `if` checks meant a forgotten `if` was a forgotten check: fail-open by omission, the same trap in different syntax. *So the bug is that the default is 'allow.'*

"Deny by default," she said. "Make the drift structurally impossible instead of carefully avoided."

"Good. And while you're in there, tell me what validates a token. Not 'Spring does it.' Show me the mechanism."

She walked it. "The `Authorization: Bearer` header gets pulled by the `BearerTokenAuthenticationFilter`, one of the boxes on your line, early in the chain with the other authentication filters. It hands the token to a `JwtDecoder`, Nimbus under the hood, which verifies the signature against the issuer's JWK set, cached so we're not calling out per request. Then the claim checks: issuer is who we trust, expiry hasn't passed, and audience is us and not some other service sharing the auth server. That last one isn't automatic. Signature, issuer, and expiry come as defaults, but the expected audience we have to declare ourselves, through the resource-server `audiences` property or our own validator. Only then does a `JwtAuthenticationConverter` map the claims to authorities. Our roles claim becomes `ROLE_*`, which is the only reason `hasRole` means anything, and the `Authentication` lands in the `SecurityContextHolder`, a `ThreadLocal`. At the other end, the `AuthorizationFilter` enforcing the URL rules runs last in the chain, once authentication has had its chance."

"And if the signature's forged?"

"The decoder throws, the filter's entry point answers `401`, and the request dies in the filter chain. The controller never runs. Security lives in filters so that a bad request never reaches the servlet."

"One more box that isn't on your line," Rachid said. "We validate the token. Who mints it?"

"Not us, and that's the half of OAuth2 people skip." *The token check she'd once hand-rolled in a servlet filter at Meridian had validated tokens the same codebase minted. Issuer and validator were one program, so the split never had a name.* "OAuth2 is a protocol with two roles. The authorization server, Keycloak in our cluster, is the thing that authenticates a person or a service and *issues* the token. We're the resource server: this API never sees a password, we only validate and enforce. How the token gets issued is a *flow*: authorization-code with PKCE for humans in browsers, client-credentials for service-to-service. The old implicit and password grants are deprecated because they leak credentials into places they don't belong. The JWT is just the token's *format*. OAuth2 is the contract about who mints it and who trusts it, which is why the DSL says `oauth2ResourceServer` and not `parseJwt`: it names our role in the protocol."

"Good. Now the question Imane's going to ask you at standup tomorrow." Rachid almost smiled. "Why is CSRF off?"

## The fix

She rebuilt it from the filter chain up and inverted the model.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .cors(Customizer.withDefaults())                  // explicit allow-list bean
        below
        .csrf(csrf -> csrf.disable())                     // ADR-010: no cookie
        credential
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers(EndpointRequest.to("health", "info",
        "prometheus")).permitAll()
            .requestMatchers("/api/v2/admin/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.GET, "/api/*/reports/**").authenticated()
            .requestMatchers(HttpMethod.POST, "/api/*/reports/**").authenticated()
            // ...and one explicit rule for every other route family, programs included
            .anyRequest().denyAll())                      // deny-by-default: silence
        means 403
        .oauth2ResourceServer(o -> o.jwt(j ->
            j.jwtAuthenticationConverter(jwtAuthenticationConverter())))
        .build();
}
```

`anyRequest().denyAll()` did the work. A new route was *unreachable* until someone wrote an explicit rule for it, so a forgotten path failed closed: `403`, not `200`.

The first rule names Actuator endpoints by id through Boot's `EndpointRequest`. `health` covers `/actuator/health` and every probe path under it, on whatever port or base path serves them. `info` and `prometheus` (the metrics scrape) open only once those endpoints are exposed.

The kitchen walkthrough had named the one claim check the framework wouldn't assume. The diff that delivered it was two lines of YAML.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.glasshouse.io/realms/glasshouse
          audiences: atrium   # ADR-010: the one claim validation the framework
                              # will NOT assume: a valid token minted for another
                              # service must bounce off this line
```

And `hasRole("ADMIN")` only meant something because the chain no longer ended in `Customizer.withDefaults()`. Out of the box the converter maps the `scope`/`scp` claim to `SCOPE_*`

authorities and nothing else: the default speaks `SCOPE_`, not `ROLE_`, and Keycloak's realm roles live in their own claim. The mapping was hers to declare, like the audience.

```
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    var roles = new JwtGrantedAuthoritiesConverter();
    roles.setAuthoritiesClaimName("roles");   // Keycloak realm roles, flattened into
        the token
    roles.setAuthorityPrefix("ROLE_");        // the default maps scope -> SCOPE_*,
        never ROLE_*
    var conv = new JwtAuthenticationConverter();
    conv.setJwtGrantedAuthoritiesConverter(roles);
    return conv;
}
```

But URL rules were coarse. "Any researcher can `GET` reports" wasn't the rule. The rule was "a researcher reads *their own* reports; a triager transitions status; an admin manages programs." That belonged on the domain methods, as defense in depth. Records crossed the boundary, not entities.

```
@Service
public class ReportServiceImpl implements ReportService {

    @PreAuthorize("hasRole('RESEARCHER') and @reportAccess.owns(#reportId,
        authentication)")
    public ReportView read(long reportId) { /* load, map to record */ }

    @PreAuthorize("hasRole('TRIAGER')")
    public void transition(long reportId, Status next) { /* ... */ }

    @PreAuthorize("hasRole('ADMIN')")
    public void updatePayoutSettings(long programId, PayoutSettingsUpdate update) { /*
        ... */ }
}
```

`@reportAccess` was a small bean SpEL could call by name. The ownership logic lived in one tested place instead of copy-pasted strings. None of it runs until the security configuration class carries `@EnableMethodSecurity`: without that annotation every `@PreAuthorize` is ignored and nothing fails.

She paused at `@PreAuthorize` and felt February all over again: the smoke-cloud drawing. "Method security is an AOP proxy," she told Imane, who'd wandered over. "Same proxy as `@Transactional`. If `ReportService` calls its own `updatePayoutSettings` through `this.`, the proxy never sees the call and the check evaporates." The February discipline already had it covered. ADR-003 had made `this.`-invoked annotated methods a review finding, so `@PreAuthorize` sim-

ply joined the list the smoke cloud predicted. She made a note for the tests anyway: assert the rule fires on the *injected* bean, never on a hand-constructed one.

The CORS fix was the smallest diff and the biggest mindset change: ACL out, named origins in.

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("https://app.glasshouse.io"));   // named, not
        reflected
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    cfg.setAllowCredentials(true);                                  // safe now: origin
        is fixed
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", cfg);
    return source;
}
```

Then came CSRF, Imane's standup question, exactly as Rachid predicted.

"Because JWT," Imane said.

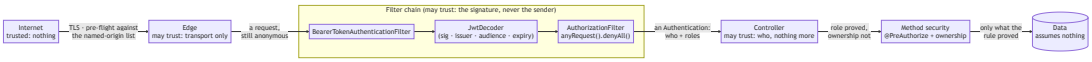
Rachid waited. He was good at waiting. Kenza let the silence work, then nudged. "Go a layer down. *Why* does a bearer token make CSRF moot?"

Imane thought. "CSRF works because… the browser sends the cookie automatically? On any request to our domain, even from an attacker's page?"

"That's it," Kenza said. "CSRF exploits *ambient* credentials: a session cookie the browser attaches to every request to our origin whether the user meant to send it or not. The attacker's page can't read the response, but it can *cause* a state-changing request, and the cookie rides along for free. That's why the CSRF token exists: it's a value that page can't obtain. But our credential lives in the `Authorization` header. JavaScript has to attach it deliberately; the browser never sends it ambiently. With no ambient credential there's no CSRF surface, so leaving it on would guard nothing. The resource-server configuration already skips the check for any request that carries a bearer token." She wrote the rule on the Wall: *cookie session → CSRF on; pure token API → CSRF off.* "Same logic kills session fixation. In the cookie world an attacker pre-plants a session ID and waits for the victim to log into it, which is why Spring Security rotates the ID at login. We have no session, so there's nothing to fixate. The skill isn't 'JWT is safer.' The skill is knowing which credential is ambient and defending *that*." The ADR recorded the rest: *off because no cookie credential; revisit the day a cookie appears.*

Before the Wall session broke up, Rachid handed her the green pen. "One more drawing: the architecture. Walk the request again, but this time don't tell me what each layer *does*. Tell me what each layer is allowed to *trust* from the one before it."

She started at the far left, where the Wall met the window.




"Zero standing trust," she said, stepping back. "Identity is cryptographic or absent, and silence past the matchers is a `403`. Each layer re-checks instead of trusting the one upstream: the chain settles *who*; the method layer re-asks *whether*, role and ownership, even though the request is already in. The chain authenticates, the method authorizes, the data layer assumes nothing."

"And the arrows. What crosses them?"

"Claims. Every arrow is a token boundary, and audience is the contract between zones, the claim naming which boundary the token was minted to cross. A token Keycloak issues for some *other* service carries a perfectly valid signature; it bounces off Atrium only because of the audience check, and that check is ours to declare, which is why it isn't a default."

"Now say the quiet part. What failed in July, the matcher or the model?"

"The model. Fail-open is never designed; it *accumulates*. Every refactor that outruns a URL string widens it, and nobody approves the purchase because nobody sees a diff. Deny-by-default makes fail-closed a *structural* property instead of a maintained one. That's the -ility on this wall. Security as architecture, not as vigilance: making the safe thing the lazy thing." She wrote that last sentence under the zones, and then, before it cooled, the half-page:

> **ADR-010: Security architecture. Context:** `@sable` filed a valid critical: an admin route reachable without a role check, plus a credentialed wildcard CORS policy. The URL rules had drifted from the routes they guarded. **Decision:** Atrium is a stateless OAuth2 resource server (Keycloak issues; Atrium validates signature, issuer, declared audience and expiry); the chain ends in `anyRequest().denyAll()`; method security adds role-plus-ownership rules as defense in depth; CSRF stays off for the token API *on the record* (no ambient cookie credential) and CORS is a named allow-list; access tokens run short with refresh rotation. **Consequences:** A forgotten route now fails closed (`403`) instead of open, at the price of an explicit rule for every new route. A leaked token stays valid until expiry, a trade-off accepted in writing rather than discovered in an incident.

Afterward, in the corridor, Rachid said quietly: "That. That's the job. Consider this your notice, months before anyone else gets theirs." And he told her, privately, long before any all-hands would, that he was stepping back at year's end, and that this rebuild had been hers for a reason.

The password path was boring in the best way: `DelegatingPasswordEncoder`, BCrypt with the `{bcrypt}` prefix at strength 12. "If this had been plaintext, or anything reversible," she told Imane, "the whole report would be a footnote. The storage itself would be the breach. BCrypt is deliberately slow, and the strength is a log scale: 12 means $2^{12}$ rounds, so each increment doubles the cost. Milliseconds per login for us; an economic dead end for whoever steals the dump. And the prefix means we can migrate to Argon2 later without invalidating a single existing hash."

The validation checklist went up on the Wall next to the filter boxes, *signature → issuer → audience → expiry → SecurityContext*, with the uncomfortable last line: a stolen token stays valid until it expires; statelessness cuts both ways.

### ⚖ Your move, architect

That last line is a fork, and ADR-010 had to take a side. Sable's *next* report, in some version of the future, is a leaked researcher token, and the question is the revocation posture. You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. Short TTL + refresh rotation, nothing else.** Access tokens live minutes; refresh tokens rotate on every use, and a replayed refresh token kills the whole family. The exposure window is priced and accepted.
- **B. A denylist on the hot path.** Revoked token IDs (`jti`) sit in a shared store, checked on every request. That is real revocation, today.
- **C. Opaque tokens + introspection.** The resource server asks the authorization server about every token it sees; Keycloak becomes the per-request source of truth.

**The trade-offs, since you've committed: B** gets real revocation but gives up **statelessness**. The shared store joins every request's critical path, adding latency and a new failure mode: when the denylist store is down, do you fail open or closed? You've just rebuilt the question this chapter answered. **C** gives up **availability**: the auth server joins every request's critical path. That is April's Cortexa lesson, where one synchronous dependency's bad afternoon spent the whole thread pool, the coupling ADR-006 taught the team to price. **A** is what they chose and what ADR-010 records: a *bounded* exposure window, fifteen minutes and in writing, in exchange for the horizontal scaling that justified JWTs in the first place. The mature answer is never "you can't revoke a JWT." It's "we decided not to, here is the price, and here is the receipt."

They shortened access tokens to fifteen minutes with refresh rotation, and recorded the heavier options (a denylist of revoked token IDs, live introspection) as the day-two path, each buying back control by selling statelessness.

She added one more mandate of her own, a security test per route: `401` without a token, `403` with the wrong role, `200` with the right one. It would have caught the drift the day it shipped. Youssef read the plan over her shoulder and said, mildly, "That's a few hundred new test classes on a CI already creeping past twenty minutes." Nobody heard him. They would, in August.

Sable's retest came back two days later, one line: *"Clean. And thank you for publishing. s."*

The postmortem went out publicly with Kenza's name on it. It covered the matcher, the CORS bean, the rebuild, the bounty paid in full. She left July carrying two things: a security model anyone could explain from the Wall, and a date in December nobody else knew yet.

## The debrief: Ch.10, The Breach That Wasn't

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 10.** JWT stateless auth: where in the filter chain is the token validated, and why is CSRF protection usually disabled?

Then the follow-ups, cold:

1. Your `anyRequest().authenticated()` let a researcher reach an admin route. What's the real fix?
2. How do you revoke a JWT before it expires?
3. Your tokens can't be revoked; defend that to a security review.
4. Who mints the token, and how does a caller get one?
5. 401 or 403, and who answers each?

Tick what you said:

- ☐ Claim: The token is validated in the security filter chain, before the `DispatcherServlet`. CSRF is off because no ambient cookie credential exists.
- ☐ Mechanism: `BearerTokenAuthenticationFilter`, then `JwtDecoder` (signature, issuer, expiry, and the audience you declared), then `AuthorizationFilter` at the end. The browser never sends a header token on its own.
- ☐ Trade-off: Stateless tokens scale horizontally with no session store, and you give up revocation: a token is good until expiry.
- ☐ Failure mode: A leaked token stays valid for its full TTL. The answers are short TTLs with refresh rotation, a denylist, or introspection.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** In a stateless JWT API the token is validated in the security *filter chain*, before the `DispatcherServlet` ever runs, and CSRF protection is disabled because there is no ambient cookie credential to forge. **Mechanism:** The `BearerTokenAuthenticationFilter` extracts the `Authorization: Bearer` header and hands it to a `JwtDecoder`, which verifies the signature against the issuer's cached JWK set, then checks issuer, expiry, and (opt-in) audience. On success a converter maps claims to authorities and populates the `SecurityContextHolder`, and the `AuthorizationFilter` at the end of the chain enforces the access rules. Browsers auto-attach session cookies cross-site, which is what CSRF exploits; a header token is never sent ambiently, so a CSRF token would guard nothing. **Trade-off:** Stateless tokens scale horizontally with no session store, but you surrender server-side revocation: the token is good until expiry no matter what you learn afterward. **Failure mode:** A leaked token stays valid for its full TTL, which is the JWT revocation problem. Mitigate with short TTLs plus refresh rotation, a per-request denylist of token IDs, or live introspection, each trading statelessness back for control.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Security** as *architecture*, not vigilance: trust zones with explicit token boundaries, and fail-closed as a structural property (`denyAll()`) versus fail-open as accumulated drift. The hole was opened by **evolvability** (a March refactor outrunning a URL string), which is why the fix changes the model. **Scalability** is what statelessness buys; the revocation window is what it costs, priced in writing. Youssef's unheard remark is **operability** debt accruing, due in August.
- **The ADR:** ADR-010, security architecture: stateless OAuth2 resource server (Keycloak issues; Atrium validates signature, issuer, declared audience and expiry), `anyRequest().denyAll()` at the end of the chain, method security with ownership rules as defense in depth, CSRF off *on the record*, CORS as a named allow-list, fifteen-minute tokens with refresh rotation.
- **Rejected alternatives:** patching the matcher string and shipping (fixes the hole, not the class of hole: the drift re-opens at `/api/v3`); per-route annotations as the *only* layer (fail-open by omission, the same drift in annotation syntax); a denylist or introspection today (each buys revocation back by selling statelessness or availability, and both are recorded as the purchasable day-two path instead).
- **Fight or lean:** *Lean* on the resource-server machinery: `BearerTokenAuthenticationFilter`, the Nimbus decoder, the cached JWK set. Hand-rolled token validation is how signature-bypass CVEs are born. *Fight* the permissive defaults you wrote yourselves (the wildcard CORS bean and the `authenticated()` fall-through were Glasshouse's ink, not Spring's), and remember the two checks the framework will *not* assume for you: the audience, and `denyAll()` itself.

**Story anchors**

- The matcher guarding `/api/admin/**` while the routes moved to `/api/v2/admin/**` → URL-string rules drift; `anyRequest().denyAll()` makes a forgotten route fail closed instead of open.
- The CORS bean reflecting any origin with credentials, "unblock the SPA team" → CORS is a browser contract, not a server ACL; name your trusted origins.
- Sable's researcher token coming back `200 OK` from the admin endpoint → authentication (who you are) is not authorization (what you may do).
- The trust-zone map on the Wall, "the chain authenticates, the method authorizes, the data layer assumes nothing" → zero-trust posture: each layer re-checks instead of trusting upstream, and the audience claim is the contract that says which boundary a token was minted to cross.

**Interview ammo**

- *Kill-shot: JWT stateless auth: where in the filter chain is the token validated, and why is CSRF usually disabled?* In the `BearerTokenAuthenticationFilter`, before the servlet: signature, issuer, expiry, opt-in audience, then `SecurityContext`. CSRF is off because a header token isn't ambient the way a cookie is, and it goes back on the day a cookie appears.
- *Follow-up: Your `anyRequest().authenticated()` let a researcher reach an admin route. What's the real fix?* Authenticated isn't authorized: invert to `denyAll()` with explicit allows so unknown routes fail closed, then layer `@PreAuthorize` ownership rules as defense in depth. Remember that method security rides the same self-invocation-prone AOP proxy as `@Transactional`.
- *Follow-up: How do you revoke a JWT before it expires?* You don't, by design. You pick a poison: short TTL plus refresh rotation (cheap, small exposure window), a denylist of token IDs (a per-request lookup), or introspection (full control, fully stateful again).
- *Architect follow-up: Your tokens can't be revoked; defend that to a security review.* Name the sale and show the receipt: ADR-010 prices a bounded fifteen-minute exposure window (refresh rotation with reuse detection killing the family) against the statelessness that lets the API scale with no shared store on the hot path. It also records denylist and introspection as purchasable day-two options, so "can't revoke" becomes "decided not to, in writing."
- *Follow-up: Who mints the token, and how does a caller get one?* The authorization server, here Keycloak, authenticates the person or the service and issues the token. Atrium is the resource server: its API never sees a password, it only validates and enforces. Humans in browsers use authorization-code with PKCE, service-to-service calls use client-credentials, and the implicit and password grants are deprecated because they leak credentials. JWT is only the token's format.

- *Follow-up: 401 or 403, and who answers each?* A missing token is a `401`, and so is a forged one, where the decoder throws and the filter's entry point answers. A valid token that no rule allows, whether the role is wrong or the route has no rule under `denyAll()`, is a `403`: the `AuthorizationFilter` at the end of the chain refuses it. Both are answered in the filter chain, so the controller never runs and no `@ControllerAdvice` shapes the response.

**Pointers** → `examples/ch02_security/`

---

# Chapter 11: Forty Minutes

August at Glasshouse was the month of the merge queue. Kenza noticed it the way you notice a limp, by how everyone had rearranged their walk around it. Imane scheduled lunch by the pipeline: push at 11:50, eat, return to a verdict. Newer hires batched unrelated changes into one PR "to save a run," so every red build was a murder mystery with four suspects. CI took forty minutes, and the team had stopped being angry about it, which was worse.

Youssef had priced this month back in July: *a few hundred new test classes on a CI already creeping past twenty minutes*. Nobody had heard him. In August everyone heard him at once.

Then, one Friday night, the leaderboard froze.

It took until Wednesday for anyone to notice, because the nightly reputation job had no metric, no alert, and the dignity to die quietly. A researcher posted a screenshot: *my points haven't moved since Friday*. Leila forwarded it with a single eyebrow of a sentence ("the leaderboard is researcher currency; please advise"), and Kenza found the stack trace in twenty minutes. Friday afternoon's migration had tightened the leaderboard's unique constraint to `(researcher_id, program_id)` so researchers could rank per program, and the nightly job's native upsert had failed every run since, correctly and loudly, into a log nobody read.

```
// LeaderboardRepository: the nightly reputation upsert
@Modifying
@Query(value = """
    INSERT INTO researcher_reputation (researcher_id, program_id, points)
    VALUES (:researcherId, :programId, :points)
    ON CONFLICT (researcher_id) DO UPDATE    -- constraint is now (researcher_id,
        program_id)
      SET points = researcher_reputation.points + EXCLUDED.points
    """, nativeQuery = true)
void upsertReputation(UUID researcherId, long programId, long points);
```

Postgres, finding no unique constraint matching `ON CONFLICT (researcher_id)`, refused the statement. The fix was one line, the conflict target widened to match the constraint, and it shipped before lunch. The horror was the test.

```
@SpringBootTest
class LeaderboardRepositoryTest {

    @Autowired LeaderboardRepository repo;
    @MockitoBean CortexaClient cortexa;            // copied from another test; unused
        here
    @MockitoBean NotificationPublisher notifications;

    @Disabled("H2 doesn't support ON CONFLICT. TODO: revisit")
    @Test
    void upsertReputation_accumulatesPoints() {
        // would have gone red the moment the constraint changed
    }
}
```

In the war room, Rachid asked for the file on the big screen. He had spent the summer mostly watching Kenza run things; he attended standup the way a lighthouse attends shipping. Now he stood, put on his glasses, and read the annotation aloud, slowly.

"*H2 doesn't support this. TODO: revisit.* Dated June." He looked over his glasses. "Of *last* year."

"Your test suite has been lying to you politely for fourteen months," he said into the silence. "It told you what H2 thinks. H2 doesn't take production traffic."

## The investigation

The disabled test was one symptom; Kenza wanted the cause. She pulled the suite's CI log that afternoon and read it the way Rachid had taught her, setting aside what had failed and asking what was really going on. What was going on was banners. The Spring Boot ASCII art scrolled past again, and again, and again.

Imane, radicalized by a summer of forty-minute waits, had already gone further. She appeared at Kenza's desk with a spreadsheet. "I audited one CI run," she said. "I grepped for context startups. Want to guess?"

"Six?"

"Seventeen. Seventeen separate Spring contexts boot in one run. Each one is eighty to ninety seconds: full component scan, every auto-configuration, a Hikari pool it mostly doesn't use, Flyway migrating an H2 database that isn't our database. And the scheduler, so the digest job *fires* mid-suite and fills the CI logs with failed email sends. Twenty-five minutes of our forty is just turning the application on and off."

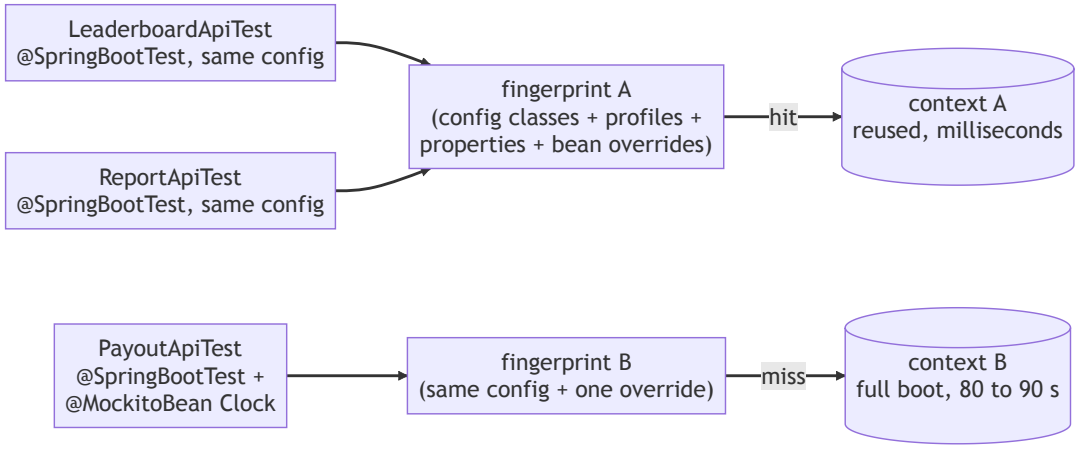
*Boot the whole app and test through it,* Kenza thought. *At Meridian that was a hand-rolled `@BeforeAll` that started Jetty and prayed, once per test class, because nobody had ever thought to share the fixture. Same tax, fancier collector.* "Why seventeen?" she asked. "Spring caches test contexts across the suite. It should boot once and reuse it."

"It *would*," Imane said. "Almost nothing hits the cache."

> ⏸ **Your call.** Two test classes boot the same app with the same profile and mint two separate contexts. Before the TestContext source opens: what could possibly make Spring treat them as *different applications*, what does that mean for forty-one classes' creative mock fields, and which -ility is bleeding out at twenty-five minutes a run? Commit.

They took it to the Wall. Kenza drew a box and labeled it the context cache key, because she'd spent an evening in the TestContext framework source and the answer was right there. The cache is a map. The key is the merged configuration fingerprint: the configuration classes, the active profiles, the test properties, the context initializers, the web environment, *and every bean override*. `@MockitoBean`, the annotation that replaced Boot's old `@MockBean` when Spring Framework pulled bean overriding into core, does more than hand you a Mockito mock. It registers a bean-override definition, and that definition is part of the key. The same key gets the cached context for free. Any difference in the key costs a full boot.

Under it she drew what the audit had been measuring, two classes paying nothing and one paying ninety seconds for a clock:




"So every test class that sprinkles its own creative combination of mocks," Kenza said, tapping Imane's spreadsheet, "is paying ninety seconds for the privilege of being unique."

The audit had it all: forty-one test classes, every one `@SpringBootTest`, copy-pasted from whichever class the author had open, each mutating the mock set just enough. This one mocked `CortexaClient`, that one mocked `CortexaClient` *and* the clock, a third added a `@TestPropertySource` for one assertion. That made seventeen distinct fingerprints and seventeen boots. The cache itself is an LRU with a default size of 32, so they weren't even close to evicting; they simply never collided. (If your pause answer was "the bean-override set is part of the key," you had the mechanism; the -ility takes the rest of the chapter to earn its name.)

"And the bugs still get through," Imane said, "because all seventeen of those contexts run on H2."

## What the senior sees

Kenza wrote the standard that week, and the first page was a drawing of a pyramid. Atrium's suite had the opposite shape. Every test sat in the heaviest, slowest tier, which made it a column, and thanks to H2 the column wasn't even load-bearing.

`@SpringBootTest` boots your *entire* `ApplicationContext`: every bean, every auto-configuration, security filter chains, schedulers. By default it runs against a mock servlet environment; a real port arrives only if you ask for `RANDOM_PORT`. It is the honest nuclear option, right for a handful of end-to-end smoke tests and wrong as the default unit of testing, because you pay whole-application startup to verify one controller's status code.

The slices exist to load *just enough*, and the discipline is knowing what each one deliberately leaves out. `@WebMvcTest` boots the `DispatcherServlet`'s world: controllers, Jackson converters, validation, `@ControllerAdvice`, plus Boot's MockMvc security setup, which in Boot 4 ships in `spring-boot-starter-security-test`. Nothing below it loads. There are no services, no repositories and no database, and collaborators arrive at the boundary as mocks.

Imane had hit the slice's sharp edge firsthand on Tuesday: her first `@WebMvcTest` security test sent a researcher token at an admin route and came back 200, where production says 403.

> **Your call.** The slice booted, MockMvc ran, the route exists. What didn't load, and why is that the slice working exactly as designed? Commit.

A slice tests a *layer*, not your wiring. `@WebMvcTest` does not load regular `@Configuration` classes, so her custom `SecurityFilterChain` was simply absent, and Boot's slice default (any authenticated request is allowed) let the researcher in. The rules came back the moment she `@Imported` the config, which is why the standard's example would carry `@Import(SecurityConfig.class)` like a load-bearing wall.

`@DataJpaTest` boots the opposite end (repositories, the `EntityManager`, a `DataSource`) and wraps each test in a transaction it rolls back at the end, so tests can't pollute each other. The rollback is also the slice's own failure mode. Every assertion runs inside one open persistence context, so lazy loading that would fail in production succeeds silently in the test. Kenza, owner of a 2 a.m. lazy-loading scar, wrote that warning herself. (May's thousand-query dashboard had taught her the mechanism: lazy loads only work while the persistence context is open; the slice's open-to-the-last-assertion transaction is the same amnesty in a test runner.) `@JsonTest` boots Jackson alone, for the serialization contracts.

The heuristic went in bold: **test behavior at the narrowest slice that exercises it, and don't test the framework at all.** Half the suite's "coverage" was asserting that Spring Data

could parse a method name or that Jackson could serialize a string. Spring's own tests already pass. Delete them and spend the budget on *your* behavior: say, an upsert's conflict target.

Rachid read the draft at the Wall that evening, green pen hovering. "Forty minutes, seventeen boots. Which -ility was paying?"

"Speed…" Kenza started, and stopped, because that was the developer answer. "Evolvability. The suite is what makes change cheap. Slow, and people batch changes; lying, and people stop believing red. Either way Atrium stops being safely changeable, and that's the only thing tests are *for*."

"And the part nobody writes down," he said, tapping the pyramid, "is that this drawing is an architecture artifact, not a testing one. Slice boundaries mirror system boundaries, so ADR-001's seams get audited for free. Imane tried the payout controller in a web slice on Tuesday; it dragged half the service layer in through a static helper. If you cannot slice-test a feature, the feature's seams are lies." He wrote that sentence under the pyramid and initialed it. ADR-001's seams were the ones he meant (package-by-feature, dependencies pointing one direction, the boundary speaking contracts), and a slice has to cut along exactly those lines.

And the H2 problem deserved its own page. H2's Postgres compatibility mode mimics syntax, not semantics: no `ON CONFLICT … DO UPDATE`, no real `JSONB`, different locking, different type behavior. A suite that runs on H2 verifies a database you don't ship. Kenza had a scar in this exact shape. *Meridian did this to itself: HSQLDB standing in for the real database to make the suite fast, green for a year, dead on the first vendor-specific MERGE in production. The instinct that an in-memory database is "close enough" is a trap, and it doesn't care whose SQL you hand-wrote.*

## The fix

The redesign landed in three moves, and Kenza ran it as the first engineering-practice change with her name on it instead of Rachid's.

Move one was to push everything down a layer. Controller tests became `@WebMvcTest`. That also paid off the per-route security-test mandate from the breach-that-wasn't (a stale matcher had let admin routes fall through to plain `authenticated()`, so any researcher token walked in; the post-mortem demanded a 401/403 test per route), because a 403 that costs ninety seconds to verify gets skipped, and a 403 that costs two hundred milliseconds gets written:

```
@WebMvcTest   // all controllers, one shared slice context
@Import({SecurityConfig.class, SharedMocks.class})
class AdminRouteSecurityTest {

    @Autowired MockMvc mvc;

    @Test
    void researcher_cannot_read_payout_settings() throws Exception {
        mvc.perform(get("/api/v2/admin/programs/77/payout-settings")
                .with(jwt().authorities(new
        SimpleGrantedAuthority("ROLE_RESEARCHER"))))
           // a 403 thrown in the filter chain never reaches @ControllerAdvice
           .andExpect(status().isForbidden());
    }

    @Test
    void anonymous_gets_401_not_a_stack_trace() throws Exception {
        mvc.perform(get("/api/v2/admin/programs/77/payout-settings"))
           .andExpect(status().isUnauthorized());
    }
}
```

`MockMvc` drives the request through the mock servlet stack, with no socket and no Tomcat. That's why it's fast, and enough for almost everything. The dozen true end-to-end smokes that survived as `@SpringBootTest(webEnvironment = RANDOM_PORT)` used `RestTestClient` instead, because those *should* speak real HTTP over a real port. Boot 4 no longer registers a test client by itself, so each smoke class adds `@AutoConfigureRestTestClient`.

Move two was one fingerprint to rule them out. The seventeen creative mock combinations collapsed into `SharedMocks`, a single `@TestConfiguration` of Mockito `@Bean` factories that every test in a tier imports identically:

```
@TestConfiguration(proxyBeanMethods = false)
class SharedMocks {

    @Bean TriageWorkflow triageWorkflow() { return Mockito.mock(TriageWorkflow.class);
        }

    @Bean CortexaClient cortexaClient() { return Mockito.mock(CortexaClient.class); }

    @Bean Clock clock() {
        return Clock.fixed(Instant.parse("2026-08-17T10:00:00Z"), ZoneOffset.UTC);
    }
    // …and the rest of the canonical set: every collaborator a controller touches,
        mocked once
}
```

A web slice doesn't load services at all, so these beans override nothing. They are the missing collaborators, supplied identically. The config class sits in the cache key the same way for every test that imports it. Scattered `@MockitoBean` fields would each have minted a fresh fingerprint. *The oldest rule in testing,* Kenza thought. *Share the expensive thing, or buy it once per class. Meridian's suite had bought Jetty seventeen times a run for years and called it thoroughness.* One more deliberate trade-off: the web slice loads *all* controllers rather than one per test class, because `@WebMvcTest(SomeController.class)` puts the controller selection in the cache key. Twelve scoped slices is twelve boots; one slightly fatter slice is one. A side effect nobody mourned: slices don't boot the scheduler, so the digest job stopped spamming CI with failed emails.

The standard drew the mock line as governance, not taste: doubles for the *ports* (clocks, vendor clients, workflows, anything Atrium talks to) and the real thing for the *dialect*, the engine Atrium ships on. Mock Postgres away and you are testing a different production; the leaderboard had spent fourteen months proving it.

Move three was to test the database you ship. The data tier moved to real Postgres via Testcontainers, with one shared base class and one container for the whole suite:

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE)   // keep the real Postgres; don't
        swap in H2
abstract class PostgresDataTest {

    @ServiceConnection   // Boot reads host/port/credentials straight off the container
    static final PostgreSQLContainer POSTGRES =
            new PostgreSQLContainer("postgres:17-alpine");

    static { POSTGRES.start(); }   // started once per JVM, shared by every subclass
}

class LeaderboardRepositoryTest extends PostgresDataTest {

    @Autowired LeaderboardRepository repo;

    @Test  // un-disabled after fourteen months
    void upsertReputation_accumulatesPoints_perProgram() {
        repo.upsertReputation(RESEARCHER, PROGRAM, 50);
        repo.upsertReputation(RESEARCHER, PROGRAM, 25);
        assertThat(repo.pointsFor(RESEARCHER, PROGRAM)).isEqualTo(75);
    }
}
```

`@ServiceConnection`, in Boot since 3.1, does the wiring that once took a hand-rolled `@DynamicPropertySource`. The container's coordinates are published as a `JdbcConnectionDetails` bean that outranks `spring.datasource.*`; nothing is written to the `Environment`. One subtlety the slice forces: `@DataJpaTest` defaults to *replacing* the `DataSource` with an embedded database (the very H2 you're escaping), so you pin the real one with `@AutoConfigureTestDatabase(replace =`

`NONE`). (Recent Boot can recognize a service-connection `DataSource` and back off on its own; stating it is the version-proof move.) Flyway runs the real migrations against the real engine. Each test still rolls back, so the container stays clean. One refinement stayed strictly local: `.withReuse(true)` on the container, paired with `testcontainers.reuse.enable=true` in `~/.testcontainers.properties`, lets the container outlive the JVM itself, so the next run finds it instead of starting fresh. Imane flipped it on for her laptop and left it off in CI, where environments are throwaway and a container that survives between runs is just stale state with a name.

The first run pulled the Postgres image, and then Kenza allowed herself one piece of theater. The hotfix had shipped weeks ago, but CI deserved its rematch: she checked out the revision that had frozen the leaderboard and ran the resurrected `ON CONFLICT` test against it. It went red, the escaped prod bug caught by the suite at last. Youssef watched the failure scroll by with something close to affection. "First time the suite has disagreed with H2 and been right." On main it was green.

That Friday, the team gathered around one laptop while Kenza deleted the H2 dependency from the `pom.xml`. The commit message entered team lore on contact: "Remove H2. It was never Postgres."

The standard needed a number, Rachid said, or it would stay an opinion, and opinions get relitigated; binders don't. Kenza condensed the week to half a page and dated it:

> **ADR-011: Test architecture. Context:** Forty-minute CI; every test class `@SpringBootTest`; seventeen context fingerprints minted by scattered `@MockitoBean` fields; H2 masking Postgres truths. The suite was both slow and untrustworthy. **Decision:** Test behavior at the narrowest slice that exercises it (`@WebMvcTest` / `@DataJpaTest` / `@JsonTest`); shared test-config classes so context fingerprints converge, under a cache-hit budget tracked in CI; Testcontainers Postgres for anything SQL-shaped; a handful of full `@SpringBootTest` journeys; security's 401/403 paths tested at the web slice. **Consequences:** CI in minutes and failures that mean something; the accepted cost is slice literacy (knowing what each slice loads and pointedly does not) as a team skill.

### ⚖ *Your move, architect*

The standard was written; the question was how to land it on a living codebase, with forty-one legacy test classes, a roadmap that wasn't pausing, and a team mid-limp. Commit before reading on:

- **A. The big-bang sprint.** Two weeks, zero features: rewrite all forty-one classes in one push; the team relearns testing in one bite.
- **B. The ratchet.** Every *new* test follows ADR-011 starting today; the CI dashboard graphs context count and cache hits; old tests migrate opportunistically, as PRs touch them.

- **C. Freeze, then contracts first.** Moratorium on new `@SpringBootTest`, and build a consumer-driven contract-test layer before touching internals.

**The trade-offs, since you've committed: A** buys a clean break at maximum **cost**: two weeks of zero features in one lump, every open branch rebasing against a moving test tree, and the lesson as one indigestible bite. **B** is what Kenza chose, the **evolvability** play applied to the suite itself. The fitness function makes regression impossible to commit unnoticed while the debt drains at the rate the code is touched. Its tax is a bilingual suite for months and slice literacy taught PR by PR, forty-one small lessons instead of one lecture. (Week one attacked the seventeen fingerprints, mostly by deleting creative mock fields, hence numbers within days.) **C** spends today protecting a **compatibility** nobody consumes yet. Contracts-first is right when external consumers make the API the asset to protect; Atrium's consumers are mostly its own SPA, so a contract layer is scaffolding for clients that don't exist. The signal that flips the answer is the day Glasshouse ships a partner API with versioned consumers Kenza doesn't employ, a day Chapter 14 starts arguing about.

The numbers came in over the next week: four context boots instead of seventeen (one E2E, one web slice, one data slice, one `@JsonTest`) and CI at nine minutes. Imane built a CI-duration panel and announced she'd be treating it like an SLO. Any PR that added a fifth fingerprint would hear from her, by way of two artifacts she wired in that same afternoon. The first made the cache testify:

```
# src/test/resources/application-test.properties: make the cache testify
logging.level.org.springframework.test.context.cache=DEBUG
# every run now logs: "...DefaultContextCache... size = 4, hitCount = 312, missCount =
4"
```

The second made ADR-011's budget executable, three lines of CI standing where a code-review opinion used to:

```
# CI step: the context budget with teeth (ADR-011, target 4)
BOOTS=$(grep -c ":: Spring Boot ::" target/test.log)   # one banner per boot: Imane's
        audit grep, promoted to a gate
[ "$BOOTS" -le 4 ] || { echo "context budget blown: $BOOTS boots"; exit 1; }
```

The `@Disabled`-with-TODO pattern was replaced in the standard by one sentence Rachid approved without a single green mark: *if you can't test it, you can't ship it.*

A fitness function asserts a property of the architecture, not a behavior of the code (ADR-002's whole move), and a slow suite is a signal that the boundaries leak. Beside the panel went the cache-hit budget: contexts-per-run, target four. CI minutes are a cost line, capacity planning for the dev loop. The panel was January's red build naming its own law, pointed at a clock. Leila did the only arithmetic she ever needed: "Thirty-one minutes a run, forty runs a day. You found us a hire."

"Nine minutes," Youssef said at the brick handover, in the tone he reserved for compliments he intended to weaponize. "So now the tests can see Postgres. Shame nobody can see *prod*. That metrics ticket from the spring turns five months old soon. I'd bake it a cake, but we have no dashboard to put the candles on. And I'm still waiting on my lock-wait graphs from May." Kenza laughed, and wrote both down, and did not yet know she would remember the sentence at 3:04 one September morning, at her kitchen table, in the dark.

## The debrief: Ch.11, Forty Minutes

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 11.** `@WebMvcTest` vs `@SpringBootTest`: what loads, what doesn't, and why does context caching matter at scale?

Then the follow-ups, cold:

1. What exactly makes the test-context cache miss or evict?
2. Why not just run H2 in Postgres compatibility mode?
3. CI takes forty minutes and the org offers to buy faster runners; what's the architecture answer?
4. `@MockBean` or `@MockitoBean`, and what does either cost?
5. MockMvc or a real port?

Tick what you said:

- ☐ Claim: Prefer the narrowest slice and reserve `@SpringBootTest` for a handful of smoke tests.
- ☐ Mechanism: `@SpringBootTest` boots the entire `ApplicationContext`, `@WebMvcTest` only the MVC layer with collaborators mocked. Contexts are cached by configuration fingerprint, bean overrides included.
- ☐ Trade-off: Slices are fast and isolated but cannot catch wiring bugs across layers, so a few full boot tests stay.
- ☐ Failure mode: Every class on `@SpringBootTest` with its own mock combination: unique fingerprints, almost nothing hits the cache, and CI spends most of its runtime booting.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Prefer the narrowest slice; reserve `@SpringBootTest` for a handful of end-to-end smokes. **Mechanism:** `@SpringBootTest` boots the entire `ApplicationContext` (full component scan, all auto-configuration, security, schedulers), while `@WebMvcTest` boots only the MVC layer (controllers, converters, validation, advice, MockMvc security) with collaborators mocked, and `@DataJpaTest` boots only repositories plus a rollback-per-test transaction. The TestContext framework caches contexts across the suite, keyed on the merged configuration fingerprint: config classes, profiles, properties, initializers, web environment, and the full set of bean overrides (`@MockitoBean`). **Trade-off:** Slices are fast and isolated but can't catch cross-layer wiring bugs, so you keep a few full-boot tests for those. **Failure mode:** A 2,000-test suite where every class is `@SpringBootTest` with its own creative mock combination produces dozens of unique fingerprints; almost nothing hits the cache, and CI spends most of its runtime booting and shutting down the same application: forty minutes of pipeline for fifteen minutes of tests.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Evolvability**, via its enabler. The suite is the system's *second architecture*, the one that makes changing the first cheap: slow, it taxes every change (people batch); lying, it taxes every belief (people ship on H2's word). And **cost**: CI minutes are money and attention, and the cache-hit budget is the dev loop's capacity plan.
- **The ADR:** ADR-011, test architecture: the narrowest slice that exercises the behavior; shared test-config so fingerprints converge under a CI-tracked cache-hit budget; Testcontainers Postgres for anything SQL-shaped; a few full `@SpringBootTest` journeys; 401/403 at the slice.
- **Rejected alternatives:** H2 in Postgres compatibility mode (it mimics the syntax without the semantics, so it verifies a database you don't ship); bigger CI runners (pays for seventeen boots instead of deleting thirteen); mocking repositories in data tests (the dialect is the one thing that must stay real, and the doubles policy is governance); the big-bang rewrite sprint (two weeks of zero features, one indigestible lesson).
- **Fight or lean:** *Lean* on the TestContext cache, the slices, and `@ServiceConnection`. Boot's test machinery is built to be cheap if you stop being creative; the cache's price is uniform configuration. *Fight* the comfortable lies: H2-as-Postgres, `@Disabled`-with-TODO, copy-pasted mock sets. Each is a convenience that spends an -ility nobody approved.

**Story anchors**

- The fourteen-month-old TODO read aloud → a `@Disabled` test reads as a deferral and works as a standing decision not to know; H2-vs-Postgres divergence is exactly what it chose not to know.

- Imane's spreadsheet of seventeen boots → the context cache key: config classes + properties + profiles + every bean override; one shared mock config is the speed lever.
- "Remove H2. It was never Postgres." → Testcontainers + `@ServiceConnection`: test against the engine that takes production traffic, and the resurrected test catches the escaped bug.
- Imane's payout controller that wouldn't fit a web slice → slice boundaries mirror system boundaries; a feature you can't slice-test has lying seams, and the suite audits ADR-001 for free.

**Interview ammo**

- *Kill-shot: `@WebMvcTest` vs `@SpringBootTest`: what loads, what doesn't, and why does context caching matter for a 2,000-test suite?* A slice loads only MVC infrastructure with mocked collaborators, and a full boot loads everything. The cache keys on the configuration fingerprint, mock set included, so uniform test configuration is the difference between four boots and seventeen.
- *Follow-up: What exactly makes the test-context cache miss or evict?* Any change to the merged configuration (classes, active profiles, test properties, initializers, web environment, or the bean-override set) is a new key and a fresh boot; `@DirtiesContext` evicts outright, and the cache is LRU with a default size of 32.
- *Follow-up: Why not just run H2 in Postgres compatibility mode?* Compatibility mode mimics syntax, not semantics: `ON CONFLICT … DO UPDATE`, `JSONB`, locking, and type behavior all diverge, so green on H2 verifies a database you don't ship. A Testcontainers Postgres shared across the suite costs seconds after the first image pull.
- *Architect follow-up: CI takes forty minutes and the org offers to buy faster runners; what's the architecture answer?* Hardware pays for the boots, design deletes them: converge fingerprints, test at the narrowest slice, track cache hits as a budgeted fitness function, and only then spend money. CI minutes are a cost -ility; don't buy hardware to subsidize a leaking architecture.
- *Follow-up: `@MockBean` or `@MockitoBean`, and what does either cost?* `@MockitoBean` replaced Boot's old `@MockBean` when Spring Framework pulled bean overriding into core. It registers a bean override, and the override set is part of the context cache key, so every new combination of mock fields boots a new context. One shared `@TestConfiguration` of `Mockito.mock(...)` beans gives every test in a tier the same key.
- *Follow-up: MockMvc or a real port?* `MockMvc` drives the request through the mock servlet stack with no socket and no Tomcat, which is why it is fast and enough for almost everything. `@SpringBootTest` also runs on a mock servlet environment unless you ask for `RANDOM_PORT`. Keep the real port for a handful of smokes that should speak real HTTP, driven by `RestTestClient`.

**Pointers** → `examples/ch11_testing/` · **run it:** `labs/lab-03-testcontainers-datajpatest` (a `@DataJpaTest` pinned to real Postgres runs an `ON CONFLICT` upsert H2 can't; needs Docker)

---

# Chapter 12: Three A.M.

The brick went off at 3:04 a.m. with the sound of a smoke detector being murdered.

Kenza was awake before she understood why, phone in hand, reading the page in the blue dark of her kitchen: `SYNTHETIC CHECK: glasshouse.io p95 > 8s (threshold 2s)`. Below it, forwarded by the overnight support rota, was a screenshot of a researcher's post: *is @glasshouse down or just thinking about it*.

She opened three terminals at the kitchen table and started looking for the system.

`kubectl get pods` said everything was `Running` and `Ready`: eight Atrium pods, green across the board and lying by omission. The logs were a firehose, eight pods' worth of interleaved lines with no request IDs and no way to tell which line belonged to which user's eight-second page load. She pulled a thread dump through `kubectl exec` and `jcmd` and found a couple of dozen threads waiting on socket reads toward Postgres. That was suspicious, except she had no idea what *normal* looked like at 3 a.m., because nothing recorded it. Was the connection pool exhausted? There was no gauge to ask. Was latency bad everywhere or only on the dashboard? There were no percentiles. There were no dashboards at all.

*At Meridian I'd have pulled up the metrics page I hand-rolled myself. Crude, but I knew what normal looked like on it,* she thought. *Here I have `kubectl`, a thread dump, and vibes.*

The Canary sat on the table. House rules were house rules, even alone, even at 3:40 a.m. She explained the symptoms to it out loud, in full, calm sentences: site slow, pods healthy, logs uncorrelated, threads waiting on the database, no evidence for or against anything. The Canary, dented and yellow, declined to escalate.

At 4:40 the synthetic checker went green. There had been no deploy and no restart, and nobody had taken any action. Kenza watched it stay green for forty minutes, then typed the worst sentence an engineer can put in an incident channel: `resolved, cause unknown`.

She did not go back to sleep.

## The investigation

The postmortem was at ten. Rachid let her walk the timeline (page, terminals, dump, recovery) without interrupting once. Then he asked his only question.

"What did you know, and when did you know it?"

"Nothing," Kenza said. "And too late."

He nodded slowly, like she'd passed a test she'd rather have failed. "Then the incident isn't the finding. Write that down as finding number one: *the system recovered and we cannot say why.*"

Youssef had brought a prop. He read it aloud: backlog ticket `ATR-1207`, *"Real application metrics,"* filed the afternoon of the promo-day meltdown, the day nearly two hundred parked

threads per pod took the site down while CPU idled and the only evidence was a thread dump. He read the subtitle too, slower. It was that postmortem's most brutal line, and overnight it had come true again: *"we graphed the LB because we had nothing else."* "Five months old," he said. "For the record: I have been making this joke since April, and it has stopped being funny."

"Then let's stop promising 'monitoring,'" Rachid said. "Monitoring is a stack, the thing you install after you've failed to design. Observability is a *property*: for every user-facing flow, what will 3 a.m. ask, and is the answer already being recorded? Last night the question was *slow everywhere, or slow somewhere*, and the answer was recorded nowhere."

"And the contract that makes it negotiable," Kenza said, the shape arriving as she spoke, "is an SLO per flow, not per server: *99.9 percent of submissions complete inside two seconds, over thirty days*. That's a sentence Leila can hand Northbeam, and it gives us an error budget: forty-three minutes a month. While it's healthy we ship features; when it burns, reliability jumps the queue, because a number we all signed says so."

Leila wrote the figures down without being asked. "I have spent two years arguing about reliability in adjectives. Give me arithmetic and I'll sign it." She did, one objective each for submission, triage and payouts. Her business case still fit in one line: "Northbeam's security team asked me, in writing, *how would you know if this happened during business hours?* I need an answer that isn't 'Twitter.'"

Three days later the cloud provider's graphs supplied the cause they couldn't see: a storage latency spike on one Postgres node, 03:01 to 04:38, overlaying the checker's red window almost perfectly. Everything waiting on that node had been slow. The storage healed, and Glasshouse healed with it. It was recovery without understanding, which is the worst kind.

The roadmap moved: observability stopped being a backlog ticket and became the sprint.

## The cluster kills the truth-tellers

One week into the buildout, the universe supplied a second lesson, free of charge.

The team had restored the database `HealthIndicator`, the one that had famously vanished under the prod profile back in March, when a dependency bump moved a class and `@ConditionalOnClass` backed off without a sound. It dutifully reported `DOWN` whenever Postgres misbehaved. Somewhere in the enthusiasm, a well-meaning commit wired the full health endpoint into the Kubernetes *liveness* probe:

```
# deployment.yaml: the well-meaning commit
livenessProbe:
  httpGet:
    path: /actuator/health   # the aggregate, which includes the database indicator
    port: 8080
  periodSeconds: 10
  failureThreshold: 3        # three misses × 10s → the kubelet kills the pod
```

> **Your call.** The indicator is honest and the YAML is idiomatic, so walk it forward: what happens to eight healthy pods the next time Postgres stumbles for ninety seconds, and which -ility is about to be destroyed by its own safety machinery?

At 12:51 on a Tuesday, Postgres had a ninety-second blip: a failover hiccup that healed itself, the kind of thing a mature system shrugs off. Every Atrium pod truthfully reported `DOWN`. Thirty seconds later the kubelets, following orders, executed all eight of them.

The blip had passed before the first replacement pod finished starting. But now eight cold JVMs were warming at once, and eight Hikari pools were refilling against a database still recovering. New pods answered their first health checks slowly. Some were killed *again*, mid-warmup, because no startup probe shielded them. A ninety-second database hiccup became a twenty-two-minute restart storm, and the health checks themselves were now the outage.

Youssef watched pods cycle on the war-room screen and delivered the line that would end up in the handbook: "The cluster is killing pods for telling the truth."

(If you ran the ledger at the pause, Postgres spent ninety seconds of availability and the probe wiring spent twenty-two minutes. That gap is an **operability** failure: machinery making decisions nobody had designed.)

## What the senior sees

Back in March, reviewing the restored indicator, Youssef had asked where Kubernetes should point at it (*liveness, readiness, we should fight about that properly someday*), and nobody had picked the fight up. It was September now, and the invoice ran to twenty-two minutes.

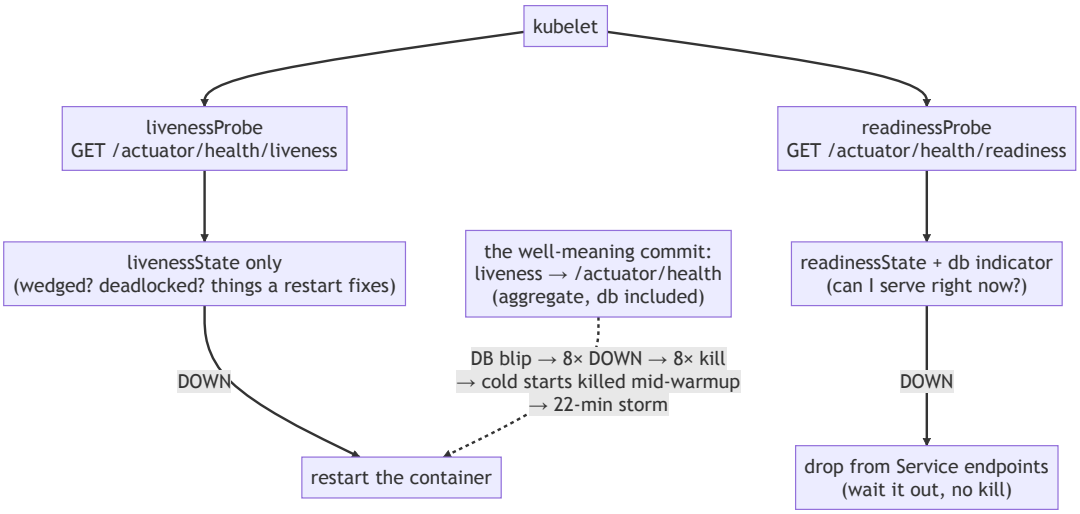
Youssef took a whiteboard marker and wrote two questions, far apart.

"Liveness asks: *should I be restarted?* The only honest reasons to say yes are about *me*: I'm deadlocked, I'm wedged, my state is unrecoverable. Things a restart fixes." He tapped the second question. "Readiness asks: *should traffic be routed to me?* That's where dependencies live. If the database is down, the correct move is to stop sending me requests and *wait*, not to shoot me. Restarting a pod has never once fixed a remote database. It just adds a stampede of cold starts to the recovery."

"So liveness must never include a dependency," Kenza said. "Because the kubelet's only remedy is the one remedy that can't help."

"And makes it worse," Youssef said. "Failure handling that amplifies the failure."

He drew the doctrine between his two questions, the well-meaning commit's wiring dashed across it like a crime-scene outline:




The fight, six months deferred, lasted about four minutes. Probe doctrine is only debatable before you've watched a kubelet execute eight pods for honesty. The drawing stayed on the Wall, beside the Request Journey, the team's socket-to-DB map of a single request.

Rachid, from the doorway: "Show me the mechanism on Boot's side."

Kenza had read it the night before. "Boot has modeled this since 2.3. The application publishes its own `LivenessState` and `ReadinessState`, and Actuator exposes them as health *groups*: `/actuator/health/liveness` and `/actuator/health/readiness`. Each group is configurable, so you decide which indicators feed which probe: liveness gets the process's own state and nothing else; readiness is where our database indicator belongs. The indicator was fine. Our mistake was pointing the wrong probe at the aggregate."

*That was the trap in my hand-wired instinct,* she realized. *At Meridian the health check was a servlet I typed myself, and I chose every probe it ran because I'd written every line of it. Here the indicator was auto-discovered into an aggregate I'd never read: convenience, billing me later.*

The same Wall session covered what Actuator was: a family of endpoints (`/health`, `/info`, `/metrics`, `/env`, `/beans`). Each is an `@Endpoint` bean, and almost all of them are unexposed over HTTP by default, opted in through `management.endpoints.web.exposure.include`. The default is deliberate. `/env` reveals your entire configuration surface (values are masked by default in current Boot, but the keys alone tell an attacker exactly what to probe for), and `/heapdump` serves the contents of memory. On a security platform, an exposed actuator was a Stonethrow submission waiting for Sable's byline: Glasshouse's bounty program pointed at itself, and the researcher it had already paid once for one leaky error page.

Then Micrometer. "It's a facade," Kenza explained at the Wall, Imane taking notes. "You record against a `MeterRegistry`; the registry implementation decides the backend. Three meter types do ninety percent of the work. A counter only goes up: it counts events, like reports submitted. A timer records durations *and* counts, and can publish percentile histograms, which is

where p99 comes from. A gauge samples a value that goes up and down, like active connections in the Hikari pool. Prometheus *pulls*: it scrapes `/actuator/prometheus` on the interval its own server config sets and stores the time series. That's every fifteen seconds in Glasshouse's deployment, though Prometheus ships defaulting to one minute. We don't push anything; we hold the meters in memory and answer the door."

## The fix

The configuration came first, as a policy argued in review and recorded in an ADR:

```
management:
  server:
    port: 8081                # management on its own port; the ingress never
  endpoints:                  # routes it, cluster-internal only
    web:
      exposure:
        include: health, info, prometheus   # /env, /beans, /heapdump stay dark
  endpoint:
    health:
      show-details: when-authorized
      probes:
        enabled: true         # Boot's default since 4.0: /health/liveness + /readiness
      group:
        readiness:
          include: readinessState, db   # dependencies live HERE, not in liveness
server:
  shutdown: graceful          # Boot's default since 3.4, pinned explicitly anyway
spring:
  lifecycle:
    timeout-per-shutdown-phase: 20s
```

The deployment's liveness probe moved to `/actuator/health/liveness` on the management port, a check that is process-only and dependency-free. Readiness got `/actuator/health/readiness`, database indicator included. One trade-off was written beside the YAML: the management port is a separate web server, so a probe can pass there while the main port has stopped accepting requests, and `management.endpoint.health.probes.add-additional-paths=true` is the Boot switch that also serves the two groups on the main port, as `/livez` and `/readyz`. A Postgres blip now meant pods stepped out of rotation, then back in when the storm passed. The database indicator completed its strange little arc: vanished in March, restored, mis-wired into the wrong probe, and finally placed correctly.

The indicator itself was the sprint's smallest artifact, a dozen lines against the read replica that returned the one number Kenza had wanted at her kitchen table:

```
@Component("dbHealthIndicator")   // claims the "db" slot in the readiness group
class ReplicaHealthIndicator implements HealthIndicator {
    private final JdbcTemplate jdbc;
    ReplicaHealthIndicator(JdbcTemplate jdbc) { this.jdbc = jdbc; }
    public Health health() {
        try {   // one cheap row: probes fire constantly, so this must cost ~nothing
            Long lag = jdbc.queryForObject(
                "SELECT extract(epoch FROM now() - pg_last_xact_replay_timestamp())",
        Long.class);
            if (lag == null) lag = 0L;   // NULL on a primary, and withDetail rejects
        null
            return Health.up().withDetail("replicaLagSeconds", lag).build();
        } catch (Exception e) {
            return Health.down(e).build();   // readiness DOWN → out of rotation, not
        executed
        }
    }
}
```

Graceful shutdown is Boot's default these days, which is exactly why nobody had ever watched it work, so the team verified it. Youssef drew the failure mode: Kubernetes sends `SIGTERM` and removes the pod from the Service's endpoints *concurrently*. The removal can lose the race, and then fresh requests are steered at a pod already shutting down. A five-second `preStop` sleep let the routing layer catch up. Graceful shutdown let in-flight requests finish, up to the timeout. They killed a pod under load and watched zero requests fail. Then they did it again, because the first time felt like luck.

Metrics came next, with a rule: business metrics ship alongside system metrics, because "the site is slow" is a symptom but "triage latency tripled" is a finding:

```
@Component
public class TriageMetrics {

    private final Counter reportsSubmitted;
    private final Timer triageLatency;

    public TriageMetrics(MeterRegistry registry) {
        this.reportsSubmitted = Counter.builder("glasshouse.reports.submitted")
                .tag("channel", "api")
                .register(registry);
        this.triageLatency = Timer.builder("glasshouse.triage.latency")
                .description("submission to triage decision")
                .publishPercentileHistogram()
                .minimumExpectedValue(Duration.ofMinutes(1))   // the default buckets
        run 1ms to 30s;
                .maximumExpectedValue(Duration.ofDays(7))      // a triage decision
        takes hours
                .register(registry);
    }

    public void reportSubmitted() { reportsSubmitted.increment(); }

    public void triaged(Duration timeToDecision) {
        triageLatency.record(timeToDecision); }
}
```

Each signed flow got one board, with the golden signals (latency, traffic, errors, saturation) beside the business meters that priced them, because an SLO the team couldn't watch burning was a contract it couldn't keep.

In a week the Grafana boards became the thing Kenza had ached for at her kitchen table: request latency percentiles per endpoint, JVM memory, and the Hikari gauges (`active`, `idle`, and `pending`), five months after the ticket was filed. Beside them sat the Postgres lock-wait panel Youssef had demanded the day `@Version` shipped (optimistic locking: stale writers get a conflict exception instead of holding row locks). "Next time someone proposes `SELECT FOR UPDATE`," he said, "I will be watching." He delivered it mildly, over the battered enamel lunch tin he brought from home every day. Fridays were couscous, his mother's recipe, and that was not up for discussion.

Imane's own enthusiasm produced the chapter's last bug. Her dashboard PR included a refinement:

```
Counter.builder("glasshouse.reports.submitted")
       .tag("researcher", report.researcherId())   // one time series PER RESEARCHER
       .register(registry)
       .increment();
```

Kenza caught it in review with the new checklist; staging supplied the proof. Every distinct researcher ID created a new time series, held in the app's registry and scraped forever. Thousands of researchers meant thousands of series, and Prometheus memory climbed a staircase as they accumulated. "Metrics are aggregates," Kenza wrote in the review. "The moment one can identify a user, you've built a worse database. IDs go in logs and traces; tags are for low-cardinality categories such as status, severity and endpoint." The team rule went on the Wall in block letters: *No user-id tags. Prometheus is not a database.* Imane fixed it in an hour, then added the rule to the checklist herself. The teaching loop, Kenza noticed, was now running without her pushing it.

Rachid's only addition to the review thread was a cost model, green even in plain text: *Cardinality is multiplication, not addition: a metric's series count is the product of its tag dimensions.* `endpoint(40) × status(5)` *is 200 series; multiply by* `researcher(30,000)` *and it's six million, each one RAM in the registry, storage in Prometheus, latency in every query. That is why the budget gets a name in the ADR instead of living in taste.*

Correlation came last, the thing that would have saved her night entirely. Micrometer Tracing went in with an OTLP exporter, and the mechanism got drawn onto the Wall's Request Journey. A tracing filter sits at the very front of the chain, before the `DispatcherServlet`. It opens a span per request and copies `traceId` and `spanId` into the MDC, the thread-local map the logging pattern reads. The export and the log format were pure configuration, which stung a little, considering the night they would have saved:

```
management:
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: http://otel-collector:4318/v1/traces  # spans ship to the collector
  tracing:
    sampling:
      probability: 0.1   # export 10% of traces; every request still gets its IDs
logging:
  structured:
    format:
      console: ecs       # structured JSON; traceId/spanId ride each line via MDC
```

Every log line now carried the IDs. The trace ID was also returned on every response as an `X-Trace-Id` header, so support could ask a researcher for the ID and find the exact request. This was the hand-rolled correlation filter every postmortem had wished for, now standing on a real tracer. The echo was the buildout's only custom code:

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE + 2)   // behind Boot's tracing filter, ahead of
        Spring Security
class TraceIdResponseFilter extends OncePerRequestFilter {
    private final Tracer tracer;
    TraceIdResponseFilter(Tracer tracer) { this.tracer = tracer; }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
            FilterChain chain) throws ServletException, IOException {
        var span = tracer.currentSpan();   // the ID support asks researchers for
        if (span != null) res.setHeader("X-Trace-Id", span.context().traceId());
        chain.doFilter(req, res);
    }
}
```

The `@Order` is what makes *every response* true. Boot registers its tracing filter at `Ordered.HIGHEST_PRECEDENCE + 1` and Spring Security's chain at -100; a filter bean with no order runs after both, so a 401 or a 403 would have left without the header. The filter also takes a `Tracer`, and a `@WebMvcTest` slice scans `Filter` beans while auto-configuring no tracer, so `SharedMocks` gained one more bean, `Tracer.NOOP`, and the web slices kept starting.

The logs themselves switched to Boot's structured JSON output (the `ecs` line above), so they could be queried instead of grepped. *MDC is the `ThreadLocal` request-ID filter I hand-rolled at Meridian, standardized,* Kenza thought, *and it keeps the filter's exact flaw: a thread-local map doesn't follow you across an executor unless you carry it.* That caveat went on the Wall too.

Even `/actuator/info` earned its keep: build-info and git commit wired in through the Maven plugin, plus a small `InfoContributor` listing active feature flags. At 3:09 a.m. Kenza had been unable to answer *what exact code is running right now*, and that was never happening again.

She spent a weekend writing it all into an on-call handbook: where the dashboards live, what normal looks like at 3 a.m., the first five queries to run. The Monday brick ceremony, the week's on-call handoff, grew a new ritual in which the outgoing engineer gives the incoming one a dashboard tour. And postmortems changed shape permanently. The template's first question, in Rachid's green ink, was his: *What did you know, and when did you know it?* House doctrine followed: *if the dashboards said nothing, that's finding #1.*

### ⚖ Your move, architect

The sprint's last argument had no code in it. Youssef had Alertmanager wired and a blank routing file, and on the table sat the question every on-call rotation eventually answers with its resignations: *what pages a human at 3 a.m.?* Commit before reading on. This fork has no compiler.

- **A. Page on causes.** CPU, Hikari `pending`, error-log spikes, GC pauses: every gauge that could precede user pain gets a threshold and a pager entry.

- **B. Page on SLO burn only.** A page means a signed objective is bleeding; every causal gauge lives on dashboards the responder opens *after* the page, to explain it.
- **C. Page on nothing.** Synthetic checks, business-hours triage, an inbox instead of a brick.

**The trade-offs, since you've committed: A** wakes humans for things users never felt, such as a CPU spike the autoscaler absorbs. Alert fatigue is an **operability** failure mode: the rotation mutes the noisy channel, and the mute eats the one page that mattered. Causes also can't be enumerated in advance. The storage spike that opened this chapter sat under no threshold anyone would have set. **B** is what went in the routing file: *symptoms page, causes explain.* Every page means user-visible pain, **availability** burning against a number somebody signed, so every page is actionable, and the burn rate sets the urgency. Its price: it only works because Leila signed SLOs, and only while the dashboards stay cause-rich enough to walk a symptom back to its cause. **C** is an honest **cost** call for an internal tool whose users sleep when you do. For a paid platform holding other companies' undisclosed vulnerabilities it's indefensible in one sentence: Northbeam asked, in writing, *how would you know*, and C's answer is *Monday.*

The decision went into the binder as the ADR the sprint's first YAML had promised, the buildout distilled to half a page:

> **ADR-012: Observability by design. Context:** A 3 a.m. page arrived with zero data and ended in `resolved, cause unknown`; a week later the cluster restart-looped healthy pods because a database check sat in the liveness probe. **Decision:** Probe doctrine: liveness is process-only, dependencies live in readiness. Every user-facing flow carries an SLO the business has signed, with golden signals and its business metrics (reports submitted, triage latency) shipped together; correlation IDs propagate from the edge filter through logs (MDC) and traces; metric tags are bounded by a named cardinality budget and are never IDs. Alerts page on symptoms (SLO burn); dashboards carry causes. **Consequences:** The next incident starts with data, and the postmortem's first question has an answer. The cost: a metrics review joins every feature's definition of done. Observability is designed in, or the feature isn't done.

"We can finally see the system," Kenza told the Canary, back on her desk where it belonged.

Two weeks later, something worth seeing arrived.

## The debrief: Ch.12, Three A.M.

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 12.** How should liveness and readiness probes be wired for a Spring Boot service, and what happens if you get it wrong?

Then the follow-ups, cold:

1. Why exactly do user-id metric tags blow up Prometheus?
2. How does one trace ID end up in every log line of a request?
3. After the outage, the VP asks for "more monitoring." What do you propose instead?
4. During the restart storm, some new pods were killed a second time. What was missing?
5. Counter, timer, gauge: what is each for, and where does p99 come from?

Tick what you said:

- ▢ Claim: Liveness means restart me, readiness means route traffic to me, and a dependency check goes in readiness, never in liveness.
- ▢ Mechanism: The kubelet kills the container on failed liveness and removes the endpoint on failed readiness. Boot's health groups decide which indicators feed which probe.
- ▢ Trade-off: Process-only liveness will not catch every wedged app, and dependency-aware readiness drains the whole fleet during a shared outage.
- ▢ Failure mode: A dependency indicator in liveness turns a ninety-second database blip into a cluster-wide restart loop of healthy pods.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Liveness and readiness answer different questions, *restart me* versus *route traffic to me*, and a dependency check goes in readiness, never in liveness. **Mechanism:** the kubelet's response to a failed liveness probe is to kill and recreate the container, while a failed readiness probe merely removes the pod from the Service's endpoints until it recovers. Boot models this with `LivenessState/ReadinessState` and exposes configurable health groups at `/actuator/health/liveness` and `/readiness`, so you choose which `HealthIndicators` feed which probe: liveness gets process-state only, readiness gets the database and friends. **Trade-off:** a process-only liveness probe won't catch every wedged-but-breathing app, so you backstop it with metrics and alerts; a dependency-aware readiness probe will drain the whole fleet from the load balancer during a shared outage, which is correct behavior but alarming if you don't expect it. **Failure mode:** wire a dependency indicator into liveness and a ninety-second database blip becomes a cluster-wide restart loop. The kubelet executes healthy pods for honestly reporting `DOWN`, and the thundering herd of cold starts hammers the recovering dependency, so the health check itself becomes the outage.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Operability**, meaning observability as a property you *design* (what will 3 a.m. ask, and is the answer already recorded?), never a stack you install after the outage. **Availability** paid twice for the deferral: a blind hour at the kitchen table, then ninety seconds of database trouble amplified to twenty-two minutes by probe wiring nobody designed. **Cost** runs underneath: cardinality is a multiplier on storage and query time, and the error budget turns reliability arguments into arithmetic the business can sign.
- **The ADR:** ADR-012, observability by design: liveness process-only, dependencies in readiness; an SLO per flow with golden signals and business metrics; correlation IDs from edge filter through MDC and traces; cardinality budgeted, never IDs; pages on SLO burn, dashboards carry causes; metrics review in the definition of done.
- **Rejected alternatives:** dependency checks in liveness (a restart has never fixed a remote database and just stampedes it with cold starts); paging on causes (unenumerable, and fatigue mutes the rotation); paging on nothing (honest for internal tools, indefensible for a paid platform); per-user metric tags (Prometheus is not a database); observability as a backlog ticket (ATR-1207 aged five months and collected an outage as interest).
- **Fight or lean:** *Lean* hard on Boot's machinery: health groups, `LivenessState/ReadinessState`, Micrometer's facade, the tracing filter and MDC plumbing, graceful shutdown. That is undifferentiated plumbing the framework should own. *Fight* the defaults wherever the app meets the cluster and the org: which indica-

tors feed which probe, which endpoints are exposed, which tags are legal, what pages a human. Those are decisions the framework cannot make for you, and the well-meaning commit is what it looks like when nobody makes them.

**Story anchors**

- The Canary on the kitchen table at 3 a.m., hearing a calm description of nothing → recovery without understanding; observability debt comes due all at once.
- Pods restart-looping *after* the database blip had already passed → a dependency check wired into the liveness probe; "the cluster is killing pods for telling the truth."
- Prometheus memory climbing a staircase under Imane's per-researcher tag → metric cardinality discipline; IDs belong in logs and traces, never in tags.
- Leila writing the SLO figures down unasked: "give me arithmetic and I'll sign it" → SLOs as the architecture's contract with the business; the error budget (forty-three minutes a month at 99.9%) decides by signature, not mood, when reliability outranks features.

**Interview ammo**

- *Kill-shot: Your pods restart-loop after a brief DB outage. Why, and how should the probes be wired?* A database `HealthIndicator` is feeding the liveness probe; restarts can't fix a remote dependency, so move dependency checks to the readiness health group and keep liveness process-only.
- *Follow-up: Why exactly do user-id metric tags blow up Prometheus?* Every distinct tag value mints a new time series held in the app's registry and stored forever in Prometheus. Unbounded IDs mean unbounded series, a slow OOM of the metrics backend.
- *Follow-up: How does one trace ID end up in every log line of a request?* A tracing filter at the front of the chain opens a span and copies trace/span IDs into the MDC, a thread-local map the log pattern renders. It is cleared on completion and explicitly propagated across thread-pool boundaries, since thread-locals don't follow work onto other threads.
- *Architect follow-up: After the outage, the VP asks for "more monitoring." What do you propose instead?* A design rule, not a stack: an SLO the business signs per flow, golden signals plus business metrics, correlation IDs from the edge through logs and traces, paging on SLO burn only, and a metrics review in the definition of done, so 3 a.m.'s question is answered before it's asked.
- *Follow-up: During the restart storm, some new pods were killed a second time. What was missing?* A startup probe. Eight cold JVMs were warming at once and refilling their Hikari pools against a database still recovering, so new pods answered their first health checks slowly, and with no startup probe to shield them some were killed again mid-warmup.

- *Follow-up: Counter, timer, gauge: what is each for, and where does p99 come from?* A counter only goes up and counts events, such as reports submitted. A timer records durations and counts and can publish percentile histograms, which is where p99 comes from. A gauge samples a value that goes up and down, such as active connections in the Hikari pool.

**Pointers** → `examples/ch05_actuator/` · **run it:** `labs/lab-04-actuator-readiness` (a dependency check in liveness restart-loops the pod; in readiness it stays alive)

---

# Chapter 13: The Cascade

The first thing Kenza noticed, at 2:14 on a Tuesday afternoon in late September, was that she could *see* it.

The Grafana board, built out of the 3 a.m. postmortem's shame, showed submission p99 climbing like a fever chart, and outbound calls to Cortexa, the CVE-enrichment vendor, timing out in red stripes. Their synthetics had alerted eleven minutes before the first researcher tweet. After nine months of debugging by flashlight, the lights were on.

Rachid was on planned leave: a cabin, a lake, deliberately no signal. Whatever happened next was Kenza's.

On the war-room screen, Youssef overlaid two graphs: Cortexa's status page (*investigating elevated load in EU region*) and Glasshouse's outbound rate to `cortexa.io`.

"Their status page says elevated load," he said. "We are the elevated load. We're sending them three times our organic traffic."

Kenza found it in four minutes. April, when the whole site 503'd waiting on this vendor, had ended with timeouts on every outbound call and a decision note in her handwriting: *someday this call gets a circuit breaker.* Someday never came. The sentence had ridden into ADR-006's margin as a promissory note, and today it came due. What came instead, in a commit labeled `temporary`, was this:

```
// EnrichmentService, "temporary" since April
ReportEnrichment enrich(String cveId) {
    for (int attempt = 1; attempt <= 3; attempt++) {
        try {
            return cortexaClient.fetch(cveId);   // connect 1s / read 2s
        } catch (RestClientException e) {
            log.warn("Cortexa attempt {} failed", attempt, e);
        }
    }
    throw new EnrichmentUnavailableException(cveId);
}
```

Three attempts, back to back: no backoff, no jitter, no budget. Every failing call became three, so a regional outage tripled Glasshouse's outbound traffic against a vendor already failing. Inside Atrium, each enrichment tied up a thread for nine seconds of stacked timeouts instead of three. It was Little's law again: in-flight equals arrival rate times residence, so tripling residence tripled the threads held. Pools filled, and the backlog spread into unrelated paths until the tracker itself slowed. It was April's pool exhaustion wearing a distributed coat: Cortexa, already drowning, held underwater by its own customer.

“First move is triage, not architecture,” Kenza said, steadier than she felt. “Kill the retries behind the feature flag. Right now we are part of their DDoS. We stop being that, then we design under calm.”

The flag flipped; within minutes the outbound graph dropped to organic levels and Cortexa’s status page began, slowly, to recover.

## The investigation

The retry storm explained the cascade’s force. It did not explain its reach. Submission was supposed to survive Cortexa entirely: April’s other fix was *submit now, enrich later*, an `@Async` hand-off to a background executor. So why was the report-detail page, a pure read path, timing out?

Imane asked the fatal question. “Wait. Isn’t enrichment *cached*? Why are we calling them at all on the detail page?”

She pulled up her own cache panel: `cve-enrichment` hit rate, a flatline at zero, flat since the day the dashboards went up. The bug was older than the measurement: her refactor, mid-August.

> **Your call.** A `@Cacheable` that compiles, runs, and returns correct data, with its hit rate flat at zero since an August refactor. Commit before Imane scrolls: what’s wrong, and which -ility has been quietly uninsured?

The room went quiet the way it had in February, when it was Kenza’s own name on the commit. Imane scrolled to the code and found it:

```
@Service
public class EnrichmentService {

    private final CortexaClient cortexaClient;   // constructor-injected

    public ReportEnrichment enrichmentFor(Report report) {
        return this.fetchEnrichment(report.cveId());      // ← the bug
    }

    @Cacheable(cacheNames = "cve-enrichment")
    public ReportEnrichment fetchEnrichment(String cveId) {
        return cortexaClient.fetch(cveId);
    }
}
```

“It compiles,” Imane said slowly. “It runs. It returns the right data. It has just… never cached anything. Every detail view, straight to the vendor, and nobody noticed because Cortexa is fast when it’s healthy.”

(If you called it at the pause: the mechanism is February's, a `this.` call that never crosses the proxy. The -ility is not performance. The cache was *availability* insurance against exactly this afternoon, and its premium lapsed without a test going red.)

Kenza fetched a printed postmortem from her desk: February, her own name in the author line. PM-32: *Half-saved triage states. Root cause: `@Transactional` method invoked via `this.`, bypassing the proxy.* It was the same mechanism. The annotation's behavior lives in the proxy the container hands your callers, and a `this.` call dispatches on the raw object behind it, so the interceptor never runs. ADR-003 had made this a named review finding, and the August review read past the line. February's ghost, back in a new coat.

"I shipped this exact bug in February," she said, handing it over. "Different annotation, same mirror. Let me show you the smoke cloud." In January it had been a printed diff in green ink on Kenza's desk. Now it was a printed postmortem on Imane's, the same treatment passed down.

February's proxy drawing was still on the Wall: the smoke cloud in front of a box labeled *your actual object*, the caption faded but legible: *this. goes around the mirror.* Kenza re-inked every line and added a list underneath: `@Transactional · @Cacheable · @Async · @PreAuthorize: same mirror, same trap.` Rachid had said that as a warning, and it took seven months to come true.

*At Meridian a retry was a for-loop you could see,* the old voice observed. *Crude, and wrong in three other ways (no backoff, no budget, no breaker), but impossible to walk around. Here, the annotation is a promise only the proxy keeps.*

## What the senior sees

"Say the mechanism back to me," Kenza said, and heard, mid-sentence, whose cadence she was borrowing.

"Spring's annotation-driven features aren't compiled into my class," Imane said. "At startup, a `BeanPostProcessor` sees the annotation and wraps my bean in a proxy, a CGLIB subclass here. The *proxy* holds the interceptor: check the cache, open the transaction, dispatch to the executor, *then* call my code. External callers get the proxy from the container. But inside my own class, `this` is the raw object. The call never crosses the proxy boundary, so the interceptor never runs. Nothing is broken. It's bypassed."

"And that's the scary part," Kenza said. "It fails *silently and correctly*. The unit test passes, because the method returns the right value. Only a hit-rate graph could catch it. Yours did; code review didn't."

She sketched the incident as a stack of absences:

**Timeouts were the foundation, and the only layer that covered every call.** Without April's timeouts (ADR-006, the latency budget), every request thread would have blocked on the unanswering vendor until none remained: the thread-pool funeral, rerun. But a timeout bounds *one* call. It says nothing about *how many* you make.

**Retries without a budget multiply load on a struggling dependency.** A retry is a bet that the failure was transient. It needs exponential backoff so attempts spread out, and jitter so a thousand clients don't synchronize into waves. Three immediate retries against a regional outage bet nothing: they only add load to something already failing.

**Nothing remembered that Cortexa was down.** Every request rediscovered the outage from scratch and paid the full timeout each time. That's the circuit breaker's job. Kenza drew the state machine next to the smoke cloud. *Closed:* calls flow, failures counted in a sliding window. Past a failure-rate threshold it *opens:* calls fail fast, with no thread consumed and no vendor touched. After a wait, *half-open* admits a few probes: if they heal it closes, and if they fail it snaps open. The breaker is the system's memory of recent pain.

**Nothing isolated the blast on the read path.** April's bulkhead was the small executor behind submission, and it covered that path only. The detail page called Cortexa on request threads, the pool every other page draws from, so a sick optional feature starved a healthy critical one. A bulkhead, a hard cap on concurrent enrichment calls, turns "Cortexa is down" into "enrichment is down," and nothing more.

"Fourth tool in that kit," Youssef said. "Rate limiter." Kenza added it. A rate limiter is a ceiling on outbound rate, enforced even when everything is green: the breaker reacts to their failures, the bulkhead caps our concurrent calls, and the rate limiter bounds what we send in the first place. "This morning's flag was a rate limiter set to zero, by hand." She penciled the numbers beside it:

```
resilience4j:
  ratelimiter:
    instances:
      cortexa:
        limit-for-period: 50
        limit-refresh-period: 1s
        timeout-duration: 0
```

Fifty permits per one-second window. `timeout-duration: 0` means the fifty-first call fails fast instead of queueing up to dogpile the next refresh. The breaker protects *you* from a failing dependency. The rate limiter protects *them*, and your contract, from you, which is why it holds even when every graph is green.

**And the async path lied about its own health.** April's `@Async` enrichment executor was bounded in threads only: nobody had set a queue capacity, and `ThreadPoolTaskExecutor` defaults it to `Integer.MAX_VALUE`. Under the storm it accumulated forty thousand pending tasks while showing eight busy threads and no errors. *Meridian's hand-rolled executor hid its backlog exactly this way,* Kenza thought. *Same lie, older clothes.* An unbounded queue converts overload into silent latency and creeping memory instead of a signal.

"One more," Youssef said. "Retries mean some calls land twice: they process our request, die before answering, we send it again. Anything that retries is at-least-once; the far side has to ab-

sorb duplicates."

"Idempotency key on every enrichment call," Kenza agreed. "Keyed by the CVE, same as the cache, so the same request twice is a no-op on their side. And write the rule down: at-least-once arrives with your first retry, not with a broker someday. Idempotency is the precondition for every retry-shaped tool on this Wall. The durable version (write the intent in the same transaction as the data, deliver later, dedupe on the consumer) is the outbox pattern. We'll need it the day anything splits out of Atrium." She underlined it twice, not knowing how soon.

## The fix

They designed under calm. The fix was February's pattern reused verbatim: when an annotation needs a proxy boundary, *extract the bean*. The vendor boundary became one small class whose only job was to be called from outside itself:

```
@Component
public class CortexaGateway {   // the resilience boundary

    private final RestClient cortexa;   // connect 1s, read 2s

    public CortexaGateway(RestClient cortexaRestClient) {
        this.cortexa = cortexaRestClient;
    }

    @Retry(name = "cortexa")
    @CircuitBreaker(name = "cortexa")
    @Bulkhead(name = "cortexa")
    public ReportEnrichment fetch(String cveId) {
        return cortexa.post().uri("/v1/enrich")
                .header("Idempotency-Key", "enrich-" + cveId)
                .body(new EnrichmentRequest(cveId))
                .retrieve()
                .body(ReportEnrichment.class);
    }
}
```

The three annotations come from `io.github.resilience4j:resilience4j-spring-boot4`, the Resilience4j starter built for Boot 4. They are aspects, so `spring-boot-starter-aspectj` sits beside it in the POM: without that starter the application still starts, nothing wraps the gateway, and all three annotations do nothing.

The verb was April's, unchanged. Cortexa enriched by POST, so the method offered no idempotency for free, and the key had to build it. The key is what makes this POST safe to repeat, and that construction, not the verb, is what licenses the `@Retry` stacked above it. A GET wouldn't have needed the header; every retried write on this Wall does. Kenza spelled it out in the PR de-

scription, because “retry only idempotent calls” gets read as a scavenger hunt when it’s a construction project.

In front of it: `EnrichmentCache`, fifteen lines, constructor-injected, its single method wearing `@Cacheable(cacheNames = "cve-enrichment", sync = true)` and delegating straight through. Two beans, two proxies, every call crossing both. The hit-rate graph moved within an hour.

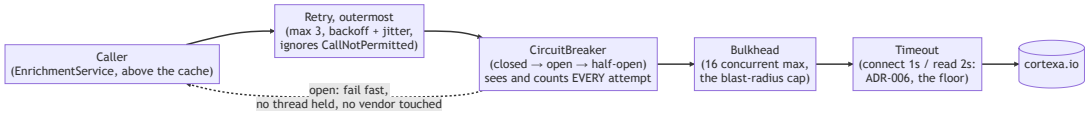
Imane asked the sharp review question: why not a `fallbackMethod` on the breaker, returning the pending badge? Kenza pointed one layer up. “Anything returned *underneath* the `@Cacheable` gets cached. Breaker opens, fallback says pending, cache stores pending, and we serve ‘pending’ for a full TTL after Cortexa heals. The fallback lives above the cache, or the cache launders the lie.” So `EnrichmentService` caught the failure above the cache and returned `ReportEnrichment.pending(cveId)`: explicit, visible, never stored. A load that throws caches nothing.

The numbers lived in configuration, tunable without a deploy:

```
resilience4j:
  retry:
    instances:
      cortexa:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2.0
        ignore-exceptions:
          - io.github.resilience4j.circuitbreaker.CallNotPermittedException
          - io.github.resilience4j.bulkhead.BulkheadFullException
  circuitbreaker:
    instances:
      cortexa:
        sliding-window-size: 50
        minimum-number-of-calls: 20
        failure-rate-threshold: 50
        slow-call-duration-threshold: 2s
        slow-call-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
        ignore-exceptions:
          - io.github.resilience4j.bulkhead.BulkheadFullException
  bulkhead:
    instances:
      cortexa:
        max-concurrent-calls: 16
```

Jitter rode along as a review note: a randomization factor on each wait, via Resilience4j’s interval functions, so a fleet of pods doesn’t retry in synchronized waves. Slow calls counted against the breaker on purpose: a timeout that “succeeds” is still pain.

Ordering got its own Wall sketch, because Kenza nearly got it wrong. By Resilience4j's default aspect order, Retry sits *outside* the breaker. Each attempt passes through the breaker and is counted. Once the breaker opens, remaining attempts fail instantly with `CallNotPermittedException`, which the retry *ignores*, so an open breaker ends the storm instead of feeding it. Flip the nesting and three attempts look like one call. The storm hides inside the breaker's statistics. "Same annotations, opposite system," she wrote. "Ordering is semantics, not style." She drew the composition on the Wall:




They kept the default order on purpose, the why written underneath: the bulkhead takes a permit *per attempt*, so the cap holds mid-storm; ADR-006's timeout prices one attempt's worst case. One consequence of that order went into the YAML: a full bulkhead throws `BulkheadFullException` back out through the breaker and the retry, so both ignore lists name it, and a call refused by our own cap is neither counted as a vendor failure nor retried.

The cache triggered the afternoon's best argument. Imane wanted Redis: shared, durable, one cache for all pods. Kenza made her argue both sides. Caffeine is in-process: nanosecond reads, no network hop, nothing new to fail. But every pod warms its own copy and there's no shared eviction. Redis buys sharing at the price of a round-trip, serialization, and one more thing on the 3 a.m. dashboard. CVE enrichment is slow-changing, harmless to duplicate and fine to lose, so Caffeine with a TTL won, with Redis kept for the day they needed shared eviction. Refreshes went through `@CachePut`, which executes the method and overwrites the entry. A manual re-run hit `@CacheEvict`. And `sync = true` earned its one-breath defense: a hot key expiring makes every concurrent request miss at once and stampede the backend, and `sync` lets one caller per key load while the rest wait, per pod.

The executor kept its name, and its queue got a bound:

```
@Bean(name = "enrichmentExecutor")
TaskExecutor enrichmentExecutor() {
    var executor = new ThreadPoolTaskExecutor();
    executor.setThreadNamePrefix("enrich-");
    executor.setCorePoolSize(8);
    executor.setMaxPoolSize(16);
    executor.setQueueCapacity(200);   // bounded: overload becomes a signal, not a
        secret
    executor.setRejectedExecutionHandler(new ThreadPoolExecutor.AbortPolicy());
    return executor;
}
```

They argued the rejection policy aloud. `CallerRunsPolicy` was tempting (backpressure for free) until Kenza traced it: under overload, the *request* thread runs the enrichment, synchronously, against a sick vendor. April, reinvented by a constructor argument. `AbortPolicy` won. The `@Async("enrichmentExecutor")` submission point catches the rejection, the report keeps its pending badge, and a `@Scheduled` sweeper re-enqueues once the breaker closes. One caveat went on the PR: Boot's default scheduler is single-threaded unless you size it, so a slow sweep delays every other job. An `AsyncUncaughtExceptionHandler` went in beside it: a `void @Async` method that throws never reaches its caller, and by default it's merely logged by a handler nobody watches. Theirs fired an alert.

Kenza capped the marker on a line written under the smoke cloud: *backpressure is a system property, not a queue setting*. "Every bound (pool, queue, retries, bulkhead) is the system saying *no* while *no* is still cheap," she said. "Unbounded anything is deferred failure. Availability isn't surviving load; it's refusing the load you can't survive."

Which left the badge. Leila fought it, because that was her job. "Researchers see 'severity pending enrichment' and read 'broken.' Synthesize a medium severity while Cortexa's down. Smooth it over."

"A fallback that lies is an incident you've scheduled for later," Kenza said. "Severity drives bounty math. The day our invented 'medium' turns out to be a critical, we're explaining money, not a badge. The fallback degrades *honestly*: what we know, what's coming, nothing invented. What a user sees while we're failing is a contract, and product co-signs it." Leila held the stare, then nodded, the same nod she'd given on timeouts in April. The badge shipped, researchers barely noticed, and trust held. Fallbacks that degrade honestly entered the house style that week.

### ⚖ Your move, architect

The badge settled new, never-enriched reports. The harder question outlived the incident: what should the detail page show for reports enriched *weeks ago*, when Cortexa goes dark and cache entries age out? Leila brought renewal numbers: an hour of erroring detail pages reads as "the platform is down." Malika brought trust: her triagers *act* on severity and must know how far to trust the page. Three policies went up on the Wall. Commit before reading on.

- **A. Fail fast, visibly.** Breaker open → the panel says so. Honest, simple, nearly free, and triage stops entirely for the duration.
- **B. Queue-and-drain.** Accept everything, enrich eventually, show pending meanwhile, and build the durable queue now.
- **C. Serve the last known enrichment, marked `STALE` with its age.** *"Enriched 41h ago · vendor degraded."* The cache becomes a record of history as well as an accelerator.

**The trade-offs, since you've committed: A** spends *availability* to buy simplicity and perfect honesty. Leila's renewal argument killed it on the record: a vendor's bad hour promoted to a triage outage is the cascade re-implemented as product policy. **B** is the right shape for *writes*: it's the

outbox pattern Chapter 14 builds. Mid-incident it's a redesign, and reads can't queue: a triager looking at a report *now* can't be handed a ticket number. **C** shipped, buying *availability* with a named *freshness* tax: the cache kept entries past their freshness window, anything stale carried an age stamp, and the failure mode was decided in advance. Stale under 48 hours is actionable, older escalates to manual, and first-contact CVEs keep the pending badge. Leila's renewals and Malika's trust resolved *on the record*: the degraded view is a designed product contract, where before it was whatever the exception handler left on screen.

The record went into the binder before the week was out, the longest entry yet, with the most scars to cite:

> **ADR-013: Resilience composition. Context:** A Cortexa regional outage met an unbudgeted retry loop ("temporary" since April); our outbound tripled, pools filled, and the cascade reached unrelated paths, while an August `@Cacheable` had never cached once (self-invocation, again). **Decision:** The composition order is law: timeouts under everything (ADR-006 is the floor); bounded, jittered retry only on idempotent calls, every attempt crossing the breaker so an open breaker ends the storm; a bulkhead caps concurrent calls per dependency; a rate limiter wherever the contract demands one; every external call carries an idempotency key; fallbacks degrade honestly (pending or stale-marked with age, never fabricated); async work rides bounded, named executors. **Consequences:** A failing dependency now costs its feature, not the site; the price is configuration that must be load-tested rather than guessed, and a review rule: a bare retry loop is a finding.

## The closed incident

Rachid came back Thursday, sunburned and unhurried, to a folder on his desk: timeline, postmortem, ADR-013, Wall photos, and the minutes of a review Kenza had run *with Cortexa's engineers*, with both companies' graphs on one screen and both sides' retries on the table. A closed incident, end to end, commanded by someone else.

He read it all, walked to her desk, and said only: "Who flipped the flag?"

"I did. Triage first. Architecture after."

He nodded once, the smallest nod she'd ever seen carry that much weight, and went to study the re-inked smoke cloud.

The only shadow: the CTO had read the postmortem, come away glowing about one word, *resilience*, and had a conference to attend in October. Next month's problem.

## The debrief: Ch.13, The Cascade

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 13.** How do you stop a failing downstream dependency from cascading through your service, and what goes wrong when the pieces are composed badly?

Then the follow-ups, cold:

1. Why do `@Cacheable`/`@Async` silently no-op on self-invocation?
2. Retry inside or outside the circuit breaker: what changes?
3. Why is an unbounded `@Async` executor queue dangerous?
4. Compose timeout, retry, breaker, bulkhead, and rate limiter for one flaky vendor: what order, and why?
5. Walk the circuit breaker's three states, and say what each number in its configuration decides.
6. Can you retry a POST?

Tick what you said:

- ☐ Claim: A timeout, a bulkhead, a circuit breaker, a bounded retry on idempotent calls only, and a fallback that degrades honestly.
- ☐ Mechanism: What a timeout and a bulkhead bound, the breaker's closed, open and half-open states, and the retry outside the breaker.
- ☐ Trade-off: The price is configuration that must be load-tested, and a stale view with its age that product has to co-sign.
- ☐ Failure mode: A bare retry loop triples traffic against a failing vendor, and a flipped nesting hides the storm in the breaker's statistics.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

**The model answer. Claim:** You compose five things in a fixed order: a timeout under every outbound call, a bulkhead, a circuit breaker, a bounded retry on idempotent calls only, and a fallback that degrades honestly. **Mechanism:** a timeout bounds one call and says nothing about how many you make; a bulkhead is a hard cap on concurrent calls; a circuit breaker counts failures in a sliding window while closed, opens past a failure-rate threshold so that calls fail fast with no thread consumed and no vendor touched, then goes half-open after a wait and lets a few probes close it or reopen it; a retry needs exponential backoff and jitter, sits outside the breaker so that every attempt is counted and an open breaker ends the retries, and is safe only on idempotent calls, which for a POST takes an idempotency key. **Trade-off:** a failing dependency now costs one feature while the site stays up; the price is configuration that must be load-tested rather than guessed, and a degraded view that product has to co-sign: the last known enrichment, marked stale with its age, buys availability with a named freshness tax. **Failure mode:** a bare retry loop with no backoff and no budget turns every failing call into three: an outage triples your traffic against a vendor already failing and stacks nine seconds of timeouts on each thread. Flip the nesting and the breaker sees three attempts as one call, so the storm hides inside its statistics.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Availability under dependency failure.** Resilience is availability on somebody else's worst day, *composed*, not installed: backpressure decides what the system refuses, idempotency what it may safely retry, honest degradation what users see while it fails. Taxes paid knowingly: **operability** (tunables that must be load-tested, one more 3 a.m. dashboard) and **cost** (per-pod caches, idle bulkhead permits).
- **The ADR:** ADR-013, resilience composition: timeouts under everything (ADR-006 the floor), bounded jittered retry only on idempotent calls with every attempt crossing the breaker, bulkhead per dependency, rate limiter where the contract demands it, idempotency keys, honest fallbacks, bounded executors.
- **Rejected alternatives:** the bare three-attempt loop (a multiplier aimed at a drowning vendor); `fallbackMethod` beneath the cache (the cache launders the lie for a full TTL); `CallerRunsPolicy` (April reinvented: a sick vendor's latency on request threads); synthesized severity (an incident scheduled for later); queue-and-drain mid-incident (Ch.14's outbox bought early, and reads can't queue).
- **Fight or lean:** *Lean* on Resilience4j's annotations and default aspect order at a dedicated gateway bean: proxy wrapping is why the container is worth inhabiting, and Retry-outside-breaker is the correct semantics. *Fight* every default that hides failure: the unbounded executor queue, the logged-and-forgotten `AsyncUncaughtExceptionHandler`, any annotation whose effect you haven't seen on a graph.

**Story anchors**

- Imane's flatlined hit-rate graph → `@Cacheable` behind a `this.` call: the proxy trap generalizes to every annotation-driven feature
- "We are the elevated load" QPS overlay → retry storms: retries without backoff, jitter, and a budget are a self-inflicted DDoS on a struggling dependency
- The "severity pending enrichment" badge → fallbacks must degrade honestly; a fallback that invents data is an incident scheduled for later
- Leila's renewals vs Malika's trust, resolved on the record → the degraded view is a designed product contract; ADR-013 made stale-marked truth house law

**Interview ammo**

- *Follow-up: Why do `@Cacheable`/`@Async` silently no-op on self-invocation?* Same mechanism as `@Transactional`: the behavior lives in a proxy interceptor created at `postProcessAfterInitialization`, and `this.` never crosses the proxy. Extract the annotated method into a separate injected bean. Nothing fails, because the method still returns correct results and the tests pass; only a flatlined hit-rate graph shows the misses.
- *Follow-up: Retry inside or outside the circuit breaker: what changes?* Resilience4j's default puts Retry outermost: the breaker counts every attempt and, once open, fails the rest fast (have the retry ignore `CallNotPermittedException`); inverted, the breaker sees three attempts as one call and the storm hides from its statistics.
- *Follow-up: Why is an unbounded `@Async` executor queue dangerous?* It turns overload into silent latency and memory growth instead of backpressure. Bound the queue, pick the rejection policy deliberately (caller-runs pushes a sick dependency's latency onto request threads), graph the depth, and register an `AsyncUncaughtExceptionHandler` so `void` async failures alert instead of just logging.
- *Architect follow-up: Compose timeout, retry, breaker, bulkhead, and rate limiter for one flaky vendor: what order, and why?* Timeouts under everything to price one attempt; bounded jittered retry only on idempotent calls, each attempt crossing the breaker (open = storm over); bulkhead caps what you'll spend waiting; rate limiter protects their contract even when green. Flip the nesting and the storm hides inside the breaker's statistics.
- *Follow-up: Walk the circuit breaker's three states, and say what each number in its configuration decides.* Closed: calls flow, and failures are counted in a sliding window of the last 50 calls, with at least 20 recorded before the breaker judges. At a 50% failure rate it opens, and calls fail fast with no thread consumed and no vendor touched. After 30 seconds it goes half-open and admits five probes: if they heal it closes, and if they fail it opens again. A call that takes longer than two seconds counts as slow, and a 50% slow-call rate opens the breaker too.

- *Follow-up: Can you retry a POST?* Only after you make it idempotent. A retried call can land twice: the far side processes the request, dies before answering, and receives it again, so anything that retries is at-least-once. The gateway sends an `Idempotency-Key` header keyed by the CVE, which makes the same request twice a no-op on the vendor's side, and that key is what licenses the `@Retry`. A GET would not have needed the header.

**Pointers** → `examples/ch12_extensions/AopAnnotationTraps.java`, `examples/ch09_microservices/CircuitBreakerSelfInvocationTrap.java`

---

# Part VI: The Architect

**The nouns, before the story** *(plain prose; skip freely if you already run Spring)*

**Bounded context:** a part of the domain with its own model and its own language. It is the natural unit to split along, if you ever split. **Modular monolith:** one deployable whose internal borders are real enough to cut later. **Strangler fig:** evolving a system by routing traffic at the edge to new code, which grows while the old code starves. **Outbox:** write the outgoing event in the same database transaction as the data, then relay it afterward, so that "saved" and "announced" can never disagree. **Saga:** a cross-service flow built from local transactions plus compensating actions, because distributed transactions across services are not a real option. **Virtual threads:** cheap JVM threads that release their OS carrier while blocked, which gives thread-per-request semantics without the pool ceiling. **Reactive (WebFlux):** event-loop concurrency with explicit pipelines. It has the highest ceilings, and it is a different programming model with its own discipline. **Native image:** compiling the application ahead of time into a binary. You get near-instant start and low memory, and you pay for them at build time.

# Chapter 14: The Split

The October all-hands had two slides nobody forgot. The first made the room go still. *Rachid: transitioning to advisory role, effective January.* Most of engineering had heard the rumor; seeing it in 48-point font was different. The second slide arrived twenty minutes later, fresh from the CTO's conference in Lisbon: a constellation of hexagons connected by arrows, and beneath it, *EVENT-DRIVEN MICROSERVICES. TARGET: Q3.*

"Industry's moving," the CTO said. "Netflix runs hundreds of services. We run one. I want a plan."

Kenza watched Rachid not react. He had eleven weeks of opinions left and was visibly rationing them. That afternoon a calendar invite appeared on her screen: *Architecture session: the split. Friday. Whole team.* Underneath was a note from Rachid: "You argue both sides. For, then against, each at full strength. No verdict until someone demands one."

She understood immediately what it was. Everyone would. The org had spent two weeks openly wondering who inherited the whiteboard chair, and Rachid had just scheduled the audition in public.

*In my old life this meeting happened too,* she thought, *except the deck won and we spent two years untangling an "order service" that couldn't take an order without four network calls.* She started building two decks of her own.

## The investigation

On Friday the Wall was on her left, a year of ink on it, and the CTO was in the front row.

Kenza argued the split first, honestly, the way the conference would have. Notifications had a scaling profile nothing else shared: a bounty award triggered email bursts, the Monday digest was a thundering herd, and all of it ran on Atrium's threads next to report submission. Blast radius was real. Last month's cascade (one slow vendor call drained the threads every endpoint shared) had proved an enrichment vendor could reach the tracker itself, and isolation is the only fence that holds at 3 a.m. Team autonomy was real too: eleven engineers stepping on one deploy pipeline. She made it sound inevitable. The CTO nodded along.

> **Your call.** Before Kenza rotates the deck: where exactly does the split break against Atrium's actual schema, and which -ility does the Q3 slide quietly spend? Commit.

Then she rotated the deck and demolished her own case.

"Triage and payout share a transaction," she said. "One commit moves a report to Awarded and writes the payout, atomically. Split those into services and that atomicity is gone. Two-phase commit exists, and banks run it, but it buys atomicity by blocking: every participant holds its locks in-doubt until a coordinator that just died comes back with the verdict. None of that is even on offer across HTTP and Kafka. Nobody in this room is signing up to operate a coordinator."

*Her hand-wired instinct whispered the Meridian move: one connection, one try block, both writes inside, commit at the bottom. That instinct was the trap, because there is no shared connection across a network. There never was; a single process had just let her pretend.*

(If you committed at the pause: the break is the shared transaction, where triage and payout are one fact in one commit, and the slide spends **operability and cost**.)

"What replaces it is a saga: a chain of local transactions with compensating actions. Payments commits and publishes an event, and Reports reacts. If Reports fails, you don't roll back, you *compensate*: another transaction that semantically undoes the first, in front of users who saw the intermediate state. Choreography means every service reacts to events and the workflow exists nowhere. Try debugging control flow that has no source file. Orchestration means you build an explicit state machine to command and compensate. It's visible, at least, but now you've written a distributed transaction monitor by hand. Either way, partial failure stops being an ops problem and becomes the UX. And we are eleven people. Every boundary we create needs timeouts, breakers, retries with budgets, dashboards, an on-call story. We just spent a month giving our *one* external vendor that treatment."

In the silence the CTO leaned back. "So which is it?"

"The seam decides, not the conference," Kenza said. "Most of Atrium has no seam, because triage, payouts, programs share transactions and a chatty domain. One piece does, and it's Notifications. Async by nature: nobody waits for an email. No shared transactions: fire and forget. Its own scaling shape. We extract exactly that, behind a gateway, and I'll write down why the rest stays whole."

Rachid asked his only question of the day, from the back row. "What would have to be true for the split to be *wrong*?"

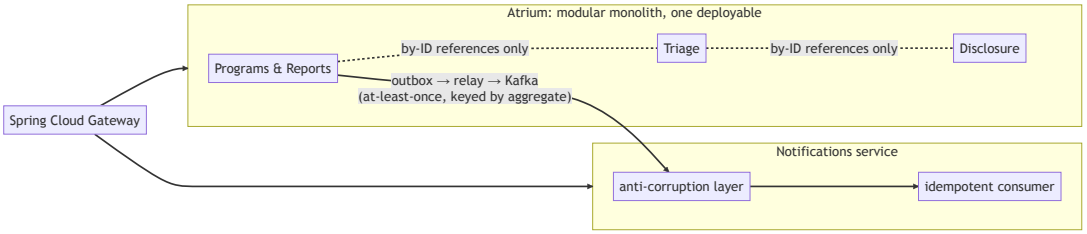
"If the seam crossed a transaction," Kenza answered. "If two services ever had to agree on one fact in the same instant. If we couldn't staff two on-call rotations. If a researcher mid-submission could ever see the partial failure." She paused. "None of those are true for Notifications. All of them are true for everything else."

That answer convinced the room more than any verdict could have. The CTO bought it: one service, a gateway, a design doc.

## The architecture under the slide

On Monday, before the doc, came the Wall. June's three chalk boxes were still up (`Program`, `Report`-owning-`Activity`, `Researcher`, arrows carrying only IDs) from back when the map had no name. Kenza gave it one, and drew the borders one size up.

"The aggregates were the dress rehearsal," she told Imane. "An aggregate is one consistency boundary: one transaction, by-ID references across the line. A **bounded context** is the next size up. It owns its *model* and its *language*: inside Triage a report is a status machine with an SLA clock, and inside Notifications it's an address and a sentence. Decompose by context. Never by entity, which builds my old life's four-call order service, and never by layer, which builds a distributed three-tier: all of the network, none of the autonomy."




Imane pointed at the box on Notifications' border. "And the translator?"

"Anti-corruption layer, the embassy translator. Events arrive speaking Atrium's language and are translated at the door, so one context's schema never becomes another's dependency. Skip it and the payload re-couples the two models, which is the monolith's coupling with a network in the middle."

One more force went up because the Lisbon slide had left it off: Conway's law. Boundaries drift toward the communication structures of the org that builds them, so every service border is also a team border and an on-call rotation. That makes team shape an *input* to the map: eleven engineers could staff two rotations, not seven. Beneath Conway's law went the ledger: **evolvability** and, someday, **organizational scalability** bought; **operability** and **cost** paid, Youssef's column and Leila's.

A week shy of November, January's sentence came due: *"every boundary you respect in January is an option you still own in November,"* Rachid had said over ADR-001, and the context map ran along exactly the seams that record had cut, feature packages and contracts-not-schema, the options exercised instead of sold. Rachid drifted past and spent none of his rationed opinions.

## What the senior sees

The design doc was titled "Why Atrium Stays Whole (Mostly)" and its first section worked through the twelve factors as a plain checklist: config in the environment, backing services as attached resources, stateless and disposable processes, logs as streams. Atrium already passed most of it (a year of scar tissue had seen to that), which was itself an argument that the monolith wasn't the problem the slide deck claimed.

Then came the infrastructure decisions, each recorded with its rejected alternative.

**Discovery: Eureka, evaluated and declined.** Eureka is a registry: services heartbeat in, and clients pull the roster and load-balance client-side. That is essential when you deploy to raw VMs

and addresses churn. “But we’re on Kubernetes,” Kenza wrote. *“*A k8s Service *is* a registry with a DNS name; kube-proxy *is* the load balancer; readiness probes *are* the health-based eviction, doctrine this team has already paid for once. Running Eureka here means operating a second discovery system to disagree with the first.” The decision was `notifications.glasshouse.svc.cluster.local`, and nothing else to patch at 3 a.m.

**Config: Spring Cloud Config, evaluated and declined.** Centralized config with a git backend and runtime refresh via `/actuator/refresh` is genuinely good if you have forty services and config that must change without redeploys. They had two services and ConfigMaps. The one feature they’d miss, broadcast refresh across instances, they didn’t need yet. “Platform primitives win until they don’t,” the doc said. “Revisit at five services.”

**The edge: Spring Cloud Gateway, adopted.** The gateway is one entry point: routing by predicate, cross-cutting filters, and rate limiting before traffic touches a business thread. The limiter was the Redis-backed token bucket: tokens refill at the replenish rate, and burst capacity caps the spike a client can spend before requests are rejected. It was sized for the Stonethrow crowd’s leaderboard-scraping habits (scripts polling the leaderboard hard enough to crowd out real traffic), because the edge is where those habits should be answered. Each bucket needs a key that identifies the caller. Gateway’s default resolver uses the authenticated principal, and the filter answers 403 when the request has none, so the route names a `KeyResolver` bean that returns the caller’s address.

```
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: notifications
              uri: http://notifications.glasshouse.svc.cluster.local:8080
              predicates:
                - Path=/api/v2/notifications/**
            - id: atrium
              uri: http://atrium.glasshouse.svc.cluster.local:8080
              predicates:
                - Path=/api/**
              filters:
                - name: RequestRateLimiter
                  args:
                    key-resolver: "#{@clientAddressKeyResolver}"   # one bucket per
       caller address
                    redis-rate-limiter.replenishRate: 50
                    redis-rate-limiter.burstCapacity: 100
```

*So Gateway is Meridian’s hand-rolled routing servlet with the route table promoted to configuration,* Kenza thought*, and the rate limiter is middleware that someone else already debugged.*

The doc gave the migration its proper name, a **strangler fig**, because the route table *was* the mechanism. Route a slice of traffic at the edge to the new service, grow it under live load, and delete the old code once nothing arrives there. The `/api/v2/notifications/**` predicate was the first slice. Reversibility was built in: at every stage the old path sat one predicate flip away, so the migration could be abandoned on any bad Tuesday. The failure mode went in too. The fig only works if the old path is *frozen*: keep feeding Atrium's notification code and you are raising twins.

**The one synchronous edge.** Notifications needed researcher email preferences, which lived in Atrium. There were two choices: carry the data on the events themselves, or call back. They chose a small synchronous call for preferences (low volume, tolerable staleness) and wrote it as an OpenFeign client: a declarative Java interface, the implementation generated at runtime as a proxy. Imane grinned when Kenza showed her: another implementation class that didn't exist, the repository trick again. Declare the interface, let the container build the proxy. The contract was explicit: connect and read timeouts, a Resilience4j breaker, a retry budget with backoff. The doc carried two footnotes. The first was load balancing: Feign normally pairs with Spring Cloud LoadBalancer to pick an instance client-side from a registry, but the k8s Service name already resolves to a balanced backend, so they pointed it at DNS and deleted a moving part. The second was a version note written conservatively. Spring's own HTTP interface clients (an `@HttpExchange` interface over `RestClient`, in the framework since version 6) give the same declarative shape natively, Framework 7 adds `@ImportHttpServices` to register such interfaces as beans, and OpenFeign upstream is feature-complete and no longer growing. The migration path was named so that whoever re-makes the decision can do it on purpose and skip the archaeology.

"The moment we split," Kenza told the room, "we became our own third-party vendor. Atrium gets the full Cortexa treatment from its own children. No exceptions because 'it's us'. The network doesn't know we're friends."

That sentence was ADR-013 reporting for duty at a boundary that hadn't existed when it was written. September's cascade (Cortexa, the enrichment vendor, slowing until everything queued behind it) had taught the composition: a retry without a budget piles load onto a dependency already failing, and the breaker is the system's memory of recent pain. A split multiplies the addresses where that composition belongs, and every arrow on the new map was one.

**And no distributed transactions.** The report-event flow would be asynchronous, on Kafka, chosen over RabbitMQ mostly for the replayable log and consumer groups. The doc was blunt that the broker mattered less than the contract: *at-least-once delivery, ordering per key at best, and consumers that survive duplicates*. That left one mechanism to get exactly right.

## The fix

The dangerous shortcut was obvious, and Kenza named it in review before anyone shipped it: save the report, then publish to Kafka from the same method. That is two systems and no shared

transaction. Commit-then-crash loses the event; publish-then-rollback invents one. Each side stays locally consistent while the pair disagrees about what happened: the dual-write lie.

The outbox pattern closes the gap by refusing to write to two systems at all. The event is written to a Postgres table *inside* the business transaction, so there is one system and one commit, atomic by construction. A relay then reads unpublished rows, publishes them to Kafka, and marks them sent.

```
create table outbox_event (
    id             uuid primary key,
    aggregate_type text        not null,
    aggregate_id   bigint      not null,
    event_type     text        not null,
    payload        jsonb       not null,
    traceparent    text,                  -- trace context of the request that wrote the
        row
    created_at     timestamptz not null default now(),
    published_at   timestamptz
);
```

The relay was the other half of the pattern, the half the Lisbon slide never drew, and Kenza wrote it herself, because every line was a decision about ordering or loss. The doc's *ordering per key at best* got earned here. Events were keyed by `aggregate_id`, so one report's history lands on one partition, in order, drained oldest-first. The poll used `skip locked`, so eight pods could read the same table without double-claiming a batch. `skip locked` prevents only the double claim: two pods holding different batches can still publish two events of one report out of order, so strict per-report order holds only while one pod drains the table. The producer was idempotent, so a broker retry couldn't reorder what the table had so carefully lined up. The `Tracer`, the `Propagator` and the `try` block around the send belong to the tracing hop at the end of this section.

```
@Component
class OutboxRelay {
    private final JdbcClient jdbc;
    private final KafkaTemplate<String, String> kafka; // enable.idempotence=true:
        retries can't reorder
    private final Tracer tracer;
    private final Propagator propagator;
    OutboxRelay(JdbcClient jdbc, KafkaTemplate<String, String> kafka, Tracer tracer,
        Propagator propagator) {
        this.jdbc = jdbc; this.kafka = kafka; this.tracer = tracer; this.propagator =
        propagator;
    }
    @Scheduled(fixedDelay = 500) @Transactional
    void relay() {
        var rows = jdbc.sql("""
                select id, aggregate_id, payload, traceparent from outbox_event
                where published_at is null order by created_at
                limit 100 for update skip locked""")      // eight pods, no double-
        claim
            .query(OutboxRow.class).list();
        for (var row : rows) {
            var span = propagator.extract(row,            // rejoin the trace that
        wrote the row
                    (r, key) -> key.equals("traceparent") ? r.traceparent() : null)
                .name("outbox-relay").start();
            try (var scope = tracer.withSpan(span)) {
                kafka.send("report-events", row.aggregateId().toString(),
        row.payload())
                     .join();  // keyed by aggregate: one partition, per-key order, and
        publish BEFORE mark
            } finally {
                span.end();
            }
            jdbc.sql("update outbox_event set published_at = now() where id = :id")
                .param("id", row.id()).update();          // crash before this line =
        redelivery, never loss
        }
    }
}
```

The fine print: the relay can publish, then crash before marking the row. On restart it publishes again. At-least-once is not a bug in the outbox; it is the contract. It surfaced in staging within a week, addressed to Imane personally: a test researcher received two identical “Your report has been triaged” emails. Her consumer had been written the innocent way:

```
@KafkaListener(topics = "report-events", groupId = "notifications")
void onReportEvent(ReportEvent event) {
    if (event.type() == ReportEventType.TRIAGED) {
        mailer.sendTriagedEmail(event.reportId(), event.researcherId());
    }
    // Looks correct. Redelivery makes it a duplicate-email generator.
}
```

"Can't Kafka just… deliver it once?" Imane asked.

"Exactly-once is a bedtime story," Youssef said. "Idempotency is the alarm clock that actually wakes you."

Kenza walked her to the fix. The outbox row's UUID rides in the event, and the consumer claims it before acting: an insert into a `processed_event` table with `ON CONFLICT DO NOTHING`. The dedup and the side-effect bookkeeping commit in one local transaction. The table lived in Notifications' own schema, in Notifications' own database, because reaching back into Atrium's tables would have rebuilt the coupling they had just paid to remove.

```
@KafkaListener(topics = "report-events", groupId = "notifications")
@Transactional
void onReportEvent(ReportEvent event) {
    boolean firstDelivery = processedEvents.claim(event.eventId());
    if (!firstDelivery) {
        log.debug("Duplicate delivery of {} ignored", event.eventId());
        return; // at-least-once upstream; duplicates become no-ops here
    }
    if (event.type() == ReportEventType.TRIAGED) {
        mailer.sendTriagedEmail(event.reportId(), event.researcherId());
    }
}
```

And `claim()` itself, the consumer's single load-bearing line, fit on the sticky note Imane stuck to her monitor afterward:

```
create table processed_event (
    event_id     uuid primary key,
    processed_at timestamptz not null default now()
);
```

```
// joins the listener's @Transactional: dedup and the handler's writes commit together,
        or neither does
boolean claim(UUID eventId) {
    return jdbc.sql("insert into processed_event (event_id) values (:id) on conflict do
        nothing")
               .param("id", eventId).update() == 1;  // affected-row count as the
        boolean:
}                                                    // 1 = first delivery, 0 =
        duplicate
```

"The duplicate still arrives," Kenza said. "It just bounces off. You don't prevent redelivery. You make redelivery boring."

One footnote went into the doc anyway: the email send itself isn't transactional. A crash in the sliver between the send and the commit can still double one. The claim doesn't abolish the window; it shrinks it from *every redelivery* to *one unlucky instant*. For side effects that live in the database, where the claim and the effect commit together, the same pattern really is exactly-once.

Leila had listened to the walkthrough while folding her agenda into smaller and smaller squares, content to let the jargon wash past her. Then she asked the only product question in the room: "What does a researcher *see* in the gap?" The honest answer was that a report flips to Awarded and its email lands seconds later, or a minute later if the relay is catching up. "Then the award banner says the email's on its way," she said, and wrote it down. Eventual consistency is a product decision as well as an infrastructure one: the architecture decides *that* the gap exists, and whoever owns the user decides what it looks like.

The last piece was the one Kenza had been privately worried about. With one service, a confused engineer greps one log. With two services and a broker between them, a lost email could have failed in any of four places, and correlation stops being nice-to-have: "which system dropped it" becomes an interrogation with no witness. The Micrometer tracing they'd built into Atrium propagated almost for free. The gateway forwarded the W3C `traceparent` header, and Atrium's instrumented web stack continued the trace. The outbox was the one hop that needed code. The relay runs on a scheduler thread that never saw the request, so the transaction that writes the outbox row also stores the request's `traceparent` value in it, produced by `Propagator.inject`, and the relay starts a span from that value before each send. The `KafkaTemplate` carried the context in record headers once Imane flipped the properties that enable observations on the template and the listener, because Boot instruments the web stack out of the box and waits to be asked about messaging. The Notifications listener picked it up on the far side, and the same trace ID sat in every log line's MDC across all four hops.

On demo day, Kenza put one researcher's submission on the big screen in Grafana: a single trace ID hopping gateway → Atrium → Kafka → Notifications, the broker's hand-off visible as a gap between spans.

"Six months ago we couldn't see one service," Youssef said. "Now we can see two."

The design doc outlived its demo as half a page in the binder:

**ADR-014: The split decision. Context:** Leadership set "event-driven microservices by Q3"; the session argued both sides at full strength against Atrium's actual transaction map. **Decision:** Atrium stays a modular monolith. Notifications (a real bounded context, async by nature, sharing no invariants) extracts by strangler fig behind the gateway; integration is event-driven through the outbox (at-least-once, idempotent consumers), saga-coordinated where a flow spans contexts; an anti-corruption layer guards Notifications' border; discovery is platform DNS, not Eureka, so there is one discovery system; no further cut before its context is mapped on the Wall. **Consequences:** one seam pays distribution's full tax (contracts, partial failure, tracing, a second pager) and proves the pattern; everything else keeps its options. A future seam clears three gates (no shared transaction, own scaling profile, staffable on-call), and the reopening signals are named: team contention at deploy time, divergent scaling, independent compliance.

### ⚖ *Your move, architect*

Strip away the Friday theater and the fork had three prongs. You have seen what they chose. Before the reveal, say in one sentence why each of the other two options loses.

- **A. No split.** The modular monolith, defended on the record, indefinitely.
- **B. Extract Notifications only.** Strangler fig behind the gateway; prove the pattern; record the signals for the next cut.
- **C. Domain-wide decomposition by Q3.** The slide as drawn, every box its own hexagon.

**The trade-offs, since you've committed: A** holds every transactional argument and keeps **operability** and **cost** at their floor, but it answers only the slide and leaves the concerns under it alone: deploy-time contention and orthogonal notification spikes return next year wearing a consultant. **C** spends **operability** and **cost** at scale: every failure mode this year billed for, from April's pool math through September's cascade, multiplied by service count, on a team still learning to run *one* distributed seam. That is a distributed monolith by default. **B** buys the **evolvability** and **organizational scalability** that are at issue, at a tax scoped to one seam, and writes down what would justify the next. The sub-fork inside B is *orchestration* (a coordinator commands steps and compensations: traceable, a coupling point) versus *choreography* (services react to events and the workflow exists nowhere: loose, scalable, emergently complex). Notifications chose choreography because the flow is a broadcast, not a workflow: nothing compensates an email, and no step orders another. Orchestration wins the day a flow carries multi-step compensation with ordering (award, ledger, transfer, each undoing the last), because you want the state machine to have a source file. The shape of that file, the day a flow earns it:

```
// orchestration, the day ADR-014's signals fire:
void run(AwardCommand cmd) {
    ledger.debit(cmd);                      // local tx 1
    try {
        transfers.initiate(cmd);            // local tx 2, another service
    } catch (TransferFailedException e) {
        ledger.credit(cmd.compensation());  // compensate: semantically undo,
        reports.reopen(cmd.reportId());     // in public; users saw the middle state
        throw e;
    }
}
```

In the back row, Rachid watched the whole demo and wrote nothing down, because there was nothing to correct. The following Monday, the recurring design-review meeting on his calendar had a new owner, and the CTO's next architecture question went to Kenza first, by hallway reflex and without anyone announcing a decision. The only cloud was an email from finance, who had noticed that two services meant two cost lines, and had questions about why so many CPUs spent their lives waiting. November could have that one.

## The debrief: Ch.14, The Split

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 14.** Should we split the monolith into microservices?

Then the follow-ups, cold:

1. How do you publish an event if-and-only-if the database transaction commits?
2. Why not Eureka and Spring Cloud Config on Kubernetes?
3. Leadership mandates "microservices by Q3"; what's your first move?
4. How does the outbox relay publish, and what does it promise about order and loss?
5. What is a Gateway route made of, and where does the rate limit sit?

Tick what you said:

- ☐ Claim: Split only where the boundary is real (independent scaling, no shared transactions, separable ownership) and defend the modular monolith everywhere else.
- ☐ Mechanism: Business state and an outbox row in one local commit, a relay publishing at least once, and consumers claiming each event ID idempotently.
- ☐ Trade-off: Independent deploys and an isolated blast radius, at the price of network latency, eventual consistency and partial failure in the UX.

- [ ] Failure mode: Splitting a chatty, transactional domain builds a distributed monolith, where sagas and compensating actions replace one ACID commit.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** Split along seams: extract a service only where the boundary is real (independent scaling, no shared transactions, separable ownership) and defend the modular monolith everywhere else. **Mechanism:** a true seam needs no distributed transaction: the producer writes business state *and* an outbox row in one local commit, a relay publishes to the broker at least once, and consumers make duplicates harmless by claiming each event ID idempotently; synchronous edges get a gateway, declarative clients, timeouts, breakers, and propagated trace context. **Trade-off:** you buy independent deploys, isolated blast radius, and per-service scaling at the price of network latency, eventual consistency, partial failure surfacing in the UX, and an observability-and-on-call tax levied on every new boundary. **Failure mode:** split a chatty, transactional domain and you build a distributed monolith: sagas and compensating actions doing badly, in public, what one ACID commit did silently for free, debugged across four logs by a team too small to staff the pager.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Evolvability** and **organizational scalability** bought (independent deploys, isolated blast radius, two teams off one pipeline) versus **operability** and **cost** spent (a second pager, versioned contracts, four-hop tracing, finance's second line). The discipline: the purchase was argued *on the record*, because a decision that can't show its losing argument is a fashion choice.
- **The ADR:** ADR-014, the monolith stays; Notifications out by strangler fig behind the gateway; outbox + idempotent consumers; ACL at the border; platform DNS over Eureka; three gates and three reopening signals for any next seam: contention, divergent scaling, independent compliance.
- **Rejected alternatives:** domain-wide decomposition and no-split (both argued at full strength above); Eureka + Cloud Config on Kubernetes (a second discovery system to disagree with the first); an orchestrated saga for notifications (a coordinator for a broadcast, with nothing to command).
- **Fight or lean:** *Lean* on the platform: k8s DNS, ConfigMaps until five services, Gateway's token bucket, Micrometer's `traceparent` propagation. *Fight* the catalog reflex: Spring Cloud will hand you a registry, a config server, a bus, forty hexagons, each a 3 a.m. system. The seam decides what you adopt, not the conference.

**Story anchors**

- The rotated slide deck, *"the seam decides, not the conference"* → monolith-first: split criteria vs. shared transactions, tiny team, chatty domain.
- Imane's two "report triaged" emails → outbox + at-least-once delivery; idempotent consumers dedup by event ID.
- One trace ID hopping four systems on the big screen → Micrometer trace propagation; correlation becomes existential after a split.
- June's three chalk boxes growing borders into the context map → bounded contexts (a model *and its language*) as the unit of decomposition; the ACL as embassy translator; January's options, finally exercised.

**Interview ammo**

- *Kill-shot: Argue both sides of "should this be microservices?" and explain saga-based consistency.* For: independent scaling, blast-radius isolation, team autonomy. Against: shared transactions, partial-failure UX, headcount. Where you must split across a transaction, a saga chains local commits with compensating actions instead of rollback: choreography (reactive events, invisible control flow) for short flows, orchestration (explicit state machine) when steps and compensations multiply.
- *Follow-up: How do you publish an event if-and-only-if the database transaction commits?* You can't atomically span DB and broker, so don't: write the event to an outbox table inside the business transaction, relay it after commit, and accept at-least-once with consumer-side dedup.
- *Follow-up: Why not Eureka and Spring Cloud Config on Kubernetes?* A k8s Service already gives DNS-based discovery, load balancing, and readiness-driven eviction, and ConfigMaps cover externalized config. Platform primitives win until you have enough services to need runtime refresh and a richer registry, and that decision should be recorded, not defaulted.
- *Architect follow-up: Leadership mandates "microservices by Q3"; what's your first move?* Map bounded contexts first; extract the one seam that crosses no transaction, strangler-fig, behind the gateway; make it pay distribution's full tax (outbox, idempotent consumers, traces) to prove the pattern; and record the gates and signals that justify the next cut.
- *Follow-up: How does the outbox relay publish, and what does it promise about order and loss?* A scheduled poll reads unpublished rows oldest first with `for update skip locked`, sends each one keyed by aggregate ID so one report's events land on one partition, and marks the row only after the send, so a crash in between means redelivery and never loss. `skip locked` prevents only the double claim: two pods holding different batches can publish two events of one report out of order.

- *Follow-up: What is a Gateway route made of, and where does the rate limit sit?* A route is an ID, a target URI, predicates that match the request (`Path=/api/v2/notifications/**`) and filters that act on it. The `RequestRateLimiter` filter is a Redis-backed token bucket, refilled at the replenish rate and capped by the burst capacity, keyed per caller by a `KeyResolver` bean, and it rejects before the request touches a business thread.

**Pointers** → `examples/ch09_microservices/`

---

# Chapter 15: Faster

The November cost review had a spreadsheet, and the spreadsheet had a row that hurt: *Atrium, 14 pods × 8 vCPU. Average CPU utilization: 9%.*

"Nine percent," the finance lead repeated, in the tone of someone reading a parking ticket aloud. "What is the other ninety-one percent doing?"

"Waiting," Youssef said. "Professionally."

It was true, and Kenza knew exactly why. In the April outage, nearly two hundred Tomcat threads per pod had parked in `NioSocketImpl.implRead` and the whole site 503'd while CPU idled. Afterwards the team had sized the pools by trauma: big pods and big pools, with enough patience tokens that they'd never run out again. The fleet had grown the same way, from eight pods to fourteen that autumn, and Postgres's `max_connections` had been raised to make room for six more connection pools. The structural fixes had come since: timeouts on every outbound call, a circuit breaker and bulkhead on the Cortexa boundary, the enrichment call made deferrable. The threads almost never all parked anymore. They just sat there, allocated and idle, while Glasshouse paid rent on them. The spreadsheet, Kenza thought, was April's napkin corner (*virtual threads, later*) arriving with an invoice attached.

Then Leila slid the Northbeam renewal rider across the table. "There's a new clause. p99 latency on report submission, five hundred milliseconds, measured monthly. Misses convert to service credits." She tapped the page. "So we need to be cheaper *and* faster. I'm told that's not how anything works."

Two camps formed by lunch. Imane, freshly WebFlux-curious, had a branch and a gleam: "Event loops. A handful of threads doing all the work, nothing ever blocks, this is what reactive was *built* for." The other camp was one line of YAML. Kenza took both to Rachid's office, and Rachid, weeks from his last day and acting like it, held up a hand before she finished the question.

"This one's yours. Show me the numbers."

## The investigation

The bake-off ran in staging against a Cortexa stub tuned to realistic latency: 180 milliseconds per enrichment call. One thing about that call had changed in October: Cortexa's v2 API came with its own Java SDK, and the Cortexa wrapper now called the SDK where the hand-written `RestClient` request used to be. The path under test was submission-plus-enrichment as it had run since the cascade (September's failure, when a slow synchronous enrichment call held request threads until the whole site queued behind it): the JPA write answering the researcher straight away, the enrichment call following on its bounded, named executor. A Gatling rig drove

it at 1×, 1.5×, and 2× of peak production load, with the Northbeam p99 measured at the submission endpoint.

There were three contenders. The baseline was platform threads and the pools they had. Contender two was the flag.

```
spring:
  threads:
    virtual:
      enabled: true   # since Boot 3.2; flips Tomcat workers, @Async,
                      # @Scheduled and listeners onto virtual threads
```

The flag reaches only the executors Boot builds. The `enrichmentExecutor` had been built by hand in April, a `ThreadPoolTaskExecutor` that stays on platform threads whatever the flag says, so for this contender it was switched as well: `setVirtualThreads(true)`. That changed one variable: the pool kept its sizes and its bounded queue, and only its workers became virtual threads.

Contender three was Imane's WebFlux spike, one endpoint rebuilt reactive end to end:

```
public Mono<SubmissionReceipt> submit(SubmissionRequest req) {
    return reportRepository.save(toEntity(req))        // R2DBC, not JPA:
        .flatMap(saved -> enrichment.score(saved.id()) // JPA would block the loop
            .timeout(Duration.ofSeconds(2))
            .onErrorResume(e -> Mono.just(Score.pending())))  // the honest badge
        .map(SubmissionReceipt::from);
}
```

Kenza stared at that snippet longer than the others. *Eight years of hand-sized Jetty pools had trained her that scale meant threads: you counted them, sized them and watched them. This thing rewrote every signature on the way down and turned control flow into a pipeline she couldn't step through in a debugger.* Her instinct voted against Imane's camp on reflex. She made a note of that, the way Rachid had taught her to note any opinion she'd held before seeing evidence.

The numbers came back Tuesday. The WebFlux spike won the synthetic throughput test clearly: on that one endpoint, a fraction of the threads at three times the load. Kenza put that result at the top of the readout, in bold, before anyone could accuse her of burying it.

Virtual threads matched it almost exactly at 1× and 1.5×, with the existing blocking code untouched. p99 sat at 240 milliseconds. Then the rig ramped to 2×, and the virtual-thread run fell off a cliff: p99 went from 240 milliseconds to eleven seconds in ninety seconds of ramp. CPU stayed near idle and throughput flatlined. It looked as if someone had reinstalled the old thread pool when nobody was looking.

> ⏸ **Your call.** CPU bored, throughput flat at 2×. That is the old pool's exact signature, on a runtime with no pool. Before the tracer goes on: what ceiling just got reinstalled, by what mechanism, and why do the SLO and the spreadsheet fail together?

Someone effectively had. Kenza reran the load with `-Djdk.tracePinnedThreads=full`. The war-room screen filled with stack traces: virtual threads blocking *without unmounting*, each trace annotated at the frame that held a monitor.

Imane found it first: `NioSocketImpl.implRead` near the top, and a dozen frames down, `com.cortexa.sdk.transport.PooledConnection.checkout`, flagged `<== monitors:1`. The vendor SDK checked connections out of its internal pool inside a `synchronized` method, then did blocking socket I/O while still holding the monitor. It did this from inside their nice bulkheaded wrapper.

She read it twice. "It's not even our code."

"It's our process," Kenza said. "Keep looking."

A JFR recording found the second one, and this one *was* their code. The `jdk.VirtualThreadPinned` events could not show it, and neither could the tracer: on JDK 21 both report a virtual thread only when it parks while pinned, and this code never parks. It showed up under `jdk.JavaMonitorEnter`, the event for a thread kept waiting at a contended monitor, after Kenza dropped that event's threshold from 20 milliseconds to zero and sorted by monitor class. A utility class from 2018, faithfully signing every outbound Cortexa request:

```
// glasshouse-common, last meaningful commit: 2018
public final class RequestSigner {

    private final MessageDigest digest;   // MessageDigest isn't thread-safe,

    RequestSigner(MessageDigest digest) {
        this.digest = digest;
    }

    public synchronized String sign(byte[] payload) {  // ...so: this
        digest.reset();
        return HexFormat.of().formatHex(digest.digest(payload));
    }
}
```

There was one shared signer bean, so one monitor, and every outbound request in the company funneled through it.

(If you called pinning at the pause: full credit only if you priced it. *Every block under a held monitor costs a carrier, of eight.*)

## What the senior sees

Rachid didn't come to the whiteboard session. He sent Imane instead ("ask Kenza, she owns this"), so Kenza drew it on the Wall, next to the connector box that began the Request Journey, the wall-length trace of one request from socket to database and back.

"A virtual thread isn't an OS thread," she said. "It's a stack the JVM keeps on the heap. It runs by *mounting* onto a carrier. The carriers are a small pool of platform threads, by default one per core, so eight on these pods. When the virtual thread hits blocking I/O, the JVM unmounts it: stack saved to the heap, carrier freed for the next one. You keep writing thread-per-request blocking code, and the ceiling on threads just stops existing. A million concurrent virtual threads is unremarkable. Fourteen pods of two hundred precious threads becomes six pods of as many cheap ones as the work needs."

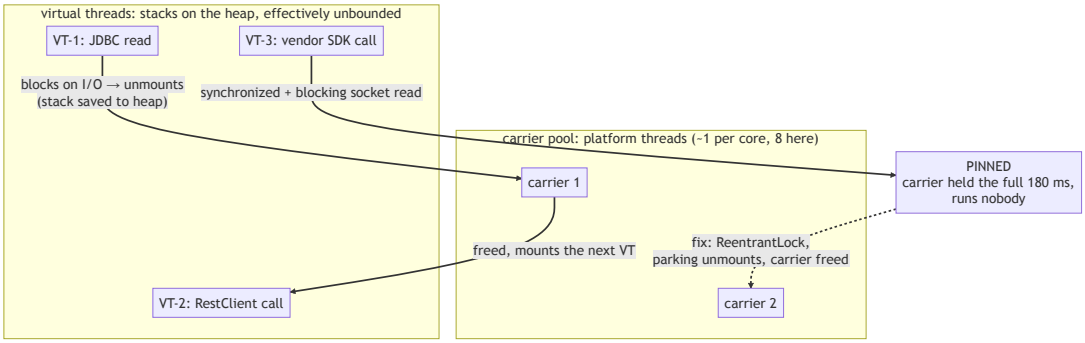
She tapped the connector box where April's napkin math had never been erased. Concurrency in flight equals arrival rate times residence time. Two hundred threads at thirty seconds of residence had once priced a whole pod at six point six requests a second. "Little's law didn't change," she said. "Re-run it with the ceiling removed: residence still costs a heap stack, just no longer a scarce thread. April's arithmetic gets its structural answer, the one filed in the napkin's corner under *later*."

"It's later," Youssef said from the doorway, sounding satisfied.

"So why the cliff?" Imane asked.

"Pinning. On Java 21, a virtual thread inside a `synchronized` block can't unmount; the JVM can't release the carrier while a monitor is held, and a thread blocked trying to *enter* a contended monitor pins its carrier too. Eight carriers. The SDK blocks on a socket read *while holding* a connection's monitor: that's one carrier gone for 180 milliseconds, doing nothing. Our signer costs less and hides better: the work inside is microseconds and nothing parks, but a thread that arrives while another is inside waits for that monitor on its carrier, and every outbound request takes it. A few pinned carriers out of eight, and the whole application is back to standing in line. Enrichment doesn't even run on the request path anymore, and it didn't matter. Carriers are a process-wide resource; the pinning taxed everyone, submissions included. We didn't remove the thread-pool ceiling. We lowered it to eight and hid it."

She drew the trap and its exit, in the do-not-erase corner:




The classic offender, she told Imane, was JDBC drivers, historically full of `synchronized`. The PostgreSQL driver had already migrated its hot paths to `java.util.concurrent` locks; that was why the database calls came back clean. Vendor SDKs were the new frontier. And the fix had a name and a date: JEP 491, delivered in JDK 24, taught the JVM to track monitor ownership per virtual thread, so `synchronized` no longer pins there (native-frame pins, such as a JNI call deep in a driver, are a separate case it doesn't touch). But Atrium ran the 21 LTS, where `synchronized` still did. You verify the fix on *your* runtime when you upgrade. *`synchronized` had always been free at Meridian,* Kenza thought. *A platform thread didn't care what it blocked inside; the pool just got slower. A virtual thread cared: the monitor nailed it to its carrier and let you find out under load.*

"And WebFlux?" Imane asked, a little mournfully.

"WebFlux solves this by never blocking at all: an event loop with a few threads, work expressed as `Mono` and `Flux` pipelines, and backpressure so a fast producer can't drown a slow consumer. When you genuinely need hundreds of thousands of concurrent connections, streaming, or flow control as a first-class concept, nothing else does it. But the costs are structural. The reactive types spread through every signature they touch, and a stack trace becomes a tour of Reactor operators. And one blocking call on the event loop, one innocent JPA query, is the cardinal sin that stalls the whole loop. Your spike was honest because it went R2DBC end to end. The real Atrium has seven years of JPA and eleven imperative programmers."

"One more thing," Kenza added, tapping the Cortexa box with its September scars: the breaker that remembers *their* pain (it opens after a failure threshold and fails fast instead of queueing) and the bulkhead capping what *we'll* spend waiting (a hard ceiling on concurrent calls into the SDK). "Virtual threads remove the *thread* ceiling, not Hikari's connections or Cortexa's tolerance. ADR-013 governs more than before: a million cheap threads crowd a scarce resource faster than two hundred precious ones ever did. The old pool was an accidental bulkhead for everything behind it; now the bulkheads we *chose* are the only ones left."

Rachid appeared in the doorway just long enough to hear the summary, nodded once, and said, "Every free lunch has a coat check." Then he left.

### ⚖ *Your move, architect*

Rewind to Tuesday's readout: three sets of numbers and no verdict, with a recommendation due Friday. This chapter *is* the memo, so write yours before reading Kenza's: your option, its price in dollars, risk, and team, and the trigger that reopens the decision.

- **A. Status quo plus autoscaling.** No engineering decision; the HPA adds pods when traffic climbs, and the cloud bill absorbs what the architecture won't.
- **B. Virtual threads plus the pinning audit.** One flag, with semantics preserved and the ceiling removed; the audit is the standing price.
- **C. The WebFlux rewrite.** The highest ceiling and the only true backpressure, paid across seven years of JPA and eleven imperative debuggers.

**The grading, not the answer:** your memo passes if it (1) prices each option in dollars, risk, *and* team; (2) names the question each one answers, since "cheaper concurrency?" is not "backpressure?"; (3) ends with the trigger that reopens it. Kenza's memo is ADR-015, three pages on. Grade yourself against it. One analysis you may not skip: why A is a decision-shaped absence of one.

## The fix

The purge came in three moves, smallest blast radius first.

Their own code came first: `synchronized` became `ReentrantLock`, because a virtual thread blocking on a `java.util.concurrent` lock parks cleanly. It unmounts, and the carrier moves on.

```
public final class RequestSigner {

    private final ReentrantLock lock = new ReentrantLock();
    private final MessageDigest digest;

    RequestSigner(MessageDigest digest) {
        this.digest = digest;
    }

    public String sign(byte[] payload) {
        lock.lock();          // a virtual thread waiting here unmounts;
        try {                 // the carrier is free to run someone else
            digest.reset();
            return HexFormat.of().formatHex(digest.digest(payload));
        } finally {
            lock.unlock();
        }
    }
}
```

Imane asked the right follow-up: why share one digest at all? A fresh `MessageDigest` per call would delete the lock entirely. They filed it; the lock shipped the same afternoon, because the cliff was the pinning, not the hashing.

The vendor SDK they couldn't edit, so they contained it: the existing Cortexa wrapper gained a small, dedicated *platform*-thread executor of sixteen named threads, and every SDK call ran there. Kenza wrote the bean herself and commented it the way you label a live wire. The flag they were about to ship had made the obvious construction wrong:

```
@Bean("cortexaSdkExecutor")              // PLATFORM threads, on purpose: under the
ExecutorService cortexaSdkExecutor() {   // virtual-threads flag, Boot's own task
    return Executors.newFixedThreadPool( // executor would hand the pinning SDK
        16,                              // virtual threads: a quarantine of nothing
        Thread.ofPlatform().name("cortexa-sdk-", 0).factory());
}
```

A platform thread that blocks is a cost you've budgeted; a pinned carrier is a tax on everyone else in the JVM. A vendor ticket went out the same day with the stack trace attached.

The first rerun barely moved. The callers were still pinned, and the tracer said where: `ConcurrentHashMap.compute`, flagged `<== monitors:1`, underneath Caffeine. It had been on the war-room screen all along, a second flag further down the same traces, and everyone had stopped reading at the first. September's `sync = true` makes the cache run its loader inside that method, under a monitor, so a virtual thread that missed the cache waited for the quarantine pool while pinned to its carrier. The third move was one attribute: `sync = true` came off the `cve-enrichment` cache. The PR carried the price and the expiry date. A key that expires under load can now be loaded by several callers at once, capped by the bulkhead's sixteen permits, and `sync` goes back on a JDK where `synchronized` no longer pins.

Then they ran JFR again on the same load profile: `jdk.VirtualThreadPinned` events were down from thousands per minute to single digits, and p99 at 2× load held at 310 milliseconds. The before/after histograms went up on the Wall, taped under the connector box.

The flag shipped to production behind a canary. Two weeks later Youssef decommissioned pods with visible pleasure: fourteen down to six, heaps re-sized for the smaller pods, and G1 kept as the collector. They discussed ZGC for the latency tail and wrote down *not yet, revisit if the SLO tightens*. A low-overhead JFR recording was left always-on; the pinning hunt had made converts of everyone. `RestClient` was confirmed as house standard for outbound calls: blocking and fluent, perfect over virtual threads. `WebClient` stayed legal only inside genuinely reactive code, and `RestTemplate` was declared what it was: deprecated in favor of `RestClient` as of Spring Framework 7, and not for new code.

Then Youssef wired the recording into alerting: a `jdk.VirtualThreadPinned` events-per-minute threshold that opens a ticket (ADR-012 keeps the pager for SLO burn). The Gatling rig was kept warm in staging. It was January's move, the red build that names its own law (an executable architecture rule that fails CI and prints the law it enforces), aimed at a runtime instead

of a package diagram: a fitness function for the concurrency model. The decision was reversible because it was measured; when the histograms move, the memo reopens.

One experiment remained, because the cost review had two rows. Notifications was small, simple, bursty and idle most of the night, everything Atrium wasn't. Imane ran it through Spring AOT and GraalVM with `./mvnw -Pnative native:compile`. The build took nine minutes, and one library needed explicit `RuntimeHints`. Native image is a closed world: no reflection, proxies or classpath scanning at runtime unless declared at build time. *You trade away the dynamism you weren't using,* Kenza thought, *and declare the reflection you were.* The result was an 80-millisecond startup against 3.2 seconds on the JVM, and a third of the memory. With startup that cheap, Notifications could scale to zero overnight and cold-start on the first morning event. That takes an autoscaler driven by the queue, such as KEDA on the topic's lag or an HPA on an external metric; a CPU-based HPA cannot scale to zero, because CPU is measured only on running pods. The same arithmetic, Kenza noted in the memo, made native image the natural choice for CLI tools (instant start, no JVM warm-up to amortize before the command exits) and for serverless functions, where billing lives and dies on the cold start. A long-running, JIT-warmed monolith like Atrium sat at the opposite end of that curve.

Atrium, they decided, would stay on the JVM, and the memo said why: nine-minute builds on every CI run for a service that deploys daily; seven years of reflection-rich code to audit; and a long-running hot service is exactly where JIT-compiled peak throughput beats an AOT binary. They took the cheap startup wins instead: layered images, so the fat dependency layers stayed cached and an ordinary code change shipped as a thin top layer. They also took a hard look at `spring.main.lazy-initialization`. Kenza rejected it for production in one line: it moves failures from startup to first request, and a startup crash is the system doing you a favor.

Kenza wrote the memo, which Rachid would later call the real deliverable, with Leila at the table, because Leila owned the column engineers pretend not to see. The memo priced every option in dollars, risk, and team: whose debugging instincts the runtime matches at 3 a.m. "Cost is an -ility," she said when Imane raised an eyebrow. "It scales, it degrades, it just acquired a contract. Architecture is capital allocation with stack traces."

The decision memo closed with the rule, boxed:

> I/O-bound workload + imperative team → virtual threads. Genuine streaming or backpressure needs → reactive, end to end or not at all. Revisit pinning assumptions on every JDK upgrade.

By Friday the memo wore a number in the binder:

> **ADR-015: Runtime selection. Context:** A cost review (fourteen pods at 9% CPU) met a contractual latency SLO; the I/O-bound monolith paid rent on parked platform threads sized by April's trauma. **Decision:** Virtual threads on Atrium, with thread-per-request semantics kept and the pool ceiling removed, after a load-tested bake-off and a pinning audit (`synchronized` hotspots → `ReentrantLock`; vendor code quarantined on a platform pool; `sync` off the enrichment cache); WebFlux declined (rewrite cost across seven years of JPA, team-model mismatch), the genuine streaming/backpressure case named; GraalVM native image for Notifications only, where scale-to-zero pays its build and closed-world tax; the JIT-warmed monolith stays on the JVM. **Consequences:** Throughput per pod rises with imperative code intact (fourteen pods → six); accepted costs: the pinning audit as a permanent review item on JDK 21, native image's build/reflection tax confined to one small service. **Re-evaluate on:** JDK releases that relax `synchronized` pinning, traffic that turns streaming, an SLO tighter than carriers can serve.

## One green line

Imane archived her WebFlux spike herself, with a writeup titled "What Reactive Would Cost Us, With Numbers," and she was prouder of the analysis than she'd been of the branch. "The process convinced me more than the verdict did," she admitted, and Kenza heard a sentence she might have said in February.

The memo came back from Rachid on paper, as everything did. It had no margin notes and no questions, only one line of green ink at the bottom:

*Ship it. And the day this stops being true, write the next one.*

It took Kenza a moment to place the sentence: ADR-015's re-evaluation clause, in green. He'd been writing decisions with expiry dates all along.

He slid the pen back into his pocket more slowly than usual, and nobody in the room pretended not to understand what had just happened. December, his last month, was four days away. Somewhere upstairs, an exec was already composing a hallway request.

## The debrief: Ch.15, Faster

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 15.** Virtual threads vs WebFlux for an I/O-bound service: when do you pick each, and what's the catch with virtual threads?

Then the follow-ups, cold:

1. Would you ever pool virtual threads?
2. If virtual threads make blocking free, why does WebFlux still exist?
3. A team lead asks “should we rewrite this service reactive?” What’s your first move?
4. On JDK 24 and later, what still pins?
5. What happens when ten thousand virtual threads meet ten database connections?

Tick what you said:

- ▢ Claim: Virtual threads for an I/O-bound service with an imperative codebase, WebFlux for genuine streaming and backpressure needs.
- ▢ Mechanism: Virtual threads mount onto a small carrier pool and unmount on blocking I/O. WebFlux removes blocking with an event loop.
- ▢ Trade-off: Virtual threads keep blocking-style code but offer no flow control. Reactive gives backpressure at the price of contagious types.
- ▢ Failure mode: Pinning: on Java 21 a virtual thread blocking inside `synchronized` holds its carrier. Detect with `jdk.VirtualThreadPinned`, fix with `ReentrantLock`.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** for an I/O-bound service with an imperative codebase, virtual threads are the right default; WebFlux is for genuine streaming and backpressure needs. **Mechanism:** virtual threads are JVM-managed stacks that mount onto a small carrier pool and unmount on blocking I/O, which gives thread-per-request semantics without the pool ceiling and with zero code rewrite (`spring.threads.virtual.enabled=true`, since Boot 3.2); WebFlux instead removes blocking entirely via an event loop and `Mono`/`Flux` pipelines with backpressure. **Trade-off:** virtual threads keep blocking-style code, debugging, and stack traces, but offer no flow control; reactive gives backpressure and extreme concurrency at the price of contagious types, operator-soup stack traces, and a hard ban on blocking the loop (JDBC is the cardinal sin). **Failure mode:** pinning. On Java 21 a virtual thread blocking inside `synchronized` (or native frames) can’t unmount and holds its carrier; a few pinned carriers out of ~one-per-core and the app silently reacquires a tiny thread-pool ceiling. Detect with JFR’s `jdk.VirtualThreadPinned`, and on JDK 21 to 23 with `-Djdk.tracePinnedThreads`; fix with `ReentrantLock` and by quarantining `synchronized`-heavy vendor code on a dedicated platform pool.

**THE ARCHITECT’S LENS**

- **-ilities at stake: Cost**, promoted to first class (the memo prices every option in dollars, risk, and team capability; runtime selection is capital allocation with stack traces), and **performance** (the contractual p99), which usually trades *against* cost; virtual threads are the rare move serving both. The bake-off and the pinning tracer are fitness functions, and their evidence keeps the decision reversible.
- **The ADR:** ADR-015, runtime selection: virtual threads on Atrium after a bake-off and a pinning audit; WebFlux declined, its genuine case named; native image for Notifications only, the monolith staying on the JVM; re-evaluation triggers (JDK pinning fixes, traffic-shape changes, a tighter SLO) written into the record. It is a decision with an expiry date.
- **Rejected alternatives:** autoscaling the status quo (a linear cloud bill standing in for a decision); the WebFlux rewrite (a backpressure answer to a question Atrium never asked, at rewrite prices); native image for the monolith (nine-minute builds, a reflection audit, and JIT beats AOT where hot services live); ZGC and `lazy-initialization` (*not yet* and *no*, with reasons; recording non-decisions is also architecture).
- **Fight or lean:** *Lean* on the platform: one flag flips Tomcat workers, `@Async`, and `@Scheduled` onto virtual threads, and `RestClient` over virtual threads is the house's outbound story. *Fight* the monitors: `synchronized` in 2018 utility code and vendor SDKs silently re-installs the ceiling; the audit, the tracer, and the quarantine pool are permanent disciplines that outlast the purge.

**Story anchors**

- Fourteen pods at 9% CPU → blocking-I/O economics: thread-per-request means paying rent on parked threads, sized by trauma instead of arithmetic.
- `<== monitors:1` on the war-room screen and Imane's "it's not even our code" → pinning: `synchronized` plus blocking I/O nails the virtual thread to its carrier; vendor SDKs and old JDBC drivers are the classic source.
- Notifications starting in 80 milliseconds after a nine-minute build → GraalVM native image: startup and memory bought with build time and a closed world; worth it for scale-to-zero, wrong for the hot monolith.
- The memo written with Leila (dollars, risk, team, an expiry clause) → cost as a first-class -ility: runtime selection as a priced decision, kept honest by ADR-015's re-evaluation triggers.

**Interview ammo**

- *Kill-shot: Virtual threads vs WebFlux for I/O-bound work, and the virtual-threads catch?* Virtual threads for I/O-bound imperative codebases (blocking model kept, pool ceiling gone via carrier mount/unmount); WebFlux only for real streaming/backpressure needs; the

catch is pinning: on Java 21, `synchronized` around blocking calls holds the carrier and silently restores a tiny ceiling.

- *Follow-up: Would you ever pool virtual threads?* Not to save threads. They're cheap and disposable by design, and pooling reinstates the very ceiling they remove. If a downstream resource is scarce, bound concurrency with a `Semaphore`, not a pool.
- *Follow-up: If virtual threads make blocking free, why does WebFlux still exist?* Because backpressure is a semantic, not a throughput hack. When a consumer must slow a producer or you're streaming unbounded data, `Flux`'s flow control is the feature, and no amount of cheap threads provides it.
- *Architect follow-up: A team lead asks "should we rewrite this service reactive?" What's your first move?* Reframe before pricing: "reactive?" usually means "cheaper concurrency?", which virtual threads answer for one flag plus a pinning audit; reserve WebFlux for the question it answers (backpressure, extreme fan-out streaming), and settle it with a bake-off and a pinning detector.
- *Follow-up: On JDK 24 and later, what still pins?* JEP 491, delivered in JDK 24, tracks monitor ownership per virtual thread, so `synchronized` no longer pins there. A native frame, such as a JNI call deep in a driver, is a separate case that JEP 491 doesn't touch. Verify the fix on your own runtime when you upgrade.
- *Follow-up: What happens when ten thousand virtual threads meet ten database connections?* The threads crowd the pool. Virtual threads remove the thread ceiling and leave Hikari's connections as scarce as before. The old thread pool was an accidental bulkhead for everything behind it, so the bulkheads you chose are the only ones left.

**Pointers** → `examples/ch12_extensions/ReactiveStackAndClients.java`, `examples/ch12_extensions/NativeImageAndAot.java`, `examples/ch06_server/VirtualThreadPinningTrap.java`

---

# Chapter 16: The Architect's Chair

The ask arrived the way the worst asks always do: in a hallway between meetings, dressed as a small thing.

"Kenza, quick one." The CTO didn't break stride. "We need duplicate-report detection. Researchers keep filing the same vuln five times, triagers are drowning, and Northbeam brought it up at the QBR. Something by Q1?"

By the time she was back at her desk, Rachid had forwarded the thread. His entire message was one line: *Design it out loud. The whole team. Friday.*

It was December, his last month. Nobody had said the word *exam*, which was how she knew it was one.

*The old instinct was to open the IDE, scaffold a* `DedupController` *and let the design emerge from the typing. It was muscle memory from her Meridian years, and it was the trap.* You didn't find out what a duplicate was by writing a loop over reports. So on Wednesday she designed nothing. She spent the day with Malika.

"What *is* a duplicate?" Kenza asked. "Same root cause? Same endpoint? Same payload?"

"Same root cause," Malika said immediately. "Two XSS reports on different pages of the same template engine are one vuln. The engine will never know that. A human has to decide."

"And what does a wrong answer cost?"

Malika thought about it. "A missed dupe wastes an hour of triage. A false merge buries a real vulnerability under a closed one." She paused. "One of those is annoying. The other one is the company."

Kenza wrote that sentence at the top of the design doc. Then came five questions, answered in writing: what is a duplicate, who decides, what does each wrong answer cost, what's the volume and latency budget (four hundred reports a week, thirty-ish dupes, suggestions worthless if they arrive after triage starts), and what number means success. Only then did she start drawing.

### ⚖ Your move, architect

This time the fork comes before the room does. You have what Kenza had on Wednesday night: Malika's asymmetry, the numbers and the binder behind you. Design the shape before she speaks. Where does similarity scoring run?

- **A. Synchronous check on the submit path.** The scorer runs inline, so suggestions exist the instant the report does, and there is no pipeline to build.
- **B. Async pipeline, advisory suggestions.** Submission commits and returns. Scoring rides an event off the outbox, candidates surface seconds later, and only a human merges.

- **C. Nightly batch dedup.** One off-peak job marks candidate pairs while everyone sleeps. It is the cheapest possible build.

**The trade-offs, since you've committed: A** couples submission **availability** and **latency** to the cost and uptime of a scoring engine, which violates ADR-006 and ADR-013 on their face: when the scorer has a bad afternoon, *submissions* do. **C** wins on **cost** and fails the requirement. Duplicates burn triager hours *same-day*, so the binding budget is the triagers' morning, not the API's p99. **B** is what Kenza drew. Her whiteboard walk is the answer key and the grilling that follows is your pressure test, so score yourself against the room.

## The investigation

Friday, ten a.m., and the whole team was in front of the Wall. Kenza opened with the five questions and Malika's asymmetry, because every decision downstream hung off it: *the system suggests; a human merges; nothing auto-closes, ever.*

The sequence she walked was fixed: clarify the spec, then contract, data model, transaction and consistency boundaries, failure modes, observability, rollout. The room had watched it assemble all year without hearing its name, the architect's kata, and the proof sat in the repo: ADR-005 was the contract step, ADR-008 the consistency step, ADR-013 the failure step. So Friday would quote the trail without relitigating it.

She sorted the decisions before drawing any of them. "Most of this is a two-way door," she said. "The threshold, the scorer behind its port, the flags: if we're wrong, we walk back through. Cheap experiments, kept options." The API contract and the merge's consistency promise were not. Those harden the moment other people build on them. ADR-005's v2 contract and ADR-008's consistency line had been the year's one-way doors, committed decisively and front-loaded, because deciding later means deciding under worse information, after others have built on your indecision. The architect's first skill was knowing which door you stood in before choosing a way to walk through it.

Then the contract:

```
GET /api/v2/reports/18342/duplicate-candidates
Authorization: Bearer eyJ...        # triager role required

HTTP/1.1 200 OK
Content-Type: application/json

{ "candidates": [
    { "reportId": 17906, "score": 0.91,
      "title": "Stored XSS in program description", "status": "TRIAGED" }
  ],
  "engineStatus": "LIVE" }          # or "DEGRADED": honest absence, never invented
data
```

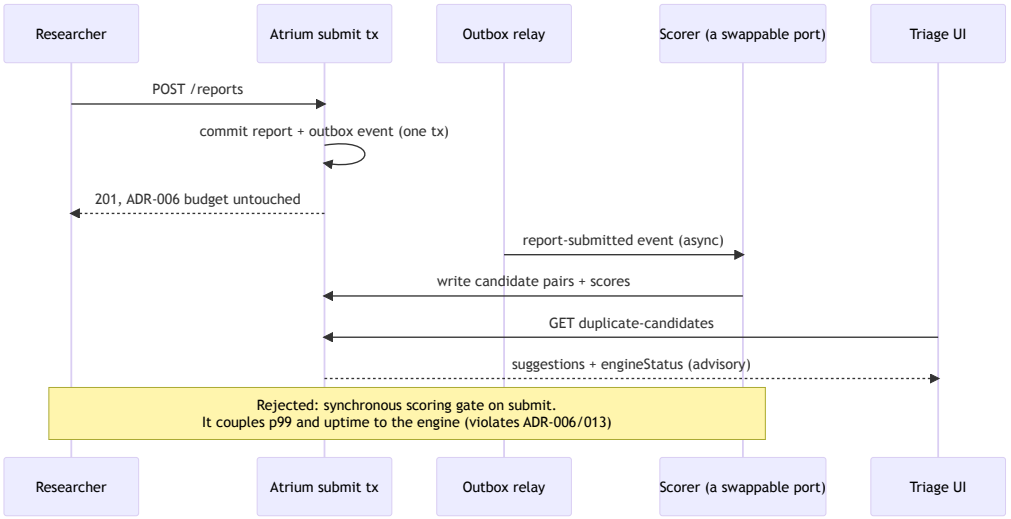
Then the data model. It held candidate pairs with scores and a human decision column, and a `merged_into` self-reference instead of deletion, because a false merge had to be reversible with full history:

```
create table report_candidate_pair (
    id            uuid primary key,
    report_id     bigint not null references report(id),
    candidate_id  bigint not null references report(id),
    score         numeric(4,3) not null,
    decided_by    uuid references users(id),
    decision      text check (decision in ('CONFIRMED','REJECTED')),
    created_at    timestamptz not null default now(),
    unique (report_id, candidate_id)
);

alter table report add column merged_into_id bigint references report(id);
-- merged reports keep their rows, their audit trail, their researcher credit
```

Transaction boundaries came next. Candidate scoring was async and eventually consistent, because submission must never block on a similarity engine. It rode the report-submitted event off the outbox the Notifications split had already laid (the event row commits in the same transaction as the report; a relay publishes it afterward). The merge was the opposite: a small transactional core in the service layer, with optimistic locking on both reports and a 409 if the version moved. The two halves sat at opposite ends of the Wall's consistency spectrum. The suggestion could lag by seconds, priced and signed for in advance. The merge could not lag at all: one commit or it's corruption.

She drew the pipeline under the contract, the rejected road crossed out beside it:



The port on the diagram was load-bearing. The design doc carried the only code Friday needed: one interface, with today's adapter behind it. The evolvability claim rested on the line between them:

```
/** The port. ADR-016: accuracy evolves behind this line. */
public interface DuplicateScorer {
    List<ScoredCandidate> score(ReportFingerprint fresh, ProgramCorpus corpus);
}

@Component
class TrigramScorer implements DuplicateScorer {       // today's adapter
    public List<ScoredCandidate> score(ReportFingerprint fresh, ProgramCorpus corpus) {
        return corpus.stream()
                .map(c -> new ScoredCandidate(c.reportId(), trigram(fresh, c)))
                .filter(s -> s.score() >= 0.65).toList();
    }
}
// next year: EmbeddingScorer implements DuplicateScorer, same port, same merge
// invariant, and shadow mode re-runs against the new engine for free
```

The grilling started before the marker was capped.

Youssef, capacity: "Why async? Sync gives the triager suggestions the second the report lands."

"Sync also couples submission latency to the scoring engine," Kenza said. "We made that mistake with Cortexa and bought a site outage. Argue it the other way: async means a triager might open a fresh report and see no suggestions yet. That costs minutes. Sync failing costs submissions. I'll take minutes." Nor did the engine need its own runtime. Scoring was batch work on virtual threads: each blocking call parks the thread and frees its carrier. The trigram pass between those calls is CPU work and keeps its carrier until it ends, because virtual threads are not time-sliced, a cost they accepted at four hundred reports a week. The pinning tax, a carrier stuck under a `synchronized` block, was already paid in November. Reactive would buy backpressure they didn't have a queue deep enough to need.

Youssef wasn't finished. "Then why isn't scoring its own service? We just split Notifications. The precedent's sitting right there."

"The seam decides, not the precedent," Kenza said. "Merge shares transactions and a schema with reports. Cut there and every false merge becomes a saga run by eleven people. Scoring is the only async piece, and it already consumes outbox events, so the cut line is drawn if it ever needs its own scaling profile. Until then it's a module with a boundary, not a service with a network bill." October's context map had already settled the placement. Duplicate detection serves Malika's queue, so it lives inside the Triage boundary. The same map had let Notifications leave, because nothing transactional held it there. ADR-014's exit criteria sat one tab back if scoring ever earned its own pager.

Imane, data model: "Merge uses JPA, but the scoring path is `JdbcClient`. Why both?"

"The merge is a write with invariants: versions, cascades, audit. That's what a persistence context is *for*. Scoring reads forty thousand rows to build pairs; dragging those through a session buys dirty-checking overhead and lazy-loading landmines for zero benefit. Different jobs, different tools. The trade-off is two data-access idioms in one codebase, which we document."

Leila, the user's side: "If a triager sees a 0.91 and trusts it blindly?"

"Then the UI's job is to make trust expensive. Side-by-side diff before confirm, the score shown as evidence, not verdict. And the precision metric tells us if we're training them to rubber-stamp."

Rachid asked exactly three questions, each a layer deeper than the last. "What happens when the similarity engine returns garbage at p99?"

"Degrade to nothing, honestly. `engineStatus: DEGRADED`, empty list, same as the severity-pending badge from the Cortexa cascade. A fallback that invents candidates is a false merge we've scheduled in advance."

"Who notices the garbage before a triager does?"

"Shadow mode does. Six weeks where the engine scores everything, shows nothing, and gets measured against the merges triagers make by hand. The metric gates the rollout: shadow, then triager-visible, then researcher-visible, each behind its own flag; rollback is turning one off."

"And when two triagers merge the same pair in the same minute?"

"`@Version` on the report, second writer gets a 409 and a refreshed view." *In eight years of hand-written UPDATEs she had never once shipped a version column. Last write had always, silently, won. Here that had a body count: a five-thousand-dollar award the bank never saw, overwritten by a second writer.* "Optimistic, not `SELECT FOR UPDATE`, because contention is rare and the hold time is human think-time. A pessimistic lock across a coffee break is a queue; a version check is one retry."

Silence. Rachid capped his pen. He had used no green ink, because there was nothing to correct. Then he picked up his chair and carried it to the back row. The chair at the whiteboard stayed empty behind her. It was simply hers.

The design doc's last page went into the binder that afternoon, sixteenth on the trail. Leila initialed the consequences line herself, because a trade-off product hasn't signed is a complaint scheduled for later.

> **ADR-016: Duplicate-report detection. Context:** An exec ask with unstated load, accuracy, and workflow expectations, clarified to ~400 reports/week, ~30 true dupes, suggestions worthless once triage starts, and a brutal cost asymmetry: a missed dupe wastes an hour, a false merge buries a live vulnerability. **Decision:** Similarity scoring runs as an async pipeline (outbox event on submission → scorer → candidate-pair suggestions surfaced to triage), never as a synchronous gate on the submit path. Suggestions are advisory and only a human merges, because the domain owns truth. The scorer is a port inside the Triage context, swappable from trigram to embeddings without moving the boundary. **Consequences:** Submission latency keeps ADR-006's budget. The feature degrades to honest absence when the engine is down, per ADR-013. Accuracy can evolve behind the port without touching the merge invariant. The price is eventual suggestions, seconds rather than instant, accepted by product on the record.

## What the senior sees

Monday's brick ceremony came with the handbook she'd written after the three-a.m. incident (a storage latency spike and a page that arrived with zero data). The handbook ended with five drills. She ran Imane through them over coffee, rapid-fire, the way Rachid had once run her.

"Intermittent `LazyInitializationException` in a scheduled job."

"No request, so no session to hide behind," Imane said. "Find where the persistence context closes, then ask who touches the entity after it. Fix the query with a projection or a fetch plan. Don't resurrect the session to comfort the bug."

"Transaction that doesn't roll back."

"Two suspects: a checked exception crossing the boundary under the unchecked-only default, or a `catch` swallowing the rollback signal before the proxy ever sees it. And before either, check that the call crosses the proxy at all. Self-invocation walks around the mirror."

"Endpoint slow under load, CPU idle."

"Thread dump before theories. Count the stacks parked in `NioSocketImpl.implRead`. The missing timeout is the bug; the pool size is just the blast radius."

"Bean not created."

"`--debug`, read the `ConditionEvaluationReport`, find the negative match. The bean didn't vanish. A condition told it not to exist, and the report names the condition."

"401 on a route that should be open."

"The verdict happened in front of the front door, before the `DispatcherServlet` saw it. List the `SecurityFilterChain` matchers in order, because first match wins. And under deny-by-default, a missing allow is a bug you can grep for."

Five symptoms had produced five chains, each running from the visible symptom to the mechanism behind it. Not once had Imane said *Spring just does that*. Somewhere around the third

drill Kenza realized that the job now was to install the shape of an answer in someone else: claim, mechanism, trade-off, failure mode. She no longer noticed herself using the four beats.

She pushed one beat further than Rachid ever had. “Again, and this time name the -ility each one threatened.”

“Operability, integrity, availability, evolvability, security,” Imane said, barely pausing on the fourth. This was succession, Kenza thought. The four beats plus a named -ility made a decision portable, able to outlive whoever made it, and architecture that lived in one head was a bus factor of one wearing a title.

## The fix

Imane’s PR for the merge flow landed Tuesday. It worked, and it demoed beautifully. Kenza printed the diff.

```
@RestController
@RequestMapping("/api/v2/reports")
class MergeController {

    @Autowired ReportRepository reports;     // field injection
    @Autowired AuditLog auditLog;
    private static final Logger log = LoggerFactory.getLogger(MergeController.class);

    @PostMapping("/{id}/merge")
    Report merge(@PathVariable Long id, @RequestBody MergeRequest req) {
        Report dupe = reports.findById(id).orElseThrow();
        Report canonical = reports.findById(req.canonicalId()).orElseThrow();
        dupe.setMergedInto(canonical);
        dupe.setStatus(Status.DUPLICATE);
        try {
            reports.save(dupe);
            auditLog.record(dupe, canonical);
        } catch (Exception e) {
            log.warn("audit failed, continuing", e);   // "handled"
        }
        return dupe;                                   // entity, straight to Jackson
    }
}
```

> **Your call.** It demoed clean and the tests are green. Before the pen moves, count what breaks here, *when* each one fires, and which -ility every line bleeds.

She left forty-one comments, all hanging off her review checklist: proxy traps, N+1, transaction boundaries, leaked entities, security gaps, untested branches. The merge had no transaction

boundary, so the status write committed on its own whether or not the audit write followed, the silent commit again. Without a version check, two triagers could repeat Malika's lost update. Without `@PreAuthorize`, any researcher could merge a competitor's report closed. The candidate loader elsewhere in the PR fetched researchers one query per row, the thousand queries in miniature. The lazy `researcher` association would fault when Jackson serialized the entity after the session closed. The `catch` swallowed the audit failure: *a caught exception is a decision; make it on purpose*. The happy path was tested; the 409, the 403, and the degraded engine were not.

The field injection would never have reached main, because the ArchUnit rule from the January incident (field injection lets a bean be built with `new` and run with null dependencies) would have failed the build. The comment went in anyway: the robot says *no*; the review teaches *why*. The only empty checklist line was proxy traps. Imane had met the smoke cloud in September (a `this.` call dispatches on the raw object, bypassing the proxy where the annotations take effect), and that lesson had only needed to cost her once.

Comment #1, on a line that worked: *"Works is not done. What happens when this entity meets Jackson after the session closes?"*

She was halfway through writing it before she recognized the sentence. She let it stand.

Imane's rewrite landed in two days: a slim controller, a `MergeService` owning the `@Transactional` core, DTO records at the boundary, the audit on `REQUIRES_NEW`, and all four failure paths tested. Her commit message entered team lore: *Done, not just working.*

Imane had one question left, at Kenza's desk, test class open. "The Testcontainers Postgres. I never configure a URL anywhere. How does Boot even *know* to wire it?"

A year ago Kenza would have said *auto-configuration* and meant it as an answer. Instead she pulled up the framework source. *In eight years at Meridian there had been no framework to read. Every line in production was theirs, and she'd assumed framework source was unreadable. Somewhere this year it had become conversation.*

```
// condensed from spring-boot-jdbc (Boot 4.0.x), package
        org.springframework.boot.jdbc.autoconfigure
@AutoConfiguration
@ConditionalOnClass(DataSource.class)
public class DataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean(JdbcConnectionDetails.class)
    PropertiesJdbcConnectionDetails jdbcConnectionDetails(DataSourceProperties props) {
        return new PropertiesJdbcConnectionDetails(props);
    }
}
```

"Read the condition," Kenza said. "Auto-config only builds connection details from properties *if nobody else has*. Your `@ServiceConnection` is found by a test context customizer, which asks a container-aware factory for a `JdbcConnectionDetails` built straight from the live container (host,

mapped port, the container's fixed credentials) and registers it as a bean definition while the context is still being prepared. The conditions get evaluated later, inside the configuration-class post-processor, the same column of the Wall as the lifecycle post-processors. So by the time Boot reads this class, your bean already exists, `@ConditionalOnMissingBean` fails, and auto-configuration backs off. The rule from the magic show was that your own bean always wins. Now you've read *why*."

While the source was open, Kenza steered her one class deeper, into `CommonAnnotationBeanPostProcessor`. "And here's your `@PostConstruct`. It isn't JVM magic, just an annotation this post-processor finds and invokes in the before-init phase. Build the bean with `new` and there's no post-processor, so no init call. It silently never runs. January's lifecycle column, in the source."

Imane stared at the screen. "It was never magic."

"No," Kenza said. "It was never magic."

The announcement came the Friday after: *principal architect*, effective January. Everyone pretended the design review hadn't been the interview, and nobody believed anyone. Rachid cleared his desk over an afternoon, refusing help. On Monday his desk was empty and Kenza's wasn't: the green pen lay beside the dented yellow Canary, under a sticky note in green ink. *It was never magic. R.*

That same week a line in the QBR notes mentioned, third-hand and unsourced, that a larger platform had been asking analysts about Glasshouse. It was only a rumor, and December had no room for weather that hadn't arrived.

The January Wall tour fell to her now. Imane had inherited the request-trace walkthrough and acquired a mentee of her own. Kenza walked the new hires down the whole year of ink: the lifecycle column, the proxy smoke cloud (*this. goes around the mirror*), the config-precedence ladder, the filter chain in front of the front door, the persistence panel, the breaker's three states. Then she drew one blank column for the year to come and traced the Request Journey a final time, socket to database and back. One thing was new: green numbers, one beneath each scene. ADR-002 sat under the lifecycle column, ADR-003 under the smoke cloud, ADR-004 beside the precedence ladder, ADR-010 on the filter chain, ADR-007 on the persistence panel, and ADR-013 inside the breaker's states. The binder and the Wall were one document now, read in two directions; the trail *was* the tour.

"So Spring just handles all that automatically," one of the new hires said.

Kenza smiled, and uncapped the green pen.

"Show me the mechanism."

## The debrief: Ch.16, The Architect's Chair

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 16.** The final gate: design a service end-to-end from a vague spec, out loud, and survive follow-ups on every layer.

Then the follow-ups, cold:

1. What happens when your similarity engine returns garbage at p99?
2. Why optimistic locking on the merge instead of `SELECT FOR UPDATE`?
3. Your similarity scorer needs to get smarter next year (trigram → embeddings) without destabilizing triage: where does that flexibility live?
4. What numbers did you design for?

Tick what you said:

- ▢ Claim: Interrogate the requirement before drawing anything, then design in a fixed sequence from clarified spec to rollout.
- ▢ Mechanism: Async events off the outbox, a small `@Transactional` merge with `@Version` and a 409, honest absence, and a precision metric from shadow mode.
- ▢ Trade-off: Eventual suggestions cost minutes, never submissions. Optimistic beats pessimistic because contention is rare. Shadow mode spends six weeks for a precision number.
- ▢ Failure mode: A wrong answer acts without a human, the fallback invents data, or rollback needs more than turning a flag off.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** a senior turns "we need duplicate detection" into a system by interrogating the requirement before drawing anything, then designing in a fixed sequence: clarified spec → API contract → data model → transaction and consistency boundaries → failure modes → observability → rollout. **Mechanism:** each layer is pinned to a real mechanism. Suggestions ride async events off the outbox so submission never blocks on the engine. The merge is a small `@Transactional` service-layer core with `@Version` optimistic locking and a 409 on conflict. Engine failure degrades to honest absence behind a status flag, and a shadow-mode precision metric gates each rollout stage. **Trade-off:** eventual consistency means a triager may briefly see no suggestions (minutes lost, never submissions). Optimistic beats pessimistic because contention is rare and hold time is human-scale. Shadow mode spends six weeks to buy a precision number before users can be harmed. **Failure mode:** the design fails if a wrong answer can act without a human (auto-close on a false merge buries a live vulnerability), if the fallback invents data instead of admitting absence, or if rollback requires anything more than turning a flag off.

**THE ARCHITECT'S LENS**

- **-ilities at stake: Judgment**, which is every -ility at once, weighed out loud: the submit path's **availability** and **performance** against suggestion freshness; **integrity** above everything, because a false merge is the company; **evolvability** behind the scorer port; **operability** in flags, shadow metrics and honest degradation.
- **The ADR:** ADR-016, duplicate detection as an async advisory pipeline behind the Triage context: outbox event → scorer port → candidate suggestions; only humans merge; ADR-006's budget kept, ADR-013's degradation honored, eventual suggestions accepted on the record.
- **Rejected alternatives:** a synchronous scoring gate on submit (couples the sacred path's latency and uptime to the engine, with ADR-006/013 violated on their face); nightly batch dedup (cheap but too slow, since the binding budget is the triagers' morning); a separate scoring service (ADR-014's exit criteria unmet, so it stays a module with a boundary, not a pager); anything that auto-closes (a false merge scheduled in advance).
- **Fight or lean:** *Lean*, almost everywhere. The feature is assembly of rails already paid for: the outbox, `@Version`, virtual threads, the slices, the flags. The one fight is with the instinct to make the machine decisive: where the domain says a human owns truth, the framework suggests and the architecture makes trust expensive.
- **Which door:** almost all of ADR-016 is two-way, since ports, thresholds and flags are cheaply reversible. The book's one-way doors were Ch.5's public API contract and Ch.8's consistency contract, committed decisively because others build on them.

**Story anchors**

- The empty chair at the whiteboard → the senior design sequence: clarify the spec, then contract → data → transactions → failures → observability → rollout.
- Comment #1 on Imane's PR → code review as architecture: proxy traps, N+1, missing transaction boundaries, leaked entities, security gaps, untested failure branches.
- The green pen on the Boot source → the senior reflex: read the `AutoConfiguration` and its conditions. `@ConditionalOnMissingBean` is *why* your own bean wins.
- The green numbers under the year of ink → the trail is the tour: every Wall scene carries its ADR, the binder and the whiteboard one document read in two directions.

**Interview ammo**

- *Kill-shot: Design duplicate-report detection end-to-end, out loud.* Clarify the cost asymmetry first (missed dupe = an hour; false merge = a buried vuln), then: suggestion API + candidate-pair model with a reversible `merged_into` FK, async scoring off events, a small optimistic-locked merge transaction, honest degradation, precision metrics, shadow-mode rollout behind flags.
- *Follow-up: What happens when your similarity engine returns garbage at p99?* Degrade to honest absence (`DEGRADED` status, empty list), never auto-close; shadow-mode precision catches the drift before a triager ever acts on it.
- *Follow-up: Why optimistic locking on the merge instead of `SELECT FOR UPDATE`?* Contention is rare and hold time is human think-time; a version check costs one 409-and-retry, while a pessimistic lock holds a row hostage across a coffee break.
- *Architect follow-up: Your similarity scorer needs to get smarter next year (trigram → embeddings) without destabilizing triage: where does that flexibility live?* Behind a port inside the Triage context: the scorer is an adapter consuming outbox events and writing candidate pairs, so accuracy evolves by swapping the implementation while the contract, the merge invariant, and the rollout flags never move. Shadow mode re-runs against the new engine for free.
- *Follow-up: What numbers did you design for?* About four hundred reports a week and about thirty true duplicates, with suggestions worthless once triage starts. The binding budget is the triagers' morning, so scoring runs async off the outbox and reads forty thousand rows through `JdbcClient` without touching the submit path's latency budget.

**Pointers** → `examples/` (all) · Appendix F

---

# Capstone

> *No new nouns. Everything in that room is something you have already debugged.*

---

# Chapter 17: The Hostile Review

The Wall had an exhibit number now.

Legal's photographer had shot it panel by panel for the data room: the lifecycle column, the smoke cloud, the breaker's three states, eight years of `DO NOT ERASE` filed under `GLASSHOUSE-TECH-047`. *Forty-seven again,* Kenza had thought. The universe kept its accounts in green ink.

The rumor from December's QBR notes had a name now: Castellan, an enterprise security conglomerate whose customer list read like a banking regulator's holiday card. It also had a process, due diligence. Finance got a week and Legal got two. The architecture got one day, and Castellan sent hired guns to spend it.

"Amina," the lead said, shaking hands like she was returning something. "I read your decision records last night. All sixteen." A beat. "It's why we only need a day." Her partner, Mounir, introduced himself with a spreadsheet open. He priced everything in run-rate and risk, and was usually right to.

They were not villains, just the outside world doing professionally what @sable had been doing for free: testing the claims, in writing.

The ADR binder sat on the conference table: the printed trail, ADR-001 through ADR-016, tabbed and dated, January's half-page about feature seams on top. Kenza sat across from it, six months a principal architect, the green pen in her shirt pocket because today it was coming along. Imane sat at the end with a laptop, officially the note-taker. Rachid was advisory now, which meant he was somewhere with no meetings; his only contribution would arrive at day's end, two lines long.

"Here's the shape of the day," Amina said. "I won't audit your code. Your code is fine; code usually is. I'm going to attack your decisions. Defend them or reopen them; both are acceptable answers. Only one thing fails this audit." She didn't say what.

## The gauntlet

She started with the oldest decision and the newest insult.

"A modular monolith. In 2027." Amina laid ADR-001 and ADR-014 side by side. "Every other target we've assessed this year showed our client a service catalog. You show me a folder of reasons not to have one. Defend not splitting."

> **Your call.** Before Kenza answers: what's the defense that isn't nostalgia, and which -ility does the monolith protect? Commit; this chapter is your exam too.

“The seam decides, not the year,” Kenza said. “Atrium is one deployable with enforced internal boundaries: feature packages have kept their internals package-private since ADR-001, and an ArchUnit rule has failed the build on every crossed dependency since January. One seam proved real, and that was Notifications: async by nature, no shared transactions, its own scaling shape. We extracted exactly that, on outbox rails, with idempotent consumers and one trace ID across all four hops. The price of staying together is one pager, one deploy pipeline, one set of ACID commits, and no independent scaling anywhere we haven’t measured the need. And what service catalogs leave off the slide is that triage, payouts, and programs share transactions and a chatty schema. Cut there and every payout becomes a saga run by eleven people, a distributed monolith doing badly, in public, what one commit does silently for free.”

“Or,” Amina said, “you lacked the organizational muscle to split, and wrote a justification afterward.”

Kenza turned ADR-014 around and pushed it across the table. “Then read the exit criteria, written *before* the split argument: we split a seam when it crosses no transaction, has its own scaling profile, and can staff its own on-call. Notifications met all three; nothing else does yet. The day one does, this document tells my successor to reopen it. That’s the difference between a defense and a justification: a defense names the conditions under which it stops being true.”

(If you committed at the pause: the -ility is **evolvability**. Enforced seams keep the split *purchasable*; the service catalog spends it up front on every boundary, real seam or not.)

Amina flipped to tab seven. “Next. You disabled `open-in-view` and now hand-write a projection for every screen. ADR-007 is very proud of this. From outside, it reads as fighting your framework. Teams that patch their platform usually don’t understand it. Why should Castellan trust one that overrides its own foundation’s defaults?”

> **Your call.** Draft the answer that proves fighting the default *was* understanding it, and name the -ility OSIV was silently spending.

“Because we fought it on the record, with the mechanism in hand, which is the opposite of not understanding it,” Kenza said. “Open-in-view keeps the Hibernate session, and the pooled connection under it, open across response rendering, so lazy loads during serialization succeed. Two consequences follow. Every missing fetch plan still works, so it ships as a hidden N+1. And request threads keep holding database connections after the transaction has ended, just to write JSON. We have an incident for each. The first is the twenty-eight-second dashboard in front of Northbeam’s CISO: four hundred and one individually flawless queries, one for the page, then two lazy loads per row, four hundred in all. The second is the April outage, where nearly two hundred threads per pod sat parked behind one slow vendor, holding things they didn’t need. It’s the same shape, one layer down. Trade-off: a DTO type per view, the mapping tax paid forever. Failure mode of the default: the exception disappears in development and the pool drains in production.” She tapped the binder. “We lean on Boot everywhere the default’s bill is one we’d pay

anyway. The ADRs are silent there, because silence means *lean*. We've fought exactly the defaults whose bill we'd already paid in incidents, and each fight has a tab. A fought default without a record is a patch; with a record, it's architecture."

"Hm." Amina flipped forward. "Tab eight. Optimistic locking on triage. Your concurrency strategy *ships errors to users*. A 409 is a failure you designed in and documented like an achievement." She was looking past Kenza, down the table. "You. You worked on the merge flow. Do you agree your users should see errors?"

> **Your call.** A designed-in 409, attacked by a professional. Defend it in four beats, and name precisely what the alternative corrupts. Then watch who answers.

Imane closed the laptop halfway, which Kenza would later describe as the sound of a torch being passed.

"Yes, and I can show you why," Imane said. "Claim: optimistic concurrency is correct wherever contention is rare and hold time is human. Mechanism: a version column rides every update, `UPDATE … WHERE id = ? AND version = ?`, and zero rows updated means someone got there first; the second writer gets a 409 and a refreshed view instead of a green toast. Trade-off: a visible conflict a handful of times a month against an invisible lost update. We've had eleven conflicts since January, every one retried inside a minute. Failure mode: the alternative had a body count before we changed it. Two triagers, same report, both saves succeeded, both got green toasts, and a five-thousand-dollar severity decision belonged to whoever clicked second. The pessimistic version just relocates the cost: a row lock held across a human coffee break is a queue with a database in it. We chose the error you can see over the corruption you can't."

Amina wrote down the last sentence. Kenza watched her do it and said nothing at all, because the proof of succession is that you don't have to.

## The concessions

After lunch, Amina stopped circling.

"Tab ten. Your tokens." She read from her notes; the ADR she had memorized. "Stateless JWTs, fifteen-minute access tokens, refresh rotation. Which means that for an offboarded employee, a stolen laptop, a compromised researcher token, your platform cannot revoke a credential; it can only wait. Castellan sells to banks. Their offboarding clauses are measured in seconds, and their auditors ask for the mechanism, not the average."

> **Your call.** Two answers exist here: the defense and the concession. Which does this room need, and which -ility has moved *under* the decision?

“The decision was right when we made it, and I’ll defend that version first,” Kenza said. “Statelessness was chosen deliberately, after a researcher token reached an admin endpoint and came back `200 OK` with a bank’s payout configuration. That token was authenticated but never authorized; no role check stood between it and the handler. The mechanism: every request is verified in the filter chain (signature against cached JWKs, issuer, audience, expiry) before the `DispatcherServlet` ever runs, and there is no server-side session store to operate or to protect. The trade-off went up on our Wall the same week: *a stolen token stays valid until it expires; statelessness cuts both ways*. We priced that window at fifteen minutes against year-one Glasshouse. The ADR records the heavier options, denylist and live introspection, as the day-two path, each buying back control by selling statelessness.”

She walked to the real Wall, not exhibit 047, and uncapped the green pen.

“And you’re right. The threat model under that decision just changed owners. Fifteen minutes was priced against our risk; under your customer base it’s mispriced, and ADR-010 has named day two since July. So I’m declaring it.” She wrote, high on the blank stretch beside the January column: **ADR-017 candidate, revocation: `jti` denylist in the bearer filter.** “Mechanism: every token carries a unique ID; the filter checks it against a shared denylist before the request proceeds. Entries expire with the TTL of the token they ban, so the set stays bounded. Cost: one lookup per request, and a policy decision nobody gets to dodge. Fail open and revocation silently vanishes whenever the store blips; fail closed and the denylist joins the availability budget. We’ll fail closed and pay for it.”

Mounir looked up, mildly interested, like a man watching someone volunteer for a tax.

Amina moved on without comment. “Tab fifteen. You bet the runtime on virtual threads, a JDK feature that spent two releases in preview before anyone would stand behind it. Pinning, a young ecosystem, and your p99 contract with Northbeam riding on it. Defend the bet.”

> **Your call.** Defend the bet, and name the evidence you’d put on the table.

“Final in 21,” Kenza said, “September of 2023, three years before we flipped the flag. But the age wasn’t the bet; reversibility was. We ran the bake-off in staging under Gatling at twice peak load, and we didn’t trust the release notes about pinning, so we measured it: `-Djdk.tracePinnedThreads`, then JFR, with `jdk.VirtualThreadPinned` for threads that park while pinned and `jdk.JavaMonitorEnter` for threads kept waiting at a monitor. The recordings pointed at two offenders. A 2018 signing utility serialized every caller through one shared digest; its `synchronized` became a `ReentrantLock`, which a virtual thread parks on cleanly. A vendor SDK we couldn’t edit was contained on a named sixteen-thread *platform* executor, because a platform thread that blocks is a budgeted cost, while a pinned carrier (a virtual thread that blocks without unmounting, welding itself to the platform thread beneath it) takes that capacity from every other virtual thread in the JVM. The pinning audit is a standing check rerun at every JDK upgrade. Newer runtimes keep retiring the `synchronized` pin, which we verify on ours and never assume,

so the risk shrinks as the bet ages. And the bet was survivable because the programming model never changed: still thread-per-request, same stack traces, same `ThreadLocal` semantics. Rollback is a property flip and a pool-sizing exercise. The binder holds one-way doors we walked deliberately, records attached; this wasn't one. We bet a *setting*. Reactive would have bet the codebase, a one-way door through every signature, for backpressure we don't have a queue deep enough to need."

"Adequate," Amina said, which Kenza understood by then to be her highest grade, and nodded to Mounir.

Mounir turned his laptop around. "My turn. Circuit breakers, bulkheads, named executors, an outbox relay, a gateway, synthetic probes. I've priced your resilience stack in engineer-months and annual run-rate, and the number is on this screen. You process four hundred reports a week. This is over-engineering for your traffic."

> **Your call.** Price the stack yourself before Kenza does: which pieces trace to a *named incident* with a postmortem attached, and, by Youssef's rule (he doesn't believe YAML until he has watched it), what would you cut? Commit; Mounir is about to find Glasshouse's.

"Resilience isn't priced against throughput," Kenza said. "It's priced against correlated failure, and ours was total, twice. The April outage: nearly two hundred Tomcat threads per pod parked in `NioSocketImpl.implRead` behind one slow vendor, CPUs idle, the whole site 503ing on launch day. Our traffic was modest; the blast radius was everything. In thread-per-request, a slow downstream doesn't slow a feature; it confiscates the threads every feature shares. That outage bought the timeout policy, ADR-006: a global floor, so a forgotten client can be slow but never unbounded. The September cascade bought the rest, ADR-013. Our own retry loop hit a failing vendor three times per request, no backoff, adding load at the worst possible moment, while a `@Cacheable` that had never cached a single entry sent every read straight to them. That was February's proxy ghost wearing a new coat: the annotation lives on the container's proxy wrapper, and a `this.` call dispatches on the raw object behind it, so the call never crossed the proxy and the annotation never ran. The breaker records recent failures and stops calls to a downstream that keeps producing them; the bulkhead turns *the vendor is down* into *enrichment is down* and nothing more. Every component on your screen traces to a named incident with a postmortem, which makes the stack insurance with the claims history attached."

Mounir nodded, conceding the column, then scrolled. "Then explain page forty-one of your runbook. *Brownout tiers: under sustained saturation, shed enrichment, digests, leaderboard, in that order.* Dated the week after that September cascade. Which drill validated it?"

Kenza opened her mouth, and closed it, and the silence did the arithmetic for everyone.

"None," she said. "It's been scheduled twice and eaten by roadmap twice. The flags exist. The tiers have never been shed in anger or on purpose."

"Then it isn't a capability," Mounir said, not unkindly; he was only pricing. "It's a hypothesis with a logo."

"It's worse than that," Kenza said, already walking to the Wall, "because undrilled flags rot. There's the flag that flips nothing since a refactor renamed the property, and the *low* tier that turns out to share a connection pool with the core path it was meant to protect. You discover those at 3 a.m. during the real brownout, and we know how that goes from the week Kubernetes restart-looped perfectly healthy pods over a database check wired into liveness instead of readiness, a ninety-second blip reading as a dead pod." She wrote the second line under the first: **ADR-017 candidate: run the brownout drill. Untested designs are hypotheses.** The ink was still wet when she sat back down. "That one's free, Mounir. You found it; we'll run it."

At five o'clock Amina closed the binder and squared it on the table.

"This morning I said only one thing fails this audit," she said. "It's defending everything. Most systems we assess are *defended*: by personality, by tenure, by whoever talks longest. Almost none are *evidenced*. You'll have our report in two weeks." She glanced once at the two wet lines of green on the Wall. "Leave those up."

Ten days later the report reached the CTO, and everyone quoted the same sentence from it. *The strongest signal in this engagement, stronger than any single decision, is that the architecture has a mechanism for being wrong: two findings conceded in the room, priced on the spot, and owned before we left the building.* An architect's authority, it turned out, was the trail of priced decisions *plus* the demonstrated ability to reopen one: defensibility as a body of evidence with a working appeals process.

### ⚖ *Your move, architect*

There are two ADR-017 candidates on the Wall, the audit adrenaline is fading, and integration planning starts Monday. The remediation priority is a real fork. Commit before reading on:

- **A. Close the revocation gap first.** Security; enterprise sales pressure; the report's headline finding.
- **B. Run the brownout drill first.** Operability; two afternoons and a staging cluster; the cheapest finding to retire, and the one that pages *you*.
- **C. Sequence both into the acquisition's first-90-days plan, with owners and dates attached.**

**The trade-offs, since you've committed:** by now you can price your own bias. **A** optimizes for the -ility with a lobbyist and starves the one with a pager; **B** does the reverse, and reads to the acquirer as ignoring their headline finding. The team chose **C**, because the architect's move is never picking a favorite -ility; it's *sequencing with names attached*. And **C** has a price tag, said out loud in the room: the acquirer's headline security finding stays open for the better part of ninety days as a known revocation gap, dated and watched, under the scrutiny of the very people who wrote it. The room accepted that as cheaper than a panicked denylist that fails open, and Castellan, reading owners and dates instead of promises, accepted it too. The drill ran in week three: Youssef,

two afternoons. One flag flipped nothing, exactly as Kenza had guessed. The second finding nobody had guessed: the shed order itself was wrong, with digests carrying a transitive dependency on enrichment, so that tier one couldn't shed without taking tier two down with it. Both were found at 1× load instead of at 3 a.m. The denylist went spike-design-ship across the ninety days, Imane's spike first, fail-closed from day one. Both lines lived in the integration plan where Castellan could watch them move. If you chose A or B, you were right about the work and wrong about the job.

Tab seventeen joined the binder that week:

> **ADR-017: Findings of the June architecture audit. Context:** Due-diligence review upheld the decision trail but surfaced two mispriced decisions: the JWT revocation window (ADR-010's recorded day-two path, now due under the acquirer's enterprise threat model) and a brownout design that has never been drilled. **Decision:** Add a fail-closed `jti` denylist in the bearer filter, its entries expiring with their token's TTL and its store added to the availability budget (Imane, 90 days); run the brownout drill as a quarterly gameday, first run within 30 days (Youssef). **Consequences:** We sell back a slice of statelessness and two afternoons a quarter; we buy revocation measured in seconds instead of TTLs, a rehearsed degradation mode, and a trail that demonstrably reopens, which the audit priced higher than any single decision in it.

One more item rode in on Monday's integration call, dressed as small. Castellan's analytics team wanted Glasshouse's report events flowing into their platform, and their lead made it sound like a favor to grant cheaply: *"Just point your outbox topic at us; we'll adapt to whatever shape it has. Fully reversible, it's only a feed. Turn it off any time."*

### ⚖ Your move, architect: name the door first

This fork has two questions, and the first one is the exam. Chapter 16 named the skill; nobody in this meeting is going to label the door for you. First commit to a classification, one-way or two-way. Then choose:

- **A. Publish the internal event schema as-is.** Fastest; zero translation code; "reversible," per the man who asked.
- **B. Publish a versioned external contract.** A purpose-built event shape behind a translation layer, the internal model free to keep moving behind it.
- **C. Decline streaming for now; ship a quarterly batch export while the integration relationship settles.**

**The trade-offs, since you've committed:** classification first. The feed *looks* two-way, a topic you can switch off, but the schema inside it is one-way: the day Castellan's dashboards parse your internal event shape, that shape freezes. Your domain model now changes at the speed of someone

else's backlog, and "turn it off any time" decodes to "break their dashboards any time," which an acquisition makes career-adjacent. The *transport*, meanwhile, is genuinely two-way: stream, batch, cadence, all renegotiable next quarter without breaking anyone. So the trap is misclassification, in either direction. **A** treats a one-way door as reversible: commit later means migrating later, under a live external contract, at twice the price and half the leverage. **C** over-commits caution on the part that was cheap to reverse. It burns a quarter of integration goodwill guarding the transport, which was never the risk. **B** is what went in the plan: a mapping class and a versioning policy, the same anti-corruption move ADR-014 bought at the Notifications border (a translation layer so another context's model can't reach your domain), pointed outward this time. Grade yourself on the classification, not the letter: if you named the schema one-way and the transport two-way, the choice followed by itself. If you picked B while calling the whole thing two-way, you got the right answer the way a coin does.

The office emptied around seven. The email arrived at 7:14, two lines, no greeting, and Kenza could hear the green ink in it:

> *Heard you conceded twice in front of the buyers. Good. An ADR you can't reopen is just dogma with a date on it. Tell Imane the chair suits her too. R.*

She didn't forward it; some things you deliver in person. The January column had six months of ink in it now, the audit's two findings drying beside it, and she took the green pen and ruled a fresh blank column on the far side, for whatever year two decided to be under new owners.

Imane appeared at her shoulder, reading the empty column. "What goes in it?"

"Whatever breaks next," Kenza said, and capped the pen. "Same two questions, every time. Show me the mechanism. And tell me which -ility it spends."

## The debrief: Ch.17, The Hostile Review

**Say it first.** Out loud, 90 seconds, before you read below the line.

> **Kill-shot 17.** What makes an architecture defensible under hostile review?

Then the follow-ups, cold:

1. An auditor attacks a decision you now believe is wrong. What do you do?
2. How do you retrofit revocation onto stateless JWTs, and what does it really cost?
3. What distinguishes an over-engineered resilience stack from a priced one?
4. Which production incident of your own would you bring to this room?

Tick what you said:

- ☐ Claim: A body of evidence: decisions priced in -ilities, recorded with rejected alternatives, plus a mechanism for reopening them.
- ☐ Mechanism: Every contested decision traces to an ADR (-ility bought, alternatives lost, conditions for revisiting), and each defense is grounded in named incidents.
- ☐ Trade-off: The trail costs a permanent writing tax and makes you precisely attackable. That precision lets a concession raise credibility.
- ☐ Failure mode: An architecture defended by tenure and talking with no records, or a trail treated as scripture with no appeals process.

Three ticks pass. With fewer, re-read the scene named in the story anchors and try again tomorrow.

---

> **The model answer. Claim:** A defensible architecture is a *body of evidence*: decisions priced in -ilities, recorded with context and rejected alternatives, plus a demonstrated mechanism for reopening any of them. **Mechanism:** every contested decision traces to an ADR naming what was true at the time, which -ility was bought, which alternatives lost, and the conditions for revisiting; each defense then runs claim → mechanism → trade-off → failure mode, grounded in named incidents. The auditor attacks the decision and meets the record, not the personality. **Trade-off:** the trail costs a permanent writing tax and makes you *precisely* attackable, since a written decision can be refuted in a way folklore can't. But that precision is what lets a review end in two findings instead of a turf war, and lets a concession raise credibility where it would otherwise spend it. **Failure mode:** two mirror-image deaths: an architecture defended by tenure and talking (no records, every question re-litigated from memory), and a trail treated as scripture (records, but no appeals process; the team that defends everything fails the audit on the one question that mattered).

**THE ARCHITECT'S LENS**

- **-ilities at stake: Defensibility**, the -ility above the others, which the whole book has been building toward: architecture as a body of evidence, where every -ility purchase has a receipt that names its return conditions. Beneath the two concessions are **security** (a revocation window priced against a threat model that changed owners) and **operability** (an undrilled degradation design is a hypothesis, not a capability), both reopened *because* the trail recorded them as reopenable.
- **The ADR:** ADR-017, the audit's findings sequenced into the first-90-days plan: a fail-closed `jti` denylist and a quarterly brownout gameday, each with an owner and a date. It is the trail's first entry written in front of outsiders.

- **Rejected alternatives:** defending everything (the trail as scripture, the one answer Amina said fails the audit); quietly fixing the JWT gap before the report landed (retires the finding but forfeits the signal: being wrong *in public* was worth more than the fix); re-litigating each attack from memory and charisma (personality-based architecture: wins mornings, loses acquisitions).
- **Fight or lean:** the year's whole pattern, compressed for the witness stand: *lean* on the platform wherever the default's -ility bill is one you'd pay anyway (the ADR binder stays silent there, because silence means lean); *fight* only on the record, mechanism in hand, bill itemized (OSIV off, ADR-007). Under hostile review, the fought defaults are your strongest exhibits, *if* each one has a tab.

**Story anchors**

- The ADR binder on the conference table, sixteen tabs, read overnight by a stranger → defensibility: the auditor attacks decisions, and decisions with records (context, -ility, rejected alternatives, exit criteria) defend themselves.
- Imane's four-beat 409 answer, "the error you can see over the corruption you can't" → the succession proof: the skeleton transmitted whole, and the architect's silence as the final teaching move.
- The two concessions drying in green on the Wall → "the architecture has a mechanism for being wrong": an architect's authority is the trail of priced decisions *plus* the demonstrated ability to reopen one.

**Interview ammo**

- *Kill-shot: The final gate: defend a whole system under hostile questioning.* Don't defend the code; defend the decisions, each in four beats, grounded in a named incident, traced to a dated record that names its own revisit conditions. When the questioner finds a decision whose context has really moved, concede on the spot and price the fix. The concession *is* the senior answer.
- *Follow-up: An auditor attacks a decision you now believe is wrong. What do you do?* Defend the decision as made (its context, its price at the time), then reopen it on the record: name what changed, sketch the remediation's mechanism and cost, attach an owner and a date. Never defend the present with the past's threat model.
- *Follow-up: How do you retrofit revocation onto stateless JWTs, and what does it really cost?* A denylist of token IDs checked in the bearer filter, entries expiring with their token's TTL so the set stays bounded. The bill is one shared-store lookup per request, plus a forced choice between fail-open (revocation silently vanishes when the store blips) and fail-closed (the store joins your availability budget).
- *Architect follow-up: What distinguishes an over-engineered resilience stack from a priced one?* In a priced stack, every component traces to a named incident or an explicitly priced risk: in-

surance with a claims history and an approved premium. Anything tracing only to a conference talk is ceremony, and anything designed but never drilled is still a hypothesis, whatever the runbook says.

- *Follow-up: Which production incident of your own would you bring to this room?* One with a name and a postmortem, told in the same four beats and tied to the decision it bought. Kenza's model is the April outage: Tomcat threads parked behind one slow vendor with CPUs idle, the whole site answering 503, and ADR-006's timeout policy as what the outage bought.

**Pointers** → the whole trail · Appendix F · Appendix G

---

# Appendices: The map behind the story

*The story in tables: where each chapter sits on the roadmap, its anchor scene, the debug drills, a study cadence with the fork index, the Wall's diagrams, the ADR trail, the principles and flashcards, what the book leaves out, and its version history.*

## Appendix A: Chapter ↔ roadmap ↔ kill-shot

The book covers the full roadmap.sh/spring-boot node tree (eleven topics) plus the senior extensions, and the architect layer above each: the -ility the chapter's decision serves and the ADR that recorded it. Use this table to jump from a weak topic to its chapter, or from a chapter to its drill files.

| Ch | Title | Roadmap topic | B.E.A.N.S. | Architect layer | `examples/` |
|---|---|---|---|---|---|
| 1 | The Review | Prerequisites: clean code, why Spring exists | B | evolvability · ADR-001 seams | `ch01_core/` |
| 2 | The Container | 1: Spring Core (IoC, DI, scopes, lifecycle) | B | enforcement · ADR-002 fitness functions | `ch01_core/` |
| 3 | Smoke and Mirrors | 1: Spring Core (AOP, proxies) | B | maintainability · ADR-003 proxy discipline | `ch01_core/` |
| 4 | The Magic Show | 3 + 4: Starters & Autoconfiguration | B | supply-chain evolvability · ADR-004 | `ch04_autoconfig/`, `ch03_starters/` |
| 5 | The Front Door | 10: Spring MVC | E | contract integrity · ADR-005 governance | `ch10_mvc/` |
| 6 | The Pool | 6: Embedded Server | E | availability · ADR-006 latency budgets | `ch06_server/` |
| 7 | The Thousand Queries | 7: Hibernate (persistence context, N+1, lazy) | A | designed performance · ADR-007 fetch policy | `ch07_jpa/` |

| Ch | Title | Roadmap topic | B.E.A.N.S. | Architect layer | `examples/` |
|---|---|---|---|---|---|
| 8 | The Silent Commit | 7: Hibernate (transactions, locking) | A | integrity · ADR-008 consistency model | `ch07_jpa/` |
| 9 | The Proxy You Didn't Write | 8: Spring Data (JPA/JDBC/Mongo) | A | data evolvability · ADR-009 aggregates | `ch08_data/` |
| 10 | The Breach That Wasn't | 2: Spring Security | N | security · ADR-010 fail-closed zones | `ch02_security/` |
| 11 | Forty Minutes | 11: Testing | N | evolvability · ADR-011 test architecture | `ch11_testing/` |
| 12 | Three A.M. | 5: Actuators / observability | N | operability · ADR-012 SLOs, probes | `ch05_actuator/` |
| 13 | The Cascade | ext: Resilience, caching, async | N | availability · ADR-013 composition | `ch12_extensions/`, `ch09_microservices/` |
| 14 | The Split | 9: Microservices & Spring Cloud | S | evolvability · ADR-014 seam gates | `ch09_microservices/` |
| 15 | Faster | ext: Performance & modern runtime | S | cost · ADR-015 reversible runtime | `ch12_extensions/`, `ch06_server/` |
| 16 | The Architect's Chair | ext: System design (the final gate) | S | judgment · ADR-016 the sequence | all |
| 17 | The Hostile Review | capstone: synthesis (every node, attacked) | all | defensibility · all ADRs | none (App. F and G) |

*Each chapter has exactly one kill-shot, numbered by chapter, and the chapter's debrief opens with it. The `examples/` column gives each chapter's drill directory: commented `.java` files, the drill artifacts the debrief pointers use throughout. The Architect layer column is Appendix F in shorthand: the -ility bought, and the record that priced it.*

*Light-touch roadmap leaves live inside chapters rather than owning one: JSP → Ch.5 (the ViewResolver "road not taken"); Eureka vs platform DNS → Ch.14 (evaluated and declined, with mechanism); Spring Data JDBC & MongoDB → Ch.9 (same grammar, three engines); OAuth2 roles & flows → Ch.10 (authorization server vs resource server). What's missing on purpose: Appendix H.*

## Appendix B: The story-anchor index *(one image per chapter: recall the scene, re-derive the mechanism, then name the -ility it threatened)*

| The scene | The mechanism | The -ility threatened | Ch |
|---|---|---|---|
| 47 green-ink comments on a working feature | "works" is not "done": the senior bar | evolvability | 1 |
| One user's draft report inside another's session | beans are singletons by default, so they must be stateless or scoped | security · scalability | 2 |
| "`this.` goes around the mirror" on the Wall | self-invocation bypasses the AOP proxy, so `@Transactional` silently does nothing | integrity | 3 |
| The health bean that vanished after a version bump | `@ConditionalOnClass` backing off silently, by design | operability | 4 |
| "Your front door leaks your floor plan" (@sable) | errors that bypass `@ControllerAdvice`; `ProblemDetail` as the contract | security | 5 |
| Full thread pool, idle CPUs, site down | thread-per-request: one slow downstream exhausts everything | availability | 6 |
| The 28-second dashboard, 401 flawless queries | N+1; the persistence context stops being invisible | performance | 7 |
| The spreadsheet that disagreed with the bank | caught checked exception = no rollback signal; unchecked-only default | integrity | 8 |
| "Where's the implementation?" | Spring Data generates the proxy at runtime; method names are queries | integrity | 9 |
| A valid critical filed against the security platform itself | authorization lives in the filter chain | security | 10 |
| Forty minutes of CI nobody trusts | context caching: every distinct config, including the bean-override set (`@MockitoBean`), is a new context | evolvability · cost | 11 |
| Kubernetes restart-looping healthy pods | dependency checks belong in readiness, never liveness | availability · operability | 12 |
| "We are the elevated load" on the QPS overlay | retry storms: retries without backoff, jitter, and a budget are a self-inflicted DDoS on a struggling dependency | availability | 13 |
| "Microservices by Q3" argued from both sides | no cross-service transactions; sagas, the outbox and idempotency replace them | integrity · operability | 14 |

| The scene | The mechanism | The -ility threatened | Ch |
|---|---|---|---|
| The carrier thread pinned under load | virtual threads unmount on blocking I/O, except inside `synchronized` (Java 21) | performance · cost | 15 |
| The green pen on Kenza's desk | the architect's job: mechanism, trade-off, and the next engineer | judgment | 16 |
| The ADR binder on the due-diligence table | architecture as a body of evidence | defensibility | 17 |

## Appendix C: The five debug drills *(Ch.16, symptom → first mechanism to check)*

| Symptom | Reason from the mechanism | Story |
|---|---|---|
| Intermittent `LazyInitializationException` | which code path touches a lazy association after its session closed? | Ch.7 |
| Transaction not rolling back | checked exception? caught exception? self-invocation? In each case the proxy never saw a rollback signal | Ch.3, 8 |
| Endpoint slow under load | pool exhaustion math first: worker threads × downstream latency × missing timeouts; then N+1 | Ch.6, 7 |
| Bean not created | `--debug` → `ConditionEvaluationReport`: which condition backed off, and why? | Ch.4 |
| 401 on a route that should be open | walk the filter chain in order: what matched before your permit rule? | Ch.10 |

## Appendix D: Suggested cadence

- **Steady:** read each chapter the evening before you work its `examples/` drills. The incident primes the mechanisms, and the Appendix A table maps each chapter to its drill directory.
- **The debrief drill:** answer the chapter's kill-shot aloud for 90 seconds, then tick the four beats you said: Claim, Mechanism, Trade-off, Failure mode. Three ticks pass. Repeat the next day, then at 3, 7, 16 and 35 days.
- **With an interview date:** follow the four-week path in "How to read this book" at the front. Then drill the anchor index (§B) until every scene surfaces its mechanism unprompted, and the decision layer the same way: the fork index (§D.1) and the flash-

cards (§G) with their answer columns covered, and one pass through the ADR trail (§F), reciting each record as *decision* + *-ility* + *priced cost* before checking the row.

- **The decision pass:** replay only the ⚖ *Your move, architect* forks, indexed below. Commit to an option, out loud, and say why the other two lose before you re-read the reveal.
- **Spaced repetition:** review the anchor index and the flashcards at the same intervals: 1, 3, 7, 16 and 35 days.

### *D.1: The decision pass, indexed (⚖ forks → verdicts)*

Here are the year's eighteen forks on one page, so the replay needs no page-hunt. The table doubles as a self-test: cover the last two columns, argue the losing options at full strength, and commit. Then uncover and grade. The ADR column is where to appeal the verdict.

| Ch | The fork | A / B / C | ADR | Chose | -ility traded |
|---|---|---|---|---|---|
| 1 | Structure the rest of reporting? | layers / feature seams / extract a service | 001 | B | evolvability |
| 2 | Stop the *next* violation? | review vigilance / module walls / fitness functions | 002 | C | enforceability |
| 3 | Transaction strategy codebase-wide? | declarative + discipline / `TransactionTemplate` / AspectJ | 003 | A | operability |
| 4 | Config strategy after the vanished bean? | ad-hoc `@Value` + wiki / typed governance / Config Server | 004 | B | interrogability |
| 5 | Where does the API version live? | path / header / query param | 005 | A | debuggability (one-way door) |
| 6 | Capacity, mid-incident? | more threads + pods / queue-and-async / timeouts + bulkhead | 006 | C | availability |
| 7 | OSIV: on / off now / off behind a window? | on / big-bang / staged | 007 | C | availability |
| 8 | Stop the silent overwrite? | `@Version` → 409 / `FOR UPDATE` / `SERIALIZABLE` | 008 | A | integrity (one-way door) |
| 9 | Dashboards across aggregates? | projection per view / clever fetching / CQRS | 009 | A | evolvability |
| 10 | Token revocation posture? | short TTL / denylist / introspection | 010 | A | statelessness; call overturned at the hostile review (ADR-017) |

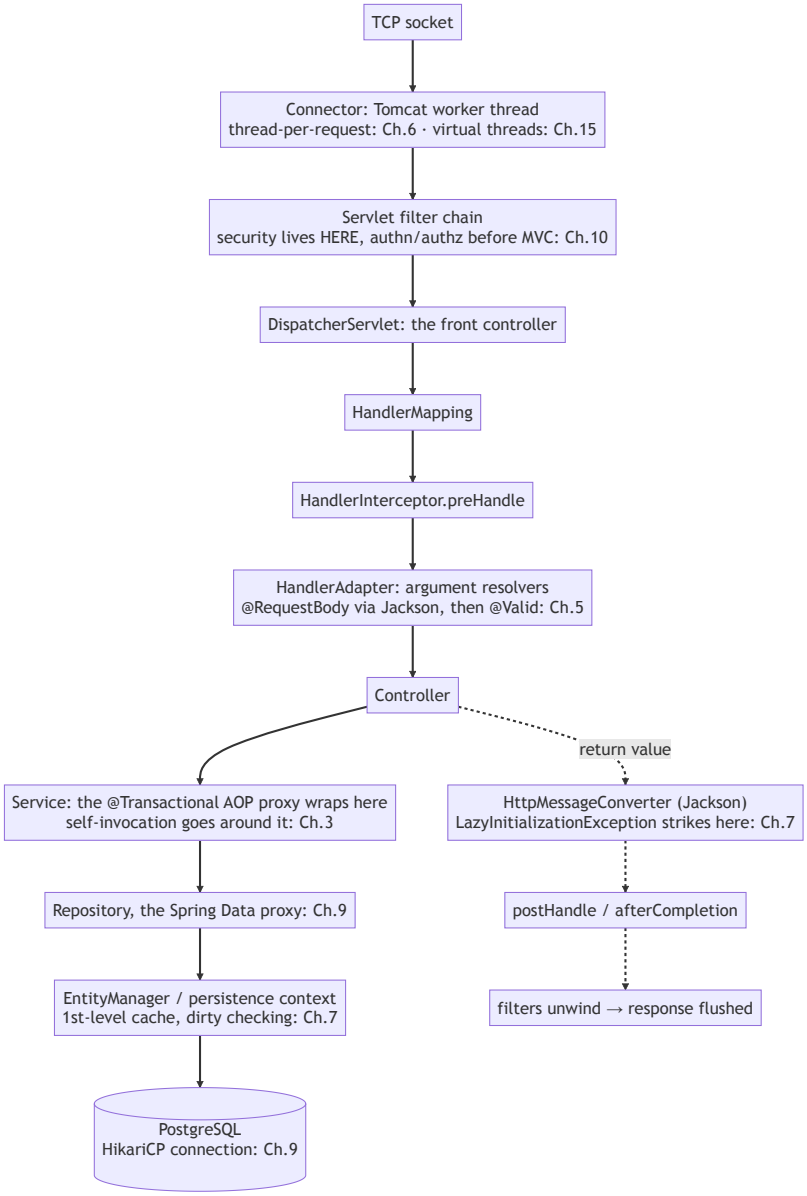
| Ch | The fork | A / B / C | ADR | Chose | -ility traded |
|---|---|---|---|---|---|
| 11 | Land the test standard? | big-bang sprint / ratchet / contracts first | 011 | B | evolvability · cost |
| 12 | What pages a human at 3 a.m.? | causes / SLO burn only / nothing | 012 | B | operability |
| 13 | The detail page when the vendor goes dark? | fail fast / queue-and-drain / `STALE` + age | 013 | C | availability |
| 14 | Split the monolith? | no split / Notifications only / domain-wide by Q3 | 014 | B | evolvability |
| 15 | Runtime: autoscale / vthreads / WebFlux? | scale out / virtual threads / reactive | 015 | B | cost · performance |
| 16 | Where does similarity scoring run? | inline on submit / async advisory / nightly batch | 016 | B | availability |
| 17 | Remediation priority? | revocation / drill / sequence both | 017 | C | judgment |
| 17 | Castellan's "it's only a feed": *name the door first* | internal schema as-is / versioned external contract / batch export | — | B | the exam: schema one-way, transport two-way; misclassification is the trap |

*Score-keeping for the pattern-matchers: the balanced option wins often but not always, and Ch.10's winning call was reopened in Ch.17. Do the analysis. Two forks are labeled one-way doors (Ch.5, Ch.8). The capstone hides a third, unlabeled: there you classify before you choose, and the grade is on the classification.*

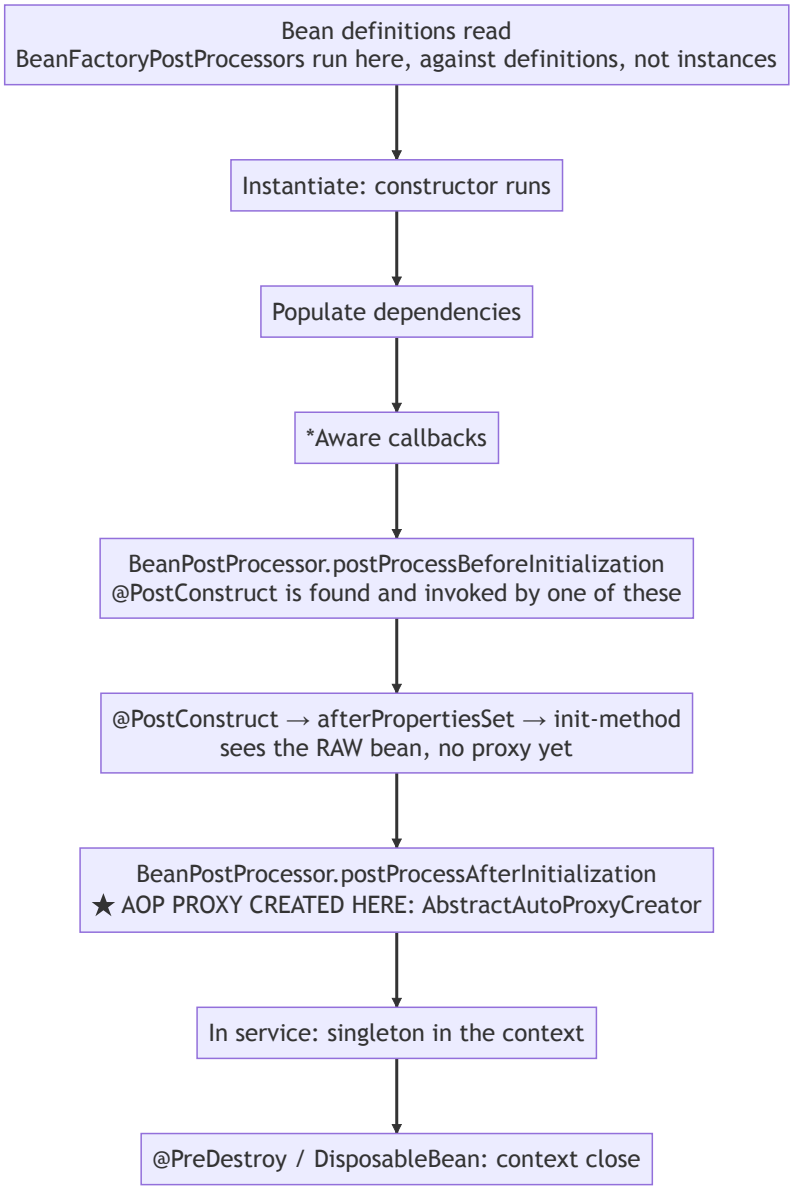
## Appendix E: The Wall, rendered

The five drawings the whole book makes on whiteboards, as actual diagrams: the original two, plus the three the architect layered over them.

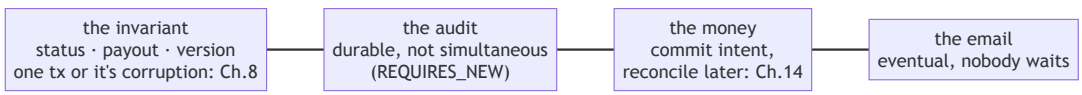
**The Request Journey.** Born in Ch.5; every senior gotcha is a stop on this line:




**The bean lifecycle.** Ch.2, with the point where the proxy is born (Ch.3):

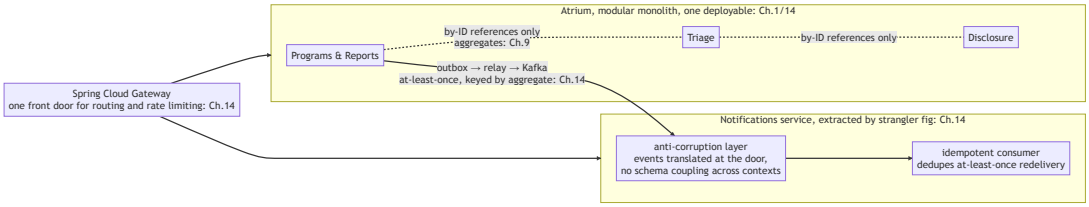



**The consistency spectrum.** Ch.8's long horizontal line under the isolation ladder, the Wall's most-recycled drawing (Ch.14 prices the "money" end with the outbox; Ch.16's design review recites it whole):

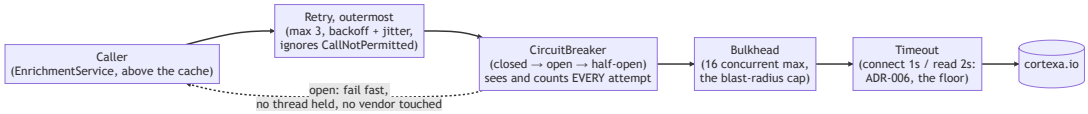


*Consistency priced per write: every step rightward buys throughput and sells simultaneity (ADR-008).*

**The context map.** Chalked as three aggregate boxes in June (Ch.9), named and bordered in October (Ch.14). It shows Atrium's bounded contexts and the one seam that was real enough to cut:




**The resilience composition.** Ch.13's ordering argument, drawn so it had a picture to lose to:




*Why this nesting (ADR-013): every retry attempt crosses the breaker and is counted, so an open breaker ends the storm instead of feeding it; the bulkhead takes a permit per attempt, so the cap holds mid-storm, and a call refused by that cap is neither counted as a vendor failure nor retried; the timeout prices one attempt's worst case. Flip retry inside the breaker and three attempts look like one call: same annotations, opposite system. Ordering is semantics, not style.*

## Appendix F: The ADR trail

This is the binder from end to end. It holds sixteen decisions minted across the year, plus the two candidates the hostile review opened. Each line is the half-page record compressed to its spine: what was decided, which -ility it bought, and the cost accepted *in writing*. A decision you can't show a record for is one you'll re-litigate forever.

| # | Title | Ch | The decision, in one line | Serves | The priced cost |
|---|---|---|---|---|---|
| ADR-001 | Feature seams in the Atrium monolith | 1 | Package by feature with package-private internals; constructors into `final` fields; records, never entities, at the boundary; domain depends on ports | evolvability (testability its leading indicator) | more boundary types and mapping code; arguments with the code that predates the rule |

| # | Title | Ch | The decision, in one line | Serves | The priced cost |
|---|---|---|---|---|---|
| ADR-002 | Executable architecture (fitness functions) | 2 | The constitution becomes a test suite: ArchUnit rules fail the build on field injection, mutable singleton state, crossed dependencies, entities from controllers; eager singleton startup never traded away | evolvability that survives turnover | rule maintenance, documented escape hatches, arguing with a robot |
| ADR-003 | Declarative cross-cutting, with eyes open | 3 | Keep annotation-driven transactions/caching/security, but any annotated method invoked via `this.` is a review *finding*; transaction boundaries live on service entry points only | maintainability of invisible control flow | a checklist line that costs review attention forever, paid knowingly because the alternative was paid in silence |
| ADR-004 | Config governance | 4 | Typed `@Validated @ConfigurationProperties` only; profiles overlay, never fork; every version rides the BOM or gets a named owner; a startup smoke test asserts critical beans exist | supply-chain evolvability · operability | the BOM constrains upgrade freedom in exchange for a tested matrix; config becomes a schema, validated at boot |
| ADR-005 | API contract governance | 5 | RFC 9457 `ProblemDetail` is the only public error shape; records, `@Valid` at binding; deliberate versioning with a published sunset process | contract integrity (interoperability + evolvability), and the security of the error surface | every new endpoint owes a version decision it never used to owe |
| ADR-006 | The latency budget | 6 | Every outbound call carries an explicit timeout derived from a per-endpoint budget; budgets sum along the Request Journey; no unbounded waits anywhere | availability | degraded modes become explicit product negotiations, and every new dependency starts with a budget argument |
| ADR-007 | The fetch policy | 7 | All associations lazy; every read path declares its fetch plan (DTO projection by default, `JOIN FETCH`/`@EntityGraph` where need- | performance as a designed property | a projection type per view, and a quarter of archaeology turn- |

| # | Title | Ch | The decision, in one line | Serves | The priced cost |
|---|---|---|---|---|---|
| | | | ed); OSIV off; hot paths assert SQL statement counts in CI | | ing hidden N+1s into visible plans |
| ADR-008 | The consistency model | 8 | Service boundaries declare `rollbackFor = Exception.class`; `@Version` optimistic locking is the default with a named retry-or-surface policy; pessimistic only for short, hot, machine-paced contention; audit writes ride `REQUIRES_NEW` | consistency/integrity, priced against throughput | conflict UX, and the discipline that isolation levels are named, never defaulted, in review |
| ADR-009 | Aggregate boundaries | 9 | Repositories per aggregate root; cross-aggregate reads via projections and read models; references by ID; migrations append-only; the pool sized by arithmetic, recorded | data-model evolvability · integrity (the transaction's natural boundary) | convenient cross-aggregate `JOIN`s stop being free and become deliberate read-model decisions |
| ADR-010 | Security architecture | 10 | Stateless OAuth2 resource server; the chain ends in `anyRequest().denyAll()`; method security as defense in depth; CSRF off for the token API *on the record*; named CORS allowlist; short tokens, refresh rotation | security as fail-closed structure | an explicit rule for every new route, and a leaked token valid until expiry, accepted in writing (reopened in Ch.17) |
| ADR-011 | Test architecture | 11 | Test at the narrowest slice that exercises the behavior; shared test configs converge context fingerprints under a CI cache-hit budget; Testcontainers Postgres for anything SQL-shaped; a handful of full journeys | evolvability, via the suite that makes change cheap | slice literacy (knowing what each slice loads and pointedly does not) as a team skill |
| ADR-012 | Observability by design | 12 | Liveness is process-only, dependencies live in readiness; every user-facing flow carries a signed SLO with golden signals and business metrics; correlation IDs end to end; a named cardinality | operability | a metrics review joins every feature's definition of done: designed in, or the feature isn't done |

| # | Title | Ch | The decision, in one line | Serves | The priced cost |
|---|---|---|---|---|---|
| | | | budget; page on symptoms, dashboard the causes | | |
| ADR-013 | Resilience composition | 13 | The order is law: timeouts under everything; bounded, jittered retry only on idempotent calls, every attempt crossing the breaker; a bulkhead per dependency; idempotency keys; fallbacks degrade honestly; bounded, named executors | availability under dependency failure | configuration that must be load-tested, not guessed, and a review rule: a bare retry loop is a finding |
| ADR-014 | The split decision | 14 | Atrium stays a modular monolith; Notifications extracts by strangler fig behind the gateway, with outbox, idempotent consumers, an ACL, platform DNS; future cuts clear three gates (no shared transaction, own scaling profile, staffable on-call) | evolvability now, organizational scalability someday | one seam pays distribution's full tax: contracts, partial failure, tracing, a second pager |
| ADR-015 | Runtime selection | 15 | Virtual threads on Atrium after a load-tested bake-off and pinning audit; WebFlux declined with its genuine case named; native image for Notifications only; re-evaluation triggers written down | cost, promoted to first class (with performance per pod) | the pinning audit as a standing review item at every JDK upgrade; native's build/closed-world tax confined to one service |
| ADR-016 | Duplicate-report detection (the design review's output) | 16 | Similarity scoring runs as an async outbox-fed pipeline, never a synchronous gate; suggestions are advisory and only a human merges; the scorer is a port inside the Triage context | integrity of the merge · submit-path availability | suggestions are eventual (seconds, not instant), accepted by product on the record |
| ADR-017 | Revocation: `jti` denylist in the bearer filter *(candidate, opened at the hostile review)* | 17 | Every token's unique ID is checked against a shared denylist in the filter; entries expire with their token's TTL; fail closed | security: revocation in seconds, not TTLs | one lookup per request; the denylist joins the availability budget; a slice of statelessness sold back |

| # | Title | Ch | The decision, in one line | Serves | The priced cost |
|---|---|---|---|---|---|
| ADR-017 | Run the brownout drill *(candidate, opened at the hostile review)* | 17 | The shedding tiers become a quarterly gameday, first run within 30 days, because untested designs are hypotheses | operability: a rehearsed degradation instead of a believed one | two afternoons a quarter, cheaper than discovering rotten flags at 3 a.m. |

*Both candidates merged the same week into tab seventeen (ADR-017, Findings of the June architecture audit), with owners and dates attached (the denylist: Imane, 90 days; the drill: Youssef, 30). It is the trail's first entry written in front of outsiders, and the one the audit priced highest: proof the architecture has a mechanism for being wrong.*

## Appendix G: The Architect's Principles & flashcards

### *The Architect's Principles*

Nineteen sentences the year wrote in green. Each one is a scene first: recall the scene, and the principle argues itself.

1. **"Works" is not "done".** Senior code is judged by how it survives, not whether it runs. Scene: the 47 green-ink comments on a feature that ran flawlessly in the demo (Ch.1).
2. **Every boundary you respect is an option you still own, and the seam, not the conference, decides when to exercise it.** Evolvability is the option ledger. Scene: the January seams the November split ran along (Ch.1/14).
3. **Testability is evolvability's leading indicator.** A class you can't construct in a unit test is architecture you can't change cheaply. Scene: `new ReportDigestService()` with nothing to pass (Ch.1).
4. **An architecture you don't enforce is a rumor.** Review vigilance decays and fitness functions don't. Scene: the red CI build naming its own law, a fortnight after its author broke it (Ch.2).
5. **No annotation is magic: the advice lives in the proxy, and `this.` walks around it.** Show me the mechanism, or the mechanism will show you. Scene: the half-saved triage states (Ch.3).
6. **A bean nobody can explain is a bean nobody owns.** Config is a schema, and absence is a failure class with a test. Scene: eight engineers silent at standup over a vanished health bean (Ch.4).
7. **Every error a user sees is a designed response or a security leak.** The error surface is part of the contract. Scene: @sable's 500 with the JDBC URL in it (Ch.5).

8. **Availability is arithmetic before it is architecture.** Pool ÷ residence time is the ceiling, and residence belongs to your slowest dependency. Scene: the napkin, 7/s × 30 s > 200 threads (Ch.6).
9. **Performance is a designed property, declared in the query and asserted in CI, never a patch.** Scene: the 28-second dashboard's 401 individually flawless statements (Ch.7).
10. **Lean on the framework where the default's -ility bill is one you'd pay anyway; fight it only on the record, mechanism in hand.** Scene: OSIV switched off with a tab in the binder, the fought default as strongest exhibit (Ch.7/17).
11. **Choose the error you can see over the corruption you can't.** Integrity means picking your consistency model on purpose. Scene: the 409 Imane defended to the auditors (Ch.8/17).
12. **The aggregate is the unit of consistency and of change.** Invariants stay inside one transaction, and IDs go across the line. Scene: three chalk boxes that grew into the context map (Ch.9/14).
13. **Fail closed is a structural property, not a vigilance program.** Security lives where forgetting is safe. Scene: `anyRequest().denyAll()` and the route that drifted (Ch.10).
14. **The test suite is the system's second architecture, the one that makes changing the first cheap.** It is evolvability's enabler. Scene: forty distrusted minutes becoming trusted ones (Ch.11).
15. **Observability is designed at feature time: what will 3 a.m. ask, and is the answer already recorded?** The -ility is operability. Scene: the page that arrived with zero data (Ch.12).
16. **Backpressure is a system property, not a queue setting, and a fallback that lies is an incident you've scheduled for later.** Availability is honestly refusing the load you can't survive. Scene: the smoke cloud and the pending badge (Ch.13).
17. **Bet settings, not codebases.** Reversibility is the -ility that keeps every other bet survivable, and cost is priced in dollars, risk, and team. Scene: virtual threads as a property flip, reactive as a one-way door (Ch.15).
18. **A defensible architecture is a body of evidence with a working appeals process, written for the next engineer, not the next meeting.** Defensibility is the -ility above the others. Scene: sixteen tabs on the due-diligence table, two concessions drying in green, the pen changing desks (Ch.16/17).
19. **Know which door you're in.** Two-way doors reward optionality; one-way doors (the public contract, the consistency model) reward decisive commitment, because deferring those means deciding later under worse information. Scene: the version-from-day-one reveal (Ch.5) and the conflict contract (Ch.8).

### *The flashcard index*

Cover the answer column. The question is the cue, because it is the only one an interviewer gives; use the scene only as a hint when the question brings nothing back. The answer is the one breath an architect gets before the follow-up.

| The scene | The interviewer asks | The architect's line |
|---|---|---|
| The 47 green-ink comments on an applauded demo (Ch.1) | Why constructor injection? Isn't field injection less ceremony? | Field injection births the object invalid and installs its organs by reflection (`AutowiredAnnotationBeanPostProcessor`); constructors make it born complete, `final`, testable with plain `new`. The ugly five-arg constructor is the SRP alarm working. |
| The circled, dated `List<Report>` return type (Ch.1) | Why DTOs at the API boundary? | Jackson walks getters after the transaction has returned, so entities detonate lazily, and they couple contract to schema; records make the boundary a deliberate contract (ADR-001). |
| The 6:50 a.m. staging corpse (Ch.2) | How does Spring resolve circular dependencies, and when can't it? | The three-level cache hands out an early reference during population; constructor cycles have nothing early to hand out, so they fail at boot, and startup-time failure is a feature. Since Boot 2.6 even resolvable cycles fail fast unless you opt back in. |
| One user's draft inside another's session (Ch.2) | What's the default bean scope, and its trap? | Singleton: one instance for everyone, so instance state is cross-user state. Beans stay stateless or get scoped. |
| Rachid's pen tapping `BPP.after` (Ch.2) | Walk the lifecycle: where is `@PostConstruct` implemented? | Instantiate → populate → `*Aware` → BPP.before → `@PostConstruct`/init against the *raw* bean → BPP.after, where the AOP proxy is born; `@PostConstruct` is invoked by a `BeanPostProcessor` (`CommonAnnotationBeanPostProcessor`). |
| "`this.` goes around the mirror" (Ch.3) | Why did `@Transactional` silently do nothing? | The advice lives in the proxy your callers hold; self-invocation dispatches on the raw target, so the method runs with whatever transaction the caller had, usually none. That is a review finding under ADR-003, and the same trap catches `@Cacheable` and `@Async`. |
| The health bean that vanished after a | How does auto-configuration decide, and debug a bean that didn't appear? | `AutoConfiguration.imports` nominates, the `@Conditional` family prunes; `--debug`'s `ConditionEvaluationReport` names which condition |

| The scene | The interviewer asks | The architect's line |
|---|---|---|
| version bump (Ch.4) | | backed off, and ADR-004 makes critical-bean absence a smoke-tested failure class. |
| Youssef's env-var flip on staging (Ch.4) | Order the config sources. | CLI > env vars > profile yml > `application.yml` > defaults, with relaxed binding mapping `SCREAMING_SNAKE` onto kebab-case. Profiles overlay a base and never fork it. |
| @sable's 500 with the JDBC URL in it (Ch.5) | Design the error surface of a public API. | One grammar, RFC 9457 `ProblemDetail` from a `@RestControllerAdvice`, because an unhandled error is information disclosure: a contract violation, not a cosmetic bug (ADR-005). |
| "Who calls your controller method? Name it." (Ch.5) | Trace a request, socket to DB. | Connector thread → filter chain (security here) → `DispatcherServlet` → HandlerMapping → interceptors → HandlerAdapter's argument resolvers → controller → `@Transactional` proxy → repository proxy → persistence context → pooled connection, then back out through the message converters. |
| 196 of 200 threads parked in the same socket read (Ch.6) | Site down, CPUs idle. Explain. | Thread-per-request: workers are the scarce resource, and one slow downstream confiscates the threads every feature shares, so the blast radius is the pool, not the feature. |
| The napkin: 7/s × 30 s > 200 (Ch.6) | How do you size the pool and set timeouts? | Little's law: demanded concurrency = arrival rate × residence. Timeouts cap residence per a recorded latency budget, so a slow dependency fails fast and partially (ADR-006). |
| The 28-second dashboard, 401 flawless queries (Ch.7) | What's N+1, and the senior fix? | Each lazy association resolves per row; the fix is a *declared* fetch plan (DTO projection by default, `JOIN FETCH`/`@EntityGraph` where the view needs entities), asserted as statement counts in CI (ADR-007). |
| The 2 a.m. digest job dying (Ch.7) | Three fixes for `LazyInitializationException`, with trade-offs. | Fetch in the query (a plan per view), project to DTOs (mapping types forever), or hold the session open via OSIV, which hides N+1s and keeps the request's connection until rendering ends. OSIV is the default ADR-007 turned off on the record. |
| The spreadsheet that disagreed with | Why didn't the transaction roll back? | Default rollback marks only unchecked exceptions, an EJB inheritance, and a *caught* exception never reaches the proxy at all; boundaries declare `rollbackFor = Exception.class`, and catching inside |

| The scene | The interviewer asks | The architect's line |
|---|---|---|
| the bank (Ch.8) | | one is a design-review question (ADR-008). If the exception first crossed an inner `REQUIRED` proxy and matched its rollback rules, the shared transaction is already rollback-only, and the outer commit throws `UnexpectedRollbackException`. |
| Two triagers, two green toasts, one overwrite (Ch.8) | Optimistic or pessimistic locking? | `@Version` turns the lost update into a visible 409 the user resolves; pessimistic relocates the cost into a row lock held across human think time. Choose by contention shape and hold time. |
| Imane's hour-long hunt for a class that doesn't exist (Ch.9) | Where's the repository implementation? | It is generated at runtime, a proxy over `SimpleJpaRepository` that parses method names into queries. Names compile but intent doesn't: the missing `Desc` shipped the oldest report as "latest." |
| The 4:55 p.m. checksum standoff (Ch.9) | Can we just edit that migration? | Never. Applied migrations are history, not code; roll forward with a new version, and `repair` only when you can explain exactly what you're repairing. |
| Three chalk boxes, arrows carrying only IDs (Ch.9) | What is an aggregate, and why repositories per root? | The unit of consistency and change: invariants inside one transaction, by-ID references across the line, cross-aggregate reads as read models. It is the context map at smaller scale (ADR-009). |
| The matcher guarding routes that had moved (Ch.10) | What ends your filter chain? | `anyRequest().denyAll()`. URL rules drift from the routes they guard, so a forgotten route must fail closed (403); fail-closed is structure, not vigilance (ADR-010). |
| A researcher token returning `200 OK` from an admin endpoint (Ch.10) | Where is a JWT validated, and why is CSRF off? | In the bearer filter before the servlet: signature against cached JWKs, issuer, audience, expiry. CSRF defends ambient cookie credentials, and a token API carries none, so it is disabled *on the record*. |
| Imane's spreadsheet of seventeen boots (Ch.11) | Why is CI forty minutes, and what's the lever? | The context cache keys on config classes + properties + profiles + every bean override, so each `@MockitoBean` fingerprint mints a fresh boot; converge on shared test configs under a CI cache-hit budget (ADR-011). |

| The scene | The interviewer asks | The architect's line |
|---|---|---|
| "Remove H2. It was never Postgres." (Ch.11) | In-memory DB or Testcontainers? | Test the engine that takes production traffic. H2 is a standing decision not to know Postgres truths; slices plus Testcontainers buy speed *and* honesty. |
| Pods restart-looping after the blip had passed (Ch.12) | Liveness vs readiness? | Liveness asks "should I be restarted?" and must stay process-only; dependencies belong in readiness, which sheds traffic instead of killing pods. Otherwise the cluster executes pods for telling the truth (ADR-012). |
| Leila writing the SLO figures down unasked (Ch.12) | Who decides reliability versus features? | The error budget, forty-three minutes a month at 99.9%, decides by signature, not mood. Alerts page on SLO burn (symptoms), and dashboards carry causes. |
| `/env` as a reconnaissance gift (Ch.12) | Which Actuator endpoints do you expose? | `health`, `info`, `prometheus`, on a management port the ingress never routes; `/env` maps your config surface and `/heapdump` hands over memory, so dark-by-default is the contract (ADR-012). |
| The cache-hit panel flat at zero (Ch.13) | Your `@Cacheable` never caches. First check? | Self-invocation, February's proxy ghost in a new coat: the call never crossed the proxy, so the annotation was decoration; one mechanism, three annotations (`@Transactional`, `@Cacheable`, `@Async`). |
| The retry loop labeled `temporary` (Ch.13) | Order retry, breaker, bulkhead, timeout. And why? | Retry outermost so every attempt crosses the breaker and an open breaker *ends* the storm; the bulkhead takes a permit per attempt; the timeout prices one attempt. Flip the nesting and three attempts look like one call (ADR-013). |
| "Microservices by Q3," argued from both sides (Ch.14) | When do you split a monolith? | When a seam clears three gates: no shared transaction, its own scaling profile, a staffable on-call. Notifications passed, nothing else did: the seam decides, not the conference (ADR-014). |
| The duplicate "report triaged" email (Ch.14) | Your event arrived twice. Whose bug? | Nobody's: at-least-once *is* the contract. The outbox writes the event inside the business transaction and a relay publishes it, so the consumer dedupes by event ID; idempotency is the receiver's half of the pattern. |
| Eureka, evaluated and de- | Why no service registry on Kubernetes? | A k8s Service *is* the registry: DNS name, kube-proxy load balancing, readiness-driven eviction. Eureka |

| The scene | The interviewer asks | The architect's line |
|---|---|---|
| clined (Ch.14) | | there is a second discovery system to disagree with the first (ADR-014). |
| `<==` `monitors:1` on the war-room screen (Ch.15) | Virtual threads vs WebFlux, and the catch? | Virtual threads keep thread-per-request semantics and unmount on blocking I/O, except when pinned (`synchronized` plus blocking, on that JDK); audit with JFR's `jdk.VirtualThreadPinned`, which fires only when a pinned thread parks, and with `jdk.JavaMonitorEnter`, its 20 ms threshold lowered, for a contended monitor whose holder never parks; quarantine vendor code on a platform pool. You bet a setting, reactive bets the codebase (ADR-015). |
| Notifications starting in 80 ms after a nine-minute build (Ch.15) | When does native image pay? | Scale-to-zero, CLIs, serverless: startup and memory bought with build time and a closed world (declare your reflection); a hot, JIT-warmed monolith is the opposite end of the curve (ADR-015). |
| The empty chair at the whiteboard (Ch.16) | "Design duplicate detection." Go. | Clarify load, accuracy, and the cost asymmetry *first*; then contract → data → transactions → failures → observability → rollout. That yields an async advisory pipeline off the outbox, and only a human merges, because the domain owns truth (ADR-016). |
| The ADR binder on the due-diligence table (Ch.17) | Defend this architecture. | Defend decisions, not code: four beats each, grounded in named incidents, traced to dated records that name their own revisit conditions. Where the context has moved, concede and price the fix in the room; the concession *is* the senior answer. |

## Appendix H: The deliberate omissions

The book leaves six topics out by choice, and each has its reason and its door.

| Left out | Why it's out | The door |
|---|---|---|
| **Reactive-first architectures** | WebFlux appears only as virtual threads' counterargument (Ch.15); event-loop-first systems, streaming backpressure, and RSocket are a different book's spine | Project Reactor reference docs |
| **Messaging depth** | Kafka and RabbitMQ appear one breath at a time (Ch.13 and 14); consumer groups, partitioning, exactly-once semantics, and stream processing deserve their own book | Spring for Apache Kafka / Spring AMQP reference docs |

| Left out | Why it's out | The door |
|---|---|---|
| **Spring Batch** | Bulk offline processing, which the story never needed | Spring Batch reference docs |
| **SSE / WebSockets** | Glasshouse's UI polls; push channels never earned a scene | Spring Framework WebSocket & SSE reference |
| **GraphQL** | Ironically, half the real bug-bounty industry exposes it. Schema-driven APIs, N+1 at the resolver layer, and persisted queries go unexplored here | Spring for GraphQL reference docs |
| **gRPC / contract-first RPC** | Internal contracts here are REST + events; binary contract-first RPC never had a seam to live in | gRPC docs · Spring gRPC project |

*Glasshouse shipped, scaled, and survived its audit without any of the six. Your next system may not be so polite, and when it isn't, the doors above open from this side.*

## Appendix I: Version history

| Version | Date | What changed |
|---|---|---|
| Storybook v1 | 2026-06-10 | The original sixteen-incident narrative (senior edition): cast, incidents, four-beat debriefs, anchor index. Preserved as `Spring-Boot-Storybook.md`. |
| Architect Edition v1.0 | 2026-06-11 | The architect layer: -ilities ledger, ADR trail (001 to 017), ⏸/⚖ devices, ARCHITECT'S LENS, Ch.17 *The Hostile Review*, Appendices F and G. |
| Architect Edition v1.1 | 2026-06-11 | Five-lens review applied: missing artifacts added (optimistic-retry wrapper, outbox relay, MDC config, full ADR file), Ch.10 security-chain converter fix, fork index (D.1). |
| Architect Edition v1.2 | 2026-06-12 | External critique applied: prose density flattened at mechanism peaks, self-healing callbacks, fork outcomes redistributed (Ch.10's call marked overturned), Appendix H (deliberate omissions), JEP 491 named, claims ledger started. |
| Mastery Book v1.0 | 2026-06-12 | Pure-Java recast: protagonist re-canonized (Meridian Freight, frameworkless Java), every cross-language beat replaced with a plain-Java naive-instinct beat teaching the same mechanism; beginner on-ramp opened; zero cross-language framing (verified by sweep). |
| Mastery Book v1.1 | 2026-06-12 | Two forks converted to one-way doors (Ch.5 public contract, Ch.8 consistency contract; middle option explicitly wrong); the door distinction taught in Ch.16 and Appendix G; per-Part plain-prose primers for the beginner on-ramp; a light plain-first touch-pass on selected mechanism paragraphs (the chapters keep their intensi- |

| Version | Date | What changed |
|---|---|---|
| | | ty by design; the primers and debriefs are the flat ground); residual cross-language stragglers removed. |
| Mastery Book v1.2 | 2026-06-12 | Honesty pass on this changelog (the v1.1 prose claim re-scoped to what the diff shows) plus a front-matter "note on temperature" stating the intensity strategy plainly; Part primers de-spoiled: twelve lines rewritten from chapter diagnosis to neutral vocabulary (the investigations stay discoveries); one new unlabeled door-classification fork in Ch.17 ("name the door first", the exam version of Ch.16's skill), indexed in D.1; VERSION-CLAIMS updated to match. |
| Mastery Book v1.3 | 2026-06-13 | Cover only; the text is unchanged. Redesigned `docs/cover.svg` (the B.E.A.N.S. building rebalanced; the beginner→architect arc added as a `MECHANIC → SENIOR → ARCHITECT` strip and a hook line; a B.E.A.N.S. legend; dead space and the ghost watermark removed) and wired it into the build: EPUB cover image + PDF cover page via `tools/build-book.py`. No prose, code, ADR, or version-sensitive claim changed. |
| Mastery Book v1.4 | 2026-06-13 | Author attribution (Otmane Omry, with role and links) added as a byline on the title front matter, and the author name added to the cover (edition line bumped to v1.4). Manning-inspired interior styling landed in the build: IntelliJ-Light syntax colours for code, justified serif body with chapter drop caps, blue section headings, sidebar-style notes, page-fitted diagrams, and a clean contents page. No chapter prose, ADR, or version-sensitive claim changed. |
| Mastery Book v1.5 | 2026-06-14 | Accuracy pass from three external model critiques (kept what verified, rejected what was stale or wrong). Corrected: the Prometheus scrape interval is now stated as Glasshouse's deployment choice (its shipped default is one minute). Sharpened: an OSIV startup-warning beat in Ch.7 (Boot logs it; nobody read it); a clause noting JEP 491 doesn't touch native-frame pinning (Ch.15); audience framing tightened (dropped the "principal-architect" reader-claim and "only prerequisite"). VERSION-CLAIMS swept: rows 28, 36 web-verified against Boot 4 / Framework 7 docs, rows 66, 71 confirmed, row 80 corrected; rows 75, 84 remain `CHECK`. No ADRs or chapter incidents changed. |
| Mastery Book v1.6 | 2026-06-14 | Second critique round. Fixed a real test trap the reviews didn't catch: Ch.11's `@DataJpaTest` Testcontainers example now pins `@AutoConfigureTestDatabase(replace = NONE)` so the slice can't silently swap in H2; it no longer leans on Boot's (version-dependent) service-connection back-off. Completed the version audit: rows 75 and 84 verified (Spring Cloud Gateway's `spring.cloud.gateway.server.webflux.*` prefix confirmed), so VERSION-CLAIMS now has zero `CHECK` rows. Added a Part II primer clause (a singleton's fields are shared across concurrent requests) to bridge Ch.2 for newcomers. README notes the book/EPUB/PDF/site are standalone (the agent team is optional). Deferred with reasons: narrative-polish rewrites and per-chapter mechanism cards. No ADRs or chapter incidents changed. |

| Version | Date | What changed |
|---|---|---|
| Mastery Book v2.0 | 2026-06-14 | Shipped the runnable-lab project deferred at v1.6: `labs/`, a multi-module Maven build (Spring Boot 4.0.7 · Java 21) where five of the book's traps run as tests. They are self-invocation (Ch.3), N+1 (Ch.7), `@DataJpaTest`-on-real-Postgres via Testcontainers (Ch.11), liveness-vs-readiness (Ch.12), and a missing read timeout (Ch.6). Each test asserts the *observable* symptom (a query count, an HTTP status, a thrown timeout, an active-transaction flag), so the proof survives refactoring; lab-03 skips cleanly without Docker, so `./mvnw test` stays green on any machine. Wired into the front matter and the relevant chapters' Pointers ("**run it:** …"). Building the suite surfaced and pinned three Boot 4 relocations the prose hadn't needed yet: the per-slice test starters (`spring-boot-starter-data-jpa-test` now carries `@DataJpaTest`/`TestEntityManager`), the `spring-boot-health` API move, and Testcontainers 2.x's `testcontainers-*` module renames. No chapter prose, ADR, or version-sensitive claim changed. |
| Mastery Book v2.1 | 2026-11-10 | Cast only. The human cast now carries Moroccan first names, and first names only: Maya Reyes → Kenza, Elias Brandt → Rachid, Priya Sharma → Imane, Idris Okafor → Youssef, Lena Park → Leila, Marisol Vega → Malika, and Chapter 17's reviewers Verena Koss → Amina and Marcus Tan → Mounir. The green-ink initials follow (E.B. → R.), and one lunch tin moved from jollof on Tuesdays to couscous on Fridays. `@sable` and every company, product, and system name are unchanged. The rename was scripted and residue-checked. No mechanism, code, decision, date, or version-sensitive claim changed. |
| Mastery Book v2.2 | 2026-11-10 | True on Boot 4. Class names and property keys were checked against the Spring Boot 4.0.7 jars, and printed code was compiled or run. Corrected, among others: `HttpClientSettings` replaces a class that no longer exists (Ch.6); the error and OTLP properties moved to `spring.web.error.*` and `management.opentelemetry.tracing.export.otlp.endpoint` (Ch.5, Ch.12); a `final` method on a CGLIB proxy runs on the proxy instance (Ch.3); OSIV keeps the connection until rendering ends (Ch.7); a caught exception after an inner `REQUIRED` call ends in `UnexpectedRollbackException` (Ch.8); the JDK 21 socket frame is `NioSocketImpl.implRead`, and `jdk.JavaMonitorEnter` finds the monitor that `jdk.VirtualThreadPinned` cannot (Ch.6, Ch.15); `sync = true` on the Caffeine cache pins a virtual thread on JDK 21, so the Chapter 15 fix now removes it. The checks run on every build: `tools/property-check.py`, `tools/symbol-check.py`, and one command for everything, `tools/build-all.sh`. |
| Mastery Book v2.3 | 2026-11-10 | Drill. Each debrief now opens with the chapter's kill-shot, numbered by chapter, then the follow-up questions and a four-line tick list. The model answer comes after a rule, so you answer first. Added: a pass gate of three ticks, a four-week path in "How to read this book", a glossary line for each noun in the Part primers, and `labs/lab-00-first-bean`, the smallest application the book can run. Removed: every pointer to material that is not published with the book. Each fork now asks |

| Version | Date | What changed |
|---|---|---|
| | | why the two losing options lose before the reveal. Story continuity was fixed across chapters: report ids, dates and pod counts. |
| **Mastery Book v2.4** | **2026-11-10** | **This version.** Companions and prose. `examples/` lost its cross-language notes, compiles against Boot 4.0.7 as part of the build, and had its comments corrected against the jars. The whole book went through a style pass: plain sentences, no em or en dashes, headings with a colon, bold kept for labels and -ility names. A script, `tools/prose-invariants.py`, proved that the pass changed no code, number, heading or drill question. `docs/VERSION-CLAIMS.md` now records what checked each claim: a jar, a run, a document, or a reading. |

*Maintenance rule: run `./tools/build-all.sh` after any edit. An edit that touches a version-sensitive fact also updates `docs/VERSION-CLAIMS.md`, and every release adds a row here.*

---

*The single source is `docs/Spring-Boot-Mastery-Book.md`; companion artifacts: `examples/` (the drills) and `labs/` (the traps as tests). The roadmap scope mirrors roadmap.sh/spring-boot.*